

Introduction to Programming Using

Python



► Y. DANIEL LIANG

ONLINE ACCESS

Thank you for purchasing a new copy of *Introduction to Programming Using Python*. Your textbook includes six months of prepaid access to the book's Companion Website. This prepaid subscription provides you with full access to the following student support areas:

- VideoNotes are step-by-step video tutorials specifically designed to enhance the programming concepts presented in this textbook.
- Web Chapters are included for more advanced courses.

Use a coin to scratch off the coating and reveal your student access code.
Do not use a knife or other sharp object as it may damage the code.

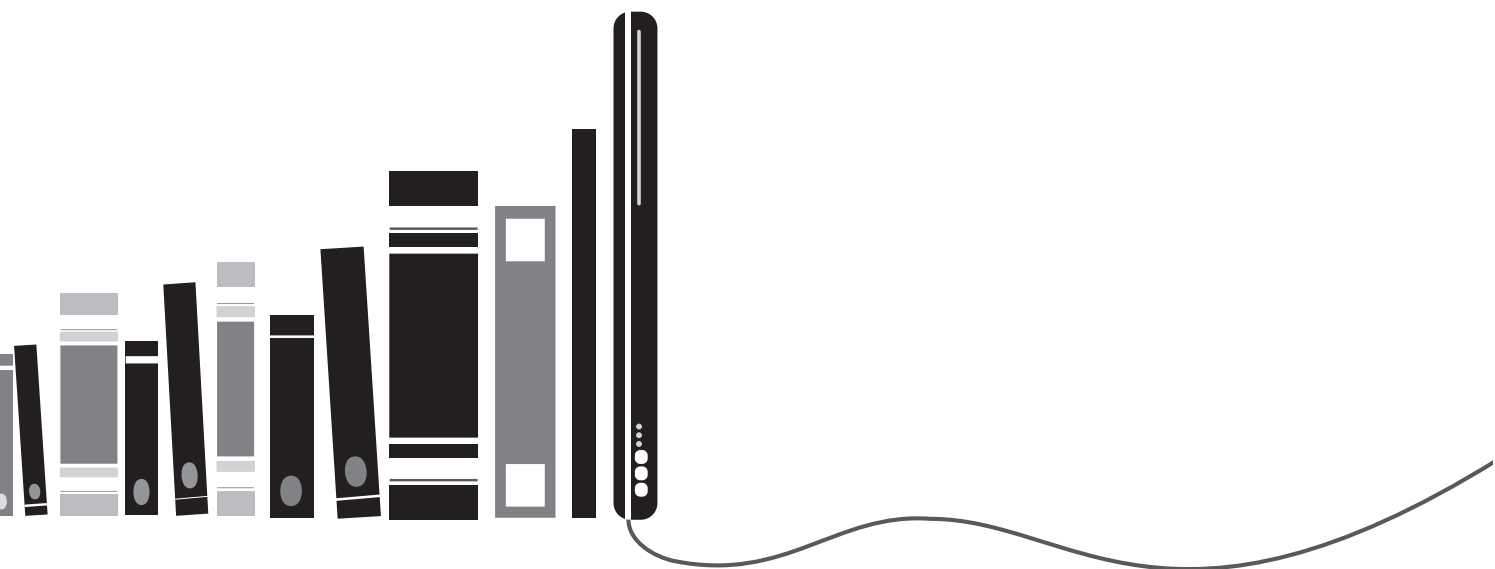
To access the *Introduction to Programming Using Python* Companion Website for the first time, you will need to register online using a computer with an Internet connection and a web browser. The process takes just a couple of minutes and only needs to be completed once.

1. Go to <http://www.pearsonhighered.com/liang>
2. Click on **Companion Website**.
3. Click on the **Register** button.
4. On the registration page, enter your student access code* found beneath the scratch-off panel. Do not type the dashes. You can use lower- or uppercase.
5. Follow the on-screen instructions. If you need help at any time during the online registration process, simply click the **Need Help?** icon.
6. Once your personal Login Name and Password are confirmed, you can begin using the *Introduction to Programming Using Python* Companion Website!

To log in after you have registered:

You only need to register for this Companion Website once. After that, you can log in any time at <http://www.pearsonhighered.com/liang> by providing your Login Name and Password when prompted.

*Important: The access code can only be used once. This subscription is valid for six months upon activation and is not transferable. If this access code has already been revealed, it may no longer be valid. If this is the case, you can purchase a subscription by going to <http://www.pearsonhighered.com/liang> and following the on-screen instructions.



get with the programming

Through the power of practice and immediate personalized feedback, MyProgrammingLab improves your performance.

MyProgrammingLab™

Learn more at www.myprogramminglab.com

This page intentionally left blank

INTRODUCTION TO
PROGRAMMING USING
PYTHON

Y. Daniel Liang

Armstrong Atlantic State University

PEARSON

Boston Columbus Indianapolis New York San Francisco Upper Saddle River
Amsterdam Cape Town Dubai London Madrid Milan Munich Paris Montreal Toronto
Delhi Mexico City Sao Paulo Sydney Hong Kong Seoul Singapore Taipei Tokyo

Editorial Director, ECS: Marcia Horton
Editor-in-Chief: Michael Hirsch
Executive Editor: Tracy Dunkelberger
Associate Editor: Carole Snyder
Director of Marketing: Patrice Jones
Marketing Manager: Yezan Alayan
Marketing Coordinator: Kathryn Ferranti
Director of Production: Vince O'Brien
Managing Editor: Jeff Holcomb
Production Project Manager: Heather McNally
Manufacturing Buyer: Lisa McDowell
Art Director and Cover Designer: Anthony Gemmellaro
Text Designer: Gillian Hall
Cover Art: "Life Aquatic" © Arthur Xanthopoulos from Damaged Photography
Media Editor: Daniel Sandin
Full-Service Project Management: Gillian Hall
Composition: Laserwords
Printer/Binder: Edwards Brothers
Cover Printer: Lehigh-Phoenix Color/Hagerstown

Credits and acknowledgments borrowed from other sources and reproduced, with permission, in this textbook appear on Credits page located in the end matter.

Microsoft® and Windows® are registered trademarks of the Microsoft Corporation in the U.S.A. and other countries. Screen shots and icons reprinted with permission from the Microsoft Corporation. This book is not sponsored or endorsed by or affiliated with the Microsoft Corporation.

Copyright © 2013 Pearson Education, Inc., publishing as Prentice Hall. All rights reserved. Manufactured in the United States of America. This publication is protected by Copyright, and permission should be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. To obtain permission(s) to use material from this work, please submit a written request to Pearson Education, Inc., Permissions Department, One Lake Street, Upper Saddle River, New Jersey 07458, or you may fax your request to 201-236-3290.

Many of the designations by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and the publisher was aware of a trademark claim, the designations have been printed in initial caps or all caps.

Library of Congress Cataloging-in-Publication Data on file.

Prentice Hall
is an imprint of



www.pearsonhighered.com

10 9 8 7 6 5 4 3 2 1

ISBN 13: 978-0-13-274718-9
ISBN 10: 0-13-274718-9

*This book is dedicated to my former colleagues
at the National Severe Storms Laboratory,
in Norman, Oklahoma.*

To Samantha, Michael, and Michelle

PREFACE

Dear Reader,

what is programming?

This book assumes that you are a new programmer with no prior knowledge of programming. So, what is programming? Programming solves problems by creating solutions—writing programs—in a programming language. The fundamentals of problem solving and programming are the same regardless of which programming language you use. You can learn programming using any high-level programming language such as Python, Java, C++, or C#. Once you know how to program in one language, it is easy to pick up other languages, because the basic techniques for writing programs are the same.

why Python?

So what are the benefits of learning programming using Python? Python is easy to learn and fun to program. Python code is simple, short, readable, intuitive, and powerful, and thus it is effective for introducing computing and problem solving to beginners.

graphics

Beginners are motivated to learn programming so they can create graphics. A big reason for learning programming using Python is that you can start programming using graphics on day one. We use Python's built-in Turtle graphics module in Chapters 1–6 because it is a good pedagogical tool for introducing fundamental concepts and techniques of programming. We introduce Python's built-in Tkinter in Chapter 9, because it is a great tool for developing comprehensive graphical user interfaces and for learning object-oriented programming. Both Turtle and Tkinter are remarkably simple and easy to use. More importantly, they are valuable pedagogical tools for teaching the fundamentals of programming and object-oriented programming.

optional Turtle

To give instructors flexibility to use this book, we cover Turtle at the end of Chapters 1–6 so they can be skipped as optional material.

problem-driven

The book teaches problem solving in a problem-driven way that focuses on problem solving rather than syntax. We stimulate student interests in programming by using interesting examples in a broad context. While the central thread of the book is on problem solving, appropriate Python syntax and library are introduced in order to solve the problems. To support the teaching of programming in a problem-driven way, the book provides a wide variety of problems at various levels of difficulty to motivate students. In order to appeal to students in all majors, the problems cover many application areas in math, science, business, financial management, gaming, animation, and multimedia.

fundamentals first

All data in Python are objects. We introduce and use objects from Chapter 3, but defining custom classes are covered in the middle of the book starting from Chapter 7. The book focuses on fundamentals first: it introduces basic programming concepts and techniques on selections, loops, and functions before writing custom classes.

examples and exercises

The best way to teach programming is *by example*, and the only way to learn programming is *by doing*. Basic concepts are explained by example and a large number of exercises with various levels of difficulty are provided for students to practice. Our goal is to produce a text that teaches problem solving and programming in a broad context using a wide variety of interesting examples and exercises.

Sincerely,

Y. Daniel Liang
y.daniel.liang@gmail.com
www.cs.armstrong.edu/liang
www.pearsonhighered.com/liang

Pedagogical Features

The book uses the following elements to get the most from the material:

- **Objectives** list what students should learn in each chapter. This will help them determine whether they have met the objectives after completing the chapter.
- The **Introduction** opens the discussion with representative problems to give the reader an overview of what to expect from the chapter.
- **Key Points** highlight the important concepts covered in each section.
- **Check Points** provide review questions to help students track their progress and evaluate their learning.
- **Problems**, carefully chosen and presented in an easy-to-follow style, teach problem solving and programming concepts. The book uses many small, simple, and stimulating examples to demonstrate important ideas.
- **Key Terms** are listed with a page number to give students a quick reference to the important terms introduced in the chapter.
- The **Chapter Summary** reviews the important subjects that students should understand and remember. It helps them reinforce the key concepts they have learned in the chapter.
- **Test Questions** are available online, grouped by sections for students to do self-test on programming concepts and techniques.
- **Programming Exercises** are grouped by sections to provide students with opportunities to apply on their own the new skills they have learned. The level of difficulty is rated as easy (no asterisk), moderate (*), hard (**), or challenging (***). The trick of learning programming is practice, practice, and practice. To that end, the book provides a great many exercises.
- **Notes, Tips, and Cautions** are inserted throughout the text to offer valuable advice and insight on important aspects of program development.



Note

Provides additional information on the subject and reinforces important concepts.



Tip

Teaches good programming style and practice.



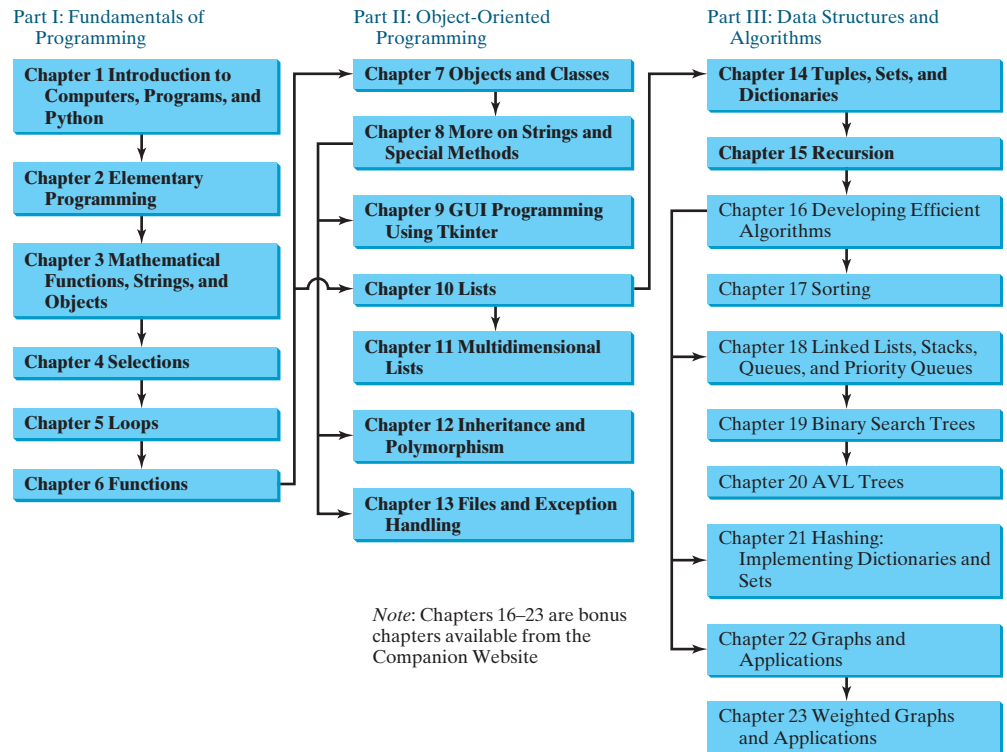
Caution

Helps students steer away from the pitfalls of programming errors.

Flexible Chapter Orderings

Graphics is a valuable pedagogical tool for learning programming. The book uses Turtle graphics in Chapters 1–6 and Tkinter in the rest of the book. However, the book is designed to give the instructors the flexibility to skip the sections on graphics or to cover them later. The following diagram shows the chapter dependencies.

Chapter 10, Lists can be covered right after Chapter 6, Functions. Chapter 14, Tuples, Sets, and Dictionaries can be covered after Chapter 10.



Organization of the Book

The chapters can be grouped into three parts that, taken together, form a comprehensive introduction to Python programming. Because knowledge is cumulative, the early chapters provide the conceptual basis for understanding programming and guide students through simple examples and exercises; subsequent chapters progressively present Python programming in detail, culminating with the development of comprehensive applications.

Part I: Fundamentals of Programming (Chapters 1–6)

The first part of the book is a stepping stone, preparing you to embark on the journey of learning programming. You will begin to know Python (Chapter 1) and will learn fundamental programming techniques with data types, variables, constants, assignments, expressions, operators, objects, and simple functions and string operations (Chapters 2–3), selection statements (Chapter 4), loops (Chapter 5), and functions (Chapter 6).

Part II: Object-Oriented Programming (Chapters 7–13)

This part introduces object-oriented programming. Python is an object-oriented programming language that uses abstraction, encapsulation, inheritance, and polymorphism to provide great flexibility, modularity, and reusability in developing software. You will learn object-oriented programming (Chapters 7–8), GUI programming using Tkinter (Chapter 9), lists (Chapter 10), multidimensional lists (Chapter 11), inheritance, polymorphism, and class design (Chapter 12), and files and exception handling (Chapter 13).

Part III: Data Structures and Algorithms (Chapters 14–15 and Bonus Chapters 16–23)

This part introduces the main subjects in a typical data structures course. Chapter 14 introduces Python built-in data structures: tuples, sets, and dictionaries. Chapter 15 introduces

recursion to write functions for solving inherently recursive problems. Chapters 16–23 are bonus chapters on the Companion Website. Chapter 16 introduces measurement of algorithm efficiency and common techniques for developing efficient algorithms. Chapter 17 discusses classic sorting algorithms. You will learn how to implement linked lists, queues, and priority queues in Chapter 18. Chapter 19 presents binary search trees, and you will learn about AVL trees in Chapter 20. Chapter 21 introduces hashing, and Chapters 22 and 23 cover graph algorithms and applications.

Student Resource Website

The Student Resource Website (www.cs.armstrong.edu/liang/py) contains the following resources:

- Answers to review questions
- Solutions to even-numbered programming exercises
- Source code for the examples in the book
- Interactive self-test questions (organized by sections for each chapter)
- Supplements on using Python IDEs, advanced topics, etc.
- Resource links
- Errata

Additional Supplements

The text covers the essential subjects. The supplements extend the text to introduce additional topics that might be of interest to readers. The supplements listed in this table are available from the Companion Website.

Part I. General Supplements

- A. Glossary
- B. Installing and Using Python
- C. Python IDLE
- D. Python on Eclipse
- E. Python on Eclipse Debugging
- F. Python Coding Style Guidelines

Part II. Advanced Python Topics

- A. Regular Expressions
- B. Obtaining Date and Time
- C. The `str` Class's `format` Method
- D. Pass Arguments from Command Line
- E. Database Programming

Instructor Resource Website

The Instructor Resource Website, accessible from www.cs.armstrong.edu/liang/py, contains the following resources:

- Microsoft PowerPoint slides with interactive buttons to view full-color, syntax-highlighted source code and to run programs without leaving the slides.
- Solutions to all the review questions and exercises. Students will have access to the solutions of even-numbered programming exercises.
- Web-based quiz generator. (Instructors can choose chapters to generate quizzes from a large database of more than 800 questions.)

- Sample exams. In general, each exam has four parts:
 - Multiple-choice questions or short-answer questions
 - Correct programming errors
 - Trace programs
 - Write programs
- Projects. In general, each project gives a description and asks students to analyze, design, and implement the project.

Some readers have requested the materials from the Instructor Resource Website. Please understand that these are for instructors only. Such requests will not be answered.

MyProgrammingLab™

Online Practice and Assessment with MyProgrammingLab

MyProgrammingLab helps students fully grasp the logic, semantics, and syntax of programming. Through practice exercises and immediate, personalized feedback, MyProgrammingLab improves the programming competence of beginning students who often struggle with the basic concepts and paradigms of popular high-level programming languages.

A self-study and homework tool, a MyProgrammingLab course consists of hundreds of small practice problems organized around the structure of this textbook. For students, the system automatically detects errors in the logic and syntax of their code submissions and offers targeted hints that enable students to figure out what went wrong—and why. For instructors, a comprehensive gradebook tracks correct and incorrect answers and stores the code inputted by students for review.

MyProgrammingLab is offered to users of this book in partnership with Turing's Craft, the makers of the CodeLab interactive programming exercise system. For a full demonstration, to see feedback from instructors and students, or to get started using MyProgrammingLab in your course, visit www.myprogramminglab.com.



VideoNote

VideoNotes

VideoNotes are Pearson's new visual tool designed for teaching students key programming concepts and techniques. These short step-by-step videos demonstrate how to solve problems from design through coding. VideoNotes allow for self-placed instruction with easy navigation including the ability to select, play, rewind, fast-forward, and stop within each VideoNote exercise.

Margin icons in your textbook let you know when a VideoNote video is available for a particular concept or homework problem.

LiveLab

This book is accompanied by a complementary Web-based course assessment and management system for instructors. The system has four main components:

- The **Automatic Grading System** can automatically grade programs.
- The **Quiz Creation/Submission/Grading System** enables instructors to create and modify quizzes that students can take and be graded upon automatically.
- The **Peer Evaluation System** enables peer evaluations.
- **Checking plagiarisms, tracking grades, attendance, etc.,** lets students track their grades, and enables instructors to view the grades of all students, to check plagiarisms, and to track students' attendance.

The main features of the Automatic Grading System include:

- Students can run and submit exercises. (The system checks whether their program runs correctly—students can continue to run and resubmit the program before the due date.)
- Instructors can review submissions, run programs with instructor test cases, correct them, and provide feedback to students.
- Instructors can create/modify their own exercises, create public and secret test cases, assign exercises, and set due dates for the whole class or for individuals.
- All the exercises in the text can be assigned to students. Additionally, LiveLab provides extra exercises that are not printed in the text.
- Instructors can sort and filter all exercises and check grades (by time frame, student, and/or exercise).
- Instructors can check plagiarisms for a programming exercise.
- Instructors can delete students from the system.
- Students and instructors can track grades on exercises.

The main features of the Quiz System are:

- Instructors can create/modify quizzes from the test bank or a text file or create completely new tests online.
- Instructors can assign the quizzes to students and set a due date and test time limit for the whole class or for individuals.
- Students and instructors can review submitted quizzes.
- Instructors can analyze quizzes and identify students' weaknesses.
- Students and instructors can track grades on quizzes.

The main features of the Peer Evaluation System include:

- Instructors can assign/unassign exercises for peer evaluation.
- Instructors can view peer evaluation reports.

Acknowledgments

I would like to thank Armstrong Atlantic State University for enabling me to teach what I write and for supporting me in writing what I teach. Teaching is the source of inspiration for the book. I am grateful to the instructors and students who have offered comments, suggestions, bug reports, and praise.

This book has been greatly enhanced thanks to the outstanding reviewers. They are:

Claude Anderson – Rose-Hulman Institute of Technology
 Lee Cornell – Minnesota State University – Mankato
 John Magee – Boston University
 Shyamal Mitra – University of Texas – Austin
 Yenumula Reddy – Grambling State University
 David Sullivan – Boston University
 Hong Wang – University of Toledo

It is a great pleasure, honor, and privilege to work with Pearson. I would like to thank Tracy Dunkelberger, Marcia Horton, Michael Hirsch, Matt Goldstein, Carole Snyder, Tim Huddleston, Yez Alayan, Jeff Holcomb, Gillian Hall, Rebecca Greenberg, and their colleagues for organizing, producing, and promoting this project.

As always, I am indebted to my wife, Samantha, for her love, support, and encouragement.

BRIEF CONTENTS

1	Introduction to Computers, Programs, and Python	1
2	Elementary Programming	31
3	Mathematical Functions, Strings, and Objects	63
4	Selections	91
5	Loops	133
6	Functions	171
7	Objects and Classes	215
8	More on Strings and Special Methods	241
9	GUI Programming Using Tkinter	271
10	Lists	313
11	Multidimensional Lists	361
12	Inheritance and Polymorphism	399
13	Files and Exception Handling	439
14	Tuples, Sets, and Dictionaries	475
15	Recursion	499

Chapters 16–23 are bonus Web chapters

16	Developing Efficient Algorithms	16-1
17	Sorting	17-1
18	Linked Lists, Stacks, Queues, and Priority Queues	18-1
19	Binary Search Trees	19-1
20	AVL Trees	20-1
21	Hashing: Implementing Dictionaries and Sets	21-1
22	Graphs and Applications	22-1
23	Weighted Graphs and Applications	23-1

APPENDIXES

A	Python Keywords	533
B	The ASCII Character Set	534
C	Number Systems	536

INDEX	541
-------	-----

CREDITS	557
---------	-----

CONTENTS

Chapter 1	Introduction to Computers, Programs, and Python	1
1.1	Introduction	2
1.2	What Is a Computer?	2
1.3	Programming Languages	9
1.4	Operating Systems	12
1.5	The History of Python	13
1.6	Getting Started with Python	14
1.7	Programming Style and Documentation	18
1.8	Programming Errors	19
1.9	Getting Started with Graphics Programming	21

Chapter 2	Elementary Programming	31
2.1	Introduction	32
2.2	Writing a Simple Program	32
2.3	Reading Input from the Console	34
2.4	Identifiers	36
2.5	Variables, Assignment Statements, and Expressions	36
2.6	Simultaneous Assignments	38
2.7	Named Constants	39
2.8	Numeric Data Types and Operators	40
2.9	Evaluating Expressions and Operator Precedence	43
2.10	Augmented Assignment Operators	44
2.11	Type Conversions and Rounding	45
2.12	Case Study: Displaying the Current Time	46
2.13	Software Development Process	48
2.14	Case Study: Computing Distances	52

Chapter 3	Mathematical Functions, Strings, and Objects	63
3.1	Introduction	64
3.2	Common Python Functions	64
3.3	Strings and Characters	67
3.4	Case Study: Minimum Number of Coins	72
3.5	Introduction to Objects and Methods	74
3.6	Formatting Numbers and Strings	76
3.7	Drawing Various Shapes	81
3.8	Drawing with Colors and Fonts	83

Chapter 4	Selections	91
4.1	Introduction	92
4.2	Boolean Types, Values, and Expressions	92
4.3	Generating Random Numbers	93
4.4	if Statements	95
4.5	Case Study: Guessing Birthdays	96
4.6	Two-Way if-else Statements	100
4.7	Nested if and Multi-Way if-elif-else Statements	102
4.8	Common Errors in Selection Statements	105
4.9	Case Study: Computing Body Mass Index	107
4.10	Case Study: Computing Taxes	108
4.11	Logical Operators	110
4.12	Case Study: Determining Leap Years	113
4.13	Case Study: Lottery	114
4.14	Conditional Expressions	116
4.15	Operator Precedence and Associativity	117
4.16	Detecting the Location of an Object	118
Chapter 5	Loops	133
5.1	Introduction	134
5.2	The while Loop	134
5.3	The for Loop	143
5.4	Nested Loops	145
5.5	Minimizing Numerical Errors	147
5.6	Case Studies	148
5.7	Keywords break and continue	151
5.8	Case Study: Displaying Prime Numbers	154
5.9	Case Study: Random Walk	156
Chapter 6	Functions	171
6.1	Introduction	172
6.2	Defining a Function	172
6.3	Calling a Function	173
6.4	Functions with/without Return Values	176
6.5	Positional and Keyword Arguments	179
6.6	Passing Arguments by Reference Values	180
6.7	Modularizing Code	183
6.8	Case Study: Converting Decimals to Hexadecimals	185
6.9	The Scope of Variables	187
6.10	Default Arguments	189
6.11	Returning Multiple Values	190
6.12	Case Study: Generating Random ASCII Characters	191
6.13	Function Abstraction and Stepwise Refinement	192
6.14	Case Study: Reusable Graphics Functions	199

Chapter 7	Objects and Classes	215
7.1	Introduction	216
7.2	Defining Classes for Objects	216
7.3	UML Class Diagrams	222
7.4	Immutable Objects vs. Mutable Objects	225
7.5	Hiding Data Fields	227
7.6	Class Abstraction and Encapsulation	229
7.7	Object-Oriented Thinking	233
Chapter 8	More on Strings and Special Methods	241
8.1	Introduction	242
8.2	The <code>str</code> Class	242
8.3	Case Study: Checking Palindromes	253
8.4	Case Study: Converting Hexadecimals to Decimals	254
8.5	Operator Overloading and Special Methods	257
8.6	Case Study: The <code>Rational</code> Class	258
Chapter 9	GUI Programming Using Tkinter	271
9.1	Introduction	272
9.2	Getting Started with Tkinter	272
9.3	Processing Events	273
9.4	The Widget Classes	275
9.5	Canvas	280
9.6	The Geometry Managers	283
9.7	Case Study: Loan Calculator	287
9.8	Displaying Images	288
9.9	Menus	290
9.10	Popup Menus	292
9.11	Mouse, Key Events, and Bindings	294
9.12	Animations	297
9.13	Scrollbars	300
9.14	Standard Dialog Boxes	301
Chapter 10	Lists	313
10.1	Introduction	314
10.2	List Basics	315
10.3	Case Study: Lotto Numbers	325
10.4	Case Study: Deck of Cards	327
10.5	Deck of Cards GUI	329
10.6	Copying Lists	330
10.7	Passing Lists to Functions	332
10.8	Returning a List from a Function	334
10.9	Case Study: Counting the Occurrences of Each Letter	335
10.10	Searching Lists	338

10.11	Sorting Lists	341
10.12	Case Study: Bouncing Balls	345
Chapter 11	Multidimensional Lists	361
11.1	Introduction	362
11.2	Processing Two-Dimensional Lists	362
11.3	Passing Two-Dimensional Lists to Functions	365
11.4	Problem: Grading a Multiple-Choice Test	367
11.5	Problem: Finding the Closest Pair	368
11.6	GUI: Finding the Closest Pair	370
11.7	Problem: Sudoku	372
11.8	Case Study: Sudoku GUI	375
11.9	Multidimensional Lists	377
Chapter 12	Inheritance and Polymorphism	399
12.1	Introduction	400
12.2	Superclasses and Subclasses	400
12.3	Overriding Methods	405
12.4	The object Class	406
12.5	Polymorphism and Dynamic Binding	409
12.6	The isinstance Function	411
12.7	Case Study: A Reusable Clock	414
12.8	Class Relationships	418
12.9	Case Study: Designing the Course Class	421
12.10	Designing a Class for Stacks	423
12.11	Case Study: The FigureCanvas Class	425
Chapter 13	Files and Exception Handling	439
13.1	Introduction	440
13.2	Text Input and Output	440
13.3	File Dialogs	448
13.4	Case Study: Counting Each Letter in a File	451
13.5	Retrieving Data from the Web	452
13.6	Exception Handling	454
13.7	Raising Exceptions	457
13.8	Processing Exceptions Using Exception Objects	460
13.9	Defining Custom Exception Classes	460
13.10	Binary IO Using Pickling	465
13.11	Case Study: Address Book	467
Chapter 14	Tuples, Sets, and Dictionaries	475
14.1	Introduction	476
14.2	Tuples	476
14.3	Sets	479
14.4	Comparing the Performance of Sets and Lists	485

14.5	Case Study: Counting Keywords	486
14.6	Dictionaries	487
14.7	Case Study: Occurrences of Words	492

Chapter 15 Recursion 499

15.1	Introduction	500
15.2	Case Study: Computing Factorials	500
15.3	Case Study: Computing Fibonacci Numbers	503
15.4	Problem Solving Using Recursion	505
15.5	Recursive Helper Functions	507
15.6	Case Study: Finding the Directory Size	510
15.7	Case Study: Towers of Hanoi	512
15.8	Case Study: Fractals	515
15.9	Case Study: Eight Queens	518
15.10	Recursion vs. Iteration	520
15.11	Tail Recursion	521

A detailed table of contents for the Web chapters is available on the companion Website:

Chapter 16 Developing Efficient Algorithms 16-1

Chapter 17 Sorting 17-1

Chapter 18 Linked Lists, Stacks, Queues, and Priority Queues 18-1

Chapter 19 Binary Search Trees 19-1

Chapter 20 AVL Trees 20-1

Chapter 21 Hashing: Implementing Dictionaries and Sets 21-1

Chapter 22 Graphs and Applications 22-1

Chapter 23 Weighted Graphs and Applications 23-1

APPENDIXES

Appendix A Python Keywords 533

Appendix B The ASCII Character Set 534

Appendix C Number Systems 536

INDEX 541

CREDITS 557

LOCATION OF VIDEONOTES IN THE TEXT 	
Chapter 1	Start with Python, p. 14 Start with Turtle graphics, p. 21
Chapter 2	Assignment statement, p. 37 Perform computation, p. 40
Chapter 3	String operations, p. 75 Draw shapes, p. 81
Chapter 4	Boolean expressions, p. 92 Coffee price, p. 123
Chapter 5	while loop, p. 134 for loop, p. 143
Chapter 6	Use functions, p. 172 Divide and conquer, p. 192
Chapter 7	Define and use classes, p. 217 Private data fields, p. 227
Chapter 8	String methods, p. 242 Define classes, p. 258
Chapter 9	Simple GUI, p. 273 Create GUI application, p. 287
Chapter 10	Use lists, p. 315 Search a list, p. 338
Chapter 11	Process a matrix, p. 362 Chessboard, p. 394
Chapter 12	Inheritance and polymorphism, p. 400 Dynamic binding, p. 409
Chapter 13	Process text file, p. 443 Handle exceptions, p. 454
Chapter 14	Use sets, p. 479 Use dictionaries, p. 488
Chapter 15	Function sum, p. 500 Function print numbers, p. 506

INTRODUCTION TO COMPUTERS, PROGRAMS, AND PYTHON

Objectives

- To demonstrate a basic understanding of computer hardware, programs, and operating systems (§§1.2–1.4).
- To describe the history of Python (§1.5).
- To explain the basic syntax of a Python program (§1.6).
- To write and run a simple Python program (§1.6).
- To explain the importance of, and provide examples of, proper programming style and documentation (§1.7).
- To explain the differences between syntax errors, runtime errors, and logic errors (§1.8).
- To create a basic graphics program using Turtle (§1.9).



1.1 Introduction



The central theme of this book is to learn how to solve problems by writing a program.

what is programming?
program

This book is about programming. So, what is programming? The term programming means to create (or develop) software, which is also called a *program*. In basic terms, software contains the instructions that tell a computer—or a computerized device—what to do.

programming languages

Software is all around you, even in devices that you might not think would need it. Of course, you expect to find and use software on a personal computer, but software also plays a role in running airplanes, cars, cell phones, and even toasters. On a personal computer, you use word processors to write documents, Web browsers to explore the Internet, and e-mail programs to send messages. These programs are all examples of software. Software developers create software with the help of powerful tools called *programming languages*.

This book teaches you how to create programs by using the Python programming language. There are many programming languages, some of which are decades old. Each language was invented for a specific purpose—to build on the strengths of a previous language, for example, or to give the programmer a new and unique set of tools. Knowing that there are so many programming languages available, it would be natural for you to wonder which one is best. But, in truth, there is no “best” language. Each one has its own strengths and weaknesses. Experienced programmers know that one language might work well in some situations, whereas a different language may be more appropriate in other situations. For this reason, seasoned programmers try to master as many different programming languages as they can, giving them access to a vast arsenal of software-development tools.

If you learn to program using one language, you should find it easy to pick up other languages. The key is to learn how to solve problems using a programming approach. That is the main theme of this book.

You are about to begin an exciting journey: learning how to program. At the outset, it is helpful to review computer basics, programs, and operating systems. If you are already familiar with such terms as CPU, memory, disks, operating systems, and programming languages, you may skip the review in Sections 1.2–1.4.

1.2 What Is a Computer?



A computer is an electronic device that stores and processes data.

hardware
software

A computer includes both *hardware* and *software*. In general, hardware comprises the visible, physical elements of the computer, and software provides the invisible instructions that control the hardware and make it perform specific tasks. Knowing computer hardware isn’t essential to learning a programming language, but it can help you better understand the effects that a program’s instructions have on the computer and its components. This section introduces computer hardware components and their functions.

A computer consists of the following major hardware components (Figure 1.1):

- A central processing unit (CPU)
- Memory (main memory)
- Storage devices (such as disks and CDs)
- Input devices (such as the mouse and keyboard)
- Output devices (such as monitors and printers)
- Communication devices (such as modems and network interface cards)

bus

A computer’s components are interconnected by a subsystem called a *bus*. You can think of a bus as a sort of system of roads running among the computer’s components; data and

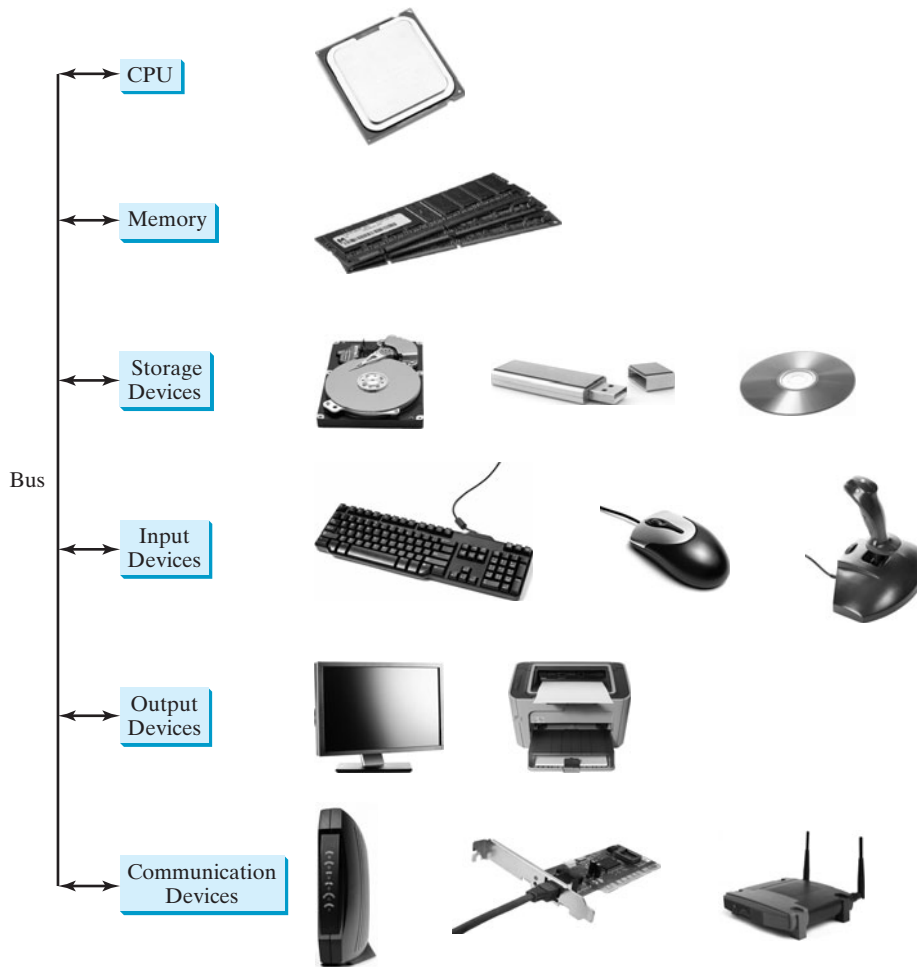


FIGURE 1.1 A computer consists of a CPU, memory, storage devices, input devices, output devices, and communication devices.

power travel along the bus from one part of the computer to another. In personal computers, the bus is built into the computer's *motherboard*, which is a circuit case that connects all of the parts of a computer together, as shown in Figure 1.2. motherboard

1.2.1 Central Processing Unit

The *central processing unit (CPU)* is the computer's brain. It retrieves instructions from memory and executes them. The CPU usually has two components: a *control unit* and an *arithmetic/logic unit*. The control unit controls and coordinates the actions of the other components. The arithmetic/logic unit performs numeric operations (addition, subtraction, multiplication, division) and logical operations (comparisons). CPU

Today's CPUs are built on small silicon semiconductor chips that contain millions of tiny electric switches, called *transistors*, for processing information.

Every computer has an internal clock, which emits electronic pulses at a constant rate. These pulses are used to control and synchronize the pace of operations. A higher clock *speed* enables more instructions to be executed in a given period of time. The unit of measurement of clock speed is the *hertz (Hz)*, with 1 hertz equaling 1 pulse per second. In the 1990s computers measured clocked speed in *megahertz*, but CPU speed has been improving continuously, and speed hertz megahertz

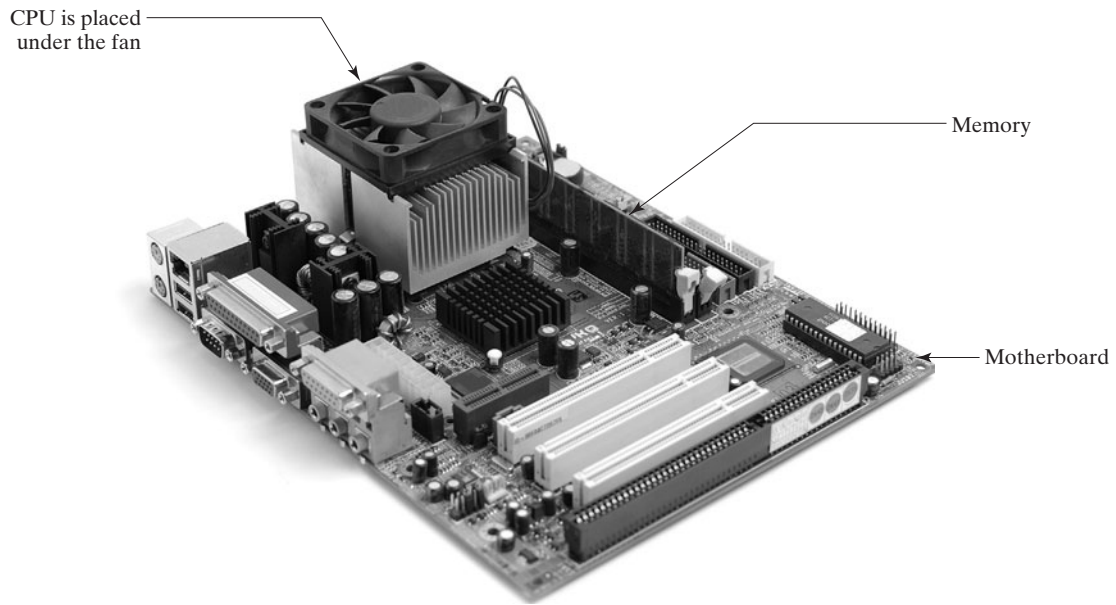


FIGURE 1.2 The motherboard connects all parts of computer together.

gigahertz

the clock speed of a computer is now usually stated in *gigahertz (GHz)*. Intel's newest processors run at about 3 GHz.

core

CPUs were originally developed with only one core. The *core* is the part of the processor that performs the reading and executing of instructions. In order to increase CPU processing power, chip manufacturers are now producing CPUs that contain multiple cores. A multicore CPU is a single component with two or more independent processors. Today's consumer computers typically have two, three, and even four separate cores. Soon, CPUs with tens or even hundreds of cores will be affordable.

1.2.2 Bits and Bytes

Before we discuss memory, let's look at how information (data and programs) are stored in a computer.

bits

A computer is really nothing more than a series of switches. Each switch exists in two states: on or off. Storing information in a computer is simply a matter of setting a sequence of switches on or off. If the switch is on, its value is 1. If the switch is off, its value is 0. These 0s and 1s are interpreted as digits in the binary number system and called *bits* (binary digits).

byte

The minimum storage unit in a computer is a *byte*. A byte is composed of eight bits. A small number such as **3** can be stored as a single byte. To store a number that cannot fit into a single byte, the computer uses several bytes.

encoding scheme

Data of various kinds, such as numbers and characters, are encoded as a series of bytes. As a programmer, you don't need to worry about the encoding and decoding of data, which the computer system performs automatically, based on the encoding scheme. An *encoding scheme* is a set of rules that govern how a computer translates characters, numbers, and symbols into data the computer can actually work with. Most schemes translate each character into a predetermined string of numbers. In the popular ASCII encoding scheme, for example, the character **C** is represented as **01000011** in one byte.

A computer’s storage capacity is measured in bytes and multiples of the byte, as follows:

- A *kilobyte (KB)* is about 1,000 bytes. kilobyte (KB)
- A *megabyte (MB)* is about 1 million bytes. megabyte (MB)
- A *gigabyte (GB)* is about 1 billion bytes. gigabyte (GB)
- A *terabyte (TB)* is about 1 trillion bytes. terabyte (TB)

A typical one-page word document might take 20 KB. So 1 MB can store 50 pages of documents and 1 GB can store 50000 pages of documents. A typical two-hour high-resolution movie might take 8 GB. So it would require 160 GB to store 20 movies.

I.2.3 Memory

A computer’s *memory* consists of an ordered sequence of bytes for storing programs as well as data that the program is working with. You can think of memory as the computer’s work area for executing a program. A program and its data must be moved into the computer’s memory before they can be executed by the CPU. memory

Every byte in the memory has a *unique address*, as shown in Figure 1.3. The address is used to locate the byte for storing and retrieving the data. Since the bytes in the memory can be accessed in any order, the memory is also referred to as *random-access memory (RAM)*. unique address
RAM

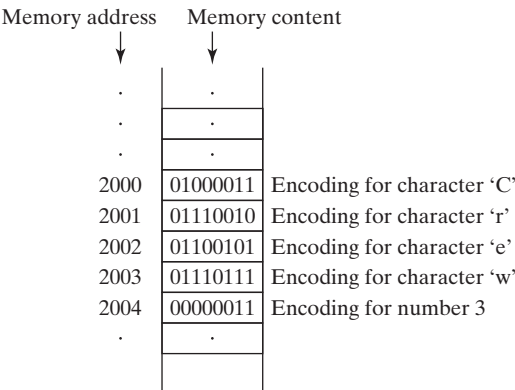


FIGURE 1.3 Memory stores data and program instructions in uniquely addressed memory locations. Each memory location can store one byte of data.

Today’s personal computers usually have at least 1 gigabyte of RAM, but they more commonly have 2 to 4 GB installed. Generally speaking, the more RAM a computer has, the faster it can operate, but there are limits to this simple rule of thumb.

A memory byte is never empty, but its initial content may be meaningless to your program. The current content of a memory byte is lost whenever new information is placed in it.

Like the CPU, memory is built on silicon semiconductor chips that have millions of transistors embedded on their surface. Compared to CPU chips, memory chips are less complicated, slower, and less expensive.

I.2.4 Storage Devices

A computer’s memory is a volatile form of data storage: any information that hasn’t been stored in memory (that is, saved) is lost when the system’s power is turned off. Programs and data are permanently stored on *storage devices* and are moved, when the computer actually uses them, to memory, which operates at much faster speeds than permanent storage devices can. storage device

There are three main types of storage devices:

- Magnetic disk drives
- Optical disc drives (CD and DVD)
- USB flash drives

Drives are devices for operating a medium, such as disks and CDs. A storage medium physically stores data or program instructions. The drive reads data from the medium and/or writes data onto the medium.

Disks

A computer usually has at least one hard disk drive (Figure 1.4). *Hard disks* are used for permanently storing data and programs. Newer computers have hard disks that can store from 200 to 800 gigabytes of data. Hard disk drives are usually encased inside the computer, but removable hard disks are also available.



FIGURE 1.4 A hard disk is a device for permanently storing programs and data.

CDs and DVDs

CD stands for compact disc. There are two types of CD drives: CD-R and CD-RW. A *CD-R* is for read-only permanent storage; the user cannot modify its contents once they are recorded. A *CD-RW* can be used like a hard disk; that is, you can write data onto the disc, and then overwrite that data with new data. A single CD can hold up to 700 MB. Most new PCs are equipped with a CD-RW drive that can work with both CD-R and CD-RW discs.

DVD stands for digital versatile disc or digital video disc. DVDs and CDs look alike, and you can use either to store data. A DVD can hold more information than a CD; a standard DVD’s storage capacity is 4.7 GB. Like CDs, there are two types of DVDs: DVD-R (read-only) and DVD-RW (rewritable).

USB Flash Drives

Universal serial bus (USB) connectors allow the user to attach many kinds of peripheral devices to the computer. You can use a USB to connect a printer, digital camera, mouse, external hard disk drive, and other devices to the computer.

A *USB flash drive* is a device for storing and transporting data. A flash drive is small—about the size of a pack of gum, as shown in Figure 1.5. It acts like a portable hard drive that can be plugged into your computer’s USB port. USB flash drives are currently available with up to 256 GB storage capacity.

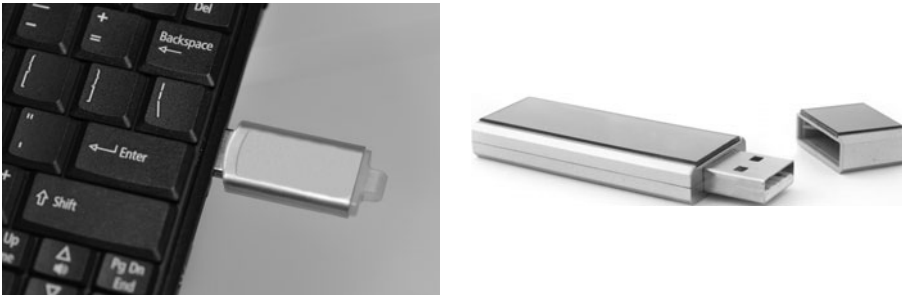


FIGURE 1.5 USB flash drives are popular portable devices for storing data.

1.2.5 Input and Output Devices

Input and output devices let the user communicate with the computer. The most common input devices are *keyboards* and *mice*. The most common output devices are *monitors* and *printers*.

The Keyboard

A keyboard is a device for entering input. A typical keyboard is shown in Figure 1.6. *Compact keyboards* are available without a numeric keypad.



FIGURE 1.6 A computer keyboard consists of the keys for sending input to a computer.

Function keys are located across the top of the keyboard and are prefaced with the letter *F*. function key
Their functions depend on the software currently being used.

8 Chapter 1 Introduction to Computers, Programs, and Python

modifier key

A *modifier key* is a special key (such as the *Shift*, *Alt*, and *Ctrl* keys) that modifies the normal action of another key when the two are pressed simultaneously.

numeric keypad

The *numeric keypad*, located on the right side of most keyboards, is a separate set of keys styled like a calculator to use for entering numbers quickly.

arrow keys

Arrow keys, located between the main keypad and the numeric keypad, are used to move the mouse pointer up, down, left, and right on the screen in many kinds of programs.

Insert key

Delete key

Page Up key

Page Down key

The *Insert*, *Delete*, *Page Up*, and *Page Down* keys are used in word processing and other programs for inserting text and objects, deleting text and objects, and moving up or down through a document one screen at a time.

The Mouse

A *mouse* is a pointing device. It is used to move a graphical pointer (usually in the shape of an arrow) called a *cursor* around the screen or to click on-screen objects (such as a button) to trigger them to perform an action.

The Monitor

The *monitor* displays information (text and graphics). The screen resolution and dot pitch determine the quality of the display.

screen resolution

pixels

The *screen resolution* specifies the number of pixels in horizontal and vertical dimensions of the display device. *Pixels* (short for “picture elements”) are tiny dots that form an image on the screen. A common resolution for a 17-inch screen, for example, is 1024 pixels wide and 768 pixels high. The resolution can be set manually. The higher the resolution, the sharper and clearer the image is.

dot pitch

The *dot pitch* is the amount of space between pixels, measured in millimeters. The smaller the dot pitch, the sharper the display.

1.2.6 Communication Devices

modem

Computers can be networked through communication devices, such as a dial-up *modem* (*modulator/demodulator*), a DSL or cable modem, a wired network interface card, or a wireless adapter.

- A dial-up modem uses a phone line and can transfer data at a speed up to 56,000 bps (bits per second).

digital subscriber line (DSL)

- A *digital subscriber line (DSL)* connection also uses a standard phone line, but it can transfer data 20 times faster than a standard dial-up modem.

cable modem

- A *cable modem* uses the cable TV line maintained by the cable company and is generally faster than DSL.

network interface card (NIC)

local area network (LAN)

- A *network interface card (NIC)* is a device that connects a computer to a *local area network (LAN)*, as shown in Figure 1.7. LANs are commonly used in universities, businesses, and government agencies. A high-speed NIC called *1000BaseT* can transfer data at 1,000 *million bits per second (mbps)*.

million bits per second
(mbps)

- Wireless networking is now extremely popular in homes, businesses, and schools. Every laptop computer sold today is equipped with a wireless adapter that enables the computer to connect to a local area network and the Internet.



Note

Answers to checkpoint questions are on the Companion Website.



Check
Point

1.1 What are hardware and software?

1.2 List five major hardware components of a computer.

1.3 What does the acronym “CPU” stand for?

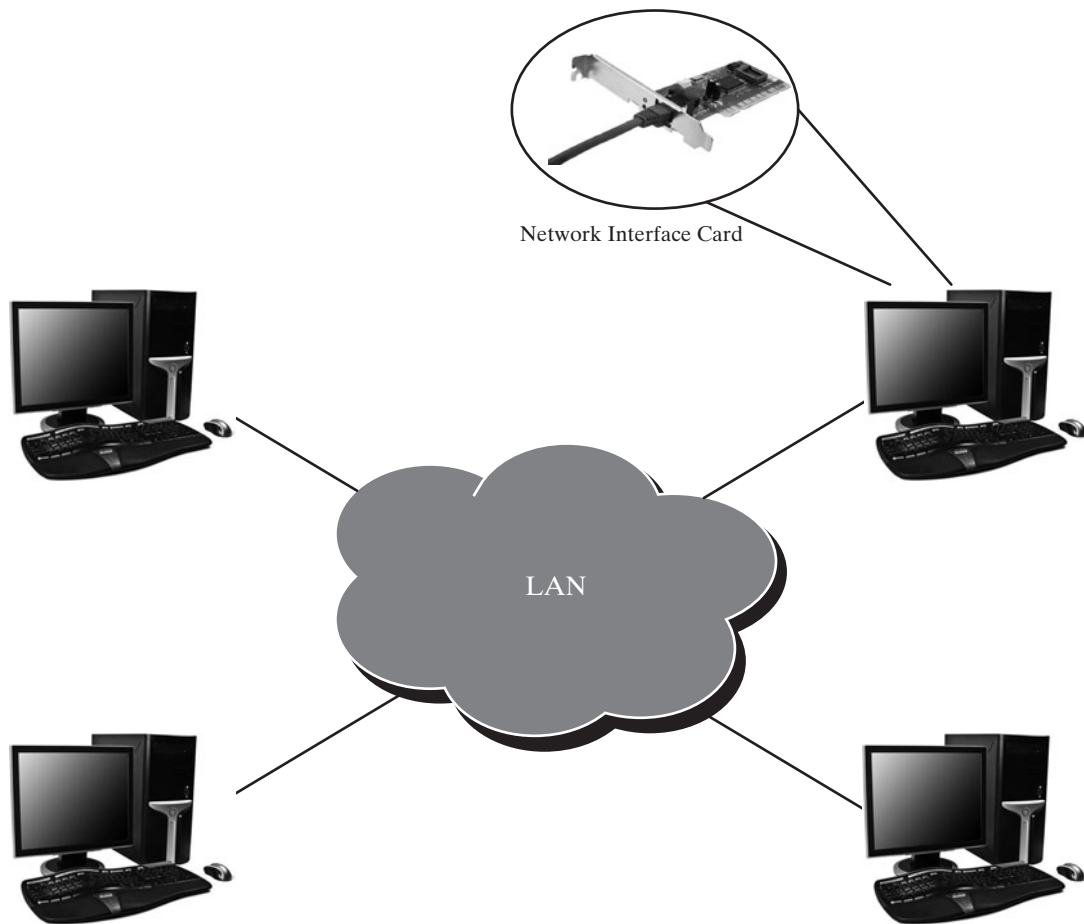


FIGURE 1.7 A local area network connects computers in close proximity to each other.

- 1.4** What unit is used to measure CPU speed?
- 1.5** What is a bit? What is a byte?
- 1.6** What is memory for? What does RAM stand for? Why is memory called RAM?
- 1.7** What unit is used to measure memory size?
- 1.8** What unit is used to measure disk size?
- 1.9** What is the primary difference between memory and a storage device?

1.3 Programming Languages

Computer programs, known as software, are instructions that tell a computer what to do.



Computers do not understand human languages, so programs must be written in a language a computer can use. There are hundreds of programming languages, and they were developed to make the programming process easier for people. However, all programs must be converted into a language the computer can understand.

1.3.1 Machine Language

A computer's native language, which differs among different types of computers, is its *machine language*—a set of built-in primitive instructions. These instructions are in the form of binary code, so if you want to give a computer an instruction in its native language, you have to enter

machine language

the instruction as binary code. For example, to add two numbers, you might have to write an instruction in binary code, like this:

```
1101101010011010
```

1.3.2 Assembly Language

assembly language

Programming in machine language is a tedious process. Moreover, programs written in machine language are very difficult to read and modify. For this reason, *assembly language* was created in the early days of computing as an alternative to machine languages. Assembly language uses a short descriptive word, known as *mnemonic*, to represent each of the machine-language instructions. For example, the mnemonic **add** typically means to add numbers and **sub** means to subtract numbers. To add the numbers **2** and **3** and get the result, you might write an instruction in assembly code like this:

```
add 2, 3, result
```

assembler

Assembly languages were developed to make programming easier. However, because the computer cannot understand assembly language, another program—called an *assembler*—is used to translate assembly-language programs into machine code, as shown in Figure 1.8.

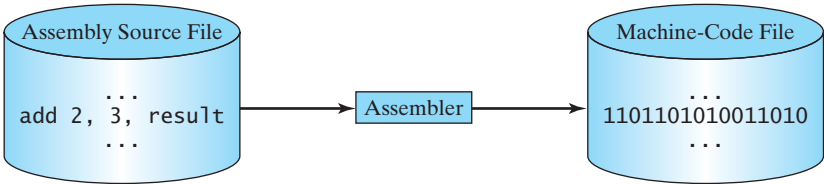


FIGURE 1.8 An assembler translates assembly-language instructions into machine code.

low-level language

Writing code in assembly language is easier than in machine language. However, it is still tedious to write code in assembly language. An instruction in assembly language essentially corresponds to an instruction in machine code. Writing in assembly requires that you know how the CPU works. Assembly language is referred to as a *low-level language*, because assembly language is close in nature to machine language and is machine dependent.

1.3.3 High-Level Language

high-level language

In the 1950s, a new generation of programming languages known as *high-level languages* emerged. They are platform-independent, which means that you can write a program in a high-level language and run it in different types of machines. High-level languages are English-like and easy to learn and use. The instructions in a high-level programming language are called *statements*. Here, for example, is a high-level language statement that computes the area of a circle with a radius of **5**:

statement

```
area = 5 * 5 * 3.1415
```

There are many high-level programming languages, and each was designed for a specific purpose. Table 1.1 lists some popular ones.

source program
source code

A program written in a high-level language is called a *source program* or *source code*. Because a computer cannot understand a source program, a source program must be translated into machine code for execution. The translation can be done using another programming tool called an *interpreter* or a *compiler*.

interpreter
compiler

- An interpreter reads one statement from the source code, translates it to the machine code or virtual machine code, and then executes it right away, as shown in Figure 1.9a.

TABLE 1.1 Popular High-Level Programming Languages

Language	Description
Ada	Named for Ada Lovelace, who worked on mechanical general-purpose computers. The Ada language was developed for the Department of Defense and is used mainly in defense projects.
BASIC	Beginner's All-purpose Symbolic Instruction Code. It was designed to be learned and used easily by beginners.
C	Developed at Bell Laboratories. C combines the power of an assembly language with the ease of use and portability of a high-level language.
C++	C++ is an object-oriented language, based on C.
C#	Pronounced "C Sharp." It is a hybrid of Java and C++ and was developed by Microsoft.
COBOL	COMmon Business Oriented Language. Used for business applications.
FORTRAN	FORmula TRANslation. Popular for scientific and mathematical applications.
Java	Developed by Sun Microsystems, now part of Oracle. It is widely used for developing platform-independent Internet applications.
Pascal	Named for Blaise Pascal, who pioneered calculating machines in the seventeenth century. It is a simple, structured, general-purpose language primarily for teaching programming.
Python	A simple general-purpose scripting language good for writing short programs.
Visual Basic	Visual Basic was developed by Microsoft and it enables the programmers to rapidly develop Windows-based applications.

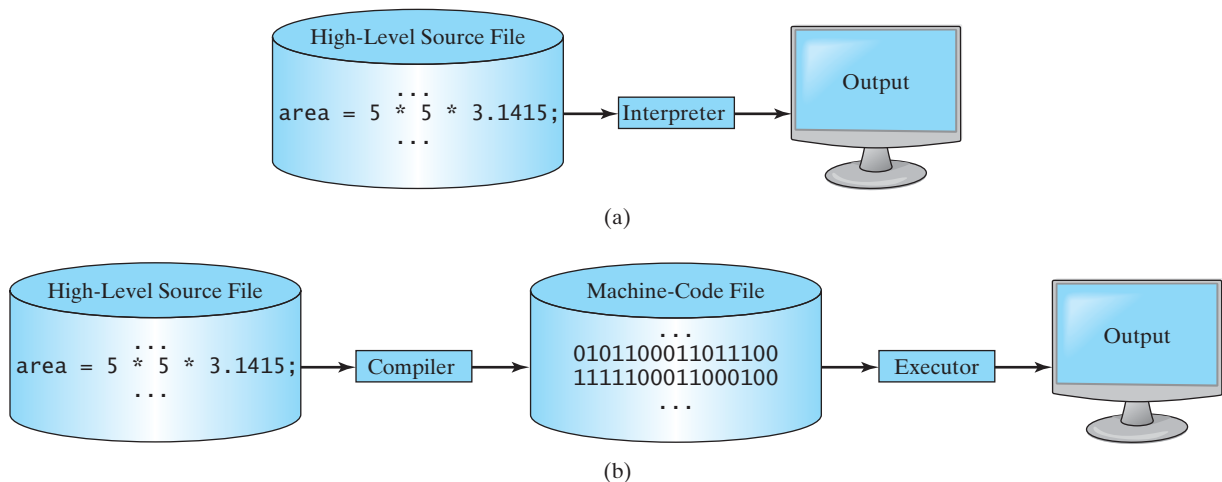


FIGURE 1.9 (a) An interpreter translates and executes a program one statement at a time. (b) A compiler translates the entire source program into a machine-language file for execution.

Note that a statement from the source code may be translated into several machine instructions.

- A compiler translates the entire source code into a machine-code file, and the machine code file is then executed, as shown in Figure 1.9b.

Python code is executed using an interpreter. Most other programming languages are processed using a compiler.

1.10 What language does the CPU understand?

1.11 What is an assembly language?



MyProgrammingLab™

- I.12** What is an assembler?
- I.13** What is a high-level programming language?
- I.14** What is a source program?
- I.15** What is an interpreter?
- I.16** What is a compiler?
- I.17** What is the difference between an interpreted language and a compiled language?

1.4 Operating Systems



*The operating system (OS) is the most important program that runs on a computer.
The OS manages and controls a computer's activities.*

operating system (OS)

The popular operating systems for general-purpose computers are Microsoft Windows, Mac OS, and Linux. Application programs, such as a Web browser or a word processor, cannot run unless an operating system is installed and running on the computer. Figure 1.10 shows the interrelationship of hardware, operating system, application software, and the user.

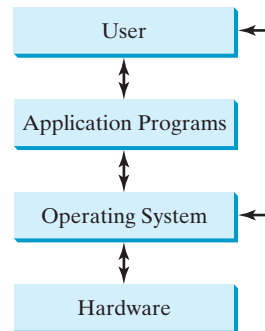


FIGURE 1.10 Users and applications access the computer's hardware via the operating system.

The major tasks of an operating system are:

- Controlling and monitoring system activities
- Allocating and assigning system resources
- Scheduling operations

1.4.1 Controlling and Monitoring System Activities

Operating systems perform basic tasks, such as recognizing input from the keyboard, sending output to the monitor, keeping track of files and folders on storage devices, and controlling peripheral devices, such as disk drives and printers. An operating system must also ensure that different programs and users working at the same time do not interfere with each other. In addition, the OS is responsible for security, ensuring that unauthorized users and programs do not access the system.

1.4.2 Allocating and Assigning System Resources

The operating system is responsible for determining what computer resources a program needs (such as CPU time, memory space, disks, input and output devices) and for allocating and assigning them to run the program.

1.4.3 Scheduling Operations

The OS is responsible for scheduling programs' activities to make efficient use of system resources. Many of today's operating systems support such techniques as *multiprogramming*, *multithreading*, and *multiprocessing* to increase system performance.

Multiprogramming allows multiple programs to run simultaneously by sharing the same CPU. The CPU is much faster than the computer's other components. As a result, it is idle most of the time—for example, while waiting for data to be transferred from a disk or waiting for other system resources to respond. A multiprogramming OS takes advantage of this situation by allowing multiple programs to use the CPU when it would otherwise be idle. For example, multiprogramming enables you to use a word processor to edit a file at the same time as your Web browser is downloading a file.

multiprogramming

Multithreading allows a single program to execute multiple tasks at the same time. For instance, a word-processing program allows users to simultaneously edit text and save it to a disk. In this example, editing and saving are two tasks within the same application. These two tasks may run concurrently.

multithreading

Multiprocessing, or *parallel processing*, uses two or more processors together to perform subtasks concurrently and then combine solutions of the subtasks to obtain a solution for the entire task. It is like a surgical operation where several doctors work together on one patient.

multiprocessing

1.18 What is an operating system? List some popular operating systems.

1.19 What are the major responsibilities of an operating system?

1.20 What are multiprogramming, multithreading, and multiprocessing?



MyProgrammingLab™

1.5 The History of Python

Python is a general-purpose, interpreted, object-oriented programming language.

Python was created by Guido van Rossum in the Netherlands in 1990 and was named after the popular British comedy troupe *Monty Python's Flying Circus*. Van Rossum developed Python as a hobby, and Python has become a popular programming language widely used in industry and academia due to its simple, concise, and intuitive syntax and extensive library.



Python is a *general-purpose programming language*. That means you can use Python to write code for any programming task. Python is now used in the Google search engine, in mission-critical projects at NASA, and in transaction processing at the New York Stock Exchange.

general-purpose programming language

Python is *interpreted*, which means that Python code is translated and executed by an interpreter, one statement at a time, as described earlier in the chapter.

interpreted

Python is an *object-oriented programming (OOP)* language. Data in Python are objects created from classes. A *class* is essentially a type or category that defines objects of the same kind with properties and methods for manipulating objects. Object-oriented programming is a powerful tool for developing reusable software. Object-oriented programming in Python will be covered in detail starting in Chapter 7.

object-oriented programming (OOP)

Python is now being developed and maintained by a large team of volunteers and is available for free from the Python Software Foundation. Two versions of Python are currently coexistent: Python 2 and Python 3. The programs written in Python 3 will not run in Python 2. Python 3 is a newer version, but it is not backward-compatible with Python 2. This means that if you write a program using the Python 2 syntax, it may not work with a Python 3 interpreter. Python provides a tool that automatically converts code written in Python 2 into syntax Python 3 can use. Python 2 will eventually be replaced by Python 3. This book teaches programming using Python 3.

Python 2 vs. Python 3



Check
Point



- I.21 Python is interpreted. What does that mean?
- I.22 Can a program written in Python 2 run in Python 3?
- I.23 Can a program written using Python 3 run in Python 2?

1.6 Getting Started with Python



Key
Point

A Python program is executed from the Python interpreter.

Let’s get started by writing a simple Python program that displays the messages **Welcome to Python** and **Python is fun** on the console. The word *console* is an old computer term that refers to the text entry and display device of a computer. Console input means to receive input from the keyboard and console output means to display output to the monitor.



Note

You can run Python on the Windows, UNIX, and Mac operating systems. For information on *installing Python*, see Supplement I.B, Installing and Using Python, on the Companion Website.

console
console input
console output

install Python



VideoNote

Start with Python

IDLE

1.6.1 Launching Python

Assume you have Python installed on the Windows OS. You can start Python in a command window by typing **python** at the command prompt, as shown in Figure 1.11, or by using IDLE, as shown in Figure 1.12. *IDLE* (Interactive DeveLopment Environment) is an integrated development environment (IDE) for Python. You can create, open, save, edit, and run Python programs in IDLE. Both the command-line Python interpreter and IDLE are available after Python is installed on your machine. Note that Python (command line)

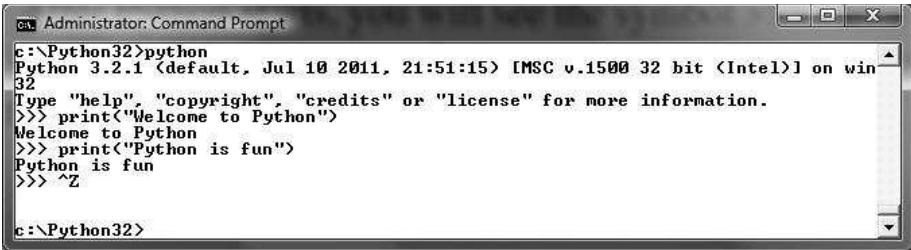


FIGURE 1.11 You can launch Python from the command window.

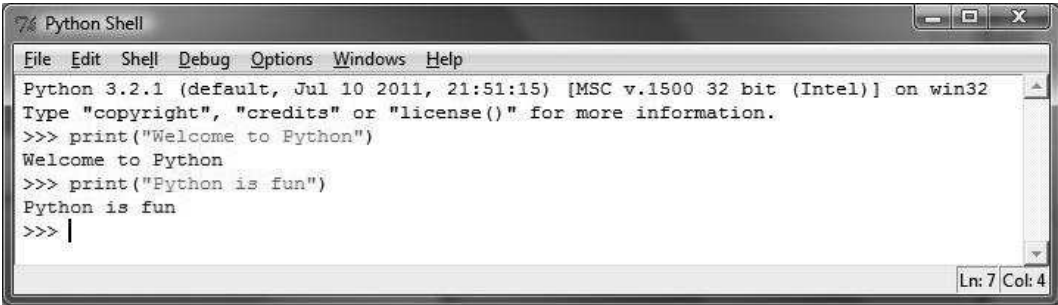


FIGURE 1.12 You can use Python from IDLE.

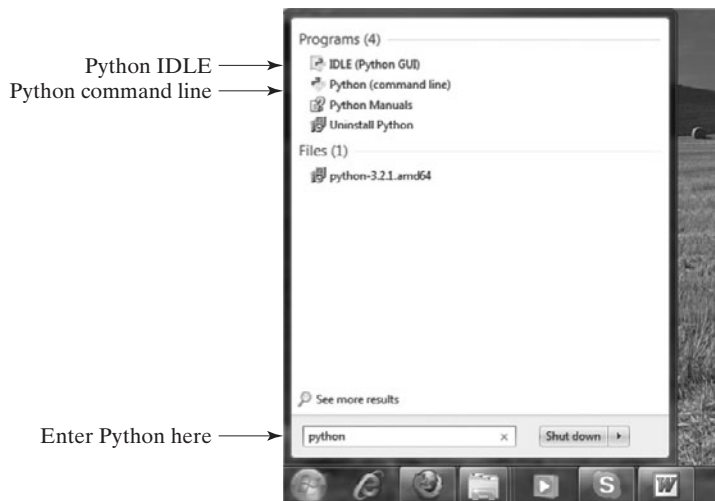


FIGURE 1.13 You can launch the Python IDLE and command line from the Start button.

and IDLE can also be accessed directly from the Windows *Start* button by searching for **Python (command line)** or **IDLE (Python GUI)** on Windows 7 or Vista, as shown in Figure 1.13.

After Python starts, you will see the symbol `>>>`. This is the *Python statement prompt*, and it is where you can enter a Python statement.

Python statement prompt `>>>`



Note

Type the commands *exactly* as they are written in this text. Formatting and other rules will be discussed later in this chapter.

Now, type `print("Welcome to Python")` and press the *Enter* key. The string **Welcome to Python** appears on the console, as shown in Figure 1.11. *String* is a programming term

string



Note

Note that Python requires double or single quotation marks around strings to delineate them from other code. As you can see in the output, Python doesn't display those quotation marks.

The `print` statement is one of Python's built-in *functions* that can be used to display a string on the console. A function performs actions. In the case of the `print` function, it displays a message to the console.

function



Note

In programming terminology, when you use a function, you are said to be "*invoking a function*" or "*calling a function*."

invoking a function
calling a function

Next, type `print("Python is fun")` and press the *Enter* key. The string **Python is fun** appears on the console, as shown in Figure 1.11. You can enter additional statements at the statement prompt `>>>`.



Note

To exit Python, press `CTRL+Z` and then the *Enter* key.

1.6.2 Creating Python Source Code Files

Entering Python statements at the statement prompt `>>>` is convenient, but the statements are not saved. To save statements for later use, you can create a text file to store the statements and use the following command to execute the statements in the file:

```
python filename.py
```

source file
script file
module
.py file
script mode
interactive mode

The text file can be created using a text editor such as Notepad. The text file, `filename`, is called a Python *source file* or *script file*, or *module*. By convention, Python files are named with the extension `.py`.

Running a Python program from a script file is known as running Python in *script mode*. Typing a statement at the statement prompt `>>>` and executing it is called running Python in *interactive mode*.



Note

Besides developing and running Python programs from the command window, you can create, save, modify, and run a Python script from IDLE. For information on using IDLE, see Supplement I.C on the Companion Website. Your instructor may also ask you to use Eclipse. Eclipse is a popular interactive development environment (IDE) used to develop programs quickly. Editing, running, debugging, and online help are integrated in one graphical user interface. If you want to develop Python programs using Eclipse, see Supplement I.D, on the Companion Website.

Python on Eclipse

Listing 1.1 shows you a Python program that displays the messages `Welcome to Python` and `Python is fun`.

LISTING 1.1 Welcome.py

comment
print a message

```
1 # Display two messages
2 print("Welcome to Python")
3 print("Python is fun")
```

line numbers
execute the program

In this text, *line numbers* are displayed for reference purposes; they are not part of the program. So, don't type line numbers in your program.
Suppose the statements are saved in a file named `Welcome.py`. To run the program, enter `python Welcome.py` at the command prompt, as shown in Figure 1.14.



FIGURE 1.14 You can run a Python script file from a command window.

comment
line comment
paragraph comment

In Listing 1.1, line 1 is a *comment* that documents what the program is and how it is constructed. Comments help programmers communicate and understand a program. They are not programming statements and thus are ignored by the interpreter. In Python, comments are preceded by a pound sign (`#`) on a line, called a *line comment*, or enclosed between three consecutive single quotation marks (`'''`) on one or several lines, called a *paragraph comment*.

When the Python interpreter sees #, it ignores all text after # on the same line. When it sees ''' , it scans for the next ''' and ignores any text between the triple quotation marks. Here are examples of comments:

```
# This program displays Welcome to Python

''' This program displays Welcome to Python and
    Python is fun
'''
```

Indentation matters in Python. Note that the statements are entered from the first column in the new line. The Python interpreter will report an error if the program is typed as follows:

```
# Display two messages
print("Welcome to Python")
print("Python is fun")
```



Don't put any punctuation at the end of a statement. For example, the Python interpreter will report errors for the following code:

```
# Display two messages
print("Welcome to Python").
print("Python is fun"),
```



Python programs are *case sensitive*. It would be wrong, for example, to replace `print` in the program with `Print`.

case sensitive

You have seen several *special characters* (`#`, `"`, `()`) in the program. They are used in almost every program. Table 1.2 summarizes their uses.

special characters

TABLE 1.2 Special Characters

Character	Name	Description
()	Opening and closing parentheses	Used with functions.
#	Pound sign	Precedes a comment line.
" "	Opening and closing quotation marks	Encloses a string (i.e., sequence of characters).
''' '''	Paragraph comments	Encloses a paragraph comment.

The program in Listing 1.1 displays two messages. Once you understand the program, it is easy to extend it to display more messages. For example, you can rewrite the program to display three messages, as shown in Listing 1.2.

LISTING 1.2 WelcomeWithThreeMessages.py

```
1 # Display three messages
2 print("Welcome to Python")
3 print("Python is fun")
4 print("Problem Driven")
```

comment
print statement

Welcome to Python
Python is fun
Problem Driven



1.6.3 Using Python to Perform Mathematical Computations

Python programs can perform all sorts of mathematical computations and display the result. To display the addition, subtraction, multiplication, and division of two numbers, **x** and **y**, use the following code:

```
print(x + y)
print(x - y)
print(x * y)
print(x / y)
```

Listing 1.3 shows an example of a program that evaluates $\frac{10.5 + 2 \times 3}{45 - 3.5}$ and prints its result.

LISTING 1.3 ComputeExpression.py

comment
compute expression

```
1 # Compute expression
2 print((10.5 + 2 * 3) / (45 - 3.5))
```



```
0.397590361446
```

As you can see, it is a straightforward process to translate an arithmetic expression to a Python expression. We will discuss Python expressions further in Chapter 2.



MyProgrammingLab™

1.24 You can run Python in two modes. Explain these two modes.

1.25 Is Python case sensitive?

1.26 What is the Python source filename extension by convention?

1.27 What is the command to run a Python source file?

1.28 What is a comment? How do you denote a comment line and a comment paragraph?

1.29 What is the statement to display the message **Hello world** on the console?

1.30 Identify and fix the errors in the following code:

```
1 # Display two messages
2 print("Welcome to Python")
3 print("Python is fun").
```

1.31 Show the output of the following code:

```
print("3.5 * 4 / 2 - 2.5 is")
print(3.5 * 4 / 2 - 2.5)
```

1.7 Programming Style and Documentation



programming style

Good programming style and proper documentation make a program easy to read and prevents errors.

Programming style deals with what programs look like. When you create programs with a professional programming style, they not only execute properly but are easy for people to read and understand. This is very important if other programmers will access or modify your programs.

documentation

Documentation is the body of explanatory remarks and comments pertaining to a program. These remarks and comments explain various parts of the program and help others understand its structure and function. As you saw earlier in the chapter, remarks and comments are

embedded within the program itself; Python's interpreter simply ignores them when the program is executed.

Programming style and documentation are as important as coding. Here are a few guidelines.

I.7.1 Appropriate Comments and Comment Styles

Include a summary comment at the beginning of the program to explain what the program does, its key features, and any unique techniques it uses. In a long program, you should also include comments that introduce each major step and explain anything that is difficult to read. It is important to make comments concise so that they do not crowd the program or make it difficult to read.

I.7.2 Proper Spacing

A consistent spacing style makes programs clear and easy to read, debug (find and fix errors), and maintain.

A single space should be added on both sides of an operator, as shown in the following statement:

<code>print(3+4*4)</code>	← Bad style
<code>print(3 + 4 * 4)</code>	← Good style

More detailed guidelines can be found in Supplement I.F, Python Coding Style Guidelines, on the Companion Website.

I.8 Programming Errors

Programming errors can be categorized into three types: syntax errors, runtime errors, and logic errors.



I.8.1 Syntax Errors

The most *common error* you will encounter are syntax errors. Like any programming language, Python has its own syntax, and you need to write code that obeys the *syntax rules*. If your program violates the rules—for example, if a quotation mark is missing or a word is misspelled—Python will report syntax errors.

Syntax errors result from errors in code construction, such as mistyping a statement, incorrect indentation, omitting some necessary punctuation, or using an opening parenthesis without a corresponding closing parenthesis. These errors are usually easy to detect, because Python tells you where they are and what caused them. For example, the following `print` statement has a syntax error:

common errors
syntax rules

syntax errors

```

C:\> Administrator: Command Prompt - python
>>> print(<Programing is fun>
File "<stdin>", line 1
    print(<Programing is fun>
          ^
SyntaxError: EOL while scanning string literal
>>> _

```

The string **Programming is fun** should be closed with a closing quotation mark.



Tip

fix syntax errors

If you don't know how to correct a syntax error, compare your program closely, character by character, with similar examples in the text. In the first few weeks of this course, you will probably spend a lot of time fixing syntax errors. Soon, you will be familiar with Python syntax and will be able to fix syntax errors quickly.

1.8.2 Runtime Errors

runtime errors

input errors

Runtime errors are errors that cause a program to terminate abnormally. They occur while a program is running if the Python interpreter detects an operation that is impossible to carry out. Input mistakes typically cause runtime errors. An *input error* occurs when the user enters a value that the program cannot handle. For instance, if the program expects to read in a number, but instead the user enters a string of text, this causes data-type errors to occur in the program.

Another common source of runtime errors is division by zero. This happens when the divisor is zero for integer divisions. For example, the expression **1 / 0** in the following statement would cause a runtime error.

```

Administrator: Command Prompt - python
>>> print(1 / 0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
>>>

```

1.8.3 Logic Errors

logic errors

Logic errors occur when a program does not perform the way it was intended to. Errors of this kind occur for many different reasons. For example, suppose you wrote the program in Listing 1.4 to convert a temperature (35 degrees) from Fahrenheit to Celsius.

LISTING 1.4 ShowLogicErrors.py

```

1 # Convert Fahrenheit to Celsius
2 print("Fahrenheit 35 is Celsius degree ")
3 print(5 / 9 * 35 - 32)

```



```

Fahrenheit 35 is Celsius degree
-12.55555555555554

```

You will get Celsius **-12.55** degrees, which is wrong. It should be **1.66**. To get the correct result, you need to use **5 / 9 * (35 - 32)** rather than **5 / 9 * 35 - 32** in the expression. That is, you need to add parentheses around **(35 - 32)** so Python will calculate that expression first before doing the division.

In Python, syntax errors are actually treated like runtime errors because they are detected by the interpreter when the program is executed. In general, syntax and runtime errors are easy to find and easy to correct, because Python gives indications as to where the errors came from and why they are wrong. Finding logic errors, on the other hand, can be very challenging.

- I.32** What are three kinds of program errors?
- I.33** If you forget to put a closing quotation mark on a string, what kind of error will be raised?
- I.34** If your program needs to read data from a file, but the file does not exist, an error would occur when running this program. What kind of error is this?
- I.35** Suppose you write a program for computing the perimeter of a rectangle and you mistakenly write your program so that it computes the area of a rectangle. What kind of error is this?



I.9 Getting Started with Graphics Programming

Turtle is Python's built-in graphics module for drawing lines, circles, and other shapes, including text. It is easy to learn and simple to use.



Beginners often enjoy learning programming by using graphics. For this reason, we provide a section on graphics programming at the end of most of the chapters in the first part of the book. However, these materials are not mandatory. They can be skipped or covered later.

There are many ways to write graphics programs in Python. A simple way to start graphics programming is to use Python's built-in **turtle** module. Later in the book, we will introduce *Tkinter* for developing comprehensive graphical user interface applications.

turtle
Tkinter

I.9.1 Drawing and Adding Color to a Figure

The following procedure will give you a basic introduction to using the **turtle** module. Subsequent chapters introduce more features.

1. Launch Python by choosing **Python** (command line) from the Windows *Start* button or by typing **python** at the command prompt.
2. At the Python statement prompt **>>>**, type the following command to import the **turtle** module. This command imports all functions defined in the **turtle** module and makes them available for you to use.

```
>>> import turtle # Import turtle module
```

3. Type the following command to show the current location and direction of the turtle, as shown in Figure 1.15a.

```
>>> turtle.showturtle()
```

Graphics programming using the Python Turtle module is like drawing with a pen. The arrowhead indicates the current position and direction of the pen. **turtle** is initially positioned at the center of the window. Here, **turtle** refers to the object for drawing graphics (objects will be introduced in Chapter 3).

4. Type the following command to draw a text string:

```
>>> turtle.write("Welcome to Python")
```

Your window should look like the one shown in Figure 1.15b.

5. Type the following command to move the arrowhead **100** pixels forward to draw a line in the direction the arrow is pointing:

```
>>> turtle.forward(100)
```

Your window should now look like the one shown in Figure 1.15c.

To draw the rest of Figure 1.15, continue with these steps.



VideoNote
Start with Turtle graphics

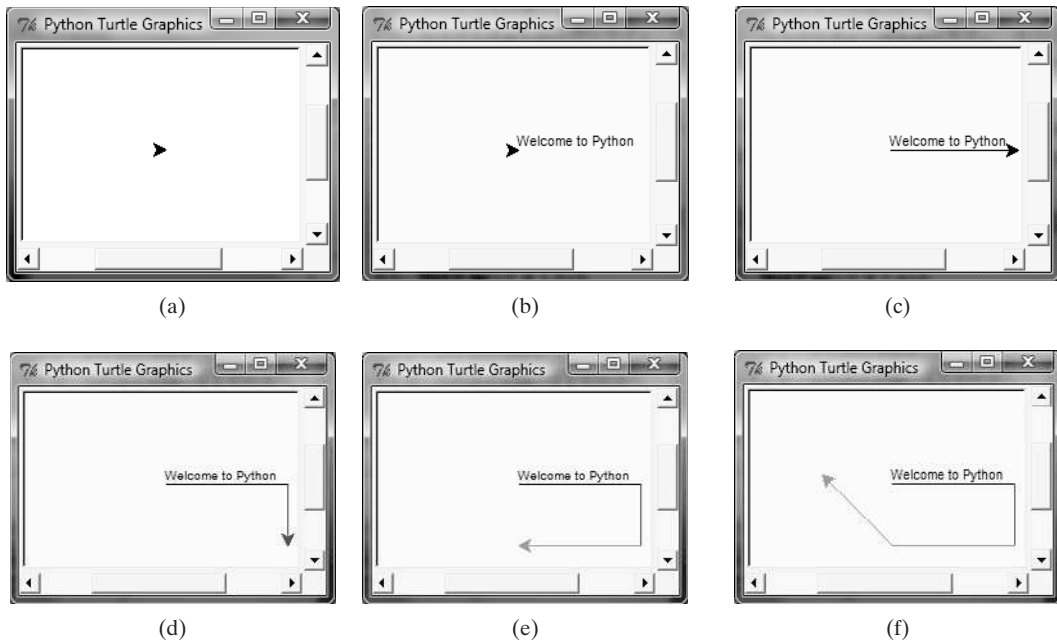


FIGURE 1.15 Graphics are dynamically displayed with each statement.

6. Type the following commands to turn the arrowhead right **90** degrees, change the **turtle**'s color to red, and move the arrowhead **50** pixels forward to draw a line, as shown in Figure 1.15d:

```
>>> turtle.right(90)
>>> turtle.color("red")
>>> turtle.forward(50)
```

7. Now, type the following commands to turn the arrowhead right **90** degrees, set the color to green, and move the arrowhead **100** pixels forward to draw a line, as shown in Figure 1.15e:

```
>>> turtle.right(90)
>>> turtle.color("green")
>>> turtle.forward(100)
```

8. Finally, type the following commands to turn the arrowhead right **45** degrees and move it **80** pixels forward to draw a line, as shown in Figure 1.15f:

```
>>> turtle.right(45)
>>> turtle.forward(80)
```

9. You can now close the Turtle Graphics window and exit Python.

1.9.2 Moving the Pen to Any Location

When the Turtle program starts, the arrowhead is at the center of the Python Turtle Graphics window at the coordinates **(0, 0)**, as shown in Figure 1.16a. You can also use the **goto(x, y)** command to move the **turtle** to any specified point **(x, y)**.

Restart Python and type the following command to move the pen to **(0, 50)** from **(0, 0)**, as shown in Figure 1.16b.

```
>>> import turtle
>>> turtle.goto(0, 50)
```

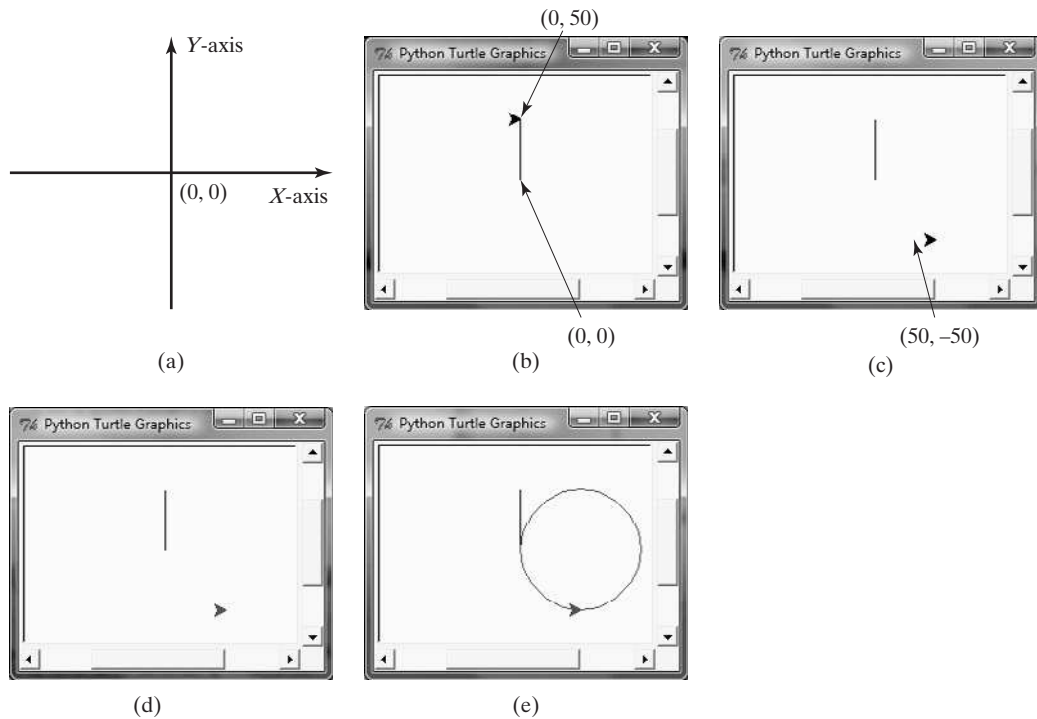


FIGURE 1.16 (a) The center of the Turtle Graphics window is at the coordinates (0, 0). (b) Move to (0, 50). (c) Move the pen to (50, -50). (d) Set color to red. (e) Draw a circle using the `circle` command.

You can also lift the pen up or put it down to control whether to draw a line when the pen is moved by using the `penup()` and `pendown()` commands. For example, the following commands move the pen to (50, -50), as shown in Figure 1.16c.

```
>>> turtle.penup()
>>> turtle.goto(50, -50)
>>> turtle.pendown()
```

You can draw a circle using the `circle` command. For example, the following commands set color red (Figure 1.16d) and draw a circle with radius 50 (Figure 1.16e).

```
>>> turtle.color("red")
>>> turtle.circle(50) # Draw a circle with radius 50
```

1.9.3 Drawing the Olympic Rings Logo

Listing 1.5 shows a program for drawing the Olympics rings logo, as shown in Figure 1.17.

LISTING 1.5 OlympicSymbol.py

```
1 import turtle
2
3 turtle.color("blue")
4 turtle.penup()
5 turtle.goto(-110, -25)
6 turtle.pendown()
7 turtle.circle(45)
8
9 turtle.color("black")
```

draw blue circle

draw black circle

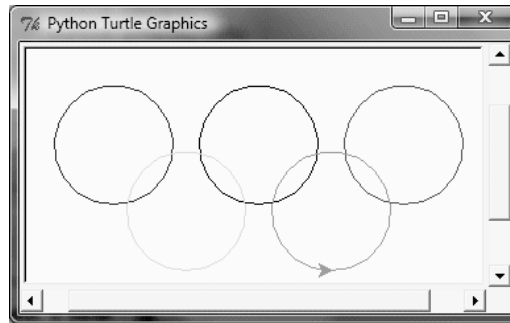


FIGURE 1.17 The program draws the Olympics rings logo.

	10	<code>turtle.penup()</code>
	11	<code>turtle.goto(0, -25)</code>
	12	<code>turtle.pendown()</code>
	13	<code>turtle.circle(45)</code>
	14	
draw red circle	15	<code>turtle.color("red")</code>
	16	<code>turtle.penup()</code>
	17	<code>turtle.goto(110, -25)</code>
	18	<code>turtle.pendown()</code>
	19	<code>turtle.circle(45)</code>
	20	
draw yellow circle	21	<code>turtle.color("yellow")</code>
	22	<code>turtle.penup()</code>
	23	<code>turtle.goto(-55, -75)</code>
	24	<code>turtle.pendown()</code>
	25	<code>turtle.circle(45)</code>
	26	
draw green circle	27	<code>turtle.color("green")</code>
	28	<code>turtle.penup()</code>
	29	<code>turtle.goto(55, -75)</code>
	30	<code>turtle.pendown()</code>
	31	<code>turtle.circle(45)</code>
	32	
pause	33	<code>turtle.done()</code>

The program imports the `turtle` module to use the Turtle Graphics window (line 1). It moves the pen to $(-110, -25)$ (line 5) and draws a blue circle with radius 45 (line 7). Similarly, it draws a black circle (lines 9–13), a red circle (lines 15–19), a yellow circle (lines 21–25), and a green circle (lines 27–31).

Line 33 invokes `turtle`'s `done()` command, which causes the program to pause until the user closes the Python Turtle Graphics window. The purpose of this is to give the user time to view the graphics. Without this line, the graphics window would be closed right after the program is finished.



MyProgrammingLab™

I.36 How do you import the `turtle` module?

I.37 How do you display text in Turtle?

I.38 How do you move the pen forward?

I.39 How do you set a new color?

I.40 How do you move the pen without drawing anything?

I.41 How do you draw a circle?

I.42 What is the purpose of `turtle.done()` in line 33 in Listing 1.5?

KEY TERMS

.py file 16	line comment 16
assembler 10	logic error 20
assembly language 10	low-level language 10
bit 4	machine language 9
bus 2	memory 5
byte 4	modem 8
cable modem 8	module 16
calling a function 15	motherboard 3
central processing unit (CPU) 3	network interface card (NIC) 8
comment 16	operating system (OS) 12
compiler 10	pixel 8
console 14	program 2
dot pitch 8	runtime errors 20
DSL (digital subscriber line) 8	screen resolution 8
encoding scheme 4	script file 16
function 15	script mode 16
hardware 2	software 2
high-level language 10	source code 10
IDLE (Interactive DeveLopment Environment) 14	source file 16
indentation 17	source program 10
interactive mode 16	statement 10
interpreter 10	storage device 5
invoking a function 15	syntax errors 19
	syntax rules 19



Note

The above terms are defined in the present chapter. Supplement I.A, Glossary, lists all the key terms and descriptions in the book, organized by chapters.

Supplement I.A

CHAPTER SUMMARY

1. A computer is an electronic device that stores and processes data.
2. A computer includes both *hardware* and *software*.
3. Hardware is the physical aspect of the computer that can be touched.
4. Computer *programs*, known as *software*, are the invisible instructions that control the hardware and make it perform tasks.
5. *Computer programming* is the writing of instructions (i.e., code) for computers to perform.
6. The *central processing unit (CPU)* is a computer's brain. It retrieves instructions from memory and executes them.
7. Computers use zeros and ones because digital devices have two stable electrical states, off and on, referred to by convention as zero and one.
8. A *bit* is a binary digit 0 or 1.

9. A *byte* is a sequence of 8 bits.
10. A kilobyte is about 1,000 bytes, a megabyte about 1 million bytes, a gigabyte about 1 billion bytes, and a terabyte about 1,000 gigabytes.
11. *Memory* stores data and program instructions for the CPU to execute.
12. A *memory unit* is an ordered sequence of bytes.
13. Memory is volatile, because information that hasn't been saved is lost when the power is turned off.
14. Programs and data are permanently stored on *storage devices* and are moved to memory when the computer actually uses them.
15. The *machine language* is a set of primitive instructions built into every computer.
16. *Assembly language* is a low-level programming language in which a mnemonic is used to represent each machine-language instruction.
17. *High-level languages* are English-like and easy to learn and program.
18. A program written in a high-level language is called *source code*.
19. A *compiler* is a software program that translates the *source program* into a *machine-language* program.
20. The *operating system (OS)* is a program that manages and controls a computer's activities.
21. You can run Python on Windows, UNIX, and Mac.
22. Python is *interpreted*, meaning that Python translates each statement and processes it one at a time.
23. You can enter Python statements interactively from the Python statement prompt `>>>` or store all your code in one file and execute it using one command.
24. To run a Python *source file* from the command line, use the **python filename.py** command.
25. In Python, *comments* are preceded by a pound sign (**#**) on a line, called a *line comment*, or enclosed between triple quotation marks (`'''` and `'''`) on one or several lines, called a *paragraph comment*.
26. Python *source programs* are case sensitive.
27. Programming errors can be categorized into three types: syntax errors, runtime errors, and logic errors. *Syntax* and *runtime errors* cause a program to terminate abnormally. *Logic errors* occur when a program does not perform the way it was intended to.

TEST QUESTIONS

Do test questions for this chapter online at www.cs.armstrong.edu/liang/py/test.html.

PROGRAMMING EXERCISES



Note

Solutions to even-numbered exercises in this book are on the Companion Website. Solutions to all exercises are on the Instructor Resource Website. The level of difficulty is rated easy (no star), moderate (*), hard (**), or challenging (***) .

level of difficulty

Section 1.6

- I.1** (Display three different messages) Write a program that displays **Welcome to Python**, **Welcome to Computer Science**, and **Programming is fun**.
- I.2** (Display the same message five times) Write a program that displays **Welcome to Python** five times.
- * I.3** (Display a pattern) Write a program that displays the following pattern:

```

FFFFFFF  U      U  NN      NN
FF        U      U  NNN     NN
FFFFFFF  U      U  NN N    NN
FF        U      U  NN  N   NN
FF        UUU     NN      NNN

```

- I.4** (Print a table) Write a program that displays the following table:

a	a ²	a ³
1	1	1
2	4	8
3	9	27
4	16	64

- I.5** (Compute expressions) Write a program that displays the result of

$$\frac{9.5 \times 4.5 - 2.5 \times 3}{45.5 - 3.5}$$

- I.6** (Summation of a series) Write a program that displays the result of $1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9$.

- I.7** (Approximate π) π can be computed using the following formula:

$$\pi = 4 \times \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \dots \right)$$

Write a program that displays the result of $4 \times \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} \right)$.

and $4 \times \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \frac{1}{13} - \frac{1}{15} \right)$.

- I.8** (Area and perimeter of a circle) Write a program that displays the area and perimeter of a circle that has a radius of **5.5** using the following formulas:

$$\begin{aligned} \text{area} &= \text{radius} \times \text{radius} \times \pi \\ \text{perimeter} &= 2 \times \text{radius} \times \pi \end{aligned}$$

- 1.9** (*Area and perimeter of a rectangle*) Write a program that displays the area and perimeter of a rectangle with the width of **4.5** and height of **7.9** using the following formula:

$$\text{area} = \text{width} \times \text{height}$$

- 1.10** (*Average speed*) Assume a runner runs **14** kilometers in **45** minutes and **30** seconds. Write a program that displays the average speed in miles per hour. (Note that **1** mile is **1.6** kilometers.)

- *1.11** (*Population projection*) The US Census Bureau projects population based on the following assumptions:

One birth every 7 seconds
 One death every 13 seconds
 One new immigrant every 45 seconds

Write a program to display the population for each of the next five years. Assume the current population is 312032486 and one year has 365 days. Hint: in Python, you can use integer division operator `//` to perform division. The result is an integer. For example, `5 // 4` is **1** (not **1.25**) and `10 // 4` is **2** (not **2.5**).

Section 1.9

- 1.12** (*Turtle: draw four squares*) Write a program that draws four squares in the center of the screen, as shown in Figure 1.18a.

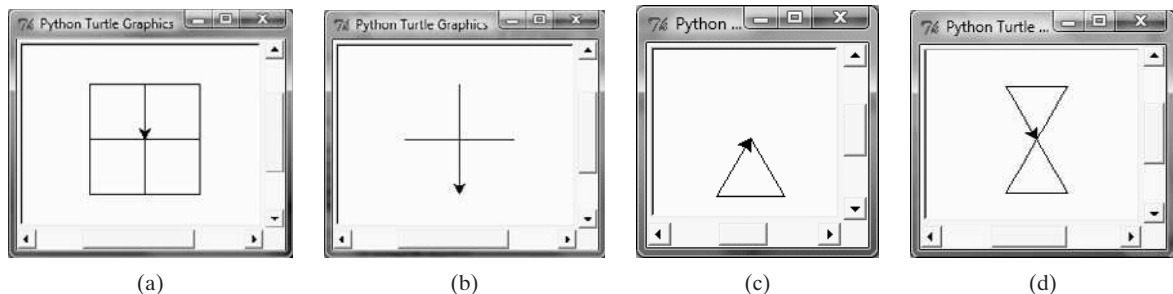


FIGURE 1.18 Four squares are drawn in (a), a cross is drawn in (b), a triangle is drawn in (c), and two triangles are drawn in (d).

- 1.13** (*Turtle: draw a cross*) Write a program that draws a cross as shown in Figure 1.18b.
- 1.14** (*Turtle: draw a triangle*) Write a program that draws a triangle as shown in Figure 1.18c.
- 1.15** (*Turtle: draw two triangles*) Write a program that draws two triangles as shown in Figure 1.18d.
- 1.16** (*Turtle: draw four circles*) Write a program that draws four circles in the center of the screen, as shown in Figure 1.19a.
- 1.17** (*Turtle: draw a line*) Write a program that draws a red line connecting two points `(-39, 48)` and `(50, -50)` and displays the coordinates of the two points, as shown in Figure 1.19b.
- **1.18** (*Turtle: draw a star*) Write a program that draws a star, as shown in Figure 1.19c. (Hint: The inner angle of each point in the star is 36 degrees.)

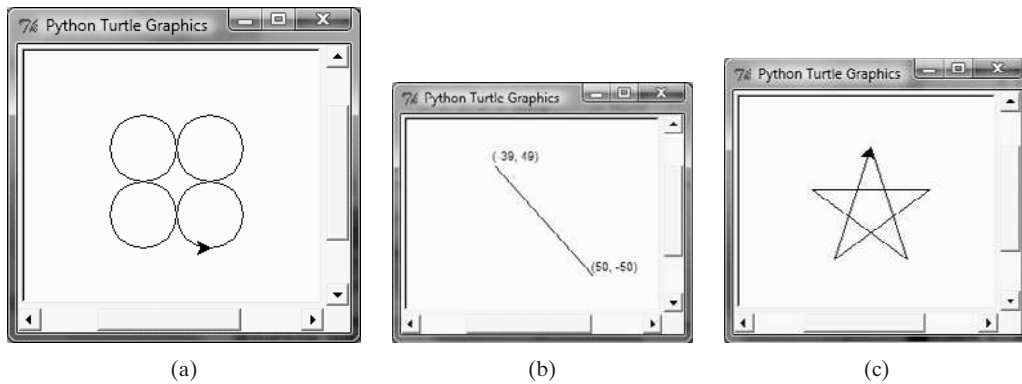


FIGURE 1.19 Four circles are drawn in (a), a line is drawn in (b), and a star is drawn in (c).

1.19 (*Turtle: draw a polygon*) Write a program that draws a polygon that connects the points $(40, -69.28)$, $(-40, -69.28)$, $(-80, -9.8)$, $(-40, 69)$, $(40, 69)$, and $(80, 0)$ in this order, as shown in Figure 1.20a.

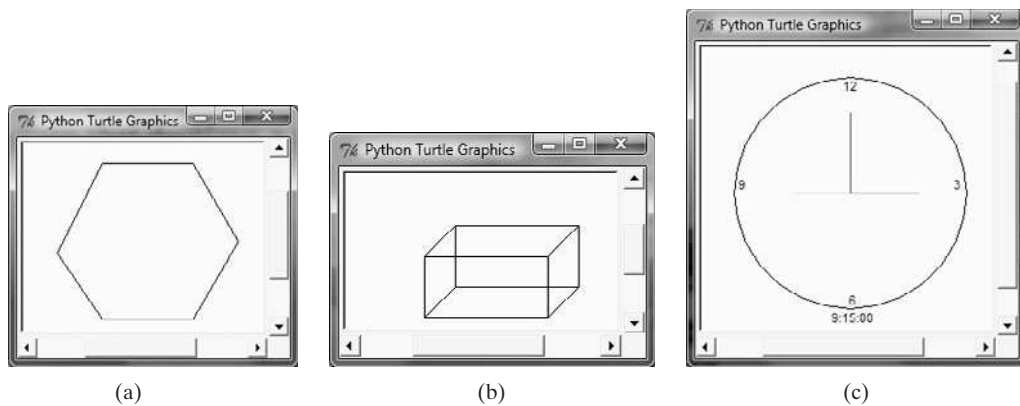


FIGURE 1.20 (a) The program displays a polygon. (b) The program displays a rectanguloid. (c) The program displays a clock for the time.

1.20 (*Turtle: display a rectanguloid*) Write a program that displays a rectanguloid, as shown in Figure 1.20b.

***1.21** (*Turtle: display a clock*) Write a program that displays a clock to show the time 9:15:00, as shown in Figure 1.20c.

This page intentionally left blank

ELEMENTARY PROGRAMMING

Objectives

- To write programs that perform simple computations (§2.2).
- To obtain input from a program's user by using the `input` function (§2.3).
- To use identifiers to name elements such as variables and functions (§2.4).
- To assign data to variables (§2.5).
- To perform simultaneous assignment (§2.6).
- To define named constants (§2.7).
- To use the operators `+`, `-`, `*`, `/`, `//`, `%`, and `**` (§2.8).
- To write and evaluate numeric expressions (§2.9).
- To use augmented assignment operators to simplify coding (§2.10).
- To perform numeric type conversion and rounding with the `int` and `round` functions (§2.11).
- To obtain the current system time by using `time.time()` (§2.12).
- To describe the software development process and apply it to develop a loan payment program (§2.13).
- To compute and display the distance between two points in graphics (§2.14).



2.1 Introduction



The focus of this chapter is on learning elementary programming techniques to solve problems.

In Chapter 1 you learned how to create and run very basic Python programs. Now you will learn how to solve problems by writing programs. Through these problems, you will learn fundamental programming techniques, such as the use of variables, operators, expressions, and input and output.

Suppose, for example, that you need to take out a student loan. Given the loan amount, loan term, and annual interest rate, can you write a program to compute the monthly payment and total payment? This chapter shows you how to write programs like this. Along the way, you learn the basic steps that go into analyzing a problem, designing a solution, and implementing the solution by creating a program.

2.2 Writing a Simple Program



Writing a program involves designing a strategy for solving the problem and then using a programming language to implement that strategy.

problem

Let's first consider the simple *problem* of computing the area of a circle. How do we write a program for solving this problem?

algorithm

Writing a program involves designing algorithms and then translating them into programming instructions, or code. When you *code*—that is, when you write a program—you translate an algorithm into a program. An *algorithm* describes how a problem is solved by listing the actions that need to be taken and the order of their execution. Algorithms can help the programmer plan a program before writing it in a programming language. Algorithms can be described in natural languages or in *pseudocode* (natural language mixed with some programming code). The algorithm for calculating the area of a circle can be described as follows:

pseudocode

1. Get the circle's radius from the user.
2. Compute the area by applying the following formula:

$$\text{area} = \text{radius} \times \text{radius} \times \pi$$

3. Display the result.



Tip

It's always good practice to outline your program (or its underlying problem) in the form of an algorithm before you begin coding.

In this problem, the program needs to read the radius, which the program's user enters from the keyboard. This raises two important issues:

- Reading the radius.
- Storing the radius in the program.

variable

Let's address the second issue first. The value for the radius is stored in the computer's memory. In order to access it, the program needs to use a *variable*. A variable is a name that references a value stored in the computer's memory. Rather than using **x** and **y** as variable names, choose *descriptive names*: in this case, for example, you can use the name **radius** for the variable that references a value for radius and **area** for the variable that references a value for area.

descriptive names

The first step is to prompt the user to designate the circle's **radius**. You will learn how to prompt the user for information shortly. For now, to learn how variables work, you can assign a fixed value to **radius** in the program as you write the code.

The second step is to compute **area** by assigning the result of the expression **radius * radius * 3.14159** to **area**.

In the final step, the program will display the value of **area** on the console by using Python's **print** function.

The complete program is shown in Listing 2.1.

LISTING 2.1 ComputeArea.py

```
1 # Assign a value to radius
2 radius = 20 # radius is now 20
3
4 # Compute area
5 area = radius * radius * 3.14159
6
7 # Display results
8 print("The area for the circle of radius", radius, "is", area)
```

radius → 20

area → 1256.636

display result

The area for the circle of radius 20 is 1256.636



Variables such as **radius** and **area** reference values stored in memory. Every variable has a name that refers to a value. You can *assign a value* to a variable using the syntax as shown in line 2.

assign value

```
radius = 20
```

This statement assigns **20** to the variable **radius**. So now **radius** references the value **20**. The statement in line 5

```
area = radius * radius * 3.14159
```

uses the value in **radius** to compute the expression and assigns the result into the variable **area**. The following table shows the value for **radius** and **area** as the program is executed. Each row in the table shows the values of variables after the statement in the corresponding line in the program is executed. This method of reviewing how a program works is called *tracing a program*. Tracing programs are helpful for understanding how programs work, and they are useful tools for finding errors in programs.

trace a program

line#	radius	area
2	20	
5		1256.636



If you have programmed in other languages, such as Java, you know you have to declare a variable with a *data type* to specify what type of values are being used, such as integers or text characters. You don't do this in Python, however, because Python automatically figures out the data type according to the value assigned to the variable.

data type

The statement in line 8 displays four items on the console. You can display any number of items in a **print** statement using the following syntax:

```
print(item1, item2, ..., itemk)
```

print(item1, ...)



MyProgrammingLab™

If an item is a number, the number is automatically converted to a string for displaying.

2.1 Show the printout of the following code:

```
width = 5.5
height = 2
print("area is", width * height)
```

2.2 Translate the following algorithm into Python code:

- Step 1: Use a variable named **miles** with initial value **100**.
- Step 2: Multiply miles by **1.609** and assign it to a variable named **kilometers**.
- Step 3: Display the value of **kilometers**.

What is **kilometers** after Step 3?

2.3 Reading Input from the Console



Reading input from the console enables the program to accept input from the user.

In Listing 2.1, a radius is set in the source code. To use a different radius, you have to modify the source code. You can use the **input** function to ask the user to input a value for the radius. The following statement prompts the user to enter a value, and then it assigns the value to the variable:

```
variable = input("Enter a value: ")
```

eval function

The value entered is a string. You can use the function **eval** to evaluate and convert it to a numeric value. For example, **eval("34.5")** returns **34.5**, **eval("345")** returns **345**, **eval("3 + 4")** returns **7**, and **eval("51 + (54 * (3 + 2))")** returns **321**.

Listing 2.2 rewrites Listing 2.1 to prompt the user to enter a radius.

LISTING 2.2 ComputeAreaWithConsoleInput.py

input radius

compute area

display result

```
1 # Prompt the user to enter a radius
2 radius = eval(input("Enter a value for radius: "))
3
4 # Compute area
5 area = radius * radius * 3.14159
6
7 # Display results
8 print("The area for the circle of radius", radius, "is", area)
```



```
Enter a value for radius: 2.5 [Enter]
The area for the circle of radius 2.5 is 19.6349375
```



```
Enter a value for radius: 23 [Enter]
The area for the circle of radius 23 is 1661.90111
```

Line 2 prompts the user to enter a value (in the form of a string) and converts it to a number, which is equivalent to

```
s = input("Enter a value for radius: ") # Read input as a string
radius = eval(s) # Convert the string to a number
```

After the user enters a number and presses the *Enter* key, the number is read and assigned to **radius**.

Listing 2.2 shows how to prompt the user for a single input. However, you can prompt for multiple inputs as well. Listing 2.3 gives an example of reading multiple inputs from the keyboard. This program reads three integers and displays their average.

LISTING 2.3 ComputeAverage.py

```
1 # Prompt the user to enter three numbers
2 number1 = eval(input("Enter the first number: "))
3 number2 = eval(input("Enter the second number: "))
4 number3 = eval(input("Enter the third number: "))
5
6 # Compute average
7 average = (number1 + number2 + number3) / 3
8
9 # Display result
10 print("The average of", number1, number2, number3,
11       "is", average)
```

input number1
input number2
input number3



```
Enter the first number: 1 ↵ Enter
Enter the second number: 2 ↵ Enter
Enter the third number: 3 ↵ Enter
The average of 1 2 3 is 2.0
```

The program prompts the user to enter three integers (lines 2–4), computes their average (line 7), and displays the result (lines 10–11).

If the user enters something other than a number, the program will terminate with a *runtime error*. In Chapter 13, you will learn how to handle the error so that the program can continue to run.

Normally a statement ends at the end of the line. In the preceding listing, the `print` statement is split into two lines (lines 10–11). This is okay, because Python scans the print statement in line 10 and knows it is not finished until it finds the closing parenthesis in line 11. We say that these two lines are *joined implicitly*.

runtime error

joining lines explicitly



Note

In some cases, the Python interpreter cannot determine the end of the statement written in multiple lines. You can place the *line continuation symbol* (`\`) at the end of a line to tell the interpreter that the statement is continued on the next line. For example, the following statement

```
sum = 1 + 2 + 3 + 4 + \
      5 + 6
```

is equivalent to

```
sum = 1 + 2 + 3 + 4 + 5 + 6
```

line continuation symbol

split a long statement



Note

Most of the programs in early chapters of this book perform three steps: Input, Process, and Output, called *IPO*. Input is to receive input from the user. Process is to produce results using the input. Output is to display the results.

IPO

2.3 How do you write a statement to prompt the user to enter a numeric value?

2.4 What happens if the user enters `5a` when executing the following code?

```
radius = eval(input("Enter a radius: "))
```

2.5 How do you break a long statement into multiple lines?



MyProgrammingLab™

2.4 Identifiers



Identifiers are the names that identify the elements such as variables and functions in a program.

identifiers

As you can see in Listing 2.3, `number1`, `number2`, `number3`, `average`, `input`, `eval`, and `print` are the names of things that appear in the program. In programming terminology, such names are called *identifiers*. All identifiers must obey the following rules:

identifier naming rules

- An identifier is a sequence of characters that consists of letters, digits, and underscores (`_`).
- An identifier must start with a letter or an underscore. It cannot start with a digit.
- An identifier cannot be a keyword. (See Appendix A, Python Keywords, for a list of keywords.) *Keywords*, also called *reserved words*, have special meanings in Python. For example, `import` is a keyword, which tells the Python interpreter to import a module to the program.
- An identifier can be of any length.

keyword

reserved word

For example, `area`, `radius`, and `number1` are legal identifiers, whereas `2A` and `d+4` are not because they do not follow the rules. When Python detects an illegal identifier, it reports a syntax error and terminates the program.



Note

case sensitive

Because Python is case sensitive, `area`, `Area`, and `AREA` are all different identifiers.



Tip

descriptive names

Descriptive identifiers make programs easy to read. Avoid using abbreviations for identifiers. Using complete words is more descriptive. For example, `numberOfStudents` is better than `numStuds`, `numOfStuds`, or `numOfStudents`. We use descriptive names for complete programs in the text. However, we will occasionally use variables names such as `i`, `j`, `k`, `x`, and `y` in the code snippets for brevity. These names also provide a generic tone to the code snippets.



Tip

variable naming convention

Use lowercase letters for variable names, as in `radius` and `area`. If a name consists of several words, concatenate them into one, making the first word lowercase and capitalizing the first letter of each subsequent word—for example, `numberOfStudents`. This naming style is known as the *camelCase* because the uppercase characters in the name resemble a camel's humps.

camelCase



2.6 Which of the following identifiers are valid? Which are Python keywords (see Appendix A)?

`miles`, `Test`, `a+b`, `b-a`, `4#R`, `$4`, `#44`, `apps`
`if`, `elif`, `x`, `y`, `radius`

MyProgrammingLab™

2.5 Variables, Assignment Statements, and Expressions



Variables are used to reference values that may be changed in the program.

why called variables?

As you can see from the programs in the preceding sections, variables are the names that reference values stored in memory. They are called “variables” because they may reference different values. For example, in the following code, `radius` is initially `1.0` (line 2) and then changed to `2.0` (line 7), and `area` is set to `3.14159` (line 3) and then reset to `12.56636` (line 8).

```

1 # Compute the first area
2 radius = 1.0
3 area = radius * radius * 3.14159
4 print("The area is", area, "for radius", radius)
5
6 # Compute the second area
7 radius = 2.0
8 area = radius * radius * 3.14159
9 print("The area is", area, "for radius", radius)

```

radius → 1.0
area → 3.14159



VideoNote
Assignment statement

radius → 2.0
area → 12.56636

The statement for assigning a value to a variable is called an *assignment statement*. In Python, the equal sign (=) is used as the *assignment operator*. The syntax for assignment statements is as follows:

assignment statement
assignment operator

variable = expression

An *expression* represents a computation involving values, variables, and operators that, taken together, evaluate to a value. For example, consider the following code:

expression

```

y = 1                # Assign 1 to variable y
radius = 1.0         # Assign 1.0 to variable radius
x = 5 * (3 / 2) + 3 * 2 # Assign the value of the expression to x
x = y + 1            # Assign the addition of y and 1 to x
area = radius * radius * 3.14159 # Compute area

```

You can use a variable in an expression. A variable can also be used in both sides of the = operator. For example,

x = x + 1

In this assignment statement, the result of $x + 1$ is assigned to x . If x is 1 before the statement is executed, then it becomes 2 after the statement is executed.

To assign a value to a variable, you must place the variable name to the left of the assignment operator. Thus, the following statement is wrong:

1 = x # Wrong



Note

In mathematics, $x = 2 * x + 1$ denotes an equation. However, in Python, $x = 2 * x + 1$ is an assignment statement that evaluates the expression $2 * x + 1$ and assigns the result to x .

If a value is assigned to multiple variables, you can use a syntax like this:

i = j = k = 1

which is equivalent to

```

k = 1
j = k
i = j

```

Every variable has a scope. The *scope of a variable* is the part of the program where the variable can be referenced. The rules that define the scope of a variable will be introduced gradually later in the book. For now, all you need to know is that a variable must be created before it can be used. For example, the following code is wrong:

scope of a variable



```

count is not defined yet.
>>> count = count + 1
NameError: count is not defined
>>>

```

To fix it, you may write the code like this:



```

>>> count = 1 # count is not created
>>> count = count + 1 # Now increment count
>>>

```



Caution

A variable must be assigned a value before it can be used in an expression. For example,

```

interestRate = 0.05
interest = interestrate * 45

```

This code is wrong, because `interestRate` is assigned a value `0.05`, but `interestrate` is not defined. Python is case-sensitive. `interestRate` and `interestrate` are two different variables.

2.6 Simultaneous Assignments

simultaneous assignment

Python also supports *simultaneous assignment* in syntax like this:

```
var1, var2, ..., varn = exp1, exp2, ..., expn
```

It tells Python to evaluate all the expressions on the right and assign them to the corresponding variable on the left simultaneously. Swapping variable values is a common operation in programming and simultaneous assignment is very useful to perform this operation. Consider two variables: `x` and `y`. How do you write the code to swap their values? A common approach is to introduce a temporary variable as follows:



```

>>> x = 1
>>> y = 2
>>> temp = x # Save x in a temp variable
>>> x = y    # Assign the value in y to x
>>> y = temp # Assign the value in temp to y

```

But you can simplify the task using the following statement to swap the values of `x` and `y`.



```
>>> x, y = y, x # Swap x with y
```

Simultaneous assignment can also be used to obtain multiple input in one statement. Listing 2.3 gives an example that prompts the user to enter three numbers and obtains their average. This program can be simplified using a simultaneous assignment statement, as shown in Listing 2.4.

LISTING 2.4 ComputeAverageWithSimultaneousAssignment.py

```

1 # Prompt the user to enter three numbers
2 number1, number2, number3 = eval(input(
3     "Enter three numbers separated by commas: "))
4
5 # Compute average
6 average = (number1 + number2 + number3) / 3
7
8 # Display result
9 print("The average of", number1, number2, number3
10     "is", average)

```

input numbers

Enter three numbers separated by commas: 1, 2, 3 

The average of 1 2 3 is 2.0



enter input in one line

2.7 What is the naming convention for variables?**2.8** What is wrong in the following statement?`2 = a`**2.9** What is `x`, `y`, and `z` after the following statement?`x = y = z = 0`**2.10** Assume that `a = 1` and `b = 2`. What is `a` and `b` after the following statement?`a, b = b, a`

MyProgrammingLab™

2.7 Named Constants

A named constant is an identifier that represents a permanent value.

The value of a variable may change during the execution of a program, but a *named constant* (or simply *constant*) represents permanent data that never changes. In our **ComputeArea** program, π is a constant. If you use it frequently, you don't want to keep typing **3.14159**; instead, you can use a descriptive name **PI** for the value. Python does not have a special syntax for naming constants. You can simply create a variable to denote a constant. However, to distinguish a constant from a variable, use all uppercase letters to name a constant. For example, you can rewrite Listing 2.1 to use a named constant for π , as follows:

```

# Assign a radius
radius = 20 # radius is now 20

# Compute area
PI = 3.14159
area = radius * radius * PI

# Display results
print("The area for the circle of radius", radius, "is", area)

```



constant naming convention

There are three benefits of using constants:

benefits of constants

1. You don't have to repeatedly type the same value if it is used multiple times.
2. If you have to change the constant's value (e.g., from **3.14** to **3.14159** for **PI**), you need to change it only in a single location in the source code.
3. Descriptive names make the program easy to read.

2.8 Numeric Data Types and Operators



Python has two numeric types—integers and floating-point numbers—for working with the operators `+`, `-`, `*`, `/`, `//`, `**`, and `%`.

floating-point numbers

integer

int

float



VideoNote

Perform computation

literal

operands

operators `+`, `-`, `*`, `/`, `//`, `**`, `%`

The information stored in a computer is generally referred to as *data*. There are two types of numeric data: integers and real numbers. Integer types (*int* for short) are for representing whole numbers. Real types are for representing numbers with a fractional part. Inside the computer, these two types of data are stored differently. Real numbers are represented as floating-point (or *float*) values. How do we tell Python whether a number is an integer or a float? A number that has a decimal point is a float even if its fractional part is `0`. For example, `1.0` is a float, but `1` is an integer. These two numbers are stored differently in the computer. In the programming terminology, numbers such as `1.0` and `1` are called literals. A *literal* is a constant value that appears directly in a program.

The operators for numeric data types include the standard arithmetic operators, as shown in Table 2.1. The *operands* are the values operated by an operator.

TABLE 2.1 Numeric Operators

Name	Meaning	Example	Result
<code>+</code>	Addition	<code>34 + 1</code>	<code>35</code>
<code>-</code>	Subtraction	<code>34.0 - 0.1</code>	<code>33.9</code>
<code>*</code>	Multiplication	<code>300 * 30</code>	<code>9000</code>
<code>/</code>	Float Division	<code>1 / 2</code>	<code>0.5</code>
<code>//</code>	Integer Division	<code>1 // 2</code>	<code>0</code>
<code>**</code>	Exponentiation	<code>4 ** 0.5</code>	<code>2.0</code>
<code>%</code>	Remainder	<code>20 % 3</code>	<code>2</code>

The `+`, `-`, and `*` operators are straightforward, but note that the `+` and `-` operators can be both unary and binary. A *unary* operator has only one operand; a *binary* operator has two. For example, the `-` operator in `-5` is a unary operator to negate the number `5`, whereas the `-` operator in `4 - 5` is a binary operator for subtracting `5` from `4`.

2.8.1 The `/`, `//`, and `**` Operators

The `/` operator performs a float division that results in a floating number. For example,



```
>>> 4 / 2
2.0
>>> 2 / 4
0.5
>>>
```

unary operator

binary operator

`/` operator

`//` operator



```
>>> 5 // 2
2
>>> 2 // 4
0
>>>
```

The `//` operator performs an integer division; the result is an integer, and any fractional part is truncated. For example,

To compute a^b (**a** with an exponent of **b**) for any numbers **a** and **b**, you can write **a ** b** ****** exponent operator in Python. For example,

```
>>> 2.3 ** 3.5
18.4521691055504
>>> (-2.5) ** 2
6.25
>>>
```

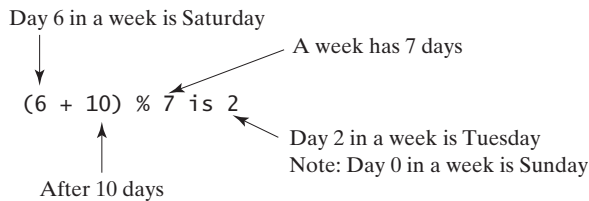


2.8.2 The % Operator

The **%** operator, known as *remainder* or *modulo* operator, yields the remainder after division. **%** operator
The left-side operand is the dividend and the right-side operand is the divisor. Therefore, **7 % 3** yields **1**, **3 % 7** yields **3**, **12 % 4** yields **0**, **26 % 8** yields **2**, and **20 % 13** yields **7**.

$$\begin{array}{r}
 2 \\
 3 \overline{) 7} \\
 \underline{6} \\
 1
 \end{array}
 \quad
 \begin{array}{r}
 0 \\
 7 \overline{) 3} \\
 \underline{0} \\
 3
 \end{array}
 \quad
 \begin{array}{r}
 3 \\
 4 \overline{) 12} \\
 \underline{12} \\
 0
 \end{array}
 \quad
 \begin{array}{r}
 3 \\
 8 \overline{) 26} \\
 \underline{24} \\
 2
 \end{array}
 \quad
 \begin{array}{r}
 \text{Divisor} \longrightarrow 13 \overline{) 20} \\
 \underline{13} \\
 7 \longleftarrow \text{Remainder}
 \end{array}
 \quad
 \begin{array}{l}
 1 \longleftarrow \text{Quotient} \\
 20 \longleftarrow \text{Dividend}
 \end{array}$$

The remainder operator is very useful in programming. For example, an even number **% 2** is always **0** and an odd number **% 2** is always **1**. Thus, you can use this property to determine whether a number is even or odd. If today is Saturday, it will be Saturday again in 7 days. Suppose you and your friends are going to meet in 10 days. What day is in 10 days? You can find that the day is Tuesday using the following expression:



Listing 2.5 shows a program that obtains minutes and remaining seconds from an amount of time in seconds. For example, **500** seconds contains **8** minutes and **20** seconds.

LISTING 2.5 DisplayTime.py

```
1 # Prompt the user for input
2 seconds = eval(input("Enter an integer for seconds: "))
3
4 # Get minutes and remaining seconds
5 minutes = seconds // 60 # Find minutes in seconds
6 remainingSeconds = seconds % 60 # Seconds remaining
7 print(seconds, "seconds is", minutes,
8       "minutes and", remainingSeconds, "seconds")
```

input seconds

get minutes

get remainingSeconds

```
Enter an integer for seconds: 500
500 seconds is 8 minutes and 20 seconds
```





line#	seconds	minutes	remainingSeconds
2	500		
5		8	
6			20

Line 2 reads an integer for `seconds`. Line 5 obtains the minutes using `seconds // 60`. Line 6 (`seconds % 60`) obtains the remaining seconds after taking away the minutes.

2.8.3 Scientific Notation

scientific notation

Floating-point values can be written in *scientific notation* in the form of $a \times 10^b$. For example, the scientific notation for 123.456 is 1.23456×10^2 and for 0.0123456 is 1.23456×10^{-2} . Python uses a special syntax to write scientific notation numbers. For example, 1.23456×10^2 is written as `1.23456E2` or `1.23456E+2`, and 1.23456×10^{-2} as `1.23456E-2`. The letter `E` (or `e`) represents an exponent and can be in either lowercase or uppercase.



Note

why called floating point?

The float type is used to represent numbers with a decimal point. Why are they called *floating-point numbers*? These numbers are stored in scientific notation in memory. When a number such as `50.534` is converted into scientific notation, such as `5.0534E+1`, its decimal point is moved (floated) to a new position.



Caution

what is overflow?

When a variable is assigned a value that is too large (*in size*) to be stored in memory, it causes *overflow*. For example, executing the following statement causes overflow.



```
>>> 245.0 ** 1000
OverflowError: 'Result too large'
>>>
```

what is underflow?

When a floating-point number is too small (that is, too close to zero), it causes *underflow* and Python approximates it to zero. Therefore, usually you don't need to be concerned with underflow.



MyProgrammingLab™

2.11 What are the results of the following expressions?

Expression	Result
42 / 5	_____
42 // 5	_____
42 % 5	_____
40 % 5	_____
1 % 2	_____
2 % 1	_____
45 + 4 * 4 - 2	_____
45 + 43 % 5 * (23 * 3 % 2)	_____
5 ** 2	_____
5.1 ** 2	_____

- 2.12** If today is Tuesday, what day of the week will it be in 100 days?
- 2.13** What is the result of `25 / 4`? How would you rewrite the expression if you wished the result to be an integer number?

2.9 Evaluating Expressions and Operator Precedence

Python expressions are evaluated in the same way as arithmetic expressions.

Writing a numeric expression in Python involves a straightforward translation of an arithmetic expression using operators. For example, the arithmetic expression

$$\frac{3 + 4x}{5} - \frac{10(y - 5)(a + b + c)}{x} + 9\left(\frac{4}{x} + \frac{9 + x}{y}\right)$$

can be translated into a Python expression as:

```
(3 + 4 * x) / 5 - 10 * (y - 5) * (a + b + c) / x +
9 * (4 / x + (9 + x) / y)
```

Though Python has its own way to evaluate an expression behind the scene, the results of a Python expression and its corresponding arithmetic expression are the same. Therefore, you can safely apply the arithmetic rules for evaluating a Python expression. Operators contained within pairs of parentheses are evaluated first. Parentheses can be nested, in which case the expression in the inner parentheses is evaluated first. When more than one operator is used in an expression, the following operator precedence rule is used to determine the order of evaluation.

evaluate an expression

operator precedence rule

- Exponentiation (`**`) is applied first.
- Multiplication (`*`), float division (`/`), integer division (`//`), and remainder operators (`%`) are applied next. If an expression contains several multiplication, division, and remainder operators, they are applied from left to right.
- Addition (`+`) and subtraction (`-`) operators are applied last. If an expression contains several addition and subtraction operators, they are applied from left to right.

Here is an example of how an expression is evaluated:

```

3 + 4 * 4 + 5 * (4 + 3) - 1
                        ↑
                        (1) inside parentheses first
3 + 4 * 4 + 5 * 7 - 1
                        ↑
                        (2) multiplication
3 + 16 + 5 * 7 - 1
                        ↑
                        (3) multiplication
3 + 16 + 35 - 1
                        ↑
                        (4) addition
19 + 35 - 1
                        ↑
                        (5) addition
54 - 1
                        ↑
                        (6) subtraction
53

```





2.14 How would you write the following arithmetic expression in Python?

$$\frac{4}{3(r + 34)} - 9(a + bc) + \frac{3 + d(2 + a)}{a + bd}$$

2.15 Suppose `m` and `r` are integers. Write a Python expression for `mr2`.

2.10 Augmented Assignment Operators



The operators `+`, `-`, `*`, `/`, `//`, `%`, and `**` can be combined with the assignment operator (`=`) to form augmented assignment operators.

Very often the current value of a variable is used, modified, and then reassigned back to the same variable. For example, the following statement increases the variable `count` by `1`:

```
count = count + 1
```

augmented assignment
compound assignment

Python allows you to combine assignment and addition operators using an augmented (or compound) assignment operator. For instance, the preceding statement can be written as:

```
count += 1
```

addition assignment operator

The `+=` operator is called the *addition assignment operator*. All augmented assignment operators are shown in Table 2.2.

TABLE 2.2 Augmented Assignment Operators

Operator	Name	Example	Equivalent
<code>+=</code>	Addition assignment	<code>i += 8</code>	<code>i = i + 8</code>
<code>-=</code>	Subtraction assignment	<code>i -= 8</code>	<code>i = i - 8</code>
<code>*=</code>	Multiplication assignment	<code>i *= 8</code>	<code>i = i * 8</code>
<code>/=</code>	Float division assignment	<code>i /= 8</code>	<code>i = i / 8</code>
<code>//=</code>	Integer division assignment	<code>i //= 8</code>	<code>i = i // 8</code>
<code>%=</code>	Remainder assignment	<code>i %= 8</code>	<code>i = i % 8</code>
<code>**=</code>	Exponent assignment	<code>i **= 8</code>	<code>i = i ** 8</code>



Caution

There are no spaces in the augmented assignment operators. For example, `+=` should be `+=`.



2.16 Assume that `a = 1`, and that each expression is independent. What are the results of the following expressions?

```
a += 4
a -= 4
a *= 4
a /= 4
a //= 4
a %= 4
a = 56 * a + 6
```

2.11 Type Conversions and Rounding

If one of the operands for the numeric operators is a float value, the result will be a float value.



Can you perform binary operations with two operands of different types? Yes. If an integer and a float are involved in a binary operation, Python automatically converts the integer to a float value. This is called *type conversion*. So, `3 * 4.5` is the same as `3.0 * 4.5`.

type conversion

Sometimes, it is desirable to obtain the integer part of a fractional number. You can use the `int(value)` function to return the integer part of a float value. For example,

int function

```
>>> value = 5.6
>>> int(value)
5
>>>
```



Note that the fractional part of the number is truncated, not rounded up.

You can also use the `round` function to round a number to the nearest whole value. For example,

round function

```
>>> value = 5.6
>>> round(value)
6
>>>
```



We will discuss the `round` function more in Chapter 3.



Note

The functions `int` and `round` do not change the variable being converted. For example, `value` is not changed after invoking the function in the following code:

```
>>> value = 5.6
>>> round(value)
6
>>> value
5.6
>>>
```



Note

The `int` function can also be used to convert an integer string into an integer. For example, `int("34")` returns `34`. So you can use the `eval` or `int` function to convert a string into an integer. Which one is better? The `int` function performs a simple conversion. It does not work for a non-integer string. For example, `int("3.4")` will cause an error. The `eval` function does more than a simple conversion. It can be used to evaluate an expression. For example, `eval("3 + 4")` returns `7`. However, there is a subtle “gotcha” for using the `eval` function. The `eval` function will produce an error for a numeric string that contains leading zeros. In contrast, the `int` function works fine for this case. For example, `eval("003")` causes an error, but `int("003")` returns `3`.

int vs. eval functions

Listing 2.6 shows a program that displays the sales tax with two digits after the decimal point.

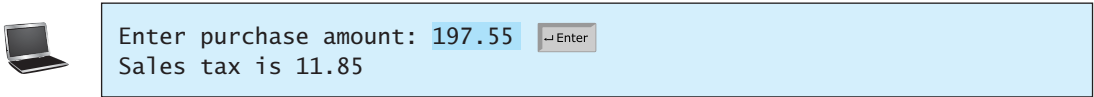
LISTING 2.6 SalesTax.py

input purchaseAmount

compute tax

format

```
1 # Prompt the user for input
2 purchaseAmount = eval(input("Enter purchase amount: "))
3
4 # Compute sales tax
5 tax = purchaseAmount * 0.06
6
7 # Display tax amount with two digits after decimal point
8 print("Sales tax is", int(tax * 100) / 100.0)
```



line#	purchaseAmount	tax	output
2	197.55		
5		11.853	
8			11.85

The value of the variable `purchaseAmount` is `197.55` (line 2). The sales tax is `6%` of the purchase, so the `tax` is evaluated as `11.853` (line 5). Note that

```
tax * 100 is 1185.3
int(tax * 100) is 1185
int(tax * 100) / 100.0 is 11.85
```

So, the statement in line 8 displays the tax `11.85` with two digits after the decimal point.



MyProgrammingLab™

- 2.17 What does a conversion from a float to an integer do with the fractional part of the float value? Does the `int(value)` function change the variable `value`?
- 2.18 Are the following statements correct? If so, show their printout.
value = 4.6
print(int(value))
print(round(value))
print(eval("4 * 5 + 2"))
print(int("04"))
print(int("4.5"))
print(eval("04"))

2.12 Case Study: Displaying the Current Time



You can use the `time()` function in the `time` module to obtain the current system time.

The problem is to develop a program that displays the current time in Greenwich Mean Time (GMT) in the format hour:minute:second, such as 13:19:18.

The `time()` function in the `time` module returns the current time in seconds with millisecond precision elapsed since the time `00:00:00` on January 1, 1970 GMT, as shown in Figure 2.1. This time is known as the *UNIX epoch*. The epoch is the point when time starts. `1970` was the year when the UNIX operating system was formally introduced. For

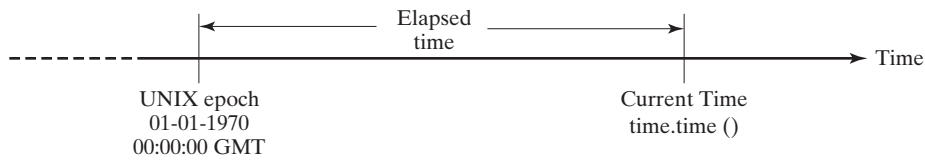


FIGURE 2.1 The `time.time()` function returns the seconds with millisecond precision since the UNIX epoch.

example, `time.time()` returns `1285543663.205`, which means `1285543663` seconds and `205` milliseconds. `time.time()`

You can use this function to obtain the current time, and then compute the current second, minute, and hour as follows.

1. Obtain the current time (since midnight, January 1, 1970) by invoking `time.time()` (for example, `1203183068.328`).
2. Obtain the total seconds `totalSeconds` using the `int` function (`int(1203183068.328)` = `1203183068`).
3. Compute the current second from `totalSeconds % 60` (`1203183068` seconds $\% 60$ = `8`, which is the current second).
4. Obtain the total minutes `totalMinutes` by dividing `totalSeconds` by `60` (`1203183068` seconds $\div 60$ = `20053051` minutes).
5. Compute the current minute from `totalMinutes % 60` (`20053051` minutes $\% 60$ = `31`, which is the current minute).
6. Obtain the total hours `totalHours` by dividing `totalMinutes` by `60` (`20053051` minutes $\div 60$ = `334217` hours).
7. Compute the current hour from `totalHours % 24` (`334217` hours $\% 24$ = `17`, which is the current hour).

Listing 2.7 gives the complete program.

LISTING 2.7 ShowCurrentTime.py

```

1  import time                                     import time module
2
3  currentTime = time.time() # Get current time      currentTime
4
5  # Obtain the total seconds since midnight, Jan 1, 1970
6  totalSeconds = int(currentTime)                  totalSeconds
7
8  # Get the current second
9  currentSecond = totalSeconds % 60                currentSecond
10
11 # Obtain the total minutes
12 totalMinutes = totalSeconds // 60                 totalMinutes
13
14 # Compute the current minute in the hour
15 currentMinute = totalMinutes % 60                 currentMinute
16
17 # Obtain the total hours
18 totalHours = totalMinutes // 60                   totalHours
19
20 # Compute the current hour
21 currentHour = totalHours % 24                     currentHour

```

display output

```
22
23 # Display results
24 print("Current time is", currentHour, ":",
25       currentMinute, ":", currentSecond, "GMT")
```



Current time is 17:31:8 GMT



	line#	3	6	9	12	15	18	21
variables								
currentTime		1203183068.328						
totalSeconds			1203183068					
currentSecond				8				
totalMinutes					20053051			
currentMinute						31		
totalHours							334217	
currentHour								17

Line 3 invokes `time.time()` to return the current time in seconds as a float value with millisecond precision. The seconds, minutes, and hours are extracted from the current time using the `//` and `%` operators (lines 6–21).

In the sample run, a single digit `8` is displayed for the second. The desirable output would be `08`. This can be fixed by using a function that formats a single digit with a prefix `0` (see Exercise 6.48).

Check Point

MyProgrammingLab™

- 2.19 What is the UNIX epoch?
- 2.20 What does `time.time()` return?
- 2.21 How do you obtain the seconds from the returned value for `time.time()`?

2.13 Software Development Process



The software development life cycle is a multistage process that includes requirements specification, analysis, design, implementation, testing, deployment, and maintenance.

requirements specification

Developing a software product is an engineering process. Software products, no matter how large or how small, have the same life cycle: requirements specification, system analysis, system design, implementation, testing, deployment, and maintenance, as shown in Figure 2.2.

Requirements specification is a formal process that seeks to understand the problem that the software will address and to document in detail what the software system needs to do. This phase involves close interaction between users and developers. Most of the examples in

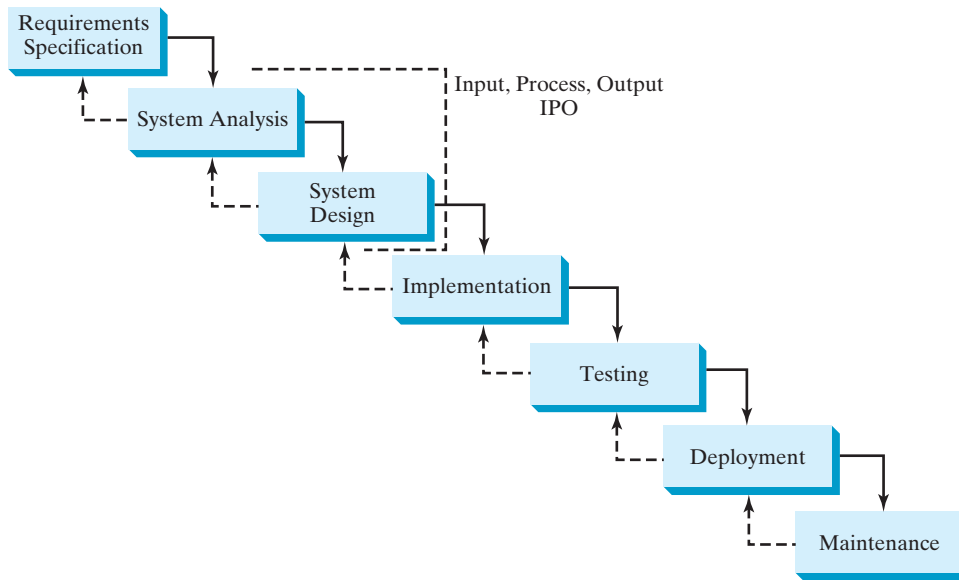


FIGURE 2.2 At any stage of the software development life cycle, it may be necessary to go back to a previous stage to correct errors or deal with other issues that might prevent the software from functioning as expected.

this book are simple, and their requirements are clearly stated. In the real world, however, problems are not always well defined. Developers need to work closely with their customers (the individuals or organizations that will use the software) and study the problem carefully to identify what the software needs to do.

System analysis seeks to analyze the data flow and to identify the system's input and output. When you do analysis, it helps to identify what the output is first, and then figure out what input data you need in order to produce the output.

System design is to design a process for obtaining the output from the input. This phase involves the use of many levels of abstraction to decompose the problem into manageable components, and design strategies for implementing each component. You can view a component as a subsystem that performs a specific function of the system. The essence of system analysis and design is input, process, and output (IPO).

Implementation involves translating the system design into programs. Separate programs are written for each component and then integrated to work together. This phase requires the use of a programming language such as Python. The implementation involves coding, self testing, and debugging (that is, finding errors, called *bugs*, in the code).

Testing ensures that the code meets the requirements specification and weeds out bugs. An independent team of software engineers not involved in the design and implementation of the product usually conducts such testing.

Deployment makes the software available for use. Depending on the type of the software, it may be installed on each user's machine or installed on a server accessible on the Internet.

Maintenance is concerned with updating and improving the product. A software product must continue to perform and improve in an ever-evolving environment. This requires periodic upgrades of the product to fix newly discovered bugs and incorporate changes.

To see the software development process in action, we will now create a program that computes loan payments. The loan can be a car loan, a student loan, or a home mortgage loan. For an introductory programming course, we focus on requirements specification, analysis, design, implementation, and testing.

system analysis

system design

IPO

implementation

testing

deployment

maintenance

Stage 1: Requirements Specification

The program must satisfy the following requirements:

- It must let the user enter the interest rate, the loan amount, and the number of years for which payments will be made.
- It must compute and display the monthly payment and total payment amounts.

Stage 2: System Analysis

The output is the monthly payment and total payment, which can be obtained using the following formula:

$$\text{monthlyPayment} = \frac{\text{loanAmount} \times \text{monthlyInterestRate}}{1 - \frac{1}{(1 + \text{monthlyInterestRate})^{\text{numberOfYears} \times 12}}}$$

$$\text{totalPayment} = \text{monthlyPayment} \times \text{numberOfYears} \times 12$$

So, the input needed for the program is the annual interest rate, the length of the loan in years, and the loan amount.

**Note**

The requirements specification says that the user must enter the interest rate, the loan amount, and the number of years for which payments will be made. During analysis, however, it is possible that you may discover that input is not sufficient or that some values are unnecessary for the output. If this happens, you can go back to modify the requirements specification.

**Note**

In the real world, you will work with customers from all walks of life. You may develop software for chemists, physicists, engineers, economists, and psychologists and of course, you will not have (or need) the complete knowledge of all these fields. Therefore, you don't have to know how the mathematical formulas are derived. Nonetheless, given the annual interest rate, number of years, and loan amount, you can use this formula to compute the monthly payment. You will, however, need to communicate with the customers and understand how the mathematic model works for the system.

Stage 3: System Design

During system design, you identify the steps in the program:

- Step 1. Prompt the user to enter the annual interest rate, number of years, and loan amount.
- Step 2. The input for the annual interest rate is a number in percent format, such as 4.5%. The program needs to convert it into a decimal by dividing it by 100. To obtain the monthly interest rate from the annual interest rate, divide it by 12, since a year has 12 months. So to obtain the monthly interest rate in decimal format, you need to divide the annual interest rate in percentage by 1200. For example, if the annual interest rate is 4.5%, then the monthly interest rate is $4.5/1200 = 0.00375$.
- Step 3. Compute the monthly payment using the formula given in Stage 2.
- Step 4. Compute the total payment, which is the monthly payment multiplied by 12 and multiplied by the number of years.
- Step 5. Display the monthly payment and total payment.

Stage 4: Implementation

Implementation is also known as *coding* (writing the code). In the formula, you have to compute $(1 + \text{monthlyInterestRate})^{\text{numberOfYears} \times 12}$. You can use the exponentiation operator to write it as

$$(1 + \text{monthlyInterestRate}) ** (\text{numberOfYears} * 12)$$

Listing 2.8 gives the complete program.

LISTING 2.8 ComputeLoan.py

```
1 # Enter annual interest rate as a percentage, e.g., 7.25
2 annualInterestRate = eval(input(                                enter interest rate
3     "Enter annual interest rate, e.g., 7.25: "))
4 monthlyInterestRate = annualInterestRate / 1200                obtain monthly interest rate
5
6 # Enter number of years
7 numberOfYears = eval(input(                                     enter years
8     "Enter number of years as an integer, e.g., 5: "))
9
10 # Enter loan amount
11 loanAmount = eval(input("Enter loan amount, e.g., 120000.95: ")) enter loan amount
12
13 # Calculate payment
14 monthlyPayment = loanAmount * monthlyInterestRate / (1         monthlyPayment
15     - 1 / (1 + monthlyInterestRate) ** (numberOfYears * 12))
16 totalPayment = monthlyPayment * numberOfYears * 12             totalPayment
17
18 # Display results
19 print("The monthly payment is", int(monthlyPayment * 100) / 100) display result
20 print("The total payment is", int(totalPayment * 100) / 100)
```

Enter annual interest rate, e.g., 7.25: 5.75

Enter number of years as an integer, e.g., 5: 15

Enter loan amount, e.g., 120000.95: 250000

The monthly payment is 2076.02

The total payment is 373684.53



	line#	2	4	7	11	14	16
variables							
annualInterestRate		5.75					
monthlyInterestRate			0.00479166666666				
numberOfYears				15			
loanAmount					250000		
monthlyPayment						2076.0252175	
totalPayment							373684.539



format numbers

Line 2 reads the annual interest rate, which is converted into the monthly interest rate in line 4.

The formula for computing the monthly payment is translated into Python code in lines 14–15.

The variable `monthlyPayment` is `2076.0252175` (line 14). Note that

```
int(monthlyPayment * 100) is 207602.52175
int(monthlyPayment * 100) / 100.0 is 2076.02
```

So, the statement in line 19 displays the tax `2076.02` with two digits after the decimal point.

Stage 5: Testing

After the program is implemented, test it with some sample input data and verify whether the output is correct. Some of the problems may involve many cases as you will see in later chapters. For this type of problems, you need to design test data that cover all cases.

incremental development and testing



Tip

The system design phase in this example identified several steps. It is a good approach to develop and test these steps incrementally by adding them one at a time. This process makes it much easier to pinpoint problems and debug the program.

2.14 Case Study: Computing Distances



This section presents two programs that compute and display the distance between two points.

Given two points, the formula for computing the distance is $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. You can use `a ** 0.5` to compute $\sqrt{a}$. The program in Listing 2.9 prompts the user to enter two points and computes the distance between them.

LISTING 2.9 ComputeDistance.py

enter x1, y1

enter x2, y2

compute distance

```
1 # Enter the first point with two float values
2 x1, y1 = eval(input("Enter x1 and y1 for Point 1: "))
3
4 # Enter the second point with two float values
5 x2, y2 = eval(input("Enter x2 and y2 for Point 2: "))
6
7 # Compute the distance
8 distance = ((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2)) ** 0.5
9
10 print("The distance between the two points is", distance)
```



```
Enter x1 and y1 for Point 1: 1.5, -3.4 Enter
Enter x2 and y2 for Point 2: 4, 5 Enter
The distance between the two points is 8.764131445842194
```

The program prompts the user to enter the coordinates of the first point (line 2) and the second point (line 5). It then computes the distance between them (line 8) and displays it (line 10).

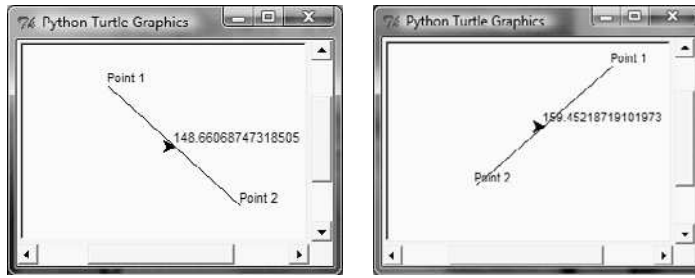


FIGURE 2.3 The program displays a line and its length.

Figure 2.3 illustrates the program in Listing 2.10. This program

1. Prompts the user to enter two points.
2. Computes the distance between the points.
3. Uses Turtle graphics to display the line that connects the two points.
4. Displays the length of the line at the center of the line.

Listing 2.10 gives the program.

LISTING 2.10 ComputeDistanceGraphics.py

1	<code>import turtle</code>	import turtle
2		
3	<code># Prompt the user for inputting two points</code>	
4	<code>x1, y1 = eval(input("Enter x1 and y1 for point 1: "))</code>	enter x1, y1
5	<code>x2, y2 = eval(input("Enter x2 and y2 for point 2: "))</code>	enter x2, y2
6		
7	<code># Compute the distance</code>	
8	<code>distance = ((x1 - x2) ** 2 + (y1 - y2) ** 2) ** 0.5</code>	compute distance
9		
10	<code># Display two points and the connecting line</code>	
11	<code>turtle.penup()</code>	
12	<code>turtle.goto(x1, y1) # Move to (x1, y1)</code>	move to point 1
13	<code>turtle.pendown()</code>	
14	<code>turtle.write("Point 1")</code>	display point 1
15	<code>turtle.goto(x2, y2) # Draw a line to (x2, y2)</code>	draw a line
16	<code>turtle.write("Point 2")</code>	display point 2
17		
18	<code># Move to the center point of the line</code>	
19	<code>turtle.penup()</code>	
20	<code>turtle.goto((x1 + x2) / 2, (y1 + y2) / 2)</code>	move to center
21	<code>turtle.write(distance)</code>	display distance
22		
23	<code>turtle.done()</code>	pause

Enter x1 and y1 for Point 1: -50, 34 Enter
 Enter x2 and y2 for Point 2: 49, -85 Enter



The program prompts the user to enter the value for two points `(x1, y1)` and `(x2, y2)`, and computes their distance (lines 4–8). It then moves to `(x1, y1)` (line 12), displays the

text **Point 1** (line 14), draws a line from **(x1, y1)** to **(x2, y2)** (line 15), and displays the text **Point 2** (line 16). Finally, it moves to the center of the line (line 20) and displays the distance (line 21).

KEY TERMS

algorithm	32	line continuation symbol	35
assignment operator (=)	37	literal	40
augmented assignment	44	operands	40
camelCase	36	operators	40
compound assignment	44	pseudocode	32
data type	33	reserved word	36
expression	37	scope of a variable	37
floating-point numbers	40	simultaneous assignment	38
identifiers	36	system analysis	49
incremental development and testing	52	system design	49
input, process, output (IPO)	35	type conversion	45
keyword	36	variable	32

CHAPTER SUMMARY

1. You can get input using the **input** function and convert a string into a numerical value using the **eval** function.
2. *Identifiers* are the names used for elements in a program.
3. An identifier is a sequence of characters of any length that consists of letters, digits, underscores (**_**), and asterisk signs (*****). An identifier must start with a letter or an underscore; it cannot start with a digit. An identifier cannot be a keyword.
4. *Variables* are used to store data in a program.
5. The equal sign (**=**) is used as the *assignment operator*.
6. A variable must be assigned a value before it can be used.
7. There are two types of numeric data in Python: integers and real numbers. Integer types (*int* for short) are for whole numbers, and real types (also called *float*) are for numbers with a decimal point.
8. Python provides *assignment operators* that perform numeric operations: **+** (addition), **-** (subtraction), ***** (multiplication), **/** (division), **//** (integer division), **%** (remainder), and ****** (exponent).
9. The numeric operators in a Python expression are applied the same way as in an arithmetic expression.
10. Python provides augmented assignment operators: **+=** (addition assignment), **-=** (subtraction assignment), ***=** (multiplication assignment), **/=** (float division assignment), **//=** (integer division assignment), and **%=** (remainder assignment). These operators combine the **+**, **-**, *****, **/**, **//**, and **%** operators and the assignment operator into one augmented operators.

11. When evaluating an expression with values of an int type and a float type, Python automatically converts the int value to a float type value.
12. You can convert a float to an int using the `int(value)` function.
13. *System analysis* seeks to analyze the data flow and to identify the system's input and output.
14. *System design* is the stage when programmers develop a process for obtaining the output from the input.
15. The essence of system analysis and design is input, process, and output. This is called *IPO*.

TEST QUESTIONS

Do test questions for this chapter online at www.cs.armstrong.edu/liang/py/test.html.

PROGRAMMING EXERCISES

MyProgrammingLab™



Pedagogical Note

Instructors may ask you to document analysis and design for selected exercises. You should use your own words to analyze the problem, including the input, output, and what needs to be computed, and describe how to solve the problem in pseudocode.

document analysis and design



Debugging Tip

Python usually gives a reason for a syntax error. If you don't know how to correct it, compare your program closely, character by character, with similar examples in the text.

learn from examples

Sections 2.2–2.10

- 2.1** (*Convert Celsius to Fahrenheit*) Write a program that reads a Celsius degree from the console and converts it to Fahrenheit and displays the result. The formula for the conversion is as follows:

$$\text{fahrenheit} = (9 / 5) * \text{celsius} + 32$$

Here is a sample run of the program:

```
Enter a degree in Celsius: 43
43 Celsius is 109.4 Fahrenheit
```

Enter



- 2.2** (*Compute the volume of a cylinder*) Write a program that reads in the radius and length of a cylinder and computes the area and volume using the following formulas:

```
area = radius * radius * π
volume = area * length
```

Here is a sample run:



```
Enter the radius and length of a cylinder: 5.5, 12 ↵ Enter
The area is 95.0331
The volume is 1140.4
```

- 2.3** (*Convert feet into meters*) Write a program that reads a number in feet, converts it to meters, and displays the result. One foot is **0.305** meters. Here is a sample run:



```
Enter a value for feet: 16.5 ↵ Enter
16.5 feet is 5.0325 meters
```

- 2.4** (*Convert pounds into kilograms*) Write a program that converts pounds into kilograms. The program prompts the user to enter a value in pounds, converts it to kilograms, and displays the result. One pound is **0.454** kilograms. Here is a sample run:



```
Enter a value in pounds: 55.5 ↵ Enter
55.5 pounds is 25.197 kilograms
```

- *2.5** (*Financial application: calculate tips*) Write a program that reads the subtotal and the gratuity rate and computes the gratuity and total. For example, if the user enters **10** for the subtotal and **15%** for the gratuity rate, the program displays **1.5** as the gratuity and **11.5** as the total. Here is a sample run:



```
Enter the subtotal and a gratuity rate: 15.69, 15 ↵ Enter
The gratuity is 2.35 and the total is 18.04
```

- **2.6** (*Sum the digits in an integer*) Write a program that reads an integer between **0** and **1000** and adds all the digits in the integer. For example, if an integer is **932**, the sum of all its digits is **14**. (Hint: Use the **%** operator to extract digits, and use the **//** operator to remove the extracted digit. For instance, **932 % 10 = 2** and **932 // 10 = 93**.) Here is a sample run:



```
Enter a number between 0 and 1000: 999 ↵ Enter
The sum of the digits is 27
```

- **2.7** (*Find the number of years and days*) Write a program that prompts the user to enter the minutes (e.g., 1 billion), and displays the number of years and days for the minutes. For simplicity, assume a year has **365** days. Here is a sample run:



```
Enter the number of minutes: 1000000000 ↵ Enter
1000000000 minutes is approximately 1902 years and 214 days
```

- 2.8** (*Science: calculate energy*) Write a program that calculates the energy needed to heat water from an initial temperature to a final temperature. Your program should

prompt the user to enter the amount of water in kilograms and the initial and final temperatures of the water. The formula to compute the energy is

$$Q = M * (\text{finalTemperature} - \text{initialTemperature}) * 4184$$

where **M** is the weight of water in kilograms, temperatures are in degrees Celsius, and energy **Q** is measured in joules. Here is a sample run:

```
Enter the amount of water in kilograms: 55.5 Enter
Enter the initial temperature: 3.5 Enter
Enter the final temperature: 10.5 Enter
The energy needed is 1625484.0
```



- *2.9** (*Science: wind-chill temperature*) How cold is it outside? The temperature alone is not enough to provide the answer. Other factors including wind speed, relative humidity, and sunshine play important roles in determining coldness outside. In 2001, the National Weather Service (NWS) implemented the new wind-chill temperature to measure the coldness using temperature and wind speed. The formula is given as follows:

$$t_{wc} = 35.74 + 0.6215t_a - 35.75v^{0.16} + 0.4275t_av^{0.16}$$

where t_a is the outside temperature measured in degrees Fahrenheit and v is the speed measured in miles per hour. t_{wc} is the wind-chill temperature. The formula cannot be used for wind speeds below 2 mph or for temperatures below -58°F or above 41°F .

Write a program that prompts the user to enter a temperature between -58°F and 41°F and a wind speed greater than or equal to 2 and displays the wind-chill temperature. Here is a sample run:

```
Enter the temperature in Fahrenheit between -58 and 41: 5.3 Enter
Enter the wind speed in miles per hour: 6 Enter
The wind chill index is -5.56707
```



- *2.10** (*Physics: find runway length*) Given an airplane's acceleration a and take-off speed v , you can compute the minimum runway length needed for an airplane to take off using the following formula:

$$\text{length} = \frac{v^2}{2a}$$

Write a program that prompts the user to enter v in meters/second (m/s) and the acceleration a in meters/second squared (m/s^2), and displays the minimum runway length. Here is a sample run:

```
Enter speed and acceleration: 60, 3.5 Enter
The minimum runway length for this airplane is 514.286 meters
```



- *2.11** (*Financial application: investment amount*) Suppose you want to deposit a certain amount of money into a savings account with a fixed annual interest rate. What amount do you need to deposit in order to have \$5,000 in the account after three years? The initial deposit amount can be obtained using the following formula:

$$\text{initialDepositAmount} = \frac{\text{finalAccountValue}}{(1 + \text{monthlyInterestRate})^{\text{numberOfMonths}}}$$

Write a program that prompts the user to enter final account value, annual interest rate in percent, and the number of years, and displays the initial deposit amount. Here is a sample run:



```
Enter final account value: 1000 ↵ Enter
Enter annual interest rate in percent: 4.25 ↵ Enter
Enter number of years: 5 ↵ Enter
Initial deposit value is 808.8639197424636
```

- 2.12** (*Print a table*) Write a program that displays the following table:

a	b	a ** b
1	2	1
2	3	8
3	4	81
4	5	1024
5	6	15625

- *2.13** (*Split digits*) Write a program that prompts the user to enter a four-digit integer and displays the number in reverse order. Here is a sample run:



```
Enter an integer: 3125 ↵ Enter
3
1
2
5
```

- *2.14** (*Geometry: area of a triangle*) Write a program that prompts the user to enter the three points (x1, y1), (x2, y2), and (x3, y3) of a triangle and displays its area. The formula for computing the area of a triangle is

$$s = (\text{side1} + \text{side2} + \text{side3}) / 2$$

$$\text{area} = \sqrt{s(s - \text{side1})(s - \text{side2})(s - \text{side3})}$$

Here is a sample run:



```
Enter three points for a triangle: 1.5, -3.4, 4.6, 5,
9.5, -3.4 ↵ Enter
The area of the triangle is 33.6
```

- 2.15** (*Geometry: area of a hexagon*) Write a program that prompts the user to enter the side of a hexagon and displays its area. The formula for computing the area of a hexagon is $Area = \frac{3\sqrt{3}}{2}s^2$, where s is the length of a side. Here is a sample run:

```
Enter the side: 5.5 
The area of the hexagon is 78.5895
```



- 2.16** (*Physics: acceleration*) Average acceleration is defined as the change of velocity divided by the time taken to make the change, as shown in the following formula:

$$a = \frac{v_1 - v_0}{t}$$

Write a program that prompts the user to enter the starting velocity v_0 in meters/second, the ending velocity v_1 in meters/second, and the time span t in seconds, and displays the average acceleration. Here is a sample run:

```
Enter v0, v1, and t: 5.5, 50.9, 4.5 
The average acceleration is 10.0889
```



- *2.17** (*Health application: compute BMI*) Body mass index (BMI) is a measure of health based on weight. It can be calculated by taking your weight in kilograms and dividing it by the square of your height in meters. Write a program that prompts the user to enter a weight in pounds and height in inches and displays the BMI. Note that one pound is **0.45359237** kilograms and one inch is **0.0254** meters. Here is a sample run:

```
Enter weight in pounds: 95.5 
Enter height in inches: 50 
BMI is 26.8573
```



Sections 2.11–2.13

- *2.18** (*Current time*) Listing 2.7, ShowCurrentTime.py, gives a program that displays the current time in GMT. Revise the program so that it prompts the user to enter the time zone in hours away from (offset to) GMT and displays the time in the specified time zone. Here is a sample run:

```
Enter the time zone offset to GMT: -5 
The current time is 4:50:34
```



- *2.19** (*Financial application: calculate future investment value*) Write a program that reads in an investment amount, the annual interest rate, and the number of years, and displays the future investment value using the following formula:

$$\text{futureInvestmentValue} = \text{investmentAmount} \times (1 + \text{monthlyInterestRate})^{\text{numberOfMonths}}$$

For example, if you enter the amount **1000**, an annual interest rate of **4.25%**, and the number of years as **1**, the future investment value is **1043.33**. Here is a sample run:



```
Enter investment amount: 1000 [Enter]
Enter annual interest rate: 4.25 [Enter]
Enter number of years: 1 [Enter]
Accumulated value is 1043.33
```

- *2.20** (*Financial application: calculate interest*) If you know the balance and the annual percentage interest rate, you can compute the interest on the next monthly payment using the following formula:

$$\text{interest} = \text{balance} * (\text{annualInterestRate} / 1200)$$

Write a program that reads the balance and the annual percentage interest rate and displays the interest for the next month. Here is a sample run:



```
Enter balance and interest rate (e.g., 3 for 3%): 1000, 3.5 [Enter]
The interest is 2.91667
```

- **2.21** (*Financial application: compound value*) Suppose you save **\$100** each month into a savings account with an annual interest rate of 5%. Therefore, the monthly interest rate is $0.05/12 = 0.00417$. After the first month, the value in the account becomes

$$100 * (1 + 0.00417) = 100.417$$

After the second month, the value in the account becomes

$$(100 + 100.417) * (1 + 0.00417) = 201.252$$

After the third month, the value in the account becomes

$$(100 + 201.252) * (1 + 0.00417) = 302.507$$

and so on.

Write a program that prompts the user to enter a monthly saving amount and displays the account value after the sixth month. Here is a sample run of the program:

Enter the monthly saving amount: 100
 After the sixth month, the account value is 608.81



- 2.22** (*Population projection*) Rewrite Exercise 1.11 to prompt the user to enter the number of years and displays the population after that many years. Here is a sample run of the program:

Enter the number of years: 5
 The population in 5 years is 325932970

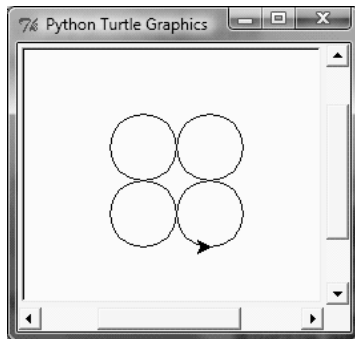


Section 2.14

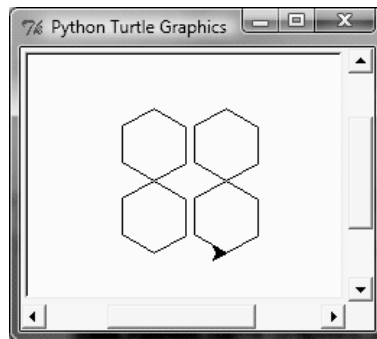
- 2.23** (*Turtle: draw four circles*) Write a program that prompts the user to enter the radius and draws four circles in the center of the screen, as shown in Figure 2.4a.

- 2.24** (*Turtle: draw four hexagons*) Write a program that draws four hexagons in the center of the screen, as shown in Figure 2.4b.

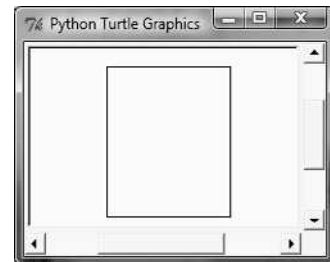
- **2.25** (*Turtle: draw a rectangle*) Write a program that prompts the user to enter the center of a rectangle, width, and height, and displays the rectangle, as shown in Figure 2.4c.



(a)



(b)



(c)

FIGURE 2.4 Four circles are drawn in (a), four hexagons are drawn in (b), and a rectangle is drawn in (c).

- **2.26** (*Turtle: draw a circle*) Write a program that prompts the user to enter the center and radius of a circle, and then displays the circle and its area, as shown in Figure 2.5.

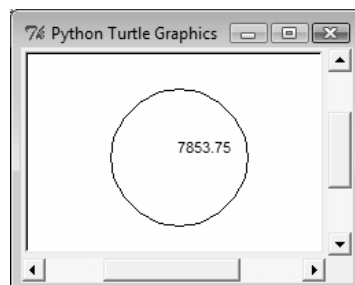


FIGURE 2.5 A circle and its area are displayed.

This page intentionally left blank

MATHEMATICAL FUNCTIONS, STRINGS, AND OBJECTS

Objectives

- To solve mathematics problems by using the functions in the **math** module (§3.2).
- To represent and process strings and characters (§§3.3–3.4).
- To encode characters using ASCII and Unicode (§§3.3.1–3.3.2).
- To use the **ord** function to obtain a numerical code for a character and the **chr** function to convert a numerical code to a character (§3.3.3).
- To represent special characters using the escape sequence (§3.3.4).
- To invoke the **print** function with the **end** argument (§3.3.5).
- To convert numbers to a string using the **str** function (§3.3.6).
- To use the **+** operator to concatenate strings (§3.3.7).
- To read strings from the keyboard (§3.3.8).
- To introduce objects and methods (§3.5).
- To format numbers and strings using the **format** function (§3.6).
- To draw various shapes (§3.7).
- To draw graphics with colors and fonts (§3.8).



3.1 Introduction

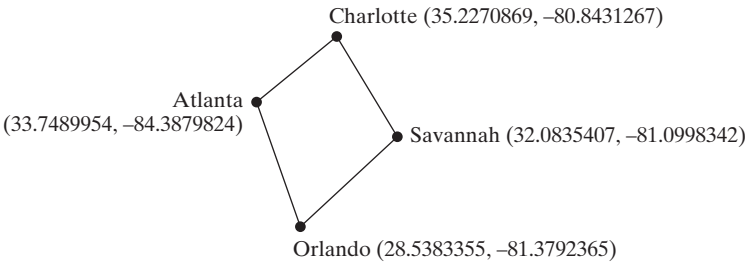


The focus of this chapter is to introduce functions, strings, and objects, and to use them to develop programs.

problem

The preceding chapter introduced fundamental programming techniques and taught you how to write simple programs to solve basic problems. This chapter introduces Python functions for performing common mathematical operations. You will learn how to create custom functions in Chapter 6.

Suppose you need to estimate the area enclosed by four cities, given the GPS locations (latitude and longitude) of these cities, as shown in the following diagram. How would you write a program to solve this problem? You will be able to write such a program after completing this chapter.



Because all data in Python are objects, it is beneficial to introduce objects early so that you can begin to use them to develop useful programs. This chapter gives a brief introduction to objects and strings; you will learn more on objects and strings in Chapters 7 and 8.

3.2 Common Python Functions



Python provides many useful functions for common programming tasks.

function

A *function* is a group of statements that performs a specific task. Python, as well as other programming languages, provides a library of functions. You have already used the functions `eval`, `input`, `print`, and `int`. These are built-in functions and they are always available in the Python interpreter. You don't have to import any modules to use these functions. Additionally, you can use the built-in functions `abs`, `max`, `min`, `pow`, and `round`, as shown in Table 3.1.

TABLE 3.1 Simple Python Built-in Functions

Function	Description	Example
<code>abs(x)</code>	Returns the absolute value for <code>x</code> .	<code>abs(-2)</code> is 2
<code>max(x1, x2, ...)</code>	Returns the largest among <code>x1</code> , <code>x2</code> , ...	<code>max(1, 5, 2)</code> is 5
<code>min(x1, x2, ...)</code>	Returns the smallest among <code>x1</code> , <code>x2</code> , ...	<code>min(1, 5, 2)</code> is 1
<code>pow(a, b)</code>	Returns <code>a^b</code> . Same as <code>a ** b</code> .	<code>pow(2, 3)</code> is 8
<code>round(x)</code>	Returns an integer nearest to <code>x</code> . If <code>x</code> is equally close to two integers, the even one is returned.	<code>round(5.4)</code> is 5 <code>round(5.5)</code> is 6 <code>round(4.5)</code> is 4
<code>round(x, n)</code>	Returns the float value rounded to <code>n</code> digits after the decimal point.	<code>round(5.466, 2)</code> is 5.47 <code>round(5.463, 2)</code> is 5.46

For example,



```
>>> abs(-3) # Returns the absolute value
3
>>> abs(-3.5) # Returns the absolute value
3.5
>>> max(2, 3, 4, 6) # Returns the maximum number
6
>>> min(2, 3, 4) # Returns the minimum number
2
>>> pow(2, 3) # Same as 2 ** 3
8
>>> pow(2.5, 3.5) # Same as 2.5 ** 3.5
24.705294220065465
>>> round(3.51) # Rounds to its nearest integer
4
>>> round(3.4) # Rounds to its nearest integer
3
>>> round(3.1456, 3) # Rounds to 3 digits after the decimal point
3.146
>>>
```

Many programs are created to solve mathematical problems. The Python `math` module provides the mathematical functions listed in Table 3.2.

Two mathematical constants, `pi` and `e`, are also defined in the `math` module. They can be accessed using `math.pi` and `math.e`. Listing 3.1 is a program that tests some math functions. Because the program uses the math functions defined in the `math` module, the `math` module is imported in line 1.

TABLE 3.2 Mathematical Functions

Function	Description	Example
<code>fabs(x)</code>	Returns the absolute value for <code>x</code> as a float.	<code>fabs(-2)</code> is 2.0
<code>ceil(x)</code>	Rounds <code>x</code> up to its nearest integer and returns that integer.	<code>ceil(2.1)</code> is 3 <code>ceil(-2.1)</code> is -2
<code>floor(x)</code>	Rounds <code>x</code> down to its nearest integer and returns that integer.	<code>floor(2.1)</code> is 2 <code>floor(-2.1)</code> is -3
<code>exp(x)</code>	Returns the exponential function of <code>x</code> (e^x).	<code>exp(1)</code> is 2.71828
<code>log(x)</code>	Returns the natural logarithm of <code>x</code> .	<code>log(2.71828)</code> is 1.0
<code>log(x, base)</code>	Returns the logarithm of <code>x</code> for the specified base.	<code>log(100, 10)</code> is 2.0
<code>sqrt(x)</code>	Returns the square root of <code>x</code> .	<code>sqrt(4.0)</code> is 2
<code>sin(x)</code>	Returns the sine of <code>x</code> . <code>x</code> represents an angle in radians.	<code>sin(3.14159 / 2)</code> is 1 <code>sin(3.14159)</code> is 0
<code>asin(x)</code>	Returns the angle in radians for the inverse of sine.	<code>asin(1.0)</code> is 1.57 <code>asin(0.5)</code> is 0.523599
<code>cos(x)</code>	Returns the cosine of <code>x</code> . <code>x</code> represents an angle in radians.	<code>cos(3.14159 / 2)</code> is 0 <code>cos(3.14159)</code> is -1
<code>acos(x)</code>	Returns the angle in radians for the inverse of cosine.	<code>acos(1.0)</code> is 0 <code>acos(0.5)</code> is 1.0472
<code>tan(x)</code>	Returns the tangent of <code>x</code> . <code>x</code> represents an angle in radians.	<code>tan(3.14159 / 4)</code> is 1 <code>tan(0.0)</code> is 0
<code>degrees(x)</code>	Converts angle <code>x</code> from radians to degrees.	<code>degrees(1.57)</code> is 90
<code>radians(x)</code>	Converts angle <code>x</code> from degrees to radians.	<code>radians(90)</code> is 1.57

LISTING 3.1 MathFunctions.py

```

import math module      1  import math # import math module to use the math functions
                        2
                        3  # Test algebraic functions
exp                      4  print("exp(1.0) =", math.exp(1))
log                     5  print("log(2.78) =", math.log(math.e))
log10                   6  print("log10(10, 10) =", math.log(10, 10))
sqrt                   7  print("sqrt(4.0) =", math.sqrt(4.0))
                        8
                        9  # Test trigonometric functions
sin                    10  print("sin(PI / 2) =", math.sin(math.pi / 2))
cos                   11  print("cos(PI / 2) =", math.cos(math.pi / 2))
tan                   12  print("tan(PI / 2) =", math.tan(math.pi / 2))
degrees               13  print("degrees(1.57) =", math.degrees(1.57))
radians               14  print("radians(90) =", math.radians(90))

```

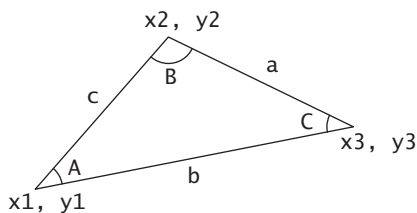


```

exp(1.0) = 2.71828182846
log(2.78) = 1.0
log10(10, 10) = 1.0
sqrt(4.0) = 2.0
sin(PI / 2) = 1.0
cos(PI / 2) = 6.12323399574e-17
tan(PI / 2) = 1.63312393532e+16
degrees(1.57) = 89.9543738355
radians(90) = 1.57079632679

```

You can use the math functions to solve many computational problems. Given the three vertices of a triangle, for example, you can compute the angles by using the following formula:



$$A = \arccos((a^2 + b^2 - c^2) / (-2 * b * c))$$

$$B = \arccos((b^2 + a^2 - c^2) / (-2 * a * c))$$

$$C = \arccos((c^2 - b^2 - a^2) / (-2 * a * b))$$

Don't be intimidated by the mathematic formula. As we discussed early in Listing 2.8, *ComuteLoan.py*, you don't have to know how the mathematical formula is derived in order to write a program for computing the loan payments. Here in this example, given the length of three sides, you can use this formula to write a program to compute the angles without having to know how the formula is derived. In order to compute the lengths of the sides, we need to know the coordinates of three corner points and compute the distances between the points.

Listing 3.2 is an example of a program that prompts the user to enter the x- and y-coordinates of the three corner points in a triangle and then displays the figure's angles.

LISTING 3.2 ComputeAngles.py

```

import math module      1  import math
                        2
enter three points      3  x1, y1, x2, y2, x3, y3 = eval(input("Enter three points: "))
                        4
compute edges           5  a = math.sqrt((x2 - x3) * (x2 - x3) + (y2 - y3) * (y2 - y3))
                        6  b = math.sqrt((x1 - x3) * (x1 - x3) + (y1 - y3) * (y1 - y3))
                        7  c = math.sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2))

```

```

8
9 A = math.degrees(math.acos((a * a - b * b - c * c) / (-2 * b * c)))
10 B = math.degrees(math.acos((b * b - a * a - c * c) / (-2 * a * c)))
11 C = math.degrees(math.acos((c * c - b * b - a * a) / (-2 * a * b)))
12
13 print("The three angles are ", round(A * 100) / 100.0,
14       round(B * 100) / 100.0, round(C * 100) / 100.0)

```

compute angles

display result

Enter three points: 1, 1, 6.5, 1, 6.5, 2.5
 The three angles are 15.26 90.0 74.74

Enter



The program prompts the user to enter three points (line 3). This prompting message is not clear. You should give the user explicit instructions on how to enter these points as follows:

```
input("Enter six coordinates of three points separated by commas\
like x1, y1, x2, y2, x3, y3: ")
```

The program computes the distances between the points (lines 5–7), and applies the formula to compute the angles (lines 9–11). The angles are rounded to display up to two digits after the decimal point (lines 13–14).

3.1 Evaluate the following functions:

- | | |
|--|---|
| (a) <code>math.sqrt(4)</code> | (j) <code>math.floor(-2.5)</code> |
| (b) <code>math.sin(2 * math.pi)</code> | (k) <code>round(3.5)</code> |
| (c) <code>math.cos(2 * math.pi)</code> | (l) <code>round(-2.5)</code> |
| (d) <code>min(2, 2, 1)</code> | (m) <code>math.fabs(2.5)</code> |
| (e) <code>math.log(math.e)</code> | (n) <code>math.ceil(2.5)</code> |
| (f) <code>math.exp(1)</code> | (o) <code>math.floor(2.5)</code> |
| (g) <code>max(2, 3, 4)</code> | (p) <code>round(-2.5)</code> |
| (h) <code>abs(-2.5)</code> | (q) <code>round(2.6)</code> |
| (i) <code>math.ceil(-2.5)</code> | (r) <code>round(math.fabs(-2.5))</code> |



3.2 True or false? The argument for trigonometric functions represents an angle in radians.

3.3 Write a statement that converts 47 degrees to radians and assigns the result to a variable.

3.4 Write a statement that converts $\pi / 7$ to an angle in degrees and assigns the result to a variable.

3.3 Strings and Characters

A string (described in Chapter 1) is a sequence of characters. Python treats characters and strings the same way.



In addition to processing numeric values, you can process *strings* in Python. A *string* is a sequence of characters and can include text and numbers. *String* values must be enclosed in matching *single quotes* (') or *double quotes* ("). Python does not have a data type for characters. A single-character string represents a character. For example,

```

letter = 'A' # Same as letter = "A"
numChar = '4' # Same as numChar = "4"
message = "Good morning" # Same as message = 'Good morning'

```

string

single quotes or double quotes

The first statement assigns a string with the character **A** to the variable **letter**. The second statement assigns a string with the digit character **4** to the variable **numChar**. The third statement assigns the string **Good morning** to the variable **message**.



Note

For consistency, this book uses double quotes for a string with more than one character and single quotes for a string with a single character or an empty string. This convention is consistent with other programming languages, so it will be easy for you to convert a Python program to a program written in other languages.

3.3.1 ASCII Code

character encoding

Computers use binary numbers internally (see Section 1.2.2). A character is stored in a computer as a sequence of 0s and 1s. Mapping a character to its binary representation is called *character encoding*. There are different ways to encode a character. The manner in which characters are encoded is defined by an encoding scheme. One popular standard is ASCII (American Standard Code for Information Interchange), a 7-bit encoding scheme for representing all uppercase and lowercase letters, digits, punctuation marks, and control characters. ASCII uses numbers 0 through 127 to represent characters. Appendix B, The ASCII Character Set, shows the ASCII code for characters.

3.3.2 Unicode Code

Python also supports Unicode. Unicode is an encoding scheme for representing international characters. ASCII is a small subset of Unicode. Unicode was established by the Unicode Consortium to support the interchange, processing, and display of written texts in the world's diverse languages. A Unicode starts with `\u`, followed by four hexadecimal digits that run from `\u0000` to `\uFFFF`. (For information on hexadecimal numbers, see Appendix C.) For example, the word “welcome” is translated into Chinese using two characters, 欢 and 迎. The Unicode representations of these two characters are `\u6B22\u8FCE`.

The program in Listing 3.3 displays two Chinese characters and three Greek letters, as shown in Figure 3.1.

LISTING 3.3 DisplayUnicode.py

```
1 import turtle
2
3 turtle.write("\u6B22\u8FCE \u03b1 \u03b2 \u03b3")
4
5 turtle.done()
```



FIGURE 3.1 You can use Unicode to display international characters in a Python GUI program.

If no Chinese font is installed on your system, you will not be able to see the Chinese characters. In this case, delete `\u6B22\u8FCE` from your program to avoid errors. The Unicode codes for the Greek letters α , β , and γ are `\u03b1`, `\u03b2`, and `\u03b3`.

3.3.3 The `ord` and `chr` Functions

Python provides the `ord(ch)` function for returning the ASCII code for the character `ch` and the `chr(code)` function for returning the character represented by the code. For example,

```
>>> ch = 'a'
>>> ord(ch)
97
>>> chr(98)
'b'
>>> ord('A')
65
>>>
```



The ASCII code for `a` is `97`, which is greater than the code for `A` (`65`). The ASCII code for lowercase letters are consecutive integers starting from the code for `a`, then for `b`, `c`, and so on, up to the letter `z`. The same is true for the uppercase letters. The difference between the ASCII code of any lowercase letter and its corresponding uppercase letter is the same: `32`. This is a useful property for processing characters. For example, you can find the uppercase representation of any lowercase letter, as shown in the following code:

```
1 >>> ord('a') - ord('A')
2 32
3 >>> ord('d') - ord('D')
4 32
5 >>> offset = ord('a') - ord('A')
6 >>> lowercaseLetter = 'h'
7 >>> uppercaseLetter = chr(ord(lowercaseLetter) - offset)
8 >>> uppercaseLetter
9 'H'
10 >>>
```



Line 6 assigns a lowercase letter to variable `lowercaseLetter`. Line 7 obtains its corresponding uppercase letter.

3.3.4 Escape Sequences for Special Characters

Suppose you want to print a message with quotation marks in the output. Can you write a statement like this?

```
print("He said, "John's program is easy to read")
```

No, this statement has an error. Python thinks the second quotation mark is the end of the string and does not know what to do with the rest of the characters.

To overcome this problem, Python uses a special notation to represent special characters, as shown in Table 3.3. This special notation, which consists of a *backslash* (`\`) followed by a letter or a combination of digits, is called an *escape sequence*.

The `\n` character is also known as a *newline*, *line break* or *end-of-line* (EOL) character, which signifies the end of a line. The `\f` character forces the printer to print from the next page. The `\r` character is used to move the cursor to the first position on the same line. The `\f` and `\r` characters are rarely used in this book.

backslash (`\`)
escape sequence
newline
line break
end-of-line (EOL) character

TABLE 3.3 Python Escape Sequences

Character Escape Sequence	Name	Numeric Value
<code>\b</code>	Backspace	8
<code>\t</code>	Tab	9
<code>\n</code>	Linefeed	10
<code>\f</code>	Formfeed	12
<code>\r</code>	Carriage Return	13
<code>\\</code>	Backslash	92
<code>\'</code>	Single Quote	39
<code>\"</code>	Double Quote	34

Now you can print the quoted message using the following statement:



```
>>> print("He said, \"John's program is easy to read\"")
He said, "John's program is easy to read"
```

Note that the symbols `\` and `"` together represent one character.

3.3.5 Printing without the Newline

When you use the `print` function, it automatically prints a linefeed (`\n`) to cause the output to advance to the next line. If you don't want this to happen after the `print` function is finished, you can invoke the `print` function by passing a special argument `end = "anyendingstring"` using the following syntax:

```
print(item, end = "anyendingstring")
```

For example, the following code

```
1 print("AAA", end = ' ')
2 print("BBB", end = '')
3 print("CCC", end = '***')
4 print("DDD", end = '***')
```

displays

```
AAA BBBCCC***DDD***
```

Line 1 prints `AAA` followed by a space character `' '`, line 2 prints `BBB`, line 3 prints `CCC` followed by `***`, and line 4 prints `DDD` followed by `***`. Note that `''` in line 2 means an empty string. So, nothing is printed for `''`.

You can also use the `end` argument for printing multiple items using the following syntax:

```
print(item1, item2, ..., end = "anyendingstring")
```

For example,

```
radius = 3
print("The area is", radius * radius * math.pi, end = ' ')
print("and the perimeter is", 2 * radius)
```

displays

```
The area is 28.26 and the perimeter is 6
```

3.3.6 The `str` Function

The `str` function can be used to convert a number into a string. For example,

```
>>> s = str(3.4) # Convert a float to string
>>> s
'3.4'
>>> s = str(3) # Convert an integer to string
>>> s
'3'
>>>
```



3.3.7 The String Concatenation Operator

You can use the `+` operator to add two numbers. The `+` operator can be used to concatenate two strings. Here are some examples:

```
1 >>> message = "Welcome " + "to " + "Python"
2 >>> message
'Welcome to Python'
3 >>> chapterNo = 3
4 >>> s = "Chapter " + str(chapterNo)
5 >>> s
'Chapter 3'
6 >>>
7 >>>
8 >>>
```



Line 1 concatenates three strings into one. In line 5, the `str` function converts the numeric value in variable `chapterNo` to a string. This string is concatenated with `"Chapter "` to obtain the new string `"Chapter 3"`.

The augmented assignment `+=` operator can also be used for string concatenation. For example, the following code concatenates the string in `message` with the string `" and Python is fun"`.

```
>>> message = "Welcome to Python"
>>> message
'Welcome to Python'
>>> message += " and Python is fun"
>>> message
'Welcome to Python and Python is fun'
>>>
```



3.3.8 Reading Strings from the Console

To read a string from the console, use the `input` function. For example, the following code reads three strings from the keyboard:

```
s1 = input("Enter a string: ")
s2 = input("Enter a string: ")
s3 = input("Enter a string: ")
print("s1 is " + s1)
print("s2 is " + s2)
print("s3 is " + s3)
```



```
Enter a string: Welcome ↵ Enter
Enter a string: to ↵ Enter
Enter a string: Python ↵ Enter
s1 is Welcome
s2 is to
s3 is Python
```



MyProgrammingLab™

3.5 Use the `ord` function to find the ASCII code for **1**, **A**, **B**, **a**, and **b**. Use the `chr` function to find the character for the decimal codes **40**, **59**, **79**, **85**, and **90**.

3.6 How do you display the characters `\` and `"`?

3.7 How do you write a character in Unicode?

3.8 Suppose you entered **A** when running the following code. What is the output?

```
x = input("Enter a character: ")
ch = chr(ord(x) + 3)
print(ch)
```

3.9 Suppose you entered **A** and **Z** when running the following code. What is the output?

```
x = input("Enter a character: ")
y = input("Enter a character: ")
print(ord(y) - ord(x))
```

3.10 What is wrong in the following code? How do you fix it?

```
title = "Chapter " + 1
```

3.11 Show the result of the following code:

```
sum = 2 + 3
print(sum)
s = '2' + '3'
print(s)
```

3.4 Case Study: Minimum Number of Coins

Now let's look at a sample program that uses the features covered in this section. Suppose you want to develop a program that classifies a given amount of money into smaller monetary units. The program lets the user enter an amount as a floating-point value representing a total in dollars and cents, and then outputs a report listing the monetary equivalent in dollars, quarters, dimes, nickels, and pennies, as shown in the sample run.

Your program should report the maximum number of dollars, then the number of quarters, dimes, nickels, and pennies, in this order, to result in the minimum number of coins.

Here are the steps in developing the program:

1. Prompt the user to enter the amount as a decimal number, such as **11.56**.
2. Convert the amount (**11.56**) into cents (**1156**).
3. Divide the cents by **100** to find the number of dollars. Obtain the remaining cents using the cents remainder `% 100`.
4. Divide the remaining cents by **25** to find the number of quarters. Obtain the remaining cents using the remaining cents remainder `% 25`.

minimum number of coins

5. Divide the remaining cents by **10** to find the number of dimes. Obtain the remaining cents using the remaining cents remainder **% 10**.
6. Divide the remaining cents by **5** to find the number of nickels. Obtain the remaining cents using the remaining cents remainder **% 5**.
7. The remaining cents are the pennies.
8. Display the result.

The complete program is shown in Listing 3.4.

LISTING 3.4 ComputeChange.py

```

1  # Receive the amount
2  amount = eval(input("Enter an amount, for example, 11.56: "))
3
4  # Convert the amount to cents
5  remainingAmount = int(amount * 100)
6
7  # Find the number of one dollars
8  numberOfOneDollars = remainingAmount // 100
9  remainingAmount = remainingAmount % 100
10
11 # Find the number of quarters in the remaining amount
12 numberOfQuarters = remainingAmount // 25
13 remainingAmount = remainingAmount % 25
14
15 # Find the number of dimes in the remaining amount
16 numberOfDimes = remainingAmount // 10
17 remainingAmount = remainingAmount % 10
18
19 # Find the number of nickels in the remaining amount
20 numberOfNickels = remainingAmount // 5
21 remainingAmount = remainingAmount % 5
22
23 # Find the number of pennies in the remaining amount
24 numberOfPennies = remainingAmount
25
26 # Display the results
27 print("Your amount", amount, "consists of\n",
28       "\t", numberOfOneDollars, "dollars\n",
29       "\t", numberOfQuarters, "quarters\n",
30       "\t", numberOfDimes, "dimes\n",
31       "\t", numberOfNickels, "nickels\n",
32       "\t", numberOfPennies, "pennies")

```

enter input

dollars

quarters

dimes

nickels

pennies

output

```

Enter an amount, for example, 11.56: 11.56
Your amount 11.56 consists of
    11 dollars
     2 quarters
     0 dimes
     1 nickels
     1 pennies

```





variables	line#	2	5	8	9	12	13	16	17	20	21	24
amount		11.56										
remainingAmount			1156		56		6		6		1	
numberOfOneDollars				11								
numberOfQuarters						2						
numberOfDimes								0				
numberOfNickels										1		
numberOfPennies												1

The variable `amount` stores the amount entered from the console (line 2). This variable is not changed, because the amount has to be used at the end of the program to display the results. The program introduces the variable `remainingAmount` (line 5) to store the changing `remainingAmount`.

The variable `amount` is a float representing dollars and cents. It is converted to an integer variable `remainingAmount`, which represents all the cents. For instance, if `amount` is `11.56`, then the initial `remainingAmount` is `1156`. `1156 // 100` is `11` (line 8). The remainder operator obtains the remainder of the division. So, `1156 % 100` is `56` (line 9).

The program extracts the maximum number of quarters from `remainingAmount` and obtains a new `remainingAmount` (lines 12–13). Continuing the same process, the program finds the maximum number of dimes, nickels, and pennies in the remaining amount.

As shown in the sample run, `0` dimes, `1` nickels, and `1` pennies are displayed in the result. It would be better not to display `0` dimes, and to display `1` nickel and `1` penny using the singular forms of the words. You will learn how to use selection statements to modify this program in the next chapter (see Exercise 4.7).



Caution

One serious problem with this example is the possible loss of precision when converting a float amount to the integer `remainingAmount`. This could lead to an inaccurate result. If you try to enter the amount `10.03`, `10.03 * 100` might be `1003.9999999999999`. You will find that the program displays `10` dollars and `2` pennies. To fix the problem, enter the amount as an integer value representing cents (see Exercise 3.8).

loss of precision

3.5 Introduction to Objects and Methods



Key Point

In Python, all data—including numbers and strings—are actually objects.

In Python, a number is an *object*, a string is an object, and every datum is an object. Objects of the same kind have the same type. You can use the `id` function and `type` function to get these pieces of information about an object. For example,

object



```
1 >>> n = 3 # n is an integer
2 >>> id(n)
3 505408904
4 >>> type(n)
5 <class 'int'>
6 >>> f = 3.0 # f is a float
```

```

7  >>> id(f)
8  26647120
9  >>> type(f)
10 <class 'float'>
11 >>> s = "Welcome" # s is a string
12 >>> id(s)
13 36201472
14 >>> type(s)
15 <class 'str'>
16 >>>

```



VideoNote
String operations

The `id` for the object is automatically assigned a unique integer by Python when the program is executed. The `id` for the object will not be changed during the execution of the program. However, Python may assign a different `id` every time the program is executed. The type for the object is determined by Python according to the value of the object. Line 2 displays the `id` for a number object `n`, line 3 shows the `id` Python has assigned for the object, and its type is displayed in line 4.

In Python, an object's type is defined by a class. For example, the class for string is `str` (line 15), for integer is `int` (line 5), and for float is `float` (line 10). The term “class” comes from object-oriented programming, which will be discussed in Chapter 7. In Python, classes and types are synonymous.



Note

The `id` and `type` functions are rarely used in programming, but they are good tools for learning more about objects.

`id` function
`type` function

A variable in Python is actually a reference to an object. Figure 3.2 shows the relationship between the variables and objects for the preceding code.

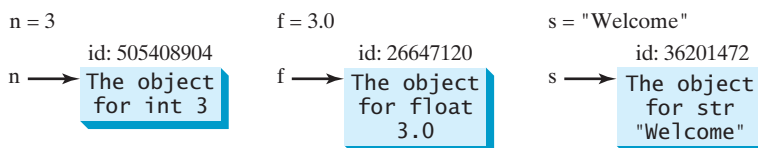


FIGURE 3.2 In Python, each variable is actually a reference to an object.

The statement `n = 3` in line 1 assigns value `3` to `n`, which actually assigns `3` to an `int` object referenced by variable `n`.



Note

For `n = 3`, we say `n` is an integer variable that holds value `3`. Strictly speaking, `n` is a variable that references an `int` object for value `3`. For simplicity, it is fine to say `n` is an `int` variable with value `3`.

objects vs. object reference
variable

You can perform operations on an object. The operations are defined using functions. The functions for the objects are called *methods* in Python. Methods can only be invoked from a specific object. For example, the string type has the methods such as `lower()` and `upper()`,

methods

which return a new string in lowercase and uppercase. Here are some examples of how to invoke these methods:



```
1 >>> s = "Welcome"
2 >>> s1 = s.lower() # Invoke the lower method
3 >>> s1
4 'welcome'
5 >>> s2 = s.upper() # Invoke the upper method
6 >>> s2
7 'WELCOME'
8 >>>
```

Line 2 invokes `s.lower()` on object `s` to return a new string in lowercase and assigns it to `s1`. Line 5 invokes `s.upper()` on object `s` to return a new string in uppercase and assigns it to `s2`.

As you can see from the preceding example, the syntax to invoke a method for an object is `object.method()`.

`strip()`

whitespace characters

Another useful string method is `strip()`, which can be used to remove (strip) the whitespace characters from both ends of a string. The characters ' ', `\t`, `\f`, `\r`, and `\n` are known as the *whitespace characters*.

For example,



```
>>> s = "\t Welcome \n"
>>> s1 = s.strip() # Invoke the strip method
>>> s1
'Welcome'
>>>
```



Note

If you use Python on Eclipse, Eclipse automatically appends `\r` in the string entered from the `input` function. Therefore, you should use the `strip()` method to remove the `\r` character as follows:

```
s = input("Enter a string").strip()
```

More details on processing strings and on object-oriented programming will be discussed in Chapter 7.



Check
Point

MyProgrammingLab™

- 3.12** What is an object? What is a method?
- 3.13** How do you find the id for an object? How do you find the type for an object?
- 3.14** Which of the following statements is the precise meaning for the statement `n = 3`?
 - (a) `n` is a variable that holds `int` value `3`.
 - (b) `n` is a variable that references an object that holds `int` value `3`.
- 3.15** Suppose `s` is `"\tGeorgia\n"`. What is `s.lower()` and `s.upper()`?
- 3.16** Suppose `s` is `" \tGood\tMorning\n"`. What is `s.strip()`?

3.6 Formatting Numbers and Strings

You can use the `format` function to return a formatted string.



Key
Point

Often it is desirable to display numbers in a certain format. For example, the following code computes interest, given the amount and the annual interest rate.



```
>>> amount = 12618.98
>>> interestRate = 0.0013
>>> interest = amount * interestRate
>>> print("Interest is", interest)
Interest is 16.404674
>>>
```

Because the interest amount is currency, it is desirable to display only two digits after the decimal point. To do this, you can write the code as follows:



```
>>> amount = 12618.98
>>> interestRate = 0.0013
>>> interest = amount * interestRate
>>> print("Interest is", round(interest, 2))
Interest is 16.4
>>>
```

However, the format is still not correct. There should be two digits after the decimal point like **16.40** rather than **16.4**. You can fix it by using the **format** function, like this:

format



```
>>> amount = 12618.98
>>> interestRate = 0.0013
>>> interest = amount * interestRate
>>> print("Interest is", format(interest, ".2f"))
Interest is 16.40
>>>
```

The syntax to invoke this function is

```
format(item, format-specifier)
```

where **item** is a number or a string and **format-specifier** is a string that specifies how the item is formatted. The function returns a string.

3.6.1 Formatting Floating-Point Numbers

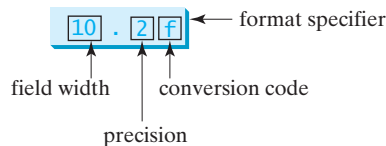
If the item is a float value, you can use the specifier to give the width and precision of the format in the form of **width.precisionf**. Here, **width** specifies the width of the resulting string, **precision** specifies the number of digits after the decimal point, and **f** is called the *conversion code*, which sets the formatting for floating point numbers. For example,

conversion code

```
print(format(57.467657, "10.2f"))
print(format(12345678.923, "10.2f"))
print(format(57.4, "10.2f"))
print(format(57, "10.2f"))
```

displays

```
|← 10 →|
□□□□ 57.47
123456782.92
□□□□ 57.40
□□□□ 57.00
```



where a square box (□) denotes a blank space. Note that the decimal point is counted as one space.

The `format("10.2f")` function formats the number into a string whose width is **10**, including a decimal point and two digits after the point. The number is rounded to two decimal places. Thus there are seven digits allocated before the decimal point. If there are fewer than seven digits before the decimal point, spaces are inserted before the number. If there are more than seven digits before the decimal point, the number's width is automatically increased. For example, `format(12345678.923, "10.2f")` returns `12345678.92`, which has a width of **11**.

You can omit the width specifier. If so, it defaults to **0**. In this case, the width is automatically set to the size needed for formatting the number. For example,

```
print(format(57.467657, "10.2f"))
print(format(57.467657, ".2f"))
```

displays

```
|← 10 →|
□□□□ 57.47
57.47
```

3.6.2 Formatting in Scientific Notation

If you change the conversion code from **f** to **e**, the number will be formatted in scientific notation. For example,

```
print(format(57.467657, "10.2e"))
print(format(0.0033923, "10.2e"))
print(format(57.4, "10.2e"))
print(format(57, "10.2e"))
```

displays

```
|← 10 →|
□□ 5.75e+01
□□ 3.39e-03
□□ 5.74e+01
□□ 5.70e+01
```

The **+** and **-** signs are counted as places in the width limit.

3.6.3 Formatting as a Percentage

You can use the conversion code **%** to format a number as a percentage. For example,

```
print(format(0.53457, "10.2%"))
print(format(0.0033923, "10.2%"))
print(format(7.4, "10.2%"))
print(format(57, "10.2%"))
```

displays

```
|← 10 →|
□□□□ 53.46%
□□□□ 0.34%
□□□ 740.00%
□□ 5700.00%
```

The format `10.2%` causes the number to be multiplied by 100 and displayed with a **%** sign following it. The total width includes the **%** sign counted as one space.

3.6.4 Justifying Format

By default, the format of a number is right justified. You can put the symbol `<` in the format specifier to specify that the item be left-justified in the resulting format within the specified width. For example,

```
print(format(57.467657, "10.2f"))
print(format(57.467657, "<10.2f"))
```

displays

```
|← 10 →|
| 57.47 |
57.47
```

3.6.5 Formatting Integers

The conversion codes `d`, `x`, `o`, and `b` can be used to format an integer in decimal, hexadecimal, octal, or binary. You can specify a width for the conversion. For example,

```
print(format(59832, "10d"))
print(format(59832, "<10d"))
print(format(59832, "10x"))
print(format(59832, "<10x"))
```

displays

```
|← 10 →|
| 59832 |
59832
|  e9b8 |
e9b8
```

The format specifier `10d` specifies that the integer is formatted into a decimal with a width of ten spaces. The format specifier `10x` specifies that the integer is formatted into a hexadecimal integer with a width of ten spaces.

3.6.6 Formatting Strings

You can use the conversion code `s` to format a string with a specified width. For example,

```
print(format("Welcome to Python", "20s"))
print(format("Welcome to Python", "<20s"))
print(format("Welcome to Python", ">20s"))
print(format("Welcome to Python and Java", ">20s"))
```

displays

```
|← 20 →|
Welcome to Python
Welcome to Python
  Welcome to Python
Welcome to Python and Java
```

The format specifier `20s` specifies that the string is formatted within a width of 20. By default, a string is left justified. To right-justify it, put the symbol `>` in the format specifier. If the string is longer than the specified width, the width is automatically increased to fit the string.

Table 3.4 summarizes the format specifiers introduced in this section.

TABLE 3.4 Frequently Used Specifiers

<i>Specifier</i>	<i>Format</i>
"10.2f"	Format the float item with width 10 and precision 2.
"10.2e"	Format the float item in scientific notation with width 10 and precision 2.
"5d"	Format the integer item in decimal with width 5.
"5x"	Format the integer item in hexadecimal with width 5.
"5o"	Format the integer item in octal with width 5.
"5b"	Format the integer item in binary with width 5.
"10.2%"	Format the number in decimal.
"50s"	Format the string item with width 50.
"<10.2f"	Left-justify the formatted item.
">10.2f"	Right-justify the formatted item.



MyProgrammingLab™

3.17 What is the return value from invoking the `format` function?

3.18 What happens if the size of the actual item is greater than the width in the format specifier?

3.19 Show the printout of the following statements:

```
print(format(57.467657, "9.3f"))
print(format(12345678.923, "9.1f"))
print(format(57.4, ".2f"))
print(format(57.4, "10.2f"))
```

3.20 Show the printout of the following statements:

```
print(format(57.467657, "9.3e"))
print(format(12345678.923, "9.1e"))
print(format(57.4, ".2e"))
print(format(57.4, "10.2e"))
```

3.21 Show the printout of the following statements:

```
print(format(5789.467657, "9.3f"))
print(format(5789.467657, "<9.3f"))
print(format(5789.4, ".2f"))
print(format(5789.4, "<.2f"))
print(format(5789.4, ">9.2f"))
```

3.22 Show the printout of the following statements:

```
print(format(0.457467657, "9.3%"))
print(format(0.457467657, "<9.3%"))
```

3.23 Show the printout of the following statements:

```
print(format(45, "5d"))
print(format(45, "<5d"))
print(format(45, "5x"))
print(format(45, "<5x"))
```

3.24 Show the printout of the following statements:

```
print(format("Programming is fun", "25s"))
print(format("Programming is fun", "<25s"))
print(format("Programming is fun", ">25s"))
```

3.7 Drawing Various Shapes

The Python **turtle** module contains methods for moving the pen, setting the pen’s size, lifting, and putting down the pen.



Chapter 1 introduced drawing with the turtle. A turtle is actually an object that is created when you import the **turtle** module. You then invoke the turtle object’s methods to perform operations. This section introduces more methods for the **turtle** object.

When a turtle object is created, its *position* is set at (0, 0)—the center of the window—and its *direction* is set to go straight to the right. The **turtle** module uses a pen to draw shapes. By default, the pen is down (like the tip of an actual pen touching a sheet of paper). When you move the turtle, it draws a line from the current position to the new position if the pen is down. Table 3.5 lists the methods for controlling the pen’s drawing state; Table 3.6 lists the methods for moving the turtle.

turtle’s position and direction
turtle’s pen



TABLE 3.5 Turtle Pen Drawing State Methods

Method	Description
turtle.pendown()	Pulls the pen down—drawing when moving.
turtle.penup()	Pulls the pen up—no drawing when moving.
turtle.pensize(width)	Sets the line thickness to the specified width.

TABLE 3.6 Turtle Motion Methods

Method	Description
turtle.forward(d)	Moves the turtle forward by the specified distance in the direction the turtle is headed.
turtle.backward(d)	Moves the turtle backward by the specified distance in the opposite direction the turtle is headed. The turtle’s direction is not changed.
turtle.right(angle)	Turns the turtle right by the specified angle.
turtle.left(angle)	Turns the turtle left by the specified angle.
turtle.goto(x, y)	Moves the turtle to an absolute position.
turtle.setx(x)	Moves the turtle’s x-coordinate to the specified position.
turtle.sety(y)	Moves the turtle’s y-coordinate to the specified position.
turtle.setheading(angle)	Sets the orientation of the turtle to a specified angle. 0-East, 90-North, 180-West, 270-South.
turtle.home()	Moves the turtle to the origin (0, 0) and east direction.
turtle.circle(r, ext, step)	Draws a circle with the specified radius, extent, and step.
turtle.dot(diameter, color)	Draws a circle with the specified diameter and color.
turtle.undo()	Undo (repeatedly) the last turtle action(s).
turtle.speed(s)	Sets the turtle’s speed to an integer between 1 and 10, with 10 being the fastest.

All these methods are straightforward. The best way to learn them is to write a test code to see how each method works.

The **circle** method has three arguments: The **radius** is required, and **extent** and **step** are optional. **extent** is an angle that determines which part of the circle is drawn. **step** determines the number of steps to use. If **step** is 3, 4, 5, 6, ..., the **circle** method will draw a maximum regular polygon with three, four, five, six, or more sides enclosed inside the circle (that is, a triangle, square, pentagon, hexagon, etc.). If **step** is not specified, the **circle** method will draw a circle.

Listing 3.5 shows sample code for drawing a triangle, a square, a pentagon, a hexagon, and a circle, as shown in Figure 3.3.

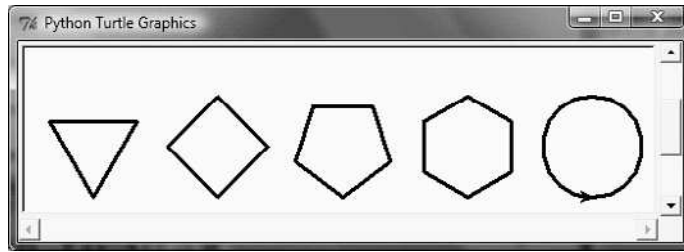


FIGURE 3.3 The program draws five shapes.

LISTING 3.5 SimpleShapes.py

```
import turtle module

set pen size
penup
move pen
pendown
draw a triangle

1  import turtle
2
3  turtle.pensize(3) # Set pen thickness to 3 pixels
4  turtle.penup() # Pull the pen up
5  turtle.goto(-200, -50)
6  turtle.pendown() # Pull the pen down
7  turtle.circle(40, steps = 3) # Draw a triangle
8
9  turtle.penup()
10 turtle.goto(-100, -50)
11 turtle.pendown()
12 turtle.circle(40, steps = 4) # Draw a square
13
14 turtle.penup()
15 turtle.goto(0, -50)
16 turtle.pendown()
17 turtle.circle(40, steps = 5) # Draw a pentagon
18
19 turtle.penup()
20 turtle.goto(100, -50)
21 turtle.pendown()
22 turtle.circle(40, steps = 6) # Draw a hexagon
23
24 turtle.penup()
25 turtle.goto(200, -50)
26 turtle.pendown()
27 turtle.circle(40) # Draw a circle
28
29 turtle.done()

pause
```

Line 1 imports the `turtle` module. Line 3 sets the pen's thickness to 3 pixels. Line 4 pulls the pen up so that you can reposition it to (-200, -50) in line 5. Line 6 puts the pen down to draw a triangle in line 7. In line 7, the `turtle` object invokes the `circle` method with a radius of 40 and 3 steps to draw a triangle. Similarly, the rest of the program draws a square (line 12), a pentagon (line 17), a hexagon (line 22), and a circle (line 27).



3.25 How do you set the turtle to its original position (0, 0)?

3.26 How do you draw a red dot with diameter 3?

3.27 What figure will the following method draw?

```
turtle.circle(50, step = 4)
```

3.28 How do you make the turtle move fast?

3.29 How do you undo the turtle's last action?

3.8 Drawing with Colors and Fonts

A turtle object contains the methods for setting colors and fonts.



The preceding section showed you how to draw shapes with the `turtle` module. You learned how to use the motion methods to move the pen and use the pen methods to raise the pen up, set it down, and control its thickness. This section introduces more pen control methods and shows you how to set colors and fonts and write text.

Table 3.7 lists the pen methods for controlling drawing, color, and filling. Listing 3.6 is a sample program that draws a triangle, a square, a pentagon, a hexagon, and a circle in different colors, as shown in Figure 3.4. The program also adds text to the drawing.

TABLE 3.7 Turtle Pen Color, Filling, and Drawing Methods

Method	Description
<code>turtle.color(c)</code>	Sets the pen color.
<code>turtle.fillcolor(c)</code>	Sets the pen fill color.
<code>turtle.begin_fill()</code>	Calls this method before filling a shape.
<code>turtle.end_fill()</code>	Fills the shapes drawn before the last call to <code>begin_fill</code> .
<code>turtle.filling()</code>	Returns the fill state: <code>True</code> if filling, <code>False</code> if not filling.
<code>turtle.clear()</code>	Clears the window. The state and the position of the turtle are not affected.
<code>turtle.reset()</code>	Clears the window and reset the state and position to the original default value.
<code>turtle.screensize(w, h)</code>	Sets the width and height of the canvas.
<code>turtle.hideturtle()</code>	Makes the turtle invisible.
<code>turtle.showturtle()</code>	Makes the turtle visible.
<code>turtle.isvisible()</code>	Returns <code>True</code> if the turtle is visible.
<code>turtle.write(s, font=("Arial", 8, "normal"))</code>	Writes the string <code>s</code> on the turtle position. Font is a triple consisting of fontname, fontsize, and fonttype.

LISTING 3.6 ColorShapes.py

<pre> 1 import turtle 2 3 turtle.pensize(3) # Set pen thickness to 3 pixels 4 turtle.penup() # Pull the pen up 5 turtle.goto(-200, -50) 6 turtle.pendown() # Pull the pen down 7 turtle.begin_fill() # Begin to fill color in a shape 8 turtle.color("red") 9 turtle.circle(40, steps = 3) # Draw a triangle 10 turtle.end_fill() # Fill the shape 11 12 turtle.penup() 13 turtle.goto(-100, -50) 14 turtle.pendown() 15 turtle.begin_fill() # Begin to fill color in a shape 16 turtle.color("blue") 17 turtle.circle(40, steps = 4) # Draw a square </pre>	<pre> import turtle module set pen size penup move pen pendown begin_fill set a color draw a triangle end_fill </pre>
--	--

```

18 turtle.end_fill() # Fill the shape
19
20 turtle.penup()
21 turtle.goto(0, -50)
22 turtle.pendown()
23 turtle.begin_fill() # Begin to fill color in a shape
24 turtle.color("green")
25 turtle.circle(40, steps = 5) # Draw a pentagon
26 turtle.end_fill() # Fill the shape
27
28 turtle.penup()
29 turtle.goto(100, -50)
30 turtle.pendown()
31 turtle.begin_fill() # Begin to fill color in a shape
32 turtle.color("yellow")
33 turtle.circle(40, steps = 6) # Draw a hexagon
34 turtle.end_fill() # Fill the shape
35
36 turtle.penup()
37 turtle.goto(200, -50)
38 turtle.pendown()
39 turtle.begin_fill() # Begin to fill color in a shape
40 turtle.color("purple")
41 turtle.circle(40) # Draw a circle
42 turtle.end_fill() # Fill the shape
43 turtle.color("green")
44 turtle.penup()
45 turtle.goto(-100, 50)
46 turtle.pendown()
47 turtle.write("Cool Colorful Shapes",
48             font = ("Times", 18, "bold"))
49 turtle.hideturtle()
50
51 turtle.done()

```

write text

pause

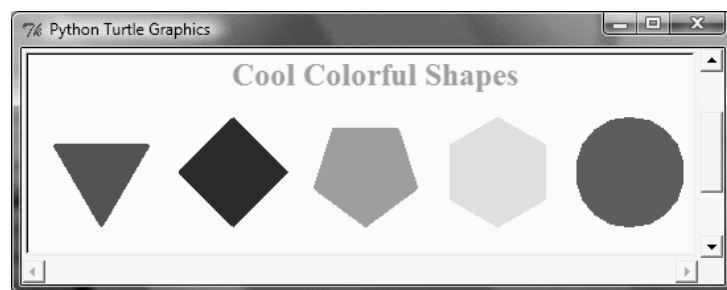


FIGURE 3.4 The program draws five shapes in different colors.

The program is similar to Listing 3.5. SimpleShapes.py, except that it fills each shape with a color and writes a string. The turtle object invokes the `begin_fill()` method in line 7 to tell Python to draw shapes filled with color. A triangle is drawn in line 9. Invoking the `end_fill()` method (line 10) completes the color filling for the shape.

The `write` method writes a string with the specified font at the current pen position (lines 47–48). Note that drawing takes place when the pen is moved if the pen is down. To avoid drawing, you need to pull the pen up. Invoking `hideturtle()` makes the turtle invisible (line 49) so you will not see the turtle in the window.

- 3.30** How do you set the turtle's color?
3.31 How do you fill a shape with color?
3.32 How do you make the turtle invisible?



MyProgrammingLab™

KEY TERMS

backslash (\) 69	methods 75
character encoding 68	newline 69
end-of-line 69	object 74
escape sequence 69	string 67
line break 69	whitespace characters 76

CHAPTER SUMMARY

1. Python provides the mathematical functions **abs**, **max**, **min**, **pow**, and **round** in the interpreter and the functions **fabs**, **ceil**, **floor**, **exp**, **log**, **sqrt**, **sin**, **asin**, **cos**, **acos**, **tan**, **degrees**, and **radians** in the **math** module.
2. A *string* is a sequence of characters. String values can be enclosed in matching single quotes (') or double quotes ("). Python does not have a data type for characters; a single-character string represents a character.
3. An *escape sequence* is a special syntax that begins with the character \ followed by a letter or a combination of digits to represent special characters, such as \', \", \t, and \n.
4. The characters ' ', \t, \f, \r, and \n are known as the *whitespace characters*.
5. All data including numbers and strings are *objects* in Python. You can invoke *methods* to perform operations on the objects.
6. You can use the **format** function to format a number or a string and return the result as a string.

TEST QUESTIONS

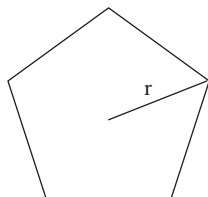
Do test questions for this chapter online at www.cs.armstrong.edu/liang/py/test.html.

PROGRAMMING EXERCISES

MyProgrammingLab™

Section 3.2

- 3.1** (*Geometry: area of a pentagon*) Write a program that prompts the user to enter the length from the center of a pentagon to a vertex and computes the area of the pentagon, as shown in the following figure.



The formula for computing the area of a pentagon is $Area = \frac{3\sqrt{3}}{2}s^2$, where s is the length of a side. The side can be computed using the formula $s = 2r \sin \frac{\pi}{5}$, where r is the length from the center of a pentagon to a vertex. Here is a sample run:



```
Enter the length from the center to a vertex: 5.5 Enter
The area of the pentagon is 108.61
```

- *3.2** (*Geometry: great circle distance*) The great circle distance is the distance between two points on the surface of a sphere. Let (x_1, y_1) and (x_2, y_2) be the geographical latitude and longitude of two points. The great circle distance between the two points can be computed using the following formula:

$$d = radius \times \arccos(\sin(x_1) \times \sin(x_2) + \cos(x_1) \times \cos(x_2) \times \cos(y_1 - y_2))$$

Write a program that prompts the user to enter the latitude and longitude of two points on the earth in degrees and displays its great circle distance. The average earth radius is 6,371.01 km. Note that you need to convert the degrees into radians using the `math.radians` function since the Python trigonometric functions use radians. The latitude and longitude degrees in the formula are for north and west. Use negative to indicate south and east degrees. Here is a sample run:



```
Enter point 1 (latitude and longitude) in degrees:
39.55, -116.25 Enter
Enter point 2 (latitude and longitude) in degrees:
41.5, 87.37 Enter
The distance between the two points is 10691.79183231593 km
```

- *3.3** (*Geography: estimate areas*) Find the GPS locations for Atlanta, Georgia; Orlando, Florida; Savannah, Georgia; and Charlotte, North Carolina from www.gps-data-team.com/map/ and compute the estimated area enclosed by these four cities. (Hint: Use the formula in Programming Exercise 3.2 to compute the distance between two cities. Divide the polygon into two triangles and use the formula in Programming Exercise 2.14 to compute the area of a triangle.)

- 3.4** (*Geometry: area of a pentagon*) The area of a pentagon can be computed using the following formula (s is the length of a side):

$$Area = \frac{5 \times s^2}{4 \times \tan\left(\frac{\pi}{5}\right)}$$

Write a program that prompts the user to enter the side of a pentagon and displays the area. Here is a sample run:



```
Enter the side: 5.5 Enter
The area of the pentagon is 53.04444136781625
```

- *3.5** (*Geometry: area of a regular polygon*) A regular polygon is an n -sided polygon in which all sides are of the same length and all angles have the same degree (i.e., the polygon is both equilateral and equiangular). The formula for computing the area of a regular polygon is

$$Area = \frac{n \times s^2}{4 \times \tan\left(\frac{\pi}{n}\right)}$$

Here, s is the length of a side. Write a program that prompts the user to enter the number of sides and their length of a regular polygon and displays its area. Here is a sample run:

```
Enter the number of sides: 5 ↵ Enter
Enter the side: 6.5 ↵ Enter
The area of the polygon is 73.69017017488385
```



Sections 3.3–3.6

- *3.6** (*Find the character of an ASCII code*) Write a program that receives an ASCII code (an integer between 0 and 127) and displays its character. For example, if the user enters 97, the program displays the character a. Here is a sample run:

```
Enter an ASCII code: 69 ↵ Enter
The character is E
```



- 3.7** (*Random character*) Write a program that displays a random uppercase letter using the `time.time()` function.
- *3.8** (*Financial application: monetary units*) Rewrite Listing 3.4, `ComputeChange.py`, to fix the possible loss of accuracy when converting a float value to an int value. Enter the input as an integer whose last two digits represent the cents. For example, the input 1156 represents 11 dollars and 56 cents.
- *3.9** (*Financial application: payroll*) Write a program that reads the following information and prints a payroll statement:

```
Employee's name (e.g., Smith)
Number of hours worked in a week (e.g., 10)
Hourly pay rate (e.g., 9.75)
Federal tax withholding rate (e.g., 20%)
State tax withholding rate (e.g., 9%)
```

A sample run is shown below:

```
Enter employee's name: Smith ↵ Enter
Enter number of hours worked in a week: 10 ↵ Enter
Enter hourly pay rate: 9.75 ↵ Enter
Enter federal tax withholding rate: 0.20 ↵ Enter
Enter state tax withholding rate: 0.09 ↵ Enter

Employee Name: Smith
```



```

Hours Worked: 10.0
Pay Rate: $9.75
Gross Pay: $97.5
Deductions:
    Federal Withholding (20.0%): $19.5
    State Withholding (9.0%): $8.77
    Total Deduction: $28.27
Net Pay: $69.22

```

***3.10** (Turtle: display Unicodes) Write a program to display Greek letters $\alpha\beta\gamma\delta\epsilon\zeta\eta\theta$. The Unicode of these characters are `\u03b1` `\u03b2` `\u03b3` `\u03b4` `\u03b5` `\u03b6` `\u03b7` `\u03b8`.

3.11 (Reverse number) Write a program that prompts the user to enter a four-digit integer and displays the number in reverse order. Here is a sample run:



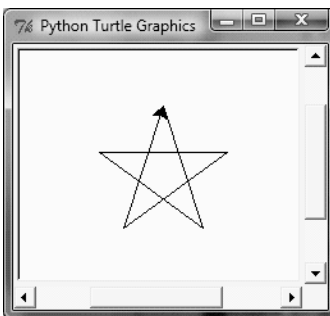
```

Enter an integer: 3125
The reversed number is 5213

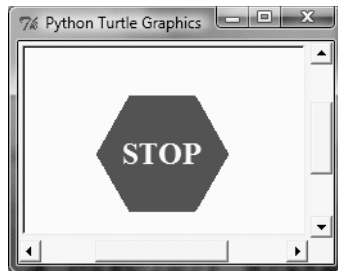
```

Sections 3.7–3.8

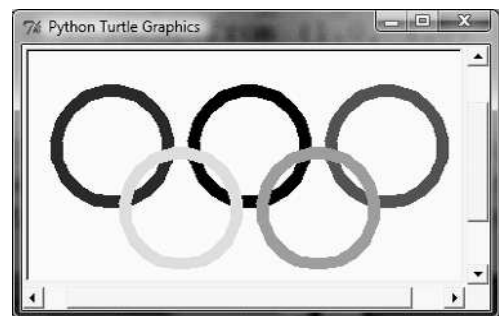
***3.12** (Turtle: draw a star) Write a program that prompts the user to enter the length of the star and draw a star, as shown in Figure 3.5a. (Hint: The inner angle of each point in the star is 36 degrees.)



(a)



(b)



(c)

FIGURE 3.5 The program (a) draws a star, (b) displays a STOP sign, and (c) draws an Olympic symbol.

***3.13** (Turtle: display a STOP sign) Write a program that displays a STOP sign, as shown in Figure 3.5b. The hexagon is in red and the text is in white.

3.14 (Turtle: draw the Olympic symbol) Write a program that prompts the user to enter the radius of the rings and draws an Olympic symbol of five rings of the same size with the colors blue, black, red, yellow, and green, as shown in Figure 3.5c.

***3.15** (Turtle: paint a smiley face) Write a program that paints a smiley face, as shown in Figure 3.6a.

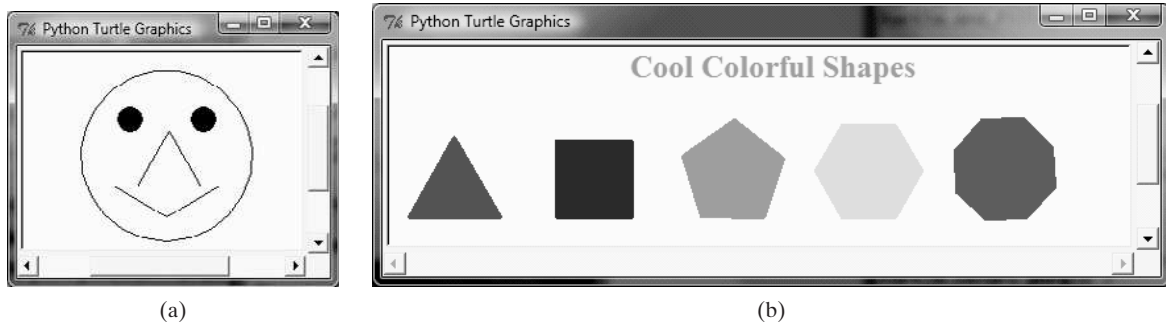


FIGURE 3.6 The program paints a smiley face in (a) and draws five shapes with bottom edges parallel to the x -axis in (b).

****3.16** (*Turtle: draw shapes*) Write a program that draws a triangle, square, pentagon, hexagon, and octagon, as shown in Figure 3.6b. Note that the bottom edges of these shapes are parallel to the x -axis. (Hint: For a triangle with a bottom line parallel to the x -axis, set the turtle's heading to 60 degrees.)

***3.17** (*Turtle: triangle area*) Write a program that prompts the user to enter the three points $p1$, $p2$, and $p3$ for a triangle and display its area below the triangle, as shown in Figure 3.7a. The formula for computing the area of a triangle is given in Exercise 2.14.

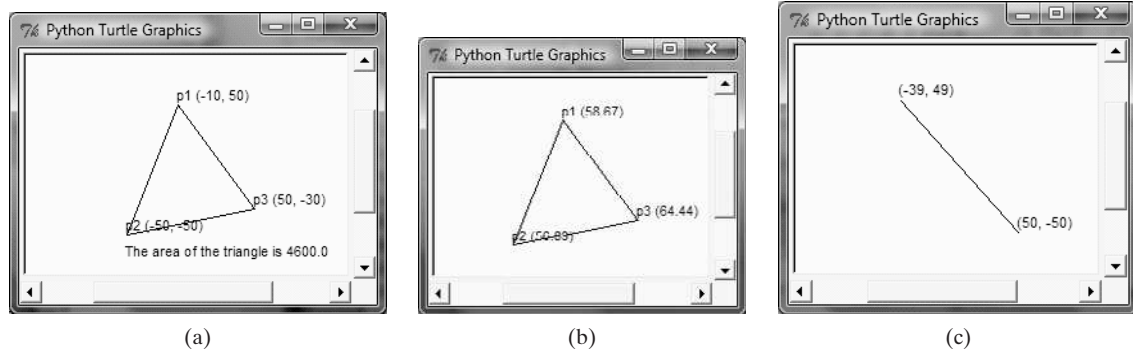


FIGURE 3.7 The program displays (a) the area of the triangle and (b) the angles for the triangle. (c) The program draws a line.

***3.18** (*Turtle: triangle angles*) Revise Listing 3.2, `ComputeAngles.py`, to write a program that prompts the user to enter the three points $p1$, $p2$, and $p3$ for a triangle and display its angles, as shown in Figure 3.7b.

****3.19** (*Turtle: draw a line*) Write a program that prompts the user to enter two points and draw a line to connect the points and displays the coordinates of the points, as shown in Figure 3.7c.

This page intentionally left blank

SELECTIONS

Objectives

- To write Boolean expressions using comparison operators (§4.2).
- To generate random numbers using the `random.randint(a, b)` or `random.random()` functions (§4.3).
- To program with Boolean expressions (`AdditionQuiz`) (§4.3).
- To implement selection control using one-way `if` statements (§4.4).
- To program with one-way `if` statements (`GuessBirthday`) (§4.5).
- To implement selection control using two-way `if-else` statements (§4.6).
- To implement selection control with nested `if` and multi-way `if-elif-else` statements (§4.7).
- To avoid common errors in `if` statements (§4.8).
- To program with selection statements (§§4.9–4.10).
- To combine conditions using logical operators (`and`, `or`, and `not`) (§4.11).
- To use selection statements with combined conditions (`LeapYear`, `Lottery`) (§§4.12–4.13).
- To write expressions that use the conditional expressions (§4.14).
- To understand the rules governing operator precedence and associativity (§4.15).
- To detect the location of an object (§4.16).



4.1 Introduction



A program can decide which statements to execute based on a condition.

problem

If you enter a negative value for `radius` in Listing 2.2, `ComputeAreaWithConsoleInput.py`, the program displays an invalid result. If the radius is negative, the program cannot compute the area. How can you deal with this situation?

selection statements

Like all high-level programming languages, Python provides *selection statements* that let you choose actions with two or more alternative courses. You can use the following selection statement to replace line 5 in Listing 2.2:

```
if radius < 0:
    print("Incorrect input")
else:
    area = radius * radius * math.pi
    print("Area is", area)
```

Boolean expressions

Selection statements use conditions, which are *Boolean expressions*. This chapter introduces Boolean types, values, comparison operators, and expressions.

4.2 Boolean Types, Values, and Expressions



A Boolean expression is an expression that evaluates to a Boolean value `True` or `False`.

comparison operators

How do you compare two values, such as whether a radius is greater than `0`, equal to `0`, or less than `0`? Python provides six *comparison operators* (also known as *relational operators*), shown in Table 4.1, which can be used to compare two values (the table assumes that a radius of `5` is being used).



VideoNote
Boolean expressions

TABLE 4.1 Comparison Operators

Python Operator	Mathematics Symbol	Name	Example (<code>radius is 5</code>)	Result
<code><</code>	<code><</code>	less than	<code>radius < 0</code>	<code>False</code>
<code><=</code>	<code>≤</code>	less than or equal to	<code>radius <= 0</code>	<code>False</code>
<code>></code>	<code>></code>	greater than	<code>radius > 0</code>	<code>True</code>
<code>>=</code>	<code>≥</code>	greater than or equal to	<code>radius >= 0</code>	<code>True</code>
<code>==</code>	<code>=</code>	equal to	<code>radius == 0</code>	<code>False</code>
<code>!=</code>	<code>≠</code>	not equal to	<code>radius != 0</code>	<code>True</code>



Caution

The *equal to* comparison operator is two equal signs (`==`), not a single equal sign (`=`). The latter symbol is for assignment.

`==` vs. `=`

Boolean value

The result of the comparison is a *Boolean value*: `True` or `False`. For example, the following statement displays the result `True`:

```
radius = 1
print(radius > 0)
```

A variable that holds a Boolean value is known as a Boolean variable. The Boolean data type is used to represent Boolean values. A Boolean variable can hold one of the two values: **True** or **False**. For example, the following statement assigns the value **True** to the variable **lightsOn**:

Boolean variable

```
lightsOn = True
```

True and **False** are literals, just like a number such as **10**. They are reserved words and cannot be used as identifiers in a program.

Boolean literals

Internally, Python uses **1** to represent **True** and **0** for **False**. You can use the **int** function to convert a Boolean value to an integer.

convert Boolean to int

For example,

```
print(int(True))
```

displays **1** and

```
print(int(False))
```

displays **0**.

You can also use the **bool** function to convert a numeric value to a Boolean value. The function returns **False** if the value is **0**; otherwise, it always returns **True**.

bool function

For example,

```
print(bool(0))
```

displays **False** and

```
print(bool(4))
```

displays **True**.

4.1 List six comparison operators.

4.2 Can the following conversions be allowed? If so, find the converted result.

```
i = int(True)
j = int(False)
```

```
b1 = bool(4)
b2 = bool(0)
```



MyProgrammingLab™

4.3 Generating Random Numbers

The **randint(a, b)** function can be used to generate a random integer between **a** and **b**, inclusively.



Suppose you want to develop a program to help a first grader practice addition. The program randomly generates two single-digit integers, **number1** and **number2**, and displays to the student a question such as **What is 1 + 7**, as shown in Listing 4.1. After the student types the answer, the program displays a message to indicate whether it is true or false.

To generate a random number, you can use the **randint(a, b)** function in the **random** module. This function returns a random integer **i** between **a** and **b**, inclusively. To obtain a random integer between **0** and **9**, use **randint(0, 9)**.

The program may be set up to work as follows:

- Step 1: Generate two single-digit integers for **number1** (e.g., 4) and **number2** (e.g., 5)
- Step 2: Prompt the student to answer, "What is 4 + 5?"
- Step 3: Check whether the student's answer is correct.

LISTING 4.1 AdditionQuiz.py

import random module

generate number1

generate number2

show question

display result

```
1 import random
2
3 # Generate random numbers
4 number1 = random.randint(0, 9)
5 number2 = random.randint(0, 9)
6
7 # Prompt the user to enter an answer
8 answer = eval(input("What is " + str(number1) + " + "
9                     + str(number2) + "? "))
10
11 # Display result
12 print(number1, "+", number2, "=", answer,
13       "is", number1 + number2 == answer)
```



What is 1 + 7? 8
1 + 7 = 8 is True



What is 4 + 8? 9
4 + 8 = 9 is False



line#	number1	number2	answer	output
4	4			
5		8		
8			9	
12				4 + 8 = 9 is False

random module

The program uses the **randint** function defined in the **random** module. The **import** statement imports the module (line 1).

randrange function

Lines 4–5 generate two numbers, **number1** and **number2**. Line 8 obtains an answer from the user. The answer is graded in line 12 using a Boolean expression **number1 + number2 == answer**.

random function

Python also provides another function, **randrange(a, b)**, for generating a random integer between **a** and **b - 1**, which is equivalent to **randint(a, b - 1)**. For example, **randrange(0, 10)** and **randint(0, 9)** are the same. Since **randint** is more intuitive, the book generally uses **randint** in the examples.

You can also use the **random()** function to generate a random float **r** such that **0 <= r < 1.0**. For example

random.random()

random.random()

random.randint(a, b)



```
1 >>> import random
2 >>> random.random()
3 0.34343
4 >>> random.random()
5 0.20119
6 >>> random.randint(0, 1)
```

```

7 0
8 >>> random.randint(0, 1)
9 1
10 >>> random.randrange(0, 1) # This will always be 0
11 0
12 >>>

```

random.randrange(a, b)

Invoking `random.random()` (lines 2 and 4) returns a random float number between **0.0** and **1.0** (excluding **1.0**). Invoking `random.randint(0, 1)` (lines 6 and 8) returns **0** or **1**. Invoking `random.randrange(0, 1)` (line 10) always returns **0**.

- 4.3** How do you generate a random integer **i** such that $0 \leq i < 20$?
- 4.4** How do you generate a random integer **i** such that $10 \leq i < 20$?
- 4.5** How do you generate a random integer **i** such that $10 \leq i \leq 50$?
- 4.6** How do you generate a random integer 0 or 1?



MyProgrammingLab™

4.4 if Statements

A one-way if statement executes the statements if the condition is true.



The preceding program displays a message such as **6 + 2 = 7 is False**. If you wish the message to be **6 + 2 = 7 is incorrect**, you have to use a selection statement to make this minor change.

Python has several types of selection statements: one-way **if** statements, two-way **if-else** statements, nested **if** statements, multi-way **if-elif-else** statements and conditional expressions. This section introduces one-way **if** statements.

A one-way **if** statement executes an action if and only if the condition is true. The syntax for a one-way **if** statement is:

```

if boolean-expression:
    statement(s) # Note that the statement(s) must be indented

```

The **statement(s)** must be indented at least one space to the right of the **if** keyword and each statement must be indented using the same number of spaces. For consistency, we indent it four spaces in this book.

The flowchart in Figure 4.1a illustrates how Python executes the syntax of an **if** statement. A *flowchart* is a diagram that describes an algorithm or process, showing the steps as boxes of various kinds, and their order by connecting these with arrows. Process operations are represented in these boxes, and arrows connecting them show flow of control. A diamond box is used to denote a Boolean condition and a rectangle box is for representing statements.

If the **boolean-expression** evaluates to true, the statements in the **if** block are executed. The **if** block contains the statements indented after the **if** statement. For example:

```

if radius >= 0:
    area = radius * radius * math.pi
    print("The area for the circle of radius", radius, "is", area)

```

The flowchart of the preceding statement is shown in Figure 4.1b. If the value of **radius** is greater than or equal to **0**, then the **area** is computed and the result is displayed; otherwise, these statements in the block are not executed.

The statements in the **if** block must be indented in the lines after the **if** line and each statement must be indented using the same number of spaces. For example, the following code is wrong, because the **print** statement in line 3 is not indented using the same number of spaces as the statement for computing area in line 2.

```

1 if radius >= 0:
2     area = radius * radius * math.pi # Compute area
3     print("The area for the circle of radius", radius, "is", area)

```

if statements

if statement

flowchart

if block



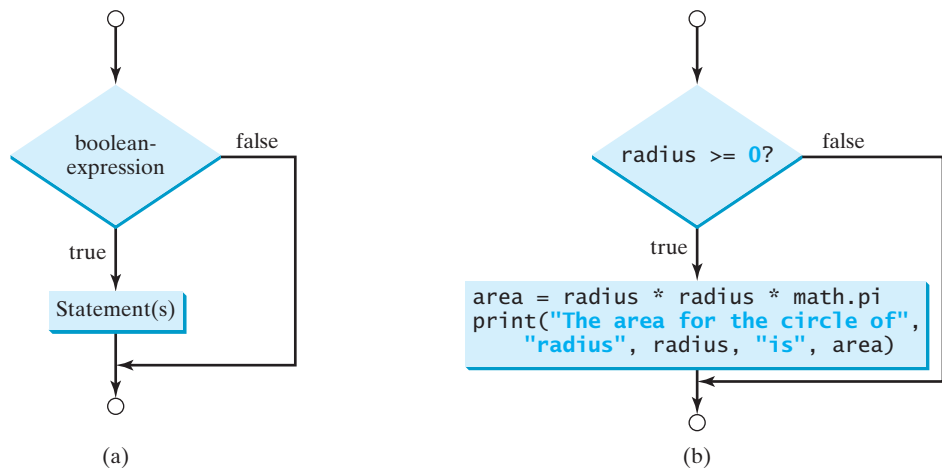


FIGURE 4.1 An **if** statement executes statements if the **boolean-expression** evaluates to **True**.

Listing 4.2 is an example of a program that prompts the user to enter an integer. If the number is a multiple of **5**, the program displays the result **HiFive**. If the number is divisible by **2**, the program displays **HiEven**.

LISTING 4.2 SimpleIfDemo.py

enter input	1 number = eval(input("Enter an integer: "))
	2
check 5	3 if number % 5 == 0:
	4 print("HiFive")
	5
check even	6 if number % 2 == 0:
	7 print("HiEven")



Enter an integer: 4

HiEven



Enter an integer: 30

HiFive
HiEven

The program prompts the user to enter an integer (line 1) and displays **HiFive** if it is divisible by **5** (lines 3–4) and **HiEven** if it is divisible by **2** (lines 6–7).



4.7 Write an **if** statement that assigns **1** to **x** if **y** is greater than **0**.

4.8 Write an **if** statement that increases **pay** by 3% if **score** is greater than **90**.

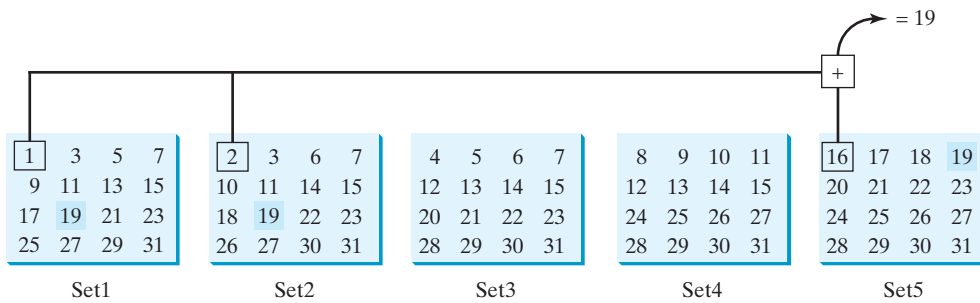
MyProgrammingLab™



4.5 Case Study: Guessing Birthdays

Guessing birthdays is an interesting problem with a simple program solution.

You can find out the date of the month when your friend was born by asking five questions. Each question asks whether the day is in one of the five sets of numbers.



The birthday is the sum of the first numbers in the sets where the date appears. For example, if the birthday is **19**, it appears in Set1, Set2, and Set5. The first numbers in these three sets are **1**, **2**, and **16**. Their sum is **19**.

Listing 4.3 is a program that prompts the user to answer whether the day is in Set1 (lines 4–13), in Set2 (lines 16–25), in Set3 (lines 28–37), in Set4 (lines 40–49), or in Set5 (lines 52–61). If the number is in the set, the program adds the first number in the set to **day** (lines 13, 25, 37, 49, and 61).

LISTING 4.3 GuessBirthday.py

```

1  day = 0 # birth day to be determined
2
3  # Prompt the user to answer the first question
4  question1 = "Is your birthday in Set1?\n" + \
5      " 1  3  5  7\n" + \
6      " 9 11 13 15\n" + \
7      "17 19 21 23\n" + \
8      "25 27 29 31" + \
9      "\nEnter 0 for No and 1 for Yes: "
10 answer = eval(input(question1))
11
12 if answer == 1:
13     day += 1
14
15 # Prompt the user to answer the second question
16 question2 = "Is your birthday in Set2?\n" + \
17     " 2  3  6  7\n" + \
18     "10 11 14 15\n" + \
19     "18 19 22 23\n" + \
20     "26 27 30 31" + \
21     "\nEnter 0 for No and 1 for Yes: "
22 answer = eval(input(question2))
23
24 if answer == 1:
25     day += 2
26
27 # Prompt the user to answer the third question
28 question3 = "Is your birthday in Set3?\n" + \
29     " 4  5  6  7\n" + \
30     "12 13 14 15\n" + \
31     "20 21 22 23\n" + \
32     "28 29 30 31" + \
33     "\nEnter 0 for No and 1 for Yes: "
34 answer = eval(input(question3))
35
36 if answer == 1:
37     day += 4

```

day to be determined

in question1?

in question2?

in question3?

in question4?

```

38
39 # Prompt the user to answer the fourth question
40 question4 = "Is your birthday in Set4?\n" + \
41     " 8  9 10 11\n" + \
42     "12 13 14 15\n" + \
43     "24 25 26 27\n" + \
44     "28 29 30 31" + \
45     "\nEnter 0 for No and 1 for Yes: "
46 answer = eval(input(question4))
47
48 if answer == 1:
49     day += 8
50
51 # Prompt the user to answer the fifth question
52 question5 = "Is your birthday in Set5?\n" + \
53     "16 17 18 19\n" + \
54     "20 21 22 23\n" + \
55     "24 25 26 27\n" + \
56     "28 29 30 31" + \
57     "\nEnter 0 for No and 1 for Yes: "
58 answer = eval(input(question5))
59
60 if answer == 1:
61     day += 16
62
63 print("\nYour birthday is "+ str(day) + "!")

```

in question5?



```

Is your birthday in Set1?
 1  3  5  7
 9 11 13 15
17 19 21 23
25 27 29 31
Enter 0 for No and 1 for Yes: 1 ↵ Enter

Is your birthday in Set2?
 2  3  6  7
10 11 14 15
18 19 22 23
26 27 30 31
Enter 0 for No and 1 for Yes: 1 ↵ Enter

Is your birthday in Set3?
 4  5  6  7
12 13 14 15
20 21 22 23
28 29 30 31
Enter 0 for No and 1 for Yes: 0 ↵ Enter

Is your birthday in Set4?
 8  9 10 11
12 13 14 15
24 25 26 27
28 29 30 31
Enter 0 for No and 1 for Yes: 0 ↵ Enter

```

```

Is your birthday in Set5?
16 17 18 19
20 21 22 23
24 25 26 27
28 29 30 31
Enter 0 for No and 1 for Yes: 1 
Your birthday is 19!

```

line#	day	answer	output
1	0		
10		1	
13	1		
22		1	
25	2		
34		0	
46		0	
58		1	
61	19		
63			Your birthday is 19



The last character `\` at the end of lines 4–8 is the line continuation symbol, which tells the interpreter that the statement is continued on the next line (see Section 2.3).

This game is easy to program. You may wonder how the game was created. The mathematics behind the game is actually quite simple. The numbers are not grouped together by accident. The way they are placed in the five sets is deliberate. The starting numbers in the five sets are **1**, **2**, **4**, **8**, and **16**, which correspond to **1**, **10**, **100**, **1000**, and **10000** in binary. A binary number for decimal integers between **1** and **31** has at most five digits, as shown in Figure 4.2a. Assume this number is $b_5b_4b_3b_2b_1$. So, $b_5b_4b_3b_2b_1 = b_50000 + b_4000 + b_300 + b_20 + b_1$, as shown in Figure 4.2b. If a day's binary number has a digit **1** in b_k , the number should appear in Set k . For example, number **19** is binary **10011**, so it appears in Set1, Set2, and Set5. It is binary **1** + **10** + **10000** = **10011** or decimal **1** + **2** + **16** = **19**. Number **31** is binary **11111**, so it appears in Set1, Set2, Set3, Set4, and Set5. It is binary **1** + **10** + **100** + **1000** + **10000** = **11111** or decimal **1** + **2** + **4** + **8** + **16** = **31**.

mathematics behind the game

Decimal	Binary
1	00001
2	00010
3	00011
...	
19	10011
...	
31	11111

(a)

b_5 0 0 0 0		10000
b_4 0 0 0		1000
b_3 0 0	10000	100
b_2 0	10	10
+ b_1	+ 1	+ 1
$b_5b_4b_3b_2b_1$	10011	11111
	19	31

(b)

FIGURE 4.2 (a) A number between **1** and **31** can be represented using a 5-digit binary number. (b) A 5-digit binary number can be obtained by adding binary numbers **1**, **10**, **100**, **1000**, or **10000**.

4.6 Two-Way if-else Statements



A two-way **if-else** statement decides which statements to execute based on whether the condition is true or false.

A one-way **if** statement takes an action if the specified condition is **True**. If the condition is **False**, nothing is done. But what if you want to take one or more alternative actions when the condition is **False**? You can use a two-way **if-else** statement. The actions that a two-way **if-else** statement specifies differ based on whether the condition is **True** or **False**.

Here is the syntax for a two-way **if-else** statement:

```
if boolean-expression:
    statement(s)-for-the-true-case
else:
    statement(s)-for-the-false-case
```

The flowchart of the statement is shown in Figure 4.3.

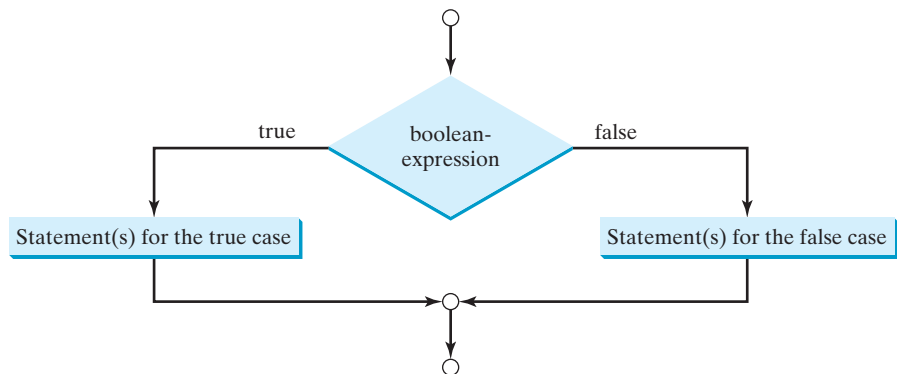


FIGURE 4.3 An **if-else** statement executes statements for the true case if the Boolean expression evaluates to **True**; otherwise, statements for the false case are executed.

If the **boolean-expression** evaluates to **True**, the statement(s) for the true case are executed; otherwise, the statement(s) for the false case are executed. For example, consider the following code:

two-way if-else statement

```
if radius >= 0:
    area = radius * radius * math.pi
    print("The area for the circle of radius", radius, "is", area)
else:
    print("Negative input")
```

If **radius >= 0** is true, **area** is computed and displayed; if it is false, the message **Negative input** is displayed.

Here is another example of the **if-else** statement. This one determines whether a number is even or odd, as follows:

```
if number % 2 == 0:
    print(number, "is even.")
else:
    print(number, "is odd.")
```

Suppose you want to develop a program for a first grader to practice subtraction. The program randomly generates two single-digit integers, **number1** and **number2**, with **number1 >= number2** and asks the student a question such as "What is 9 - 2?" After the student enters the answer, the program displays a message indicating whether it is correct.

The program may work as follows:

Step 1: Generate two single-digit integers for **number1** and **number2**.

Step 2: If **number1 < number2**, swap **number1** with **number2**.

Step 3: Prompt the student to answer, “What is number1 – number2?”

Step 4: Check the student’s answer and display whether the answer is correct.

The complete program is shown in Listing 4.4.

LISTING 4.4 SubtractionQuiz.py

```

1  import random                                import random module
2
3  # 1. Generate two random single-digit integers
4  number1 = random.randint(0, 9)                random numbers
5  number2 = random.randint(0, 9)
6
7  # 2. If number1 < number2, swap number1 with number2
8  if number1 < number2:                          swap if necessary
9      number1, number2 = number2, number1 # Simultaneous assignment
10
11 # 3. Prompt the student to answer "What is number1 - number2?"
12 answer = eval(input("What is " + str(number1) + " - " +      get answer
13                     str(number2) + "? "))
14
15 # 4. Check the answer and display the result
16 if number1 - number2 == answer:                  check the answer
17     print("You are correct!")
18 else:
19     print("Your answer is wrong.\n", number1, '-',
20           number2, "is", number1 - number2, '.')
```

What is 6 - 6? 0

You are correct!



What is 9 - 2? 5

Your answer is wrong.
9 - 2 is 7.



line#	number1	number2	answer	output
4	2			
5		9		
9	9	2		
12			5	
19				Your answer is wrong. 9 - 2 is 7.



If **number1 < number2**, the program uses simultaneous assignment to swap the two variables (lines 8–9).

4.9 Write an **if** statement that increases **pay** by 3% if **score** is greater than **90**, otherwise it increases **pay** by 1%.



MyProgrammingLab™ 4.10 What is the printout of the code in (a) and (b) if **number** is **30** and **35**, respectively?

```
if number % 2 == 0:
    print(number, "is even.")
print(number, "is odd.")
```

(a)

```
if number % 2 == 0:
    print(number, "is even.")
else:
    print(number, "is odd.")
```

(b)

4.7 Nested if and Multi-Way if-elif-else Statements



*One **if** statement can be placed inside another **if** statement to form a nested **if** statement.*

The statement in an **if** or **if-else** statement can be any legal Python statement, including another **if** or **if-else** statement. The inner **if** statement is said to be *nested* inside the outer **if** statement. The inner **if** statement can contain another **if** statement; in fact, there is no limit to the depth of the nesting. For example, the following is a nested **if** statement:

```
if i > k:
    if j > k:
        print("i and j are greater than k")
else:
    print("i is less than or equal to k")
```

The **if j > k** statement is nested inside the **if i > k** statement.

The nested **if** statement can be used to implement multiple alternatives. The statement given in Figure 4.4a, for instance, assigns a letter value to the variable **grade** according to the score, with multiple alternatives.

```
if score >= 90.0:
    grade = 'A'
else:
    if score >= 80.0:
        grade = 'B'
    else:
        if score >= 70.0:
            grade = 'C'
        else:
            if score >= 60.0:
                grade = 'D'
            else:
                grade = 'F'
```

(a)

Equivalent

This is better

```
if score >= 90.0:
    grade = 'A'
elif score >= 80.0:
    grade = 'B'
elif score >= 70.0:
    grade = 'C'
elif score >= 60.0:
    grade = 'D'
else:
    grade = 'F'
```

(b)

FIGURE 4.4 A preferred format for multiple alternatives is shown in (b) using a multi-way **if-elif-else** statement.

The execution of how this **if** statement proceeds is shown in Figure 4.5. The first condition (**score >= 90**) is tested. If it is **True**, the grade becomes **A**. If it is **False**, the second condition (**score >= 80**) is tested. If the second condition is **True**, the grade becomes **B**. If that condition is **False**, the third condition and the rest of the conditions (if necessary) are tested until a condition is met or all of the conditions prove to be **False**. If all of the conditions are **False**, the grade becomes **F**. Note that a condition is tested only when all of the conditions that come before it are **False**.

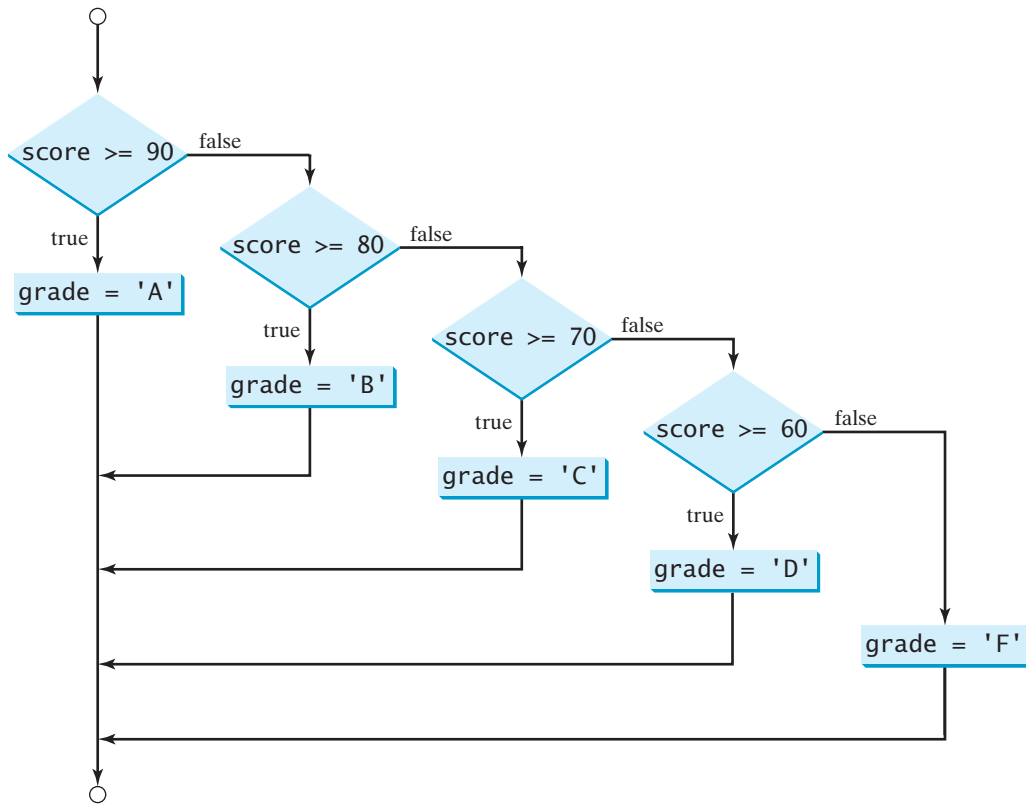


FIGURE 4.5 You can use a multi-way **if-elif-else** statement to assign a grade.

The **if** statement in Figure 4.4a is equivalent to the **if** statement in Figure 4.4b. In fact, Figure 4.4b is the preferred coding style for multiple alternative **if** statements. This style, called *multi-way if statements*, avoids deep indentation and makes the program easier to read. The multi-way **if** statements uses the syntax **if-elif-else; elif** (short for *else if*) is a Python keyword.

multi-way if statement

Now let's write a program to find out the Chinese zodiac sign for a given year. The Chinese zodiac sign is based on a 12-year cycle, and each year in this cycle is represented by an animal—monkey, rooster, dog, pig, rat, ox, tiger, rabbit, dragon, snake, horse, and sheep—as shown in Figure 4.6.

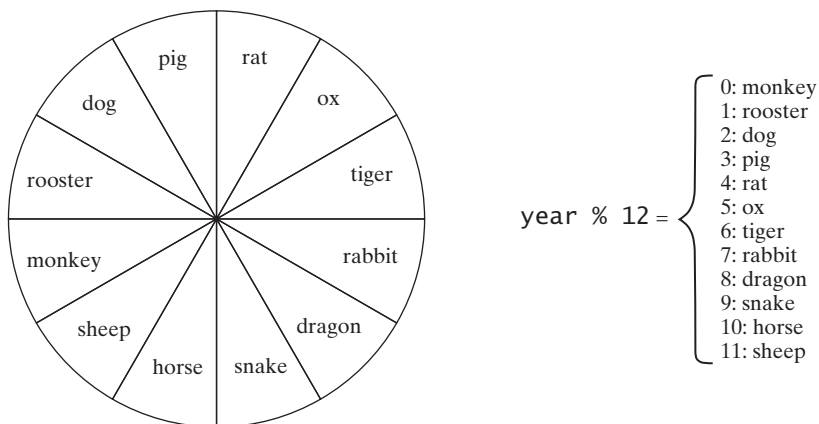


FIGURE 4.6 The Chinese zodiac is based on a 12-year cycle.

The value of `year % 12` determines the zodiac sign. 1900 is the year of the rat since `1900 % 12` is 4. Listing 4.5 shows a program that prompts the user to specify a year, and then it displays the animal for that year.

LISTING 4.5 ChineseZodiac.py

enter year
12-year cycle
determine zodiac sign

```
1 year = eval(input("Enter a year: "))
2 zodiacYear = year % 12
3 if zodiacYear == 0:
4     print("monkey")
5 elif zodiacYear == 1:
6     print("rooster")
7 elif zodiacYear == 2:
8     print("dog")
9 elif zodiacYear == 3:
10    print("pig")
11 elif zodiacYear == 4:
12    print("rat")
13 elif zodiacYear == 5:
14    print("ox")
15 elif zodiacYear == 6:
16    print("tiger")
17 elif zodiacYear == 7:
18    print("rabbit")
19 elif zodiacYear == 8:
20    print("dragon")
21 elif zodiacYear == 9:
22    print("snake")
23 elif zodiacYear == 10:
24    print("horse")
25 else:
26    print("sheep")
```



Enter a year: 1963
rabbit



Enter a year: 1877
ox



MyProgrammingLab™

4.11 Suppose `x = 3` and `y = 2`; show the output, if any, of the following code. What is the output if `x = 3` and `y = 4`? What is the output if `x = 2` and `y = 2`? Draw a flow-chart for the code.

```
if x > 2:
    if y > 2:
        z = x + y
        print("z is", z)
    else:
        print("x is", x)
```

4.12 Suppose `x = 2` and `y = 4`. Show the output, if any, of the following code. What is the output if `x = 3` and `y = 2`? What is the output if `x = 3` and `y = 3`? (Hint: Indent the statement correctly first.)

```

if x > 2:
    if y > 2:
        z = x + y
        print("z is", z)
else:
    print("x is", x)

```

4.13 What is wrong in the following code?

```

if score >= 60.0:
    grade = 'D'
elif score >= 70.0:
    grade = 'C'
elif score >= 80.0:
    grade = 'B'
elif score >= 90.0:
    grade = 'A'
else:
    grade = 'F'

```

4.8 Common Errors in Selection Statements

Most common errors in selection statements are caused by incorrect indentation.

Consider the following code in (a) and (b).



```

radius = -20

if radius >= 0:
    area = radius * radius * math.pi
print("The area is", area)

```

(a) Wrong

```

radius = -20

if radius >= 0:
    area = radius * radius * math.pi
    print("The area is", area)

```

(b) Correct

In (a), the **print** statement is not in the **if** block. To place it in the **if** block, you have to indent it, as shown in (b).

Consider another example in the following code in (a) and (b). The code in (a) below has two **if** clauses and one **else** clause. Which **if** clause is matched by the **else** clause? The indentation indicates that the **else** clause matches the first **if** clause in (a) and the second **if** clause in (b).

```

i = 1
j = 2
k = 3

if i > j:
    if i > k:
        print('A')
else:
    print('B')

```

(a)

```

i = 1
j = 2
k = 3

if i > j:
    if i > k:
        print('A')
    else:
        print('B')

```

(b)

Since **(i > j)** is false, the code in (a) displays **B**, but nothing is displayed from the statement in (b).

assign Boolean variable



Tip

Often new programmers write the code that assigns a test condition to a Boolean variable, like the code in (a):

```
if number % 2 == 0:
    even = True
else:
    even = False
```

(a)

Equivalent
This is shorter

```
even = number % 2 == 0
```

(b)

The code can be simplified by assigning the test value directly to the variable, as shown in (b).



MyProgrammingLab™

4.14 Which of the following statements are equivalent? Which ones are correctly indented?

```
if i > 0:
    x = 0
    y = 1
else:
    y = 0
    z = 0
```

(a)

```
if i > 0:
    x = 0
    y = 1
else:
    y = 0
    z = 0
```

(b)

```
if i > 0:
    x = 0
    y = 1
else:
    y = 0
    z = 0
```

(c)

```
if i > 0:
    x = 0
    y = 1
else:
    y = 0
    z = 0
```

(d)

4.15 Rewrite the following statement using a Boolean expression:

```
if count % 10 == 0:
    newLine = True
else:
    newLine = False
```

4.16 Are the following statements correct? Which one is better?

```
if age < 16:
    print("Cannot get a driver's license")
if age >= 16:
    print("Can get a driver's license")
```

(a)

```
if age < 16:
    print("Cannot get a driver's license")
else:
    print("Can get a driver's license")
```

(b)

4.17 What is the output of the following code if **number** is 14, 15, and 30?

```
if number % 2 == 0:
    print(number, "is even")
if number % 5 == 0:
    print(number, "is multiple of 5")
```

(a)

```
if number % 2 == 0:
    print(number, "is even")
elif number % 5 == 0:
    print(number, "is multiple of 5")
```

(b)

4.9 Case Study: Computing Body Mass Index

You can use nested **if** statements to write a program that interprets body mass index.



Body mass index (BMI) is a measure of health based on weight. It can be calculated by taking your weight in kilograms and dividing it by the square of your height in meters. The interpretation of BMI for people 16 years and older is as follows:

BMI	Interpretation
Below 18.5	Underweight
18.5–24.9	Normal
25.0–29.9	Overweight
Above 30.0	Obese

Write a program that prompts the user to enter a weight in pounds and height in inches and then displays the BMI. Note that one pound is **0.45359237** kilograms and one inch is **0.0254** meters. Listing 4.6 gives the program.

LISTING 4.6 ComputeBMI.py

```

1  # Prompt the user to enter weight in pounds
2  weight = eval(input("Enter weight in pounds: "))
3
4  # Prompt the user to enter height in inches
5  height = eval(input("Enter height in inches: "))
6
7  KILOGRAMS_PER_POUND = 0.45359237 # Constant
8  METERS_PER_INCH = 0.0254        # Constant
9
10 # Compute BMI
11 weightInKilograms = weight * KILOGRAMS_PER_POUND
12 heightInMeters = height * METERS_PER_INCH
13 bmi = weightInKilograms / (heightInMeters * heightInMeters)
14
15 # Display result
16 print("BMI is", format(bmi, ".2f"))
17 if bmi < 18.5:
18     print("Underweight")
19 elif bmi < 25:
20     print("Normal")
21 elif bmi < 30:
22     print("Overweight")
23 else:
24     print("Obese")

```

input weight

input height

compute bmi

display output

```

Enter weight in pounds: 146
Enter height in inches: 70
BMI is 20.95
Normal

```





line#	weight	height	weightInKilograms	heightInMeters	bmi	output
2	146					
5		70				
11			66.22448602			
12				1.778		
13					20.9486	
16						BMI is 20.95
22						Normal

named constants

The two named constants, `KILOGRAMS_PER_POUND` and `METERS_PER_INCH`, are defined in lines 7–8. Named constants were introduced in Section 2.6. Using named constants here makes programs easy to read. Unfortunately, there is no special syntax for defining named constants in Python. Named constants are treated just like variables in Python. This book uses the format of writing constants in all uppercase letters to distinguish them from variables, and separates the words in constants with an underscore (`_`).

4.10 Case Study: Computing Taxes



You can use nested `if` statements to write a program for computing taxes.

The United States federal personal income tax is calculated based on filing status and taxable income. There are four filing statuses: single filers, married filing jointly, married filing separately, and head of household. The tax rates vary every year. Table 4.2 shows the rates for 2009. If you are, say, single with a taxable income of \$10,000, the first \$8,350 is taxed at 10% and the other \$1,650 is taxed at 15%. So, your tax is \$1,082.50.

TABLE 4.2 2009 U.S. Federal Personal Tax Rates

Marginal Tax Rate	Single	Married Filing Jointly	Married Filing Separately	Head of Household
10%	\$0 – \$8,350	\$0 – \$16,700	\$0 – \$8,350	\$0 – \$11,950
15%	\$8,351 – \$33,950	\$16,701 – \$67,900	\$8,351 – \$33,950	\$11,951 – \$45,500
25%	\$33,951 – \$82,250	\$67,901 – \$137,050	\$33,951 – \$68,525	\$45,501 – \$117,450
28%	\$82,251 – \$171,550	\$137,051 – \$208,850	\$68,526 – \$104,425	\$117,451 – \$190,200
33%	\$171,551 – \$372,950	\$208,851 – \$372,950	\$104,426 – \$186,475	\$190,201 – \$372,950
35%	\$372,951+	\$372,951+	\$186,476+	\$372,951+

You are to write a program to compute personal income tax. Your program should prompt the user to enter the filing status and taxable income and then compute the tax. Enter **0** for single filers, **1** for married filing jointly, **2** for married filing separately, and **3** for head of household.

Your program computes the tax for the taxable income based on the filing status. The filing status can be determined using `if` statements outlined as follows:

```

if status == 0:
    # Compute tax for single filers
elif status == 1:
    # Compute tax for married filing jointly
elif status == 2:

```

```

    # Compute tax for married filing separately
elif status == 3:
    # Compute tax for head of household
else:
    # Display wrong status

```

For each filing status there are six tax rates. Each rate is applied to a certain amount of taxable income. For example, of a taxable income of \$400,000 for single filers, \$8,350 is taxed at 10%, (33,950 – 8,350) at 15%, (82,250 – 33,950) at 25%, (171,550 – 82,250) at 28%, (372,950 – 171,550) at 33%, and (400,000 – 372,950) at 35%.

Listing 4.7 gives the solution to compute taxes for single filers. The complete solution is left as Programming Exercise 4.13 at the end of this chapter.

LISTING 4.7 ComputeTax.py

```

1  import sys                                import sys module
2
3  # Prompt the user to enter filing status
4  status = eval(input(                      input status
5      "(0-single filer, 1-married jointly,\n" +
6      "2-married separately, 3-head of household)\n" +
7      "Enter the filing status: "))
8
9  # Prompt the user to enter taxable income
10 income = eval(input("Enter the taxable income: "))    input income
11
12 # Compute tax
13 tax = 0
14
15 if status == 0: # Compute tax for single filers        compute tax
16     if income <= 8350:
17         tax = income * 0.10
18     elif income <= 33950:
19         tax = 8350 * 0.10 + (income - 8350) * 0.15
20     elif income <= 82250:
21         tax = 8350 * 0.10 + (33950 - 8350) * 0.15 + \
22             (income - 33950) * 0.25
23     elif income <= 171550:
24         tax = 8350 * 0.10 + (33950 - 8350) * 0.15 + \
25             (82250 - 33950) * 0.25 + (income - 82250) * 0.28
26     elif income <= 372950:
27         tax = 8350 * 0.10 + (33950 - 8350) * 0.15 + \
28             (82250 - 33950) * 0.25 + (171550 - 82250) * 0.28 + \
29             (income - 171550) * 0.33
30     else:
31         tax = 8350 * 0.10 + (33950 - 8350) * 0.15 + \
32             (82250 - 33950) * 0.25 + (171550 - 82250) * 0.28 + \
33             (372950 - 171550) * 0.33 + (income - 372950) * 0.35;
34 elif status == 1: # Compute tax for married file jointly
35     print("Left as exercise")
36 elif status == 2: # Compute tax for married separately
37     print("Left as exercise")
38 elif status == 3: # Compute tax for head of household
39     print("Left as exercise")
40 else:
41     print("Error: invalid status")
42     sys.exit()                                exit program
43
44 # Display the result
45 print("Tax is", format(tax, ".2f"))          display output

```



```
(0-single filer, 1-married jointly,
2-married separately, 3-head of household)
Enter the filing status: 0 
Enter the taxable income: 400000 
Tax is 117683.50
```



line#	status	income	tax	output
4	0			
10		400000		
13			0	
17			117683.5	
45				Tax is 117683.50

The program receives the filing status and taxable income. The multiple alternative **if** statements (lines 15, 34, 36, 38, and 40) check the filing status and compute the tax based on the filing status.

`sys.exit()`

sys.exit() (line 42) is defined in the **sys** module. Invoking this function terminates the program.

test all cases

To test a program, you need to provide input that covers all cases. For this program, your input should cover all statuses (0, 1, 2, and 3). For each status, test the tax for each of the six brackets. So, there are a total of 24 cases.



Tip

For all programs, you should write a small amount of code and test it before moving on to add more code. This is called *incremental development and testing*. This approach makes debugging easier, because the errors are likely in the new code you just added.

incremental development and testing



MyProgrammingLab™

4.18 Are the following two statements equivalent?

```
if income <= 10000:
    tax = income * 0.1
elif income <= 20000:
    tax = 1000 + \
        (income - 10000) * 0.15
```

```
if income <= 10000:
    tax = income * 0.1
elif income > 10000 and
    income <= 20000:
    tax = 1000 + \
        (income - 10000) * 0.15
```

4.19 What is wrong in the following code?

```
income = 232323

if income <= 10000:
    tax = income * 0.1
elif income > 10000 and income <= 20000:
    tax = 1000 + (income - 10000) * 0.15

print(tax)
```

4.11 Logical Operators



The logical operators **not**, **and**, and **or** can be used to create a composite condition.

Sometimes, a combination of several conditions determines whether a statement is executed. You can use logical operators to combine these conditions to form a compound expression.

Logical operators, also known as *Boolean operators*, operate on Boolean values to create a new Boolean value. Table 4.3 lists the Boolean operators. Table 4.4 defines the **not** operator, which negates **True** to **False** and **False** to **True**. Table 4.5 defines the **and** operator. The **and** of two Boolean operands is true if and only if both operands are true. Table 4.6 defines the **or** operator. The **or** of two Boolean operands is true if at least one of the operands is true.

TABLE 4.3 Boolean Operators

Operator	Description
not	logical negation
and	logical conjunction
or	logical disjunction

TABLE 4.4 Truth Table for Operator **not**

p	not p	Example (assume age = 24, gender = 'F')
True	False	not (age > 18) is False , because (age > 18) is True .
False	True	not (gender == 'M') is True , because (gender == 'M') is False .

TABLE 4.5 Truth Table for Operator **and**

p ₁	p ₂	p ₁ and p ₂	Example (assume age = 24, gender = 'F')
False	False	False	(age > 18) and (gender == 'F') is True , because (age > 18) and (gender == 'F') are both True .
False	True	False	
True	False	False	(age > 18) and (gender != 'F') is False , because (gender != 'F') is False .
True	True	True	

TABLE 4.6 Truth Table for Operator **or**

p ₁	p ₂	p ₁ and p ₂	Example (assume age = 24, gender = 'F')
False	False	False	(age > 34) or (gender == 'F') is True , because (gender == 'F') is True .
False	True	True	
True	False	True	(age > 34) or (gender == 'M') is False , because (age > 34) and (gender == 'M') are both False .
True	True	True	

The program in Listing 4.8 checks whether a number is divisible by **2 and 3**, by **2 or 3**, and by **2 or 3** but *not both*.

LISTING 4.8 TestBooleanOperators.py

```

1 # Receive an input
2 number = eval(input("Enter an integer: "))
3
4 if number % 2 == 0 and number % 3 == 0:
5     print(number, "is divisible by 2 and 3")

```

input
and

```

6
7 if number % 2 == 0 or number % 3 == 0:
8     print(number, "is divisible by 2 or 3")
9
10 if (number % 2 == 0 or number % 3 == 0) and \
11     not (number % 2 == 0 and number % 3 == 0):
12     print(number, "is divisible by 2 or 3, but not both")

```



```

Enter an integer: 18 Enter
18 is divisible by 2 and 3
18 is divisible by 2 or 3

```



```

Enter an integer: 15 Enter
15 is divisible by 2 or 3
15 is divisible by 2 or 3, but not both

```

In line 4, `number % 2 == 0 and number % 3 == 0` checks whether the number is divisible by 2 and 3. `number % 2 == 0 or number % 3 == 0` (line 7) checks whether the number is divisible by 2 or 3. The Boolean expression in lines 10–11

```

(number % 2 == 0 and number % 3 == 0) and
not (number % 2 == 0 and number % 3 == 0)

```

checks whether the number is divisible by 2 or 3 but not both.



Note

De Morgan's law, named after Indian-born British mathematician and logician Augustus De Morgan (1806–1871), can be used to simplify Boolean expressions. The law states that:

```

not (condition1 and condition2) is the same as
not condition1 or not condition2
not (condition1 or condition2) is the same as
not condition1 and not condition2

```

So, line 11 in the preceding example,

```
not (number % 2 == 0 and number % 3 == 0)
```

can be simplified by using an equivalent expression:

```
(number % 2 != 0 or number % 3 != 0)
```

As another example,

```
not (number == 2 or number == 3)
```

is better written as

```
number != 2 and number != 3
```

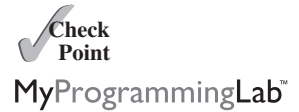
If one of the operands of an `and` operator is `False`, the expression is `False`; if one of the operands of an `or` operator is `True`, the expression is `True`. Python uses these properties to improve the performance of these operators. When evaluating `p1 and p2`, Python first evaluates `p1` and then, if `p1` is `True`, evaluates `p2`; if `p1` is `False`, it does not evaluate `p2`. When evaluating `p1 or p2`, Python first evaluates `p1` and then, if `p1` is `False`,

evaluates **p2**; if **p1** is **True**, it does not evaluate **p2**. Therefore, **and** is referred to as the *conditional* or *short-circuit AND* operator, and **or** is referred to as the *conditional* or *short-circuit OR* operator.

conditional operator
short-circuit evaluation

4.20 Assuming that **x** is **1**, show the result of the following Boolean expressions.

```
True and (3 > 4)
not (x > 0) and (x > 0)
(x > 0) or (x < 0)
(x != 0) or (x == 0)
(x >= 0) or (x < 0)
(x != 1) == not (x == 1)
```



4.21 Write a Boolean expression that evaluates to **True** if variable **num** is between **1** and **100**.

4.22 Write a Boolean expression that evaluates to **True** if variable **num** is between **1** and **100** or the number is negative.

4.23 Assuming **x** = 4 and **y** = 5, show the result of the following Boolean expressions:

```
x >= y >= 0
x <= y >= 0
x != y == 5
(x != 0) or (x == 0)
```

4.24 Are the following expressions equivalent?

- (a) `(x >= 1) and (x < 10)`
- (b) `(1 <= x < 10)`

4.25 What is the value of the expression `ch >= 'A' and ch <= 'Z'` if **ch** is **'A'**, **'p'**, **'E'**, or **'5'**?

4.26 Suppose, when you run the following program, you enter input **2**, **3**, **6** from the console. What is the output?

```
x, y, z = eval(input("Enter three numbers: "))

print("(x < y and y < z) is", x < y and y < z)
print("(x < y or y < z) is", x < y or y < z)
print("not (x < y) is", not (x < y))
print("(x < y < z) is", x < y < z)
print("not(x < y < z) is", not (x < y < z))
```

4.27 Write a Boolean expression that evaluates true if **age** is greater than **13** and less than **18**.

4.28 Write a Boolean expression that evaluates true if **weight** is greater than **50** or **height** is greater than **160**.

4.29 Write a Boolean expression that evaluates true if **weight** is greater than **50** and **height** is greater than **160**.

4.30 Write a Boolean expression that evaluates true if either **weight** is greater than **50** or **height** is greater than **160**, but not both.

4.12 Case Study: Determining Leap Years

A year is a leap year if it is divisible by 4 but not by 100 or if it is divisible by 400.

You can use the following Boolean expressions to determine whether a year is a leap year:

```
# A leap year is divisible by 4
isLeapYear = (year % 4 == 0)
# A leap year is divisible by 4 but not by 100
isLeapYear = isLeapYear and (year % 100 != 0)
```



```
# A leap year is divisible by 4 but not by 100 or divisible by 400
isLeapYear = isLeapYear or (year % 400 == 0)
```

or you can combine all these expressions into one, like this:

```
isLeapYear = (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)
```

Listing 4.9 is an example of a program that lets the user enter a year and then determines whether it is a leap year.

LISTING 4.9 LeapYear.py

input	1 year = eval(input("Enter a year: "))
	2
	3 # Check if the year is a leap year
leap year?	4 isLeapYear = (year % 4 == 0 and year % 100 != 0) or \
	5 (year % 400 == 0)
	6
	7 # Display the result
display result	8 print(year, "is a leap year?", isLeapYear)



```
Enter a year: 2008
2008 is a leap year? True
```



```
Enter a year: 1900
1900 is a leap year? False
```



```
Enter a year: 2002
2002 is a leap year? False
```

4.13 Case Study: Lottery



The lottery program in this case study involves generating random numbers, comparing digits, and using Boolean operators.

Suppose you want to develop a program to play a lottery. The program randomly generates a two-digit number, prompts the user to enter a two-digit number, and determines whether the user wins according to the following rules:

1. If the user's input matches the lottery in the exact order, the award is \$10,000.
2. If all the digits in the user's input match all the digits in the lottery number, the award is \$3,000.
3. If one digit in the user's input matches a digit in the lottery number, the award is \$1,000.

The complete program is shown in Listing 4.10.

LISTING 4.10 Lottery.py

generate a lottery	1 import random
	2
	3 # Generate a lottery number
	4 lottery = random.randint(0, 99)
	5
	6 # Prompt the user to enter a guess
enter a guess	7 guess = eval(input("Enter your lottery pick (two digits): "))

```
8
9 # Get digits from lottery
10 lotteryDigit1 = lottery // 10
11 lotteryDigit2 = lottery % 10
12
13 # Get digits from guess
14 guessDigit1 = guess // 10
15 guessDigit2 = guess % 10
16
17 print("The lottery number is", lottery)
18
19 # Check the guess
20 if guess == lottery:
21     print("Exact match: you win $10,000")
22 elif (guessDigit2 == lotteryDigit1 and \
23       guessDigit1 == lotteryDigit2):
24     print("Match all digits: you win $3,000")
25 elif (guessDigit1 == lotteryDigit1
26       or guessDigit1 == lotteryDigit2
27       or guessDigit2 == lotteryDigit1
28       or guessDigit2 == lotteryDigit2):
29     print("Match one digit: you win $1,000")
30 else:
31     print("Sorry, no match")
```

exact match?
match all digits?
match one digit?

Enter your lottery pick (two digits): 45

The lottery number is 12

Sorry, no match



Enter your lottery pick (two digits): 23

The lottery number is 34

Match one digit: you win \$1,000



line#	4	7	10	11	14	15	29
variable							
lottery	34						
guess		23					
lotteryDigit1			3				
lotteryDigit2				4			
guessDigit1					2		
guessDigit2						3	
output							Match one digit: you win \$1,000



The program generates a lottery number using the `random.randint(0, 99)` function (line 4) and prompts the user to enter a guess (line 7). Note that `guess % 10` obtains the last digit from `guess` and `guess // 10` obtains the first digit from `guess`, since `guess` is a two-digit number (lines 14–15).

The program checks the guess against the lottery number in this order:

1. First check whether the guess matches the lottery number exactly (line 20).
2. If not, check whether the reversal of the guess matches the lottery number (lines 22–23).
3. If not, check whether one digit is in the lottery number (lines 25–28).
4. If not, nothing matches and display **Sorry, no match** (lines 30–31).

4.14 Conditional Expressions



A conditional expression evaluates an expression based on a condition.

You might want to assign a value to a variable that is restricted by certain conditions. For example, the following statement assigns **1** to **y** if **x** is greater than **0**, and **-1** to **y** if **x** is less than or equal to **0**.

```
if x > 0:
    y = 1
else:
    y = -1
```

Alternatively, as in this next example, you can use a conditional expression to achieve the same result.

```
y = 1 if x > 0 else -1
```

conditional expression

Conditional expressions are in a completely different style. The syntax is:

```
expression1 if boolean-expression else expression2
```

The result of this conditional expression is **expression1** if **boolean-expression** is true; otherwise, the result is **expression2**.

Suppose you want to assign the larger number of variables **number1** and **number2** to **max**. You can simply write a statement using the conditional expression:

```
max = number1 if number1 > number2 else number2
```

For another example, the following statement displays the message **number is even** if **number** is even, and otherwise displays **number is odd**.

```
print("number is even" if number % 2 == 0 else "number is odd")
```



MyProgrammingLab™

4.31 Suppose that when you run the following program you enter the input **2, 3, 6** from the console. What is the output?

```
x, y, z = eval(input("Enter three numbers: "))
print("sorted" if x < y and y < z else "not sorted")
```

4.32 Rewrite the following **if** statements using a conditional expression:

```
if ages >= 16:
    ticketPrice = 20
else:
    ticketPrice = 10
```

```
if count % 10 == 0:
    print(count)
else:
    print(count, end = " ")
```

4.33 Rewrite the following conditional expressions using **if/else** statements:

- (a) `score = 3 * scale if x > 10 else 4 * scale`
- (b) `tax = income * 0.2 if income > 10000 else income * 0.17 + 1000`
- (c) `print(i if number % 3 == 0 else j)`

4.15 Operator Precedence and Associativity

Operator precedence and associativity determine the order in which operators are evaluated.



Operator precedence and operator associativity determine the order in which Python evaluates operators. Suppose that you have this expression:

`3 + 4 * 4 > 5 * (4 + 3) - 1`

What is its value? What is the execution order of the operators?

Arithmetically, the expression in the parentheses is evaluated first. (Parentheses can be nested, in which case the expression in the inner parentheses is executed first.) When evaluating an expression without parentheses, the operators are applied according to the precedence rule and the associativity rule.

The precedence rule defines precedence for operators. Table 4.7 contains the operators you have learned so far, with the operators listed in decreasing order of precedence from top to bottom. The logical operators have lower precedence than the relational operators and the relational operators have lower precedence than the arithmetic operators. Operators with the same precedence appear in the same group.

operator precedence
operator associativity

precedence

TABLE 4.7 Operator Precedence Chart

Precedence	Operator
	<code>+, -</code> (Unary plus and minus)
	<code>**</code> (Exponentiation)
	<code>not</code>
	<code>*, /, //, %</code> (Multiplication, division, integer division, and remainder)
	<code>+, -</code> (Binary addition and subtraction)
	<code><, <=, >, >=</code> (Comparison)
	<code>==, !=</code> (Equality)
	<code>and</code>
	<code>or</code>
	<code>=, +=, -=, *=, /=, //=, %=</code> (Assignment operators)

If operators with the same precedence are next to each other, their *associativity* determines the order of evaluation. All binary operators are *left-associative*. For example, since `+` and `-` are of the same precedence and are left-associative, the expression

associativity

$$a - b + c - d \text{ is equivalent to } ((a - b) + c) - d$$



Note

Python has its own way to evaluate an expression internally. The result of a Python evaluation is the same as that of its corresponding arithmetic evaluation.

behind the scenes

4.34 List the precedence order of the Boolean operators. Evaluate the following expressions:

`True or True and False`
`True and True or False`

4.35 True or false? All the binary operators except `=` are left-associative.



MyProgrammingLab™

4.36 Evaluate the following expressions:

$2 * 2 - 3 > 2$ and $4 - 2 > 5$
 $2 * 2 - 3 > 2$ or $4 - 2 > 5$

4.37 Is $(x > 0$ and $x < 10)$ the same as $((x > 0)$ and $(x < 10))$? Is $(x > 0$ or $x < 10)$ the same as $((x > 0)$ or $(x < 10))$? Is $(x > 0$ or $x < 10$ and $y < 0)$ the same as $(x > 0$ or $(x < 10$ and $y < 0))$?

4.16 Detecting the Location of an Object



Detecting whether an object is inside another object is a common task in game programming.

In game programming, often you need to determine whether an object is inside another object. This section gives an example of testing whether a point is inside a circle. The program prompts the user to enter the center of a circle, the radius, and a point. The program then displays the circle and the point along with a message indicating whether the point is inside or outside the circle, as shown in Figure 4.7a–b.

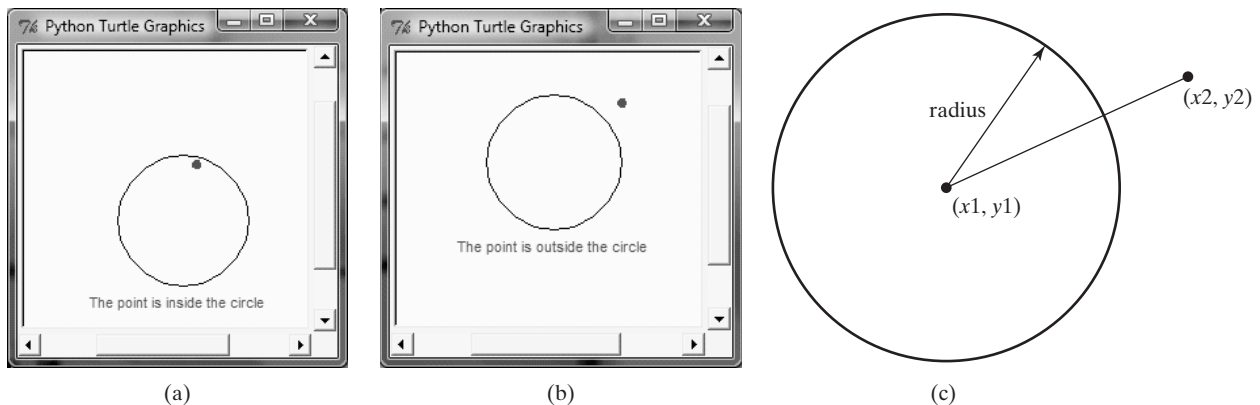


FIGURE 4.7 The program displays a circle, a point, and a message indicating whether the point is inside or outside the circle.

A point is in the circle if its distance to the center of the circle is less than or equal to the radius of the circle, as shown in Figure 4.7c. The formula for computing the distance is $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. Listing 4.11 gives the program.

LISTING 4.11 PointInCircle.py

```
import turtle

enter input
1  import turtle
2
3  x1, y1 = eval(input("Enter the center of a circle x, y: "))
4  radius = eval(input("Enter the radius of the circle: "))
5  x2, y2 = eval(input("Enter a point x, y: "))
6
7  # Draw the circle
8  turtle.penup()          # Pull the pen up
9  turtle.goto(x1, y1 - radius)
10 turtle.pendown()        # Pull the pen down
11 turtle.circle(radius)
12 # Draw the point
```

```

13 turtle.penup()          # Pull the pen up
14 turtle.goto(x2, y2)
15 turtle.pendown()        # Pull the pen down
16 turtle.begin_fill()     # Begin to fill color in a shape
17 turtle.color("red")
18 turtle.circle(3)
19 turtle.end_fill()       # Fill the shape
20
21 # Display the status
22 turtle.penup()          # Pull the pen up
23 turtle.goto(x1 - 70, y1 - radius - 20)
24 turtle.pendown()
25
26 d = ((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1)) ** 0.5
27 if d <= radius:
28     turtle.write("The point is inside the circle")
29 else:
30     turtle.write("The point is outside the circle")
31
32 turtle.hideturtle()
33
34 turtle.done()

```

draw a point

compute distance

in the circle

not in the circle

pause

The program obtains the circle's center location and radius (lines 3–4) and the location of a point (line 5). It displays the circle (lines 8–11) and the point (lines 13–19). The program computes the distance between the center of the circle and the point (line 26) and determines whether the point is inside or outside the circle.

The code in lines 16–19 draws a dot, which can be simplified using the **dot** method, presented in Table 3.6, as follows:

```
turtle.dot(6, "red")
```

This method draws a red dot with diameter 6.

KEY TERMS

Boolean expressions	92	random module	94
Boolean value	92	selection statements	92
operator associativity	117	short-circuit evaluation	113
operator precedence	117		

CHAPTER SUMMARY

1. A Boolean type variable can store a **True** or **False** value.
2. The relational operators (**<**, **<=**, **==**, **!=**, **>**, **>=**), which work with numbers and characters, yield a *Boolean value*.
3. The Boolean operators **and**, **or**, and **not** operate with Boolean values and variables.
4. When evaluating **p1 and p2**, Python first evaluates **p1** and then evaluates **p2** if **p1** is **True**; if **p1** is **False**, it does not evaluate **p2**. When evaluating **p1 or p2**, Python first evaluates **p1** and then evaluates **p2** if **p1** is **False**; if **p1** is **True**, it does not evaluate **p2**. Therefore, **and** is referred to as the conditional or *short-circuit AND operator*, and **or** is referred to as the conditional or short-circuit OR operator.

5. *Selection statements* are used for programming with alternative courses. There are several types of selection statements: **if** statements, **if-else** statements, nested **if-elif-else** statements, and conditional expressions.
6. The various **if** statements all make control decisions based on a *Boolean expression*. Based on the **True** or **False** evaluation of the expression, these statements take one of two possible courses.
7. The operators in arithmetic expressions are evaluated in the order determined by the rules of parentheses, *operator precedence*, and *operator associativity*.
8. Parentheses can be used to force the order of evaluation to occur in any sequence.
9. Operators with higher precedence are evaluated earlier. For operators of the same precedence, their associativity determines the order of evaluation.

TEST QUESTIONS

Do test questions for this chapter online at www.cs.armstrong.edu/liang/py/test.html.

MyProgrammingLab™

PROGRAMMING EXERCISES



Pedagogical Note

For each exercise, you should carefully analyze the problem requirements and design strategies for solving the problem before coding.

think before coding



Debugging Tip

Before you ask for help, read and explain the program to yourself, and trace it using several representative inputs by hand or using an IDE debugger. You learn how to program by debugging your own mistakes.

learn from mistakes

Section 4.2

- *4.1** (*Algebra: solve quadratic equations*) The two roots of a quadratic equation, for example, $ax^2 + bx + c = 0$, can be obtained using the following formula:

$$r_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad \text{and} \quad r_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

$b^2 - 4ac$ is called the discriminant of the quadratic equation. If it is positive, the equation has two real roots. If it is zero, the equation has one root. If it is negative, the equation has no real roots.

Write a program that prompts the user to enter values for a , b , and c and displays the result based on the discriminant. If the discriminant is positive, display two roots. If the discriminant is **0**, display one root. Otherwise, display **The equation has no real roots**. Here are some sample runs.

Enter a, b, c: 1.0, 3, 1 ↵ Enter
The roots are -0.381966 and -2.61803



Enter a, b, c: 1, 2.0, 1 ↵ Enter
The root is -1



Enter a, b, c: 1, 2, 3 ↵ Enter
The equation has no real roots



- *4.2** (*Game: add three numbers*) The program in Listing 4.1 generates two integers and prompts the user to enter the sum of these two integers. Revise the program to generate three single-digit integers and prompt the user to enter the sum of these three integers.

Sections 4.3–4.8

- *4.3** (*Algebra: solve 2×2 linear equations*) You can use Cramer's rule to solve the following 2×2 system of linear equation:

$$\begin{array}{l} ax + by = e \\ cx + dy = f \end{array} \quad x = \frac{ed - bf}{ad - bc} \quad y = \frac{af - ec}{ad - bc}$$

Write a program that prompts the user to enter **a**, **b**, **c**, **d**, **e**, and **f** and display the result. If $ad - bc$ is 0, report that **The equation has no solution**.

Enter a, b, c, d, e, f: 9.0, 4.0, 3.0, -5.0, -6.0, -21.0 ↵ Enter
x is -2.0 and y is 3.0



Enter a, b, c, d, e, f: 1.0, 2.0, 2.0, 4.0, 4.0, 5.0 ↵ Enter
The equation has no solution



- **4.4** (*Game: learn addition*) Write a program that generates two integers under 100 and prompts the user to enter the sum of these two integers. The program then reports true if the answer is correct, false otherwise. The program is similar to Listing 4.1.
- *4.5** (*Find future dates*) Write a program that prompts the user to enter an integer for today's day of the week (Sunday is 0, Monday is 1, ..., and Saturday is 6). Also prompt the user to enter the number of days after today for a future day and display the future day of the week. Here is a sample run:

Enter today's day: 1 ↵ Enter
Enter the number of days elapsed since today: 3 ↵ Enter
Today is Monday and the future day is Thursday





```
Enter today's day: 0 ↵ Enter
Enter the number of days elapsed since today: 31 ↵ Enter
Today is Sunday and the future day is Wednesday
```

- *4.6** (*Health application: BMI*) Revise Listing 4.6, `ComputeBMI.py`, to let users enter their weight in pounds and their height in feet and inches. For example, if a person is 5 feet and 10 inches, you will enter 5 for feet and 10 for inches. Here is a sample run:



```
Enter weight in pounds: 140 ↵ Enter
Enter feet: 5 ↵ Enter
Enter inches: 10 ↵ Enter
BMI is 20.087702275404553
You are Normal
```

- 4.7** (*Financial application: monetary units*) Modify Listing 3.4, `ComputeChange.py`, to display the nonzero denominations only, using singular words for single units such as 1 dollar and 1 penny, and plural words for more than one unit such as 2 dollars and 3 pennies.
- *4.8** (*Sort three integers*) Write a program that prompts the user to enter three integers and displays them in increasing order.
- *4.9** (*Financial: compare costs*) Suppose you shop for rice and find it in two different-sized packages. You would like to write a program to compare the costs of the packages. The program prompts the user to enter the weight and price of each package and then displays the one with the better price. Here is a sample run:



```
Enter weight and price for package 1: 50, 24.59 ↵ Enter
Enter weight and price for package 2: 25, 11.99 ↵ Enter
Package 1 has the better price.
```

- 4.10** (*Game: multiplication quiz*) Listing 4.4, `SubtractionQuiz.py`, randomly generates a subtraction question. Revise the program to randomly generate a multiplication question with two integers less than 100.

Sections 4.9–4.16

- *4.11** (*Find the number of days in a month*) Write a program that prompts the user to enter the month and year and displays the number of days in the month. For example, if the user entered month 2 and year 2000, the program should display that February 2000 has 29 days. If the user entered month 3 and year 2005, the program should display that March 2005 has 31 days.
- 4.12** (*Check a number*) Write a program that prompts the user to enter an integer and checks whether the number is divisible by both 5 and 6, divisible by 5 or 6, or just one of them (but not both). Here is a sample run:



```
Enter an integer: 10 ↵ Enter
Is 10 divisible by 5 and 6? False
Is 10 divisible by 5 or 6? True
Is 10 divisible by 5 or 6, but not both? True
```



VideoNote
Coffee price

***4.13** (*Financial application: compute taxes*) Listing 4.7, `ComputeTax.py`, gives the source code to compute taxes for single filers. Complete Listing 4.7 to give the complete source code for the other filing statuses.

4.14 (*Game: heads or tails*) Write a program that lets the user guess whether a flipped coin displays the head or the tail. The program randomly generates an integer **0** or **1**, which represents head or tail. The program prompts the user to enter a guess and reports whether the guess is correct or incorrect.

****4.15** (*Game: lottery*) Revise Listing 4.10, `Lottery.py`, to generate a three-digit lottery number. The program prompts the user to enter a three-digit number and determines whether the user wins according to the following rules:

1. If the user input matches the lottery number in the exact order, the award is \$10,000.
2. If all the digits in the user input match all the digits in the lottery number, the award is \$3,000.
3. If one digit in the user input matches a digit in the lottery number, the award is \$1,000.

4.16 (*Random character*) Write a program that displays a random uppercase letter.

***4.17** (*Game: scissor, rock, paper*) Write a program that plays the popular scissor-rock-paper game. (A scissor can cut a paper, a rock can knock a scissor, and a paper can wrap a rock.) The program randomly generates a number **0**, **1**, or **2** representing scissor, rock, and paper. The program prompts the user to enter a number **0**, **1**, or **2** and displays a message indicating whether the user or the computer wins, loses, or draws. Here are sample runs:

```
scissor (0), rock (1), paper (2): 1 ↵ Enter
The computer is scissor. You are rock. You won.
```



```
scissor (0), rock (1), paper (2): 2 ↵ Enter
The computer is paper. You are paper too. It is a draw.
```



***4.18** (*Financials: currency exchange*) Write a program that prompts the user to enter the currency exchange rate between U.S. dollars and Chinese Renminbi (RMB). Prompt the user to enter **0** to convert from U.S. dollars to Chinese RMB and **1** for vice versa. Prompt the user to enter the amount in U.S. dollars or Chinese RMB to convert it to Chinese RMB or U.S. dollars, respectively. Here are some sample runs:

```
Enter the exchange rate from dollars to RMB: 6.81 ↵ Enter
Enter 0 to convert dollars to RMB and 1 vice versa: 0 ↵ Enter
Enter the dollar amount: 100 ↵ Enter
$100.0 is 681.0 yuan
```



```
Enter the exchange rate from dollars to RMB: 6.81 ↵ Enter
Enter 0 to convert dollars to RMB and 1 vice versa: 1 ↵ Enter
Enter the RMB amount: 10000 ↵ Enter
10000.0 yuan is $1468.43
```





```
Enter the exchange rate from dollars to RMB: 6.81 ↵ Enter
Enter 0 to convert dollars to RMB and 1 vice versa: 5 ↵ Enter
Incorrect input
```

- **4.19** (*Compute the perimeter of a triangle*) Write a program that reads three edges for a triangle and computes the perimeter if the input is valid. Otherwise, display that the input is invalid. The input is valid if the sum of every pair of two edges is greater than the remaining edge. Here is a sample run:



```
Enter three edges: 1, 1, 1 ↵ Enter
The perimeter is 3
```



```
Enter three edges: 1, 3, 1 ↵ Enter
The input is invalid
```

- *4.20** (*Science: wind-chill temperature*) Exercise 2.9 gives a formula to compute the wind-chill temperature. The formula is valid for temperatures in the range between -58°F and 41°F and for wind speed greater than or equal to 2. Write a program that prompts the user to enter a temperature and a wind speed. The program displays the wind-chill temperature if the input is valid; otherwise, it displays a message indicating whether the temperature and/or wind speed is invalid.

Comprehensive

- **4.21** (*Science: day of the week*) Zeller's congruence is an algorithm developed by Christian Zeller to calculate the day of the week. The formula is

$$h = \left(q + \left\lfloor \frac{26(m+1)}{10} \right\rfloor + k + \left\lfloor \frac{k}{4} \right\rfloor + \left\lfloor \frac{j}{4} \right\rfloor + 5j \right) \% 7$$

where

- **h** is the day of the week (0: Saturday, 1: Sunday, 2: Monday, 3: Tuesday, 4: Wednesday, 5: Thursday, 6: Friday).
- **q** is the day of the month.
- **m** is the month (3: March, 4: April, ..., 12: December). January and February are counted as months 13 and 14 of the previous year.
- **j** is the century (i.e., $\left\lfloor \frac{\text{year}}{100} \right\rfloor$).
- **k** is the year of the century (i.e., $\text{year} \% 100$).

Write a program that prompts the user to enter a year, month, and day of the month, and then it displays the name of the day of the week. Here are some sample runs:



```
Enter year: (e.g., 2008): 2013 ↵ Enter
Enter month: 1-12: 1 ↵ Enter
Enter the day of the month: 1-31: 25 ↵ Enter
Day of the week is Friday
```

```

Enter year: (e.g., 2008): 2012 Enter
Enter month: 1-12: 5 Enter
Enter the day of the month: 1-31: 12 Enter
Day of the week is Saturday
    
```



(Hint: $\lfloor n \rfloor = n // 1$ for a positive n . January and February are counted as 13 and 14 in the formula, so you need to convert the user input 1 to 13 and 2 to 14 for the month and change the year to the previous year.)

****4.22** (Geometry: point in a circle?) Write a program that prompts the user to enter a point (x, y) and checks whether the point is within the circle centered at $(0, 0)$ with radius 10. For example, $(4, 5)$ is inside the circle and $(9, 9)$ is outside the circle, as shown in Figure 4.8a.

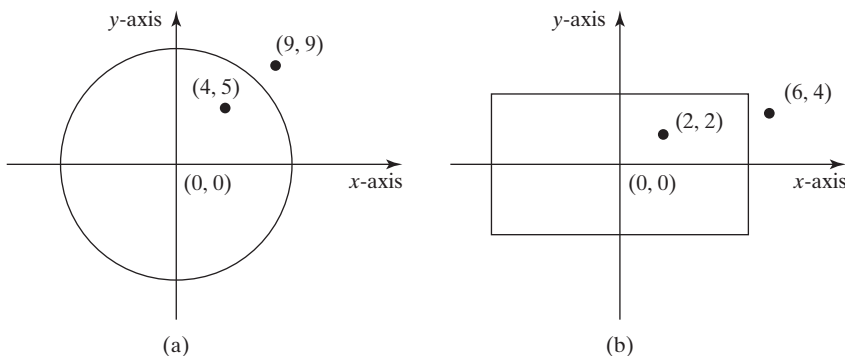


FIGURE 4.8 (a) Points inside and outside of the circle; (b) points inside and outside of the rectangle.

(Hint: A point is in the circle if its distance to $(0, 0)$ is less than or equal to 10. The formula for computing the distance is $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. Test your program to cover all cases.) Two sample runs are shown next.

```

Enter a point with two coordinates: 4, 5 Enter
Point (4.0, 5.0) is in the circle
    
```



```

Enter a point with two coordinates: 9, 9 Enter
Point (9.0, 9.0) is not in the circle
    
```



****4.23** (Geometry: point in a rectangle?) Write a program that prompts the user to enter a point (x, y) and checks whether the point is within the rectangle centered at $(0, 0)$ with width 10 and height 5. For example, $(2, 2)$ is inside the rectangle and $(6, 4)$ is outside the rectangle, as shown in Figure 4.8b. (Hint: A point is in the rectangle if its horizontal distance to $(0, 0)$ is less than or equal to $10 / 2$ and its vertical distance to $(0, 0)$ is less than or equal to $5.0 / 2$. Test your program to cover all cases.) Here are two sample runs:

```

Enter a point with two coordinates: 2, 2 Enter
Point (2.0, 2.0) is in the rectangle
    
```





```
Enter a point with two coordinates: 6, 4
Point (6.0, 4.0) is not in the rectangle
```

- **4.24** (*Game: pick a card*) Write a program that simulates picking a card from a deck of 52 cards. Your program should display the rank (Ace, 2, 3, 4, 5, 6, 7, 8, 9, 10, Jack, Queen, King) and suit (Clubs, Diamonds, Hearts, Spades) of the card. Here is a sample run of the program:



```
The card you picked is the Jack of Hearts
```

- *4.25** (*Geometry: intersecting point*) Two points on line 1 are given as (x_1, y_1) and (x_2, y_2) and on line 2 as (x_3, y_3) and (x_4, y_4) , as shown in Figure 4.9a–b.

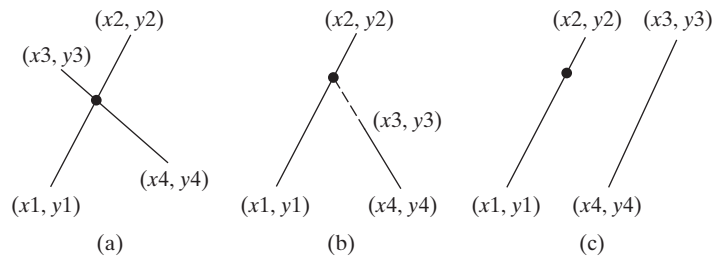


FIGURE 4.9 Two lines intersect in (a–b) and two lines are parallel in (c).

The intersecting point of the two lines can be found by solving the following linear equation:

$$(y_1 - y_2)x - (x_1 - x_2)y = (y_1 - y_2)x_1 - (x_1 - x_2)y_1$$

$$(y_3 - y_4)x - (x_3 - x_4)y = (y_3 - y_4)x_3 - (x_3 - x_4)y_3$$

This linear equation can be solved using Cramer's rule (see Exercise 4.3). If the equation has no solutions, the two lines are parallel (Figure 4.9c). Write a program that prompts the user to enter four points and displays the intersecting point. Here are sample runs:



```
Enter x1, y1, x2, y2, x3, y3, x4, y4:
2, 2, 5, -1, 4, 2, -1, -2
The intersecting point is at (2.88889, 1.1111)
```



```
Enter x1, y1, x2, y2, x3, y3, x4, y4:
2, 2, 7, 6, 4, 2, -1, -2
The two lines are parallel
```

- 4.26** (*Palindrome number*) Write a program that prompts the user to enter a three-digit integer and determines whether it is a palindrome number. A number is a palindrome

if it reads the same from right to left and from left to right. Here is a sample run of this program:

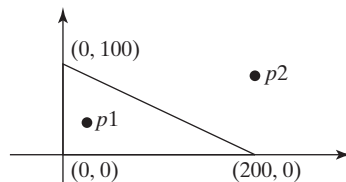
Enter a three-digit integer: 121
121 is a palindrome



Enter a three-digit integer: 123
123 is not a palindrome



****4.27** (*Geometry: points in triangle?*) Suppose a right triangle is placed in a plane as shown below. The right-angle point is at $(0, 0)$, and the other two points are at $(200, 0)$, and $(0, 100)$. Write a program that prompts the user to enter a point with x - and y -coordinates and determines whether the point is inside the triangle. Here are some sample runs:



Enter a point's x - and y -coordinates: 100.5, 25.5
The point is in the triangle



Enter a point's x - and y -coordinates: 100.5, 50.5
The point is not in the triangle



****4.28** (*Geometry: two rectangles*) Write a program that prompts the user to enter the center x -, y -coordinates, width, and height of two rectangles and determines whether the second rectangle is inside the first or overlaps with the first, as shown in Figure 4.10. Test your program to cover all cases.

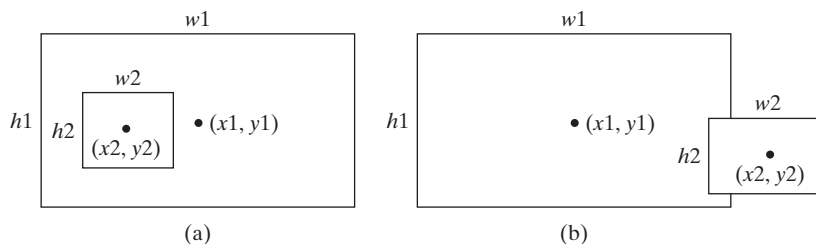


FIGURE 4.10 (a) A rectangle is inside another one. (b) A rectangle overlaps another one.

Here are some sample runs:



```
Enter r1's center x-, y-coordinates, width, and height:
2.5, 4, 2.5, 43 ↵ Enter
Enter r2's center x-, y-coordinates, width, and height:
1.5, 5, 0.5, 3 ↵ Enter
r2 is inside r1
```



```
Enter r1's center x-, y-coordinates, width, and height:
1, 2, 3, 5.5 ↵ Enter
Enter r2's center x-, y-coordinates, width, and height:
3, 4, 4.5, 5 ↵ Enter
r2 overlaps r1
```



```
Enter r1's center x-, y-coordinates, width, and height:
1, 2, 3, 3 ↵ Enter
Enter r2's center x-, y-coordinates, width, and height:
40, 45, 3, 2 ↵ Enter
r2 does not overlap r1
```

****4.29** (*Geometry: two circles*) Write a program that prompts the user to enter the center coordinates and radii of two circles and determines whether the second circle is inside the first or overlaps with the first, as shown in Figure 4.11. (Hint: circle2 is inside circle1 if the distance between the two centers $\leq |r1 - r2|$ and circle2 overlaps circle1 if the distance between the two centers $\leq r1 + r2$. Test your program to cover all cases.)

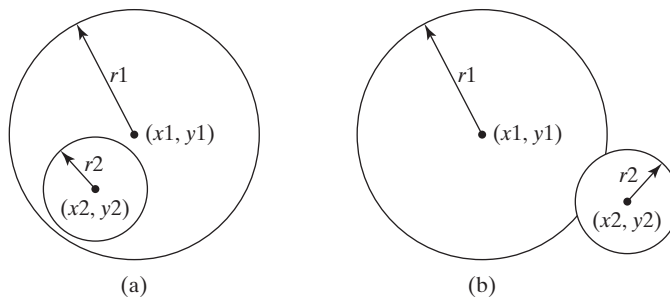


FIGURE 4.11 (a) A circle is inside another circle. (b) A circle overlaps another circle.

Here are some sample runs:



```
Enter circle1's center x-, y-coordinates, and radius:
0.5, 5.1, 13 ↵ Enter
Enter circle2's center x-, y-coordinates, and radius:
1, 1.7, 4.5 ↵ Enter
circle2 is inside circle1
```



```
Enter circle1's center x-, y-coordinates, and radius:
4.4, 5.7, 5.5 ↵ Enter
Enter circle2's center x-, y-coordinates, and radius:
6.7, 3.5, 3 ↵ Enter
circle2 overlaps circle1
```

```

Enter circle1's center x-, y-coordinates, and radius:
4.4, 5.5, 1 ↵ Enter
Enter circle2's center x-, y-coordinates, and radius:
5.5, 7.2, 1 ↵ Enter
circle2 does not overlap circle1
    
```



***4.30** (Current time) Revise Programming Exercise 2.18 to display the hour using a 12-hour clock. Here is a sample run:

```

Enter the time zone offset to GMT: -5 ↵ Enter
The current time is 4:50:34 AM
    
```



***4.31** (Geometry: point position) Given a directed line from point $p_0(x_0, y_0)$ to $p_1(x_1, y_1)$, you can use the following condition to decide whether a point $p_2(x_2, y_2)$ is on the left side of the line, on the right side of the line, or on the same line (see Figure 4.12):

$$(x_1 - x_0)(y_2 - y_0) - (x_2 - x_0)(y_1 - y_0) \begin{cases} > 0 & p_2 \text{ is on the left side of the line} \\ = 0 & p_2 \text{ is on the same line} \\ < 0 & p_2 \text{ is on the right side of the line} \end{cases}$$

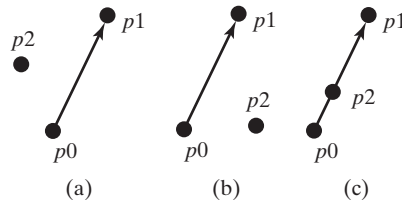


FIGURE 4.12 (a) p_2 is on the left side of the line. (b) p_2 is on the right side of the line. (c) p_2 is on the same line.

Write a program that prompts the user to enter the x- and y-coordinates for the three points p_0 , p_1 , and p_2 and displays whether p_2 is on the left side of the line from p_0 to p_1 , on the right side, or on the same line. Here are some sample runs:

```

Enter coordinates for the three points p0, p1, and p2:
3.4, 2, 6.5, 9.5, -5, 4 ↵ Enter
p2 is on the left side of the line from p0 to p1
    
```



```

Enter coordinates for the three points p0, p1, and p2:
1, 1, 5, 5, 2, 2 ↵ Enter
p2 is on the same line from p0 to p1
    
```



```

Enter coordinates for the three points p0, p1, and p2:
3.4, 2, 6.5, 9.5, 5, 2.5 ↵ Enter
p2 is on the right side of the line from p0 to p1
    
```



- *4.32** (*Geometry: point on line segment*) Exercise 4.31 shows how to test whether a point is on an unbounded line. Revise Exercise 4.31 to test whether a point is on a line segment. Write a program that prompts the user to enter the x- and y-coordinates for the three points p0, p1, and p2 and displays whether p2 is on the line segment from p0 to p1. Here are some sample runs:



```
Enter coordinates for the three points p0, p1, and p2:
1, 1, 2.5, 2.5, 1.5, 1.5 ↵ Enter
(1.5, 1.5) is on the line segment from (1.0, 1.0) to (2.5, 2.5)
```



```
Enter coordinates for the three points p0, p1, and p2:
1, 1, 2, 2, 3.5, 3.5 ↵ Enter
(3.5, 3.5) is not on the line segment from (1.0, 1.0) to
(2.0, 2.0)
```

- *4.33** (*Decimal to hex*) Write a program that prompts the user to enter an integer between 0 and 15 and displays its corresponding hex number. Here are some sample runs:



```
Enter a decimal value (0 to 15): 11 ↵ Enter
The hex value is B
```



```
Enter a decimal value (0 to 15): 5 ↵ Enter
The hex value is 5
```



```
Enter a decimal value (0 to 15): 31 ↵ Enter
Invalid input
```

- *4.34** (*Hex to decimal*) Write a program that prompts the user to enter a hex character and displays its corresponding decimal integer. Here are some sample runs:



```
Enter a hex character: A ↵ Enter
The decimal value is 10
```



```
Enter a hex character: a ↵ Enter
The decimal value is 10
```



```
Enter a hex character: 5 ↵ Enter
The decimal value is 5
```

Enter a hex character: G ↵ Enter
Invalid input



- *4.35** (*Turtle: point position*) Write a program that prompts the user to enter the x - and y -coordinates for the three points p_0 , p_1 , and p_2 , and displays a message to indicate whether p_2 is on the left side, the right side, or on the line from p_0 to p_1 , as shown in Figure 4.13. See Exercise 4.31 for determining the point position.

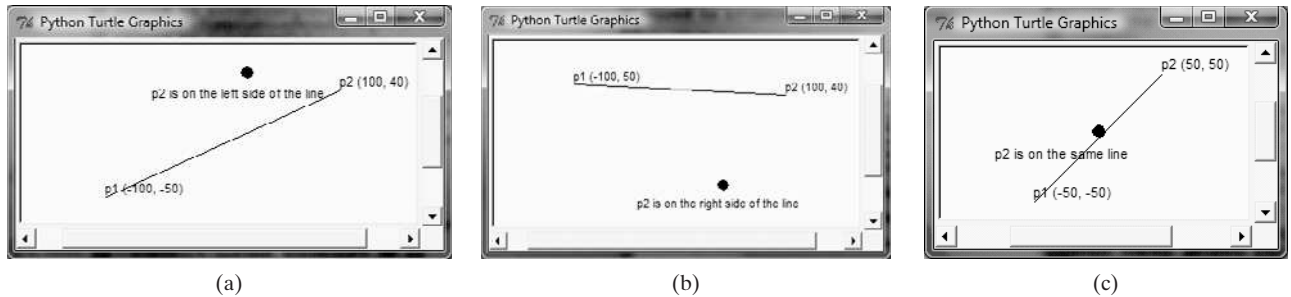


FIGURE 4.13 The program displays the point position graphically.

- **4.36** (*Turtle: point in a circle?*) Modify Listing 4.11 to let the program randomly generate a point within the square whose center is the same as the circle center and whose side is the diameter of the circle. Draw the circle and the point. Display a message to indicate whether the point is inside the circle.
- **4.37** (*Turtle: point in a rectangle?*) Write a program that prompts the user to enter a point (x, y) and checks whether the point is within the rectangle centered at $(0, 0)$ with width **100** and height **50**. Display the point, the rectangle, and a message indicating whether the point is inside the rectangle in the window, as shown in Figure 4.14.

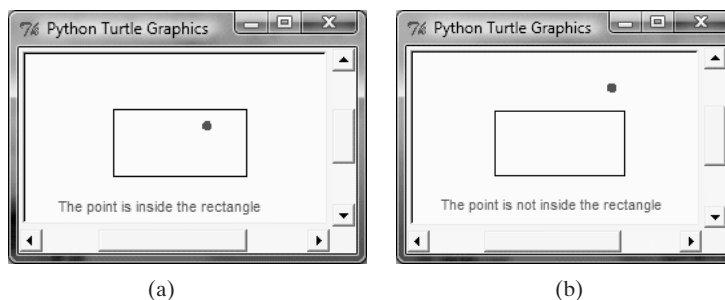


FIGURE 4.14 The program displays the rectangle, a point, and a message whether a point is in or outside of the rectangle.

- *4.38** (*Geometry: two rectangles*) Write a program that prompts the user to enter the center x -, y -coordinates, width, and height of two rectangles and determines whether the second rectangle is inside the first or overlaps with the first, as shown in Figure 4.15.

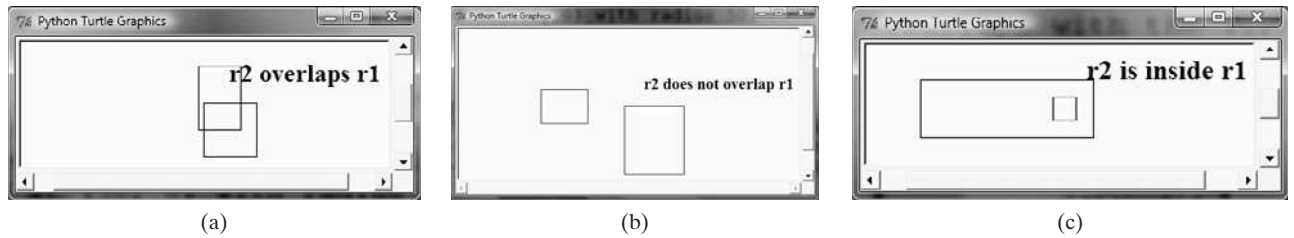


FIGURE 4.15 The program checks whether a rectangle is inside another one, overlaps another one, or does not overlap.

***4.39** (*Turtle: two circles*) Write a program that prompts the user to enter the center coordinates and radii of two circles and determines whether the second circle is inside the first or overlaps with the first, as shown in Figure 4.16.

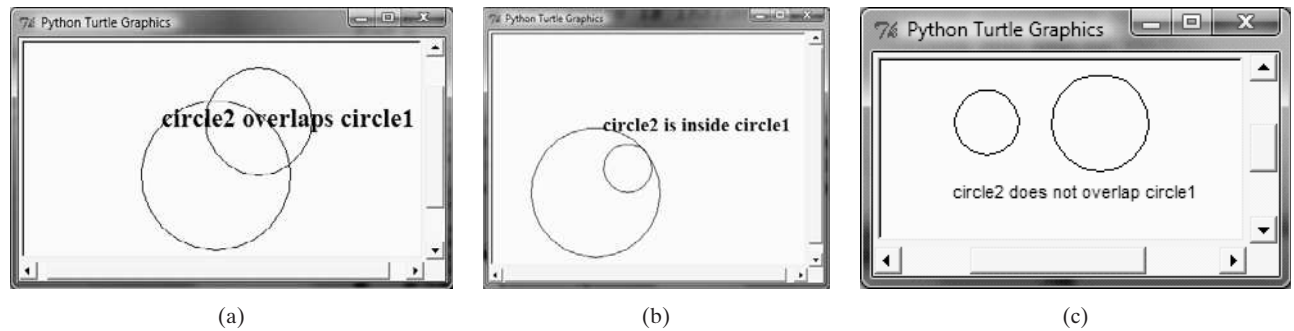


FIGURE 4.16 The program displays two circles and a status message.

CHAPTER 5

LOOPS

Objectives

- To write programs for executing statements repeatedly by using a **while** loop (§5.2).
- To develop loops following the loop design strategy (§§5.2.1–5.2.3).
- To control a loop with the user's confirmation (§5.2.4).
- To control a loop with a sentinel value (§5.2.5).
- To obtain a large amount of input from a file by using input redirection instead of typing from the keyboard and to save output to a file by using output redirection (§5.2.6).
- To use **for** loops to implement counter-controlled loops (§5.3).
- To write nested loops (§5.4).
- To learn the techniques for minimizing numerical errors (§5.5).
- To learn loops from a variety of examples ([GCD](#), [FutureTuition](#), [MonteCarloSimulation](#), [PrimeNumber](#)) (§§5.6, 5.8).
- To implement program control with **break** and **continue** (§5.7).
- To use a loop to simulate a random walk (§5.9).



5.1 Introduction



A loop can be used to tell a program to execute statements repeatedly.

Suppose that you need to display a string (e.g., **Programming is fun!**) 100 times. It would be tedious to type the statement 100 times:

```
100 times { print("Programming is fun!")
            print("Programming is fun!")
            ...
            print("Programming is fun!")
```

problem
why loop?
loop

So, how do you solve this problem?

Python provides a powerful construct called a *loop*, which controls how many times in succession an operation (or a sequence of operations) is performed. By using a loop statement, you don't have to code the print statement a hundred times; you simply tell the computer to display a string that number of times. The loop statement can be written as follows:

```
count = 0
while count < 100:
    print("Programming is fun!")
    count = count + 1
```

loop body

The variable **count** is initially **0**. The loop checks whether **count < 100** is true. If so, it executes the *loop body*—the part of the loop that contains the statements to be repeated—to display the message **Programming is fun!** and increments **count** by **1**. It repeatedly executes the loop body until **count < 100** becomes false (i.e., when **count** reaches **100**). At this point the loop terminates and the next statement after the loop statement is executed.

condition-controlled loop
count-controlled loop

A loop is a construct that controls the repeated execution of a block of statements. The concept of looping is fundamental to programming. Python provides two types of loop statements: **while** loops and **for** loops. The **while** loop is a *condition-controlled loop*; it is controlled by a true/false condition. The **for** loop is a *count-controlled loop* that repeats a specified number of times.

5.2 The while Loop



A while loop executes statements repeatedly as long as a condition remains true.

The syntax for the **while** loop is:

while loop

```
while loop-continuation-condition:
    # Loop body
    Statement(s)
```

iteration
loop-continuation-
condition

Figure 5.1a shows the **while**-loop flowchart. A single execution of a loop body is called an *iteration* (or repetition) of the loop. Each loop contains a **loop-continuation-condition**, a Boolean expression that controls the body's execution. It is evaluated each time to determine if the loop body should be executed. If its evaluation is **True**, the loop body is executed; otherwise, the entire loop terminates and the program control turns to the statement that follows the **while** loop.

The loop that displays **Programming is fun!** 100 times is an example of a **while** loop. Its flowchart is shown in Figure 5.1b. The **loop-continuation-condition** is **count < 100** and the loop body contains two statements:

```
count = 0
while count < 100:
    print("Programming is fun!")
    count = count + 1
```

} loop body

← loop-continuation-condition



VideoNote
while loop

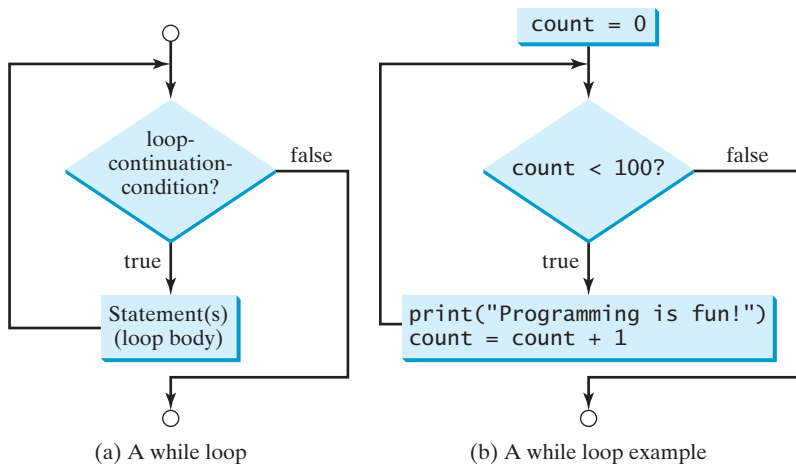


FIGURE 5.1 The **while** loop repeatedly executes the statements in the loop body as long as the **loop-continuation-condition** evaluates to **True**.

Here is another example illustrating how a loop works:

```
sum = 0
i = 1
while i < 10:
    sum = sum + i
    i = i + 1
print("sum is", sum) # sum is 45
```

If **i < 10** is true, the program adds **i** to **sum**. The variable **i** is initially set to **1**, then incremented to **2**, **3**, and so on, up to **10**. When **i** is **10**, **i < 10** is false, and the loop exits. So **sum** is **1 + 2 + 3 + ... + 9 = 45**.

Suppose the loop is mistakenly written as follows:

```
sum = 0
i = 1
while i < 10:
    sum = sum + i
i = i + 1
```

Note that the entire loop body must be indented inside the loop. Here the statement **i = i + 1** is not in the loop body. This loop is infinite, because **i** is always **1** and **i < 10** will always be true.

BAD CODE



Note

Make sure that the **loop-continuation-condition** eventually becomes false so that the loop will terminate. A common programming error involves *infinite loops* (i.e., the loop runs forever). If your program takes an unusual long time to run and does not stop, it may have an infinite loop. If you run the program from the command window, press CTRL+C to stop it.

infinite loop



Caution

Programmers often mistakenly execute a loop one time more or less than intended. This kind of mistake is commonly known as the *off-by-one error*. For example, the following loop displays **Programming is fun!** 101 times rather than 100 times. The error lies in the condition, which should be **count < 100** rather than **count <= 100**.

off-by-one error

```
count = 0
while count <= 100:
    print("Programming is fun!")
    count = count + 1
```

Recall that Listing 4.4, `SubtractionQuiz.py`, gives a program that prompts the user to enter an answer for a question on subtraction. Using a loop, you can now rewrite the program to let the user enter a new answer until it is correct, as shown in Listing 5.1.

LISTING 5.1 RepeatSubtractionQuiz.py

```

1  import random
2
3  # 1. Generate two random single-digit integers
4  number1 = random.randint(0, 9)
5  number2 = random.randint(0, 9)
6
7  # 2. If number1 < number2, swap number1 with number2
8  if number1 < number2:
9      number1, number2 = number2, number1
10
11 # 3. Prompt the student to answer "What is number1 - number2?"
12 answer = eval(input("What is " + str(number1) + " - "
13                     + str(number2) + "? "))
14
15 # 4. Repeatedly ask the question until the answer is correct
16 while number1 - number2 != answer:
17     answer = eval(input("Wrong answer. Try again. What is "
18                         + str(number1) + " - " + str(number2) + "? "))
19
20 print("You got it!")

```

random number1
random number2

swap numbers

enter answer

check answer
enter answer again



```

What is 4 - 3? 4 Enter
Wrong answer. Try again. What is 4 - 3? 5 Enter
Wrong answer. Try again. What is 4 - 3? 1 Enter
You got it!

```

The loop in lines 16–18 repeatedly prompts the user to enter an answer when `number1 - number2 != answer` is true. Once `number1 - number2 != answer` is false, the loop exits.

5.2.1 Case Study: Guessing Numbers

The problem is to guess what number a computer has in mind. You will write a program that randomly generates an integer between 0 and 100, inclusive. The program prompts the user to enter numbers continuously until it matches the randomly generated number. For each user input, the program reports whether it is too low or too high, so the user can choose the next input intelligently. Here is a sample run:



```

Guess a magic number between 0 and 100
Enter your guess: 50 Enter
Your guess is too high
Enter your guess: 25 Enter
Your guess is too low
Enter your guess: 42 Enter
Your guess is too high
Enter your guess: 39 Enter
Yes, the number is 39

```

intelligent guess

The magic number is between 0 and 100. To minimize the number of guesses, enter 50 first. If your guess is too high, the magic number is between 0 and 49. If your guess is too low,

the magic number is between 51 and 100. So, after one guess, you can eliminate half the numbers from further consideration.

How do you write this program? Do you immediately begin coding? No. It is important to *think before coding*. Think about how you would solve the problem without writing a program. You need to first generate a random number between 0 and 100, inclusive, then prompt the user to enter a guess, and then compare the guess with the random number.

think before coding

It is a good practice to code *incrementally*—that is, one step at a time. For programs involving loops, if you don't know how to write a loop right away, you might first write the program so it executes the code once, and then figure out how to execute it repeatedly in a loop. For this program, you can create an initial draft, as shown in Listing 5.2.

code incrementally

LISTING 5.2 GuessNumberOneTime.py

```

1  import random
2
3  # Generate a random number to be guessed
4  number = random.randint(0, 100)
5
6  print("Guess a magic number between 0 and 100")
7
8  # Prompt the user to guess the number
9  guess = eval(input("Enter your guess: "))
10
11 if guess == number:
12     print("Yes, the number is", number)
13 elif guess > number:
14     print("Your guess is too high")
15 else:
16     print("Your guess is too low")

```

generate a number

enter a guess

correct guess?

too high?

too low?

When this program runs, it prompts the user to enter a guess only once. To let the user enter a guess repeatedly, you can change the code in lines 11–16 to create a loop, as follows:

```

1  while True:
2      # Prompt the user to guess the number
3      guess = eval(input("Enter your guess: "))
4
5      if guess == number:
6          print("Yes, the number is", number)
7      elif guess > number:
8          print("Your guess is too high")
9      else:
10         print("Your guess is too low")

```

This loop repeatedly prompts the user to enter a guess. However, the loop still needs to terminate; when **guess** matches **number**, the loop should end. So, revise the loop as follows:

```

1  while guess != number:
2      # Prompt the user to guess the number
3      guess = eval(input("Enter your guess: "))
4
5      if guess == number:
6          print("Yes, the number is", number)
7      elif guess > number:
8          print("Your guess is too high")
9      else:
10         print("Your guess is too low")

```

The complete code is given in Listing 5.3.

LISTING 5.3 GuessNumber.py

generate a number

enter a guess

too high?

too low?

```
1  import random
2
3  # Generate a random number to be guessed
4  number = random.randint(0, 100)
5
6  print("Guess a magic number between 0 and 100")
7
8  guess = -1
9  while guess != number:
10     # Prompt the user to guess the number
11     guess = eval(input("Enter your guess: "))
12
13     if guess == number:
14         print("Yes, the number is", number)
15     elif guess > number:
16         print("Your guess is too high")
17     else:
18         print("Your guess is too low")
```



	line#	number	guess	output
	4	39		
	8		-1	
iteration 1 {	11		50	
	16			Your guess is too high
iteration 2 {	11		25	
	18			Your guess is too low
iteration 3 {	11		42	
	16			Your guess is too high
iteration 4 {	11		39	
	14			Yes, the number is 39

The program generates the magic number in line 4 and prompts the user to enter a guess continuously in a loop (lines 9–18). For each guess, the program determines whether the user’s number is correct, too high, or too low (lines 13–18). When the guess is correct, the program exits the loop (line 9). Note that `guess` is initialized to `-1`. This is to avoid initializing it to a value between `0` and `100`, because that could be the number to be guessed.

5.2.2 Loop Design Strategies

Writing a loop that works correctly is not an easy task for novice programmers. Consider the three steps involved when writing a loop:

- Step 1: Identify the statements that need to be repeated.
- Step 2: Wrap these statements in a loop like this:

```
while True:
    Statements
```

Step 3: Code the loop-continuation-condition and add appropriate statements for controlling the loop.

```
while loop-continuation-condition:
    Statements
    Additional statements for controlling the loop
```

5.2.3 Case Study: Multiple Subtraction Quiz

The subtraction quiz program in Listing 4.4, `SubtractionQuiz.py`, generates just one question for each run. You can use a loop to generate questions repeatedly. How do you write the code to generate five questions? Follow the loop design strategy. First, identify the statements that need to be repeated. These are the statements for obtaining two random numbers, prompting the user with a subtraction question, and grading the question. Second, wrap the statements in a loop. Third, add a loop-control variable and the loop-continuation-condition to execute the loop five times.

Listing 5.4 is a program that generates five questions and, after a student answers all of them, reports the number of correct answers. The program also displays the time spent on the test, as shown in the sample run.

LISTING 5.4 `SubtractionQuizLoop.py`

```
1  import random
2  import time
3
4  correctCount = 0 # Count the number of correct answers          correct count
5  count = 0 # Count the number of questions                      total count
6  NUMBER_OF_QUESTIONS = 5 # Constant
7
8  startTime = time.time() # Get start time                        get start time
9
10 while count < NUMBER_OF_QUESTIONS:                               loop
11     # Generate two random single-digit integers
12     number1 = random.randint(0, 9)
13     number2 = random.randint(0, 9)
14
15     # If number1 < number2, swap number1 with number2
16     if number1 < number2:
17         number1, number2 = number2, number1
18
19     # Prompt the student to answer "What is number1 - number2?"
20     answer = eval(input("What is " + str(number1) + " - " +      display a question
21         str(number2) + "? "))
22
23     # Grade the answer and display the result
24     if number1 - number2 == answer:                               grade an answer
25         print("You are correct!")
26         correctCount += 1                                         increase correct count
27     else:
28         print("Your answer is wrong.\n", number1, "-",
29             number2, "is", number1 - number2)
30
31     # Increase the count
32     count += 1                                                    increase control variable
33
34 endTime = time.time() # Get end time                             get end time
35 testTime = int(endTime - startTime) # Get test time              test time
36 print("Correct count is", correctCount, "out of",                display result
37     NUMBER_OF_QUESTIONS, "\nTest time is", testTime, "seconds")
```



```

What is 1 - 1? 0 Enter
You are correct!

What is 7 - 2? 5 Enter
You are correct!

What is 9 - 3? 4 Enter
Your answer is wrong.
9 - 3 is 6

What is 6 - 6? 0 Enter
You are correct!

What is 9 - 6? 2 Enter
Your answer is wrong.
9 - 6 is 3

Correct count is 3 out of 5
Test time is 10 seconds

```

The program uses the control variable `count` to control the execution of the loop. `count` is initially `0` (line 5) and is increased by `1` in each iteration (line 32). A subtraction question is displayed and processed in each iteration. The program obtains the time before the test starts in line 8 and the time after the test ends in line 34, and computes the test time in seconds in line 35. The program displays the correct count and test time after all the quizzes have been taken (lines 36–37).

5.2.4 Controlling a Loop with User Confirmation

The preceding example executes the loop five times. If you want the user to decide whether to take another question, you can offer a user *confirmation*. The template of the program can be coded as follows:

```

continueLoop = 'Y'
while continueLoop == 'Y':
    # Execute the loop body once
    ...

    # Prompt the user for confirmation
    continueLoop = input("Enter Y to continue and N to quit: ")

```

You can rewrite Listing 5.4 with user confirmation to let the user decide whether to advance to the next question.

5.2.5 Controlling a Loop with a Sentinel Value

Another common technique for controlling a loop is to designate a special input value, known as a *sentinel value*, which signifies the end of the input. A loop that uses a sentinel value in this way is called a *sentinel-controlled loop*.

The program in Listing 5.5 reads and calculates the sum of an unspecified number of integers. The input `0` signifies the end of the input. You don't need to use a new variable for each input value. Instead, use a variable named `data` (line 1) to store the input value and use a variable named `sum` (line 5) to store the total. Whenever a value is read, assign it to `data` (line 9) and add it to `sum` (line 7) if it is not zero.

confirmation

sentinel value
sentinel-controlled loop

LISTING 5.5 SentinelValue.py

```
1 data = eval(input("Enter an integer (the input ends " +
2   "if it is 0): "))
3
4 # Keep reading data until the input is 0
5 sum = 0
6 while data != 0:
7     sum += data
8
9     data = eval(input("Enter an integer (the input ends " +
10    "if it is 0): "))
11
12 print("The sum is", sum)
```

input data

loop

output result

Enter an integer (the input ends if it is 0): 2

Enter an integer (the input ends if it is 0): 3

Enter an integer (the input ends if it is 0): 4

Enter an integer (the input ends if it is 0): 0

The sum is 9



	line#	data	sum	output
	1	2		
	5		0	
iteration 1 {	7		2	
	9	3		
iteration 2 {	7		5	
	9	4		
iteration 3 {	7		9	
	9	0		
	12			The sum is 9



If **data** is not **0**, it is added to the **sum** (line 7) and the next item of input data is read (lines 9–10). If **data** is **0**, the loop body is no longer executed and the **while** loop terminates. The input value **0** is the sentinel value for this loop. Note that if the first input read is **0**, the loop body never executes, and the resulting sum is **0**.



Caution

Don't use floating-point values for equality checking in a loop control. Since those values are approximated, they could lead to imprecise counter values. This example uses **int** value for **data**. Consider the following code for computing **1 + 0.9 + 0.8 + ... + 0.1**:

numeric error

```
item = 1
sum = 0

while item != 0: # No guarantee item will be 0
    sum += item
    item -= 0.1

print(sum)
```

The variable `item` starts with `1` and is reduced by `0.1` every time the loop body is executed. The loop should terminate when `item` becomes `0`. However, there is no guarantee that `item` will be exactly `0`, because the floating-point arithmetic is approximated. This loop seems okay on the surface, but it is actually an infinite loop.

5.2.6 Input and Output Redirections

In Listing 5.5, if you have a lot of data to enter, it would be cumbersome to type all the entries from the keyboard. You can store the data in a text file (named `input.txt`, for example) and run the program by using the following command:

```
python SentinelValue.py < input.txt
```

input redirection

This command is called *input redirection*. Instead of having the user type the data from the keyboard at runtime, the program takes the input from the file `input.txt`. Suppose the file contains the following numbers, one number per line:

```
2
3
4
0
```

The program should get `sum` to be `9`.

output redirection

Similarly, *output redirection* can send the output to a file instead of displaying it on the screen. The command for output redirection is:

```
python Script.py > output.txt
```

Input and output redirection can be used in the same command. For example, the following command gets input from `input.txt` and sends output to `output.txt`:

```
python SentinelValue.py < input.txt > output.txt
```

Run the program and see what contents show up in `output.txt`.



MyProgrammingLab™

- 5.1** Analyze the following code. Is `count < 100` always `True`, always `False`, or sometimes `True` or sometimes `False` at Point A, Point B, and Point C?

```
count = 0
while count < 100:
    # Point A
    print("Programming is fun!")
    count += 1
    # Point B

    # Point C
```

- 5.2** What is wrong if `guess` is initialized to `0` in line 8 in Listing 5.3?
- 5.3** How many times are the following loop bodies repeated? What is the printout of each loop?

```
i = 1
while i < 10:
    if i % 2 == 0:
        print(i)
```

(a)

```
i = 1
while i < 10:
    if i % 2 == 0:
        print(i)
        i += 1
```

(b)

```
i = 1
while i < 10:
    if i % 2 == 0:
        print(i)
    i += 1
```

(c)

5.4 Show the errors in the following code:

```
count = 0
while count < 100:
    print(count)
```

(a)

```
count = 0
while count < 100:
    print(count)
    count -= 1
```

(b)

```
count = 0
while count < 100:
    count += 1
```

(c)

5.5 Suppose the input is 2 3 4 5 0 (one number per line). What is the output of the following code?

```
number = eval(input("Enter an integer: "))
max = number

while number != 0:
    number = eval(input("Enter an integer: "))
    if number > max:
        max = number

print("max is", max)
print("number", number)
```

5.3 The for Loop

A Python **for** loop iterates through each value in a sequence.

Often you know exactly how many times the loop body needs to be executed, so a control variable can be used to count the executions. A loop of this type is called a counter-controlled loop. In general, the loop can be written as follows:

```
i = initialValue # Initialize loop-control variable
while i < endValue:
    # Loop body
    ...
    i += 1 # Adjust loop-control variable
```

A **for** loop can be used to simplify the preceding loop:

```
for i in range(initialValue, endValue):
    # Loop body
```

In general, the syntax of a **for** loop is:

```
for var in sequence:
    # Loop body
```

A sequence holds multiple items of data, stored one after the other. Later in the book, we will introduce strings, lists, and tuples. They are sequence-type objects in Python. The variable **var** takes on each successive value in the sequence, and the statements in the body of the loop are executed once for each value.

The function **range(a, b)** returns the sequence of integers **a**, **a + 1**, ..., **b - 2**, and **b - 1**. For example,



counter-controlled loop



VideoNote
for loop

for loop

range(a, b) function



```
>>> for v in range(4, 8):
...     print(v)
...
4
5
6
7
>>>
```

range(a) function
range(a, b, k) function
step value

The **range** function has two more versions. You can also use **range(a)** or **range(a, b, k)**. **range(a)** is the same as **range(0, a)**. **k** is used as *step value* in **range(a, b, k)**. The first number in the sequence is **a**. Each successive number in the sequence will increase by the step value **k**. **b** is the limit. The last number in the sequence must be less than **b**. For example,



```
>>> for v in range(3, 9, 2):
...     print(v)
...
3
5
7
>>>
```

count backward

The step value in **range(3, 9, 2)** is **2**, and the limit is **9**. So, the sequence is **3, 5**, and **7**.

The **range(a, b, k)** function can count backward if **k** is negative. In this case, the sequence is still **a, a + k, a + 2k**, and so on for a negative **k**. The last number in the sequence must be greater than **b**. For example,



```
>>> for v in range(5, 1, -1):
...     print(v)
...
5
4
3
2
>>>
```



Note

The numbers in the **range** function must be integers. For example, **range(1.5, 8.5)**, **range(8.5)**, or **range(1.5, 8.5, 1)** would be wrong.



MyProgrammingLab™

5.6 Suppose the input is **2 3 4 5 0** (one number per line). What is the output of the following code?

```
number = 0
sum = 0

for count in range(5):
    number = eval(input("Enter an integer: "))
    sum += number

print("sum is", sum)
print("count is", count)
```

5.7 Can you convert any **for** loop to a **while** loop? List the advantages of using **for** loops.

5.8 Convert the following **for** loop statement to a **while** loop:

```
sum = 0
for i in range(1001):
    sum = sum + i
```

5.9 Can you always convert any **while** loop into a **for** loop? Convert the following **while** loop into a **for** loop.

```
i = 1
sum = 0

while sum < 10000:
    sum = sum + i
    i += 1
```

5.10 Count the number of iterations in the following loops:

```
count = 0
while count < n:
    count += 1
```

(a)

```
for count in range(n):
    print(count)
```

(b)

```
count = 5
while count < n:
    count += 1
```

(c)

```
count = 5
while count < n:
    count = count + 3
```

(d)

5.4 Nested Loops

A loop can be nested inside another loop.

Nested loops consist of an outer loop and one or more inner loops. Each time the outer loop is repeated, the inner loops are reentered and started anew.

Listing 5.6 presents a program that uses nested **for** loops to display a multiplication table.



**Key
Point**
nested loops

LISTING 5.6 MultiplicationTable.py

```
1 print("          Multiplication Table")
2 # Display the number title
3 print(" |", end = '')
4 for j in range(1, 10):
5     print(" ", j, end = '')
6 print() # Jump to the new line
7 print("-----")
8
9 # Display table body
10 for i in range(1, 10):
11     print(i, "|", end = '')
12     for j in range(1, 10):
13         # Display the product and align properly
14         print(format(i * j, "4d"), end = '')
15     print() # Jump to the new line
```

table title

table body

nested loop

display a line



Multiplication Table									
	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9
2	2	4	6	8	10	12	14	16	18
3	3	6	9	12	15	18	21	24	27
4	4	8	12	16	20	24	28	32	36
5	5	10	15	20	25	30	35	40	45
6	6	12	18	24	30	36	42	48	54
7	7	14	21	28	35	42	49	56	63
8	8	16	24	32	40	48	56	64	72
9	9	18	27	36	45	54	63	72	81

The program displays a title (line 1) on the first line in the output. The first **for** loop (lines 4–5) displays the numbers **1** through **9** on the second line. A line of dashes (–) is displayed on the third line (line 7).

The next loop (lines 10–15) is a nested **for** loop with the control variable **i** in the outer loop and **j** in the inner loop. For each **i**, the product **i * j** is displayed on a line in the inner loop, with **j** being **1, 2, 3, . . . , 9**.

To align the numbers properly, the program formats **i * j** using **format(i * j, "4d")** (line 14). Recall that **"4d"** specifies a decimal integer format with width **4**.

Normally, the **print** function automatically jumps to the next line. Invoking **print(item, end = '')** (lines 3, 5, 11, and 14) prints the item without advancing to the next line. Note that the **print** function with the **end** argument was introduced in Section 3.3.5.

```
end = ''
```



Note

Be aware that a nested loop may take a long time to run. Consider the following loop nested in three levels:

```
for i in range(1000):
    for j in range(1000):
        for k in range(1000):
            Perform an action
```

The action is performed 1,000,000,000 times. If it takes 1 millisecond to perform the action, the total time to run the loop would be more than 277 hours.



5.11 Show the output of the following programs. (Hint: Draw a table and list the variables in the columns to trace these programs.)

MyProgrammingLab™

```
for i in range(1, 5):
    j = 0
    while j < i:
        print(j, end = " ")
        j += 1
```

(a)

```
i = 0
while i < 5:
    for j in range(i, 1, -1):
        print(j, end = " ")
    print("****")
    i += 1
```

(b)

```
i = 5
while i >= 1:
    num = 1
    for j in range(1, i + 1):
        print(num, end = "xxx")
        num *= 2
    print()
    i -= 1
```

(c)

```
i = 1
while i <= 5:
    num = 1
    for j in range(1, i + 1):
        print(num, end = "G")
        num += 2
    print()
    i += 1
```

(d)

5.5 Minimizing Numerical Errors

Using floating-point numbers in the loop-continuation-condition may cause numeric errors.



Numerical errors involving floating-point numbers are inevitable. This section provides an example showing you how to minimize such errors.

The program in Listing 5.7 sums a series that starts with **0.01** and ends with **1.0**. The numbers in the series will increment by **0.01**, as follows: **0.01 + 0.02 + 0.03** and so on.

LISTING 5.7 TestSum.py

```
1 # Initialize sum
2 sum = 0
3
4 # Add 0.01, 0.02, ..., 0.99, 1 to sum
5 i = 0.01
6 while i <= 1.0:
7     sum += i
8     i = i + 0.01
9
10 # Display result
11 print("The sum is", sum)
```

loop

The sum is 49.5



numeric error

The result displayed is **49.5**, but the correct result is actually **50.5**. What went wrong? For each iteration in the loop, **i** is incremented by **0.01**. When the loop ends, the **i** value is slightly larger than **1** (not exactly **1**). This causes the last **i** value not to be added into **sum**. The fundamental problem is that the floating-point numbers are represented by approximation.

To fix the problem, use an integer count to ensure that all the numbers are added to **sum**. Here is the new loop:

```
# Initialize sum
sum = 0

# Add 0.01, 0.02, ..., 0.99, 1 to sum
count = 0
i = 0.01
while count < 100:
    sum += i
    i = i + 0.01
    count += 1 # Increase count

# Display result
print("The sum is", sum)
```

Or, use a **for** loop as follows:

```
# Initialize sum
sum = 0

# Add 0.01, 0.02, ..., 0.99, 1 to sum
i = 0.01
for count in range(100):
    sum += i
    i = i + 0.01

# Display result
print("The sum is", sum)
```

After this loop, **sum** is **50.5**.

5.6 Case Studies



Loops are fundamental in programming. The ability to write loops is essential in learning programming.

If you can write programs using loops, you know how to program! For this reason, this section presents three additional examples of solving problems using loops.

5.6.1 Problem: Finding the Greatest Common Divisor

GCD

The greatest common divisor (GCD) of the two integers **4** and **2** is **2**. The greatest common divisor of the two integers **16** and **24** is **8**. How do you find the greatest common divisor? Let the two input integers be **n1** and **n2**. You know that number **1** is a common divisor, but it may not be the greatest common divisor. So you can check whether **k** (for **k** = **2**, **3**, **4**, and so on) is a common divisor for **n1** and **n2**, until **k** is greater than **n1** or **n2**. Store the common divisor in a variable named **gcd**. Initially, **gcd** is **1**. Whenever a new common divisor is found, it becomes the new **gcd**. When you have checked all the possible common divisors from **2** up to **n1** or **n2**, the value in the variable **gcd** is the greatest common divisor. The idea can be translated into the following loop:

```
gcd = 1 # Initial gcd is 1
int k = 2 # Possible gcd
while k <= n1 and k <= n2:
    if n1 % k == 0 and n2 % k == 0:
        gcd = k
    k += 1 # Next possible gcd

# After the loop, gcd is the greatest common divisor for n1 and n2
```

Listing 5.8 presents a program that prompts the user to enter two positive integers and finds their greatest common divisor.

LISTING 5.8 GreatestCommonDivisor.py

input
input

gcd

output

```
1 # Prompt the user to enter two integers
2 n1 = eval(input("Enter first integer: "))
3 n2 = eval(input("Enter second integer: "))
4
5 gcd = 1
6 k = 2
7 while k <= n1 and k <= n2:
8     if n1 % k == 0 and n2 % k == 0:
9         gcd = k
10    k += 1
11
12 print("The greatest common divisor for",
13       n1, "and", n2, "is", gcd)
```



```
Enter first integer: 125  ↵ Enter
Enter second integer: 2525  ↵ Enter
The greatest common divisor for 125 and 2525 is 25
```

think before you type

How would you approach writing this program? Would you immediately begin to write the code? No. It is important to *think before you type*. Thinking enables you to generate a logical

solution for the problem without wondering how to write the code. Once you have a logical solution, type the code to translate the solution into a program.

A problem often has multiple solutions. The GCD problem can be solved in many ways. Exercise 5.16 at the end of this chapter suggests another solution. A more efficient solution is to use the classic Euclidean algorithm. See www.cut-the-knot.org/blue/Euclid.shtml for more information.

multiple solutions

5.12 If you think that a divisor for a number `n1` cannot be greater than `n1 / 2`, you might attempt to improve the program using the following loop:

```
k = 2
while k <= n1 / 2 and k <= n2 / 2:
    if n1 % k == 0 and n2 % k == 0:
        gcd = k
    k += 1
```



MyProgrammingLab™

erroneous solutions

This revision is wrong. Can you find the reason?

5.6.2 Problem: Predicting the Future Tuition

Suppose that the tuition for a university is \$10,000 this year and increases 7% every year. In how many years will the tuition have doubled?

Before you attempt to write a program, first consider how to solve this problem by hand. The tuition for the second year is the tuition for the first year * 1.07. The tuition for a future year is the tuition of its preceding year * 1.07. So, the tuition for each year can be computed as follows:

```
year = 0 # Year 0
tuition = 10000

year += 1 # Year 1
tuition = tuition * 1.07

year += 1 # Year 2
tuition = tuition * 1.07

year += 1 # Year 3
tuition = tuition * 1.07
...
```

Keep computing `tuition` for a new year until it is at least 20000. By then you will know how many years it will take for the tuition to be doubled. You can now translate the logic into the following loop:

```
year = 0 # Year 0
tuition = 10000
while tuition < 20000:
    year += 1
    tuition = tuition * 1.07
```

The complete program is shown in Listing 5.9.

LISTING 5.9 FutureTuition.py

```
1 year = 0 # Year 0
2 tuition = 10000 # Year 1
3
4 while tuition < 20000:
```

loop

```

5         year += 1
6         tuition = tuition * 1.07
7
8     print("Tuition will be doubled in", year, "years")
9     print("Tuition will be $" + format(tuition, ".2f"),
10          "in", year, "years")

```



```

Tuition will be doubled in 11 years
Tuition will be $21048.52 in 11 years

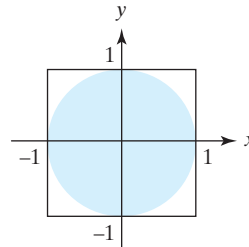
```

The **while** loop (lines 4–6) is used to repeatedly compute the tuition for a new year. The loop terminates when **tuition** is greater than or equal to **20000**.

5.6.3 Problem: Monte Carlo Simulation

A Monte Carlo simulation uses random numbers and probability to solve problems. It has a wide range of applications in computational mathematics, physics, chemistry, and finance. We now look at an example of using a Monte Carlo simulation for estimating π .

First, draw a circle with its bounding square.



Assume the radius of the circle is **1**. So, the circle area is π and the square area is **4**. Randomly generate a point in the square. The probability that the point falls in the circle is $\text{circleArea} / \text{squareArea} = \pi / 4$.

Write a program that randomly generates **1000000** points that fall in the square and let **numberOfHits** denote the number of points that fall in the circle. So, **numberOfHits** is approximately $1000000 * (\pi / 4)$. π can be approximated as $4 * \text{numberOfHits} / 1000000$. The complete program is shown in Listing 5.10.

LISTING 5.10 MonteCarloSimulation.py

```

1  import random
2
3  NUMBER_OF_TRIALS = 1000000 # Constant
4  numberOfHits = 0
5
6  for i in range(NUMBER_OF_TRIALS):
7      x = random.random() * 2 - 1
8      y = random.random() * 2 - 1
9
10     if x * x + y * y <= 1:
11         numberOfHits += 1
12
13 pi = 4 * numberOfHits / NUMBER_OF_TRIALS
14
15 print("PI is", pi)

```



```

PI is 3.14124

```

The program repeatedly generates a random point (**x**, **y**) in the square in lines 7–8:

```
x = random.random() * 2 - 1
y = random.random() * 2 - 1
```

Recall that **random()** returns a random float **r** such that $0 \leq r < 1.0$.

If $x^2 + y^2 \leq 1$, the point is inside the circle and **numberOfHits** is incremented by 1. π is approximately $4 * \text{numberOfHits} / \text{NUMBER_OF_TRIALS}$ (line 13).

5.7 Keywords **break** and **continue**

The **break** and **continue** keywords provide additional controls to a loop.



Pedagogical Note

Two keywords, **break** and **continue**, can be used in loop statements to provide additional controls. Using **break** and **continue** can simplify programming in some cases. Overusing or improperly using them, however, can make programs difficult to read and debug. (Note to readers: You may skip this section without affecting your understanding of the rest of the book.)

You can use the keyword **break** in a loop to immediately terminate a loop. Listing 5.11 presents a program to demonstrate the effect of using **break** in a loop.

LISTING 5.11 TestBreak.py

```
1  sum = 0
2  number = 0
3
4  while number < 20:
5      number += 1
6      sum += number
7      if sum >= 100:
8          break
9
10 print("The number is", number)
11 print("The sum is", sum)
```

break out of the loop

The number is 14
The sum is 105



The program adds integers from 1 to 20 in this order to **sum** until **sum** is greater than or equal to 100. Without lines 7–8, this program would calculate the sum of the numbers from 1 to 20. But with lines 7–8, the loop terminates when **sum** becomes greater than or equal to 100. Without lines 7–8, the output would be:

The number is 20
The sum is 210



You can also use the **continue** keyword in a loop. When it is encountered, it ends the current iteration and program control goes to the end of the loop body. In other words, **continue** breaks out of an iteration, while the **break** keyword breaks out of a loop. The program in Listing 5.12 shows the effect of using **continue** in a loop.

continue statement

LISTING 5.12 TestContinue.py

jump to the end of the iteration

```

1  sum = 0
2  number = 0
3
4  while number < 20:
5      number += 1
6      if number == 10 or number == 11:
7          continue
8      sum += number
9
10 print("The sum is", sum)

```



The sum is 189

The program adds all the integers from 1 to 20 except 10 and 11 to `sum`. The `continue` statement is executed when `number` becomes 10 or 11. The `continue` statement ends the current iteration so that the rest of the statement in the loop body is not executed; therefore, `number` is not added to `sum` when it is 10 or 11.

Without lines 6 and 7, the output would be as follows:



The sum is 210

In this case, all the numbers are added to `sum`, even when `number` is 10 or 11. Therefore, the result is 210.

You can always write a program without using `break` or `continue` in a loop (see Checkpoint Question 5.15). In general, it is appropriate to use `break` and `continue` if their use simplifies coding and makes programs easy to read.

Suppose you need to write a program to find the smallest factor other than 1 for an integer `n` (assume `n >= 2`). You can write a simple and intuitive code using the `break` statement as follows:

```

n = eval(input("Enter an integer >= 2: "))
factor = 2
while factor <= n:
    if n % factor == 0:
        break
    factor += 1
print("The smallest factor other than 1 for", n, "is", factor)

```

You may rewrite the code without using `break` as follows:

```

n = eval(input("Enter an integer >= 2: "))
found = False
factor = 2
while factor <= n and not found:
    if n % factor == 0:
        found = True
    else:
        factor += 1
print("The smallest factor other than 1 for", n, "is", factor)

```

Obviously, the `break` statement makes the program simpler and easier to read in this example. However, you should use `break` and `continue` with caution. Too many `break`

and **continue** statements will produce a loop with many exit points and make the program difficult to read.



Note

Some programming languages have a **goto** statement. The **goto** statement indiscriminately transfers control to any statement in the program and executes it. This makes your program vulnerable to errors. The **break** and **continue** statements in Python are different from **goto** statements. They operate only in a loop statement. The **break** statement breaks out of the loop, and the **continue** statement breaks out of the current iteration in the loop.

goto

- 5.13** What is the keyword **break** for? What is the keyword **continue** for? Will the following program terminate? If so, give the output.



MyProgrammingLab™

```
balance = 1000
while True:
    if balance < 9:
        break
    balance = balance - 9
print("Balance is", balance)
```

(a)

```
balance = 1000
while True:
    if balance < 9:
        continue
    balance = balance - 9
print("Balance is", balance)
```

(b)

- 5.14** The **for** loop on the left is converted into the **while** loop on the right. What is wrong? Correct it.

```
for i in range(4):
    if i % 3 == 0:
        continue
    sum += i
```

Converted

Wrong
conversion

```
i = 0
while i < 4:
    if i % 3 == 0:
        continue
    sum += i
    i += 1
```

- 5.15** Rewrite the programs **TestBreak** and **TestContinue** in Listings 5.11 and 5.12 without using **break** and **continue** statements.

- 5.16** After the **break** statement in (a) is executed in the following loop, which statement is executed? Show the output. After the **continue** statement in (b) is executed in the following loop, which statement is executed? Show the output.

```
for i in range(1, 4):
    for j in range(1, 4):
        if i * j > 2:
            break

    print(i * j)

print(i)
```

(a)

```
for i in range(1, 4):
    for j in range(1, 4):
        if i * j > 2:
            continue

    print(i * j)

print(i)
```

(b)



5.8 Case Study: Displaying Prime Numbers

This section presents a program that displays the first fifty prime numbers in five lines, each containing ten numbers.

An integer greater than 1 is *prime* if its only positive divisor is 1 or itself. For example, 2, 3, 5, and 7 are prime numbers, but 4, 6, 8, and 9 are not.

The problem can be broken into the following tasks:

- Determine whether a given number is prime.
- For `number = 2, 3, 4, 5, 6, ...`, test whether the number is prime.
- Count the prime numbers.
- Display each prime number, and display ten numbers per line.

Obviously, you need to write a loop and repeatedly test whether a new number is prime. If the number is prime, increase the count by 1. The count is 0 initially. When it reaches 50, the loop terminates.

Here is the algorithm for the problem:

```
Set the number of prime numbers to be displayed as
a constant NUMBER_OF_PRIMES
Use count to track the number of prime numbers and
set an initial count to 0
Set an initial number to 2
```

```
while count < NUMBER_OF_PRIMES:
    Test if number is prime

    if number is prime:
        Display the prime number and increase count

    Increment number by 1
```

To test whether a number is prime, check whether it is divisible by 2, 3, 4, ..., up to `number/2`. If a divisor is found, the number is not a prime. The algorithm can be described as follows:

```
Use a Boolean variable isPrime to denote whether
the number is prime; Set isPrime to True initially

for divisor in range(2, number / 2 + 1):
    if number % divisor == 0:
        Set isPrime to False
        Exit the loop
```

The complete program is given in Listing 5.13.

LISTING 5.13 PrimeNumber.py

```
1  NUMBER_OF_PRIMES = 50 # Number of primes to display
2  NUMBER_OF_PRIMES_PER_LINE = 10 # Display 10 per line
3  count = 0 # Count the number of prime numbers
4  number = 2 # A number to be tested for primeness
5
6  print("The first 50 prime numbers are")
7
8  # Repeatedly find prime numbers
```

count prime numbers

```

9  while count < NUMBER_OF_PRIMES:
10     # Assume the number is prime
11     isPrime = True # Is the current number prime?
12
13     # Test if number is prime
14     divisor = 2
15     while divisor <= number / 2:
16         if number % divisor == 0:
17             # If true, the number is not prime
18             isPrime = False # Set isPrime to false
19             break # Exit the for loop
20         divisor += 1
21
22     # Display the prime number and increase the count
23     if isPrime:
24         count += 1 # Increase the count
25
26         print(format(number, "5d"), end = '')
27         if count % NUMBER_OF_PRIMES_PER_LINE == 0:
28             # Display the number and advance to the new line
29             print() # Jump to the new line
30
31     # Check if the next number is prime
32     number += 1

```

check primeness

exit loop

display if prime

The first 50 prime numbers are

2	3	5	7	11	13	17	19	23	29
31	37	41	43	47	53	59	61	67	71
73	79	83	89	97	101	103	107	109	113
127	131	137	139	149	151	157	163	167	173
179	181	191	193	197	199	211	223	227	229



This is a complex example for novice programmers. The key to developing a programmatic solution for this problem—and for many other problems—is to break it into sub-problems and develop solutions for each of them in turn. Do not attempt to develop a complete solution in the first trial. Instead, begin by writing the code to determine whether a given number is prime, and then expand the program to test whether other numbers are prime in a loop.

To determine whether a number is prime, check whether it is divisible by a number between 2 and `number/2` inclusive. If so, it is not a prime number; otherwise, it is a prime number. For a prime number, display it. If the count is divisible by 10, advance to a new line. The program ends when the count reaches 50.

The program uses the `break` statement in line 19 to exit the `for` loop as soon as the number is found to be a nonprime. You can rewrite the loop (lines 15–20) without using the `break` statement as follows:

```

while divisor <= number / 2 and isPrime:
    if number % divisor == 0:
        # If True, the number is not prime
        isPrime = False # Set isPrime to False
    divisor += 1

```

However, using the `break` statement makes the program simpler and easier to read in this case.

5.9 Case Study: Random Walk



You can use Turtle graphics to simulate a random walk.

In this section, we will write a Turtle program that simulates a random walk in a lattice (e.g., like walking around a garden and turning to look at certain flowers) that starts from the center and ends at a point on the boundary, as shown in Figure 5.2. Listing 5.14 gives the program.

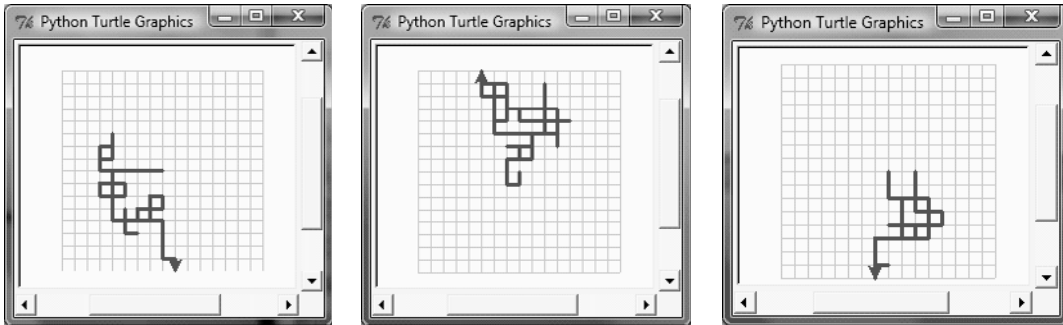


FIGURE 5.2 The program simulates random walks in a lattice.

LISTING 5.14 RandomWalk.py

```
import turtle
import randint

set turtle speed
set color
draw horizontal lines
draw vertical lines
move to center
current position
check boundaries
walk east

1 import turtle
2 from random import randint
3
4 turtle.speed(1) # Set turtle speed to slowest
5
6 # Draw 16-by-16 lattice
7 turtle.color("gray") # Color for lattice
8 x = -80
9 for y in range(-80, 80 + 1, 10):
10     turtle.penup()
11     turtle.goto(x, y) # Draw a horizontal line
12     turtle.pendown()
13     turtle.forward(160)
14
15 y = 80
16 turtle.right(90)
17 for x in range(-80, 80 + 1, 10):
18     turtle.penup()
19     turtle.goto(x, y) # Draw a vertical line
20     turtle.pendown()
21     turtle.forward(160)
22
23 turtle.pensize(3)
24 turtle.color("red")
25
26 turtle.penup()
27 turtle.goto(0, 0) # Go to the center
28 turtle.pendown()
29
30 x = y = 0 # Current pen location at the center of lattice
31 while abs(x) < 80 and abs(y) < 80:
32     r = randint(0, 3)
33     if r == 0:
34         x += 10 # Walk right
```

```

35         turtle.setheading(0)
36         turtle.forward(10)
37     elif r == 1:
38         y -= 10 # Walk down
39         turtle.setheading(270)
40         turtle.forward(10)
41     elif r == 2:
42         x -= 10 # Walk left
43         turtle.setheading(180)
44         turtle.forward(10)
45     elif r == 3:
46         y += 10 # Walk up
47         turtle.setheading(90)
48         turtle.forward(10)
49
50 turtle.done()

```

walk south

walk west

walk north

pause

Assume the size of the lattice is 16 by 16 and the distance between two lines in the lattice is 10 pixels (lines 6–21). The program first draws the lattice in a gray color. It sets the color to gray (line 7), uses the **for** loop (lines 9–13) to draw the horizontal lines, and the **for** loop (lines 17–21) to draw the vertical lines.

The program moves the pen to the center (line 27), and starts to simulate a random walk in a **while** loop (lines 31–48). The variables **x** and **y** are used to track the current position in the lattice. Initially, it is at (0, 0) (line 30). A random number from 0 to 3 is generated in line 32. These four numbers each correspond to a direction: east, south, west, and north. Consider four cases:

- If a walk is to the east, **x** is increased by 10 (line 34) and the pen is moved to the right (lines 35–36).
- If a walk is to the south, **y** is decreased by 10 (line 38) and the pen is moved downward (lines 39–40).
- If a walk is to the west, **x** is decreased by 10 (line 42) and the pen is moved to the left (lines 43–44).
- If a walk is to the north, **y** is increased by 10 (line 46) and the pen is moved upward (lines 47–48).

The walk stops when **abs(x)** or **abs(y)** is 80 (i.e., the walk reaches the boundary of the lattice).

A more interesting walk is called a *self-avoiding walk*. It is a random walk in a lattice that does not visit the same point twice. You will learn how to write a program to simulate a self-avoiding walk later in the book.

KEY TERMS

break keyword	151	loop body	134
condition-controlled loop	134	loop-continuation-condition	134
continue keyword	151	nested loop	145
count-controlled loop	134	off-by-one error	135
infinite loop	135	output redirection	142
input redirection	142	sentinel value	140
iteration	134		
loop	134		

CHAPTER SUMMARY

1. There are two types of repetition statements: the **while** loop and the **for** loop.
2. The part of the *loop* that contains the statements to be repeated is called the *loop body*.
3. A one-time execution of a loop body is referred to as an *iteration of the loop*.
4. An *infinite loop* is a loop statement that executes infinitely.
5. In designing loops, you need to consider both the loop-control structure and the loop body.
6. The **while** loop checks the **loop-continuation-condition** first. If the condition is true, the loop body is executed; otherwise, the loop terminates.
7. A *sentinel value* is a special value that signifies the end of the input.
8. The **for** loop is a *count-controlled loop* and is used to execute a loop body a predictable number of times.
9. Two keywords, **break** and **continue**, can be used in a loop.
10. The **break** keyword immediately ends the innermost loop, which contains the break.
11. The **continue** keyword ends only the current iteration.

TEST QUESTIONS

Do test questions for this chapter online at www.cs.armstrong.edu/liang/py/test.html.

MyProgrammingLab™

read and think before coding

explore solutions

PROGRAMMING EXERCISES



Pedagogical Note

For each problem, read it several times until you understand it. Think how to solve the problem before coding. Translate your logic into a program.

A problem often can be solved in many different ways. You should explore various solutions.

Sections 5.2–5.7

- *5.1** (*Count positive and negative numbers and compute the average of numbers*)
Write a program that reads an unspecified number of integers, determines how many positive and negative values have been read, and computes the total and average of the input values (not counting zeros). Your program ends with the input **0**. Display the average as a floating-point number. Here is a sample run:



```
Enter an integer, the input ends if it is 0: 1 Enter
Enter an integer, the input ends if it is 0: 2 Enter
Enter an integer, the input ends if it is 0: -1 Enter
Enter an integer, the input ends if it is 0: 3 Enter
```

```
Enter an integer, the input ends if it is 0: 0 Enter
The number of positives is 3
The number of negatives is 1
The total is 5
The average is 1.25
```

```
Enter an integer, the input ends if it is 0: 0 Enter
You didn't enter any number
```



5.2 (*Repeat additions*) Listing 5.4, `SubtractionQuizLoop.py`, generates five random subtraction questions. Revise the program to generate ten random addition questions for two integers between 1 and 15. Display the correct count and test time.

5.3 (*Conversion from kilograms to pounds*) Write a program that displays the following table (note that 1 kilogram is 2.2 pounds):

Kilograms	Pounds
1	2.2
3	6.6
...	
197	433.4
199	437.8

5.4 (*Conversion from miles to kilometers*) Write a program that displays the following table (note that 1 mile is 1.609 kilometers):

Miles	Kilometers
1	1.609
2	3.218
...	
9	15.481
10	16.090

***5.5** (*Conversion from kilograms to pounds and pounds to kilograms*) Write a program that displays the following two tables side by side (note that 1 kilogram is 2.2 pounds and that 1 pound is .45 kilograms):

Kilograms	Pounds		Pounds	Kilograms
1	2.2		20	9.09
3	6.6		25	11.36
...				
197	433.4		510	231.82
199	437.8		515	235.09

***5.6** (*Conversion from miles to kilometers and kilometers to miles*) Write a program that displays the following two tables side by side (note that 1 mile is 1.609 kilometers and that 1 kilometer is .621 mile):

Miles	Kilometers		Kilometers	Miles
1	1.609		20	12.430
2	3.218		25	15.538
...				
9	15.481		60	37.290
10	16.090		65	40.398

- 5.7** (Use trigonometric functions) Print the following table to display the `sin` value and `cos` value of degrees from 0 to 360 with increments of 10 degrees. Round the value to keep four digits after the decimal point.

Degree	Sin	Cos
0	0.0000	1.0000
10	0.1736	0.9848
...		
350	-0.1736	0.9848
360	0.0000	1.0000

- 5.8** (Use the `math.sqrt` function) Write a program that prints the following table using the `sqrt` function in the `math` module.

Number	Square Root
0	0.0000
2	1.4142
...	
18	5.2426
20	5.4721

- **5.9** (Financial application: compute future tuition) Suppose that the tuition for a university is \$10,000 this year and increases 5% every year. Write a program that computes the tuition in ten years and the total cost of four years' worth of tuition starting ten years from now.
- 5.10** (Find the highest score) Write a program that prompts the user to enter the number of students and each student's score, and displays the highest score. Assume that the input is stored in a file named `score.txt`, and the program obtains the input from the file.
- *5.11** (Find the two highest scores) Write a program that prompts the user to enter the number of students and each student's score, and displays the highest and second-highest scores.
- 5.12** (Find numbers divisible by 5 and 6) Write a program that displays, ten numbers per line, all the numbers from 100 to 1,000 that are divisible by 5 and 6. The numbers are separated by exactly one space.
- 5.13** (Find numbers divisible by 5 or 6, but not both) Write a program that displays, ten numbers per line, all the numbers from 100 to 200 that are divisible by 5 or 6, but not both. The numbers are separated by exactly one space.
- 5.14** (Find the smallest n such that $n^2 > 12,000$) Use a `while` loop to find the smallest integer n such that n^2 is greater than 12,000.
- 5.15** (Find the largest n such that $n^3 < 12,000$) Use a `while` loop to find the largest integer n such that n^3 is less than 12,000.
- *5.16** (Compute the greatest common divisor) For Listing 5.8, another solution to find the greatest common divisor of two integers `n1` and `n2` is as follows: First find `d` to be the minimum of `n1` and `n2`, and then check whether `d`, `d - 1`, `d - 2`, ..., `2`, or `1` is a divisor for both `n1` and `n2` in this order. The first such common divisor is the greatest common divisor for `n1` and `n2`.

Section 5.8

- *5.17** (Display the ASCII character table) Write a program that displays the characters in the ASCII character table from `!` to `~`. Display ten characters per line. The characters are separated by exactly one space.

****5.18** (*Find the factors of an integer*) Write a program that reads an integer and displays all its smallest factors, also known as *prime factors*. For example, if the input integer is 120, the output should be as follows:

2, 2, 2, 3, 5

****5.19** (*Display a pyramid*) Write a program that prompts the user to enter an integer from 1 to 15 and displays a pyramid, as shown in the following sample run:

```

Enter the number of lines: 7
1
  2 1 2
    3 2 1 2 3
      4 3 2 1 2 3 4
        5 4 3 2 1 2 3 4 5
          6 5 4 3 2 1 2 3 4 5 6
            7 6 5 4 3 2 1 2 3 4 5 6 7
    
```



***5.20** (*Display four patterns using loops*) Use nested loops that display the following patterns in four separate programs:

Pattern A	Pattern B	Pattern C	Pattern D
1	1 2 3 4 5 6	1	1 2 3 4 5 6
1 2	1 2 3 4 5	2 1	1 2 3 4 5
1 2 3	1 2 3 4	3 2 1	1 2 3 4
1 2 3 4	1 2 3	4 3 2 1	1 2 3
1 2 3 4 5	1 2	5 4 3 2 1	1 2
1 2 3 4 5 6	1	6 5 4 3 2 1	1

****5.21** (*Display numbers in a pyramid pattern*) Write a nested **for** loop that displays the following output:

```

          1
        1 2 1
      1 2 4 2 1
    1 2 4 8 4 2 1
  1 2 4 8 16 8 4 2 1
1 2 4 8 16 32 16 8 4 2 1
1 2 4 8 16 32 64 32 16 8 4 2 1
    
```

***5.22** (*Display prime numbers between 2 and 1,000*) Modify Listing 5.13 to display all the prime numbers between 2 and 1,000, inclusive. Display eight prime numbers per line.

Comprehensive

****5.23** (*Financial application: compare loans with various interest rates*) Write a program that lets the user enter the loan amount and loan period in number of years and displays the monthly and total payments for each interest rate starting from 5% to 8%, with an increment of 1/8. Here is a sample run:



```

Loan Amount: 10000  ↵ Enter
Number of Years: 5  ↵ Enter
Interest Rate      Monthly Payment    Total Payment
5.000%            188.71              11322.74
5.125%            189.28              11357.13
5.250%            189.85              11391.59
...
7.875%            202.17              12129.97
8.000%            202.76              12165.83

```

For the formula to compute monthly payment, see Listing 2.8, ComputeLoan.py.

****5.24** (*Financial application: loan amortization schedule*) The monthly payment for a given loan pays the principal and the interest. The monthly interest is computed by multiplying the monthly interest rate and the balance (the remaining principal). The principal paid for the month is therefore the monthly payment minus the monthly interest. Write a program that lets the user enter the loan amount, number of years, and interest rate, and then displays the amortization schedule for the loan. Here is a sample run:



```

Loan Amount: 10000  ↵ Enter
Number of Years: 1  ↵ Enter
Annual Interest Rate: 7  ↵ Enter

Monthly Payment: 865.26
Total Payment: 10383.21

Payment#    Interest    Principal    Balance
1           58.33       806.93       9193.07
2           53.62       811.64       8381.43
...
11          10.00       855.26       860.27
12           5.01       860.25        0.01

```



Note

The balance after the last payment may not be zero. If so, the last payment should be the normal monthly payment plus the final balance.



Hint

Write a loop to display the table. Since the monthly payment is the same for each month, it should be computed before the loop. The balance is initially the loan amount. For each iteration in the loop, compute the interest and principal and update the balance. The loop may look like this:

```

for i in range(1, numberOfYears * 12 + 1):
    interest = monthlyInterestRate * balance
    principal = monthlyPayment - interest
    balance = balance - principal
    print(i, "\t\t", interest, "\t\t", principal, "\t\t",
          balance)

```

- *5.25** (*Demonstrate cancellation errors*) A *cancellation error* occurs when you are manipulating a very large number with a very small number. The large number may cancel out the smaller number. For example, the result of **100000000.0** + **0.00000001** is equal to **100000000.0**. To avoid cancellation errors and obtain more accurate results, carefully select the order of computation. For example, in computing the following series, you will obtain more accurate results by computing from right to left rather than from left to right:

$$1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$$

Write a program that compares the results of the summation of the preceding series, computing both from left to right and from right to left with **n** = **50000**.

- *5.26** (*Sum a series*) Write a program to sum the following series:

$$\frac{1}{3} + \frac{3}{5} + \frac{5}{7} + \frac{7}{9} + \frac{9}{11} + \frac{11}{13} + \dots + \frac{95}{97} + \frac{97}{99}$$

- **5.27** (*Compute π*) You can approximate π by using the following series:

$$\pi = 4 \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \dots + \frac{(-1)^{i+1}}{2i-1} \right)$$

Write a program that displays the π value for **i** = **10000**, **20000**, . . . , and **100000**.

- **5.28** (*Compute e*) You can approximate e by using the following series:

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \frac{1}{4!} + \dots + \frac{1}{i!}$$

Write a program that displays the e value for **i** = **10000**, **20000**, . . . , and **100000**. (Hint: Since $i! = i \times (i-1) \times \dots \times 2 \times 1$, then $\frac{1}{i!}$ is $\frac{1}{i(i-1)!}$.)

Initialize **e** and **item** to be **1** and keep adding a new **item** to **e**. The new item is the previous item divided by **i** for **i** = **2, 3, 4, . . .**)

- 5.29** (*Display leap years*) Write a program that displays, ten per line, all the leap years in the twenty-first century (from year 2001 to 2100). The years are separated by exactly one space.

- **5.30** (*Display the first days of each month*) Write a program that prompts the user to enter the year and first day of the year, and displays the first day of each month in the year on the console. For example, if the user entered year 2013, and 2 for Tuesday, January 1, 2013, your program should display the following output:

```
January 1, 2013 is Tuesday
...
December 1, 2013 is Sunday
```

- **5.31** (*Display calendars*) Write a program that prompts the user to enter the year and first day of the year, and displays on the console the calendar table for the year. For example, if the user entered year 2005, and 6 for Saturday, January 1,

2005, your program should display the calendar for each month in the year, as follows:

January 2005						
Sun	Mon	Tue	Wed	Thu	Fri	Sat
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31					
...						

December 2005						
Sun	Mon	Tue	Wed	Thu	Fri	Sat
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

- *5.32** (*Financial application: compound value*) Suppose you save \$100 *each* month into a savings account with the annual interest rate 5%. So, the monthly interest rate is $0.05/12 = 0.00417$. After the first month, the value in the account becomes

$$100 * (1 + 0.00417) = 100.417$$

After the second month, the value in the account becomes

$$(100 + 100.417) * (1 + 0.00417) = 201.252$$

After the third month, the value in the account becomes

$$(100 + 201.252) * (1 + 0.00417) = 302.507$$

and so on.

Write a program that prompts the user to enter an amount (e.g., 100), the annual interest rate (e.g., 5), and the number of months (e.g., 6), and displays the amount in the savings account after the given month.

- *5.33** (*Financial application: compute CD value*) Suppose you put \$10,000 into a CD with an annual percentage yield of 5.75%. After one month, the CD is worth

$$10000 + 10000 * 5.75 / 1200 = 10047.91$$

After two months, the CD is worth

$$10047.91 + 10047.91 * 5.75 / 1200 = 10096.06$$

After three months, the CD is worth

$$10096.06 + 10096.06 * 5.75 / 1200 = 10145.43$$

and so on.

Write a program that prompts the user to enter an amount (e.g., 10,000), the annual percentage yield (e.g., 5.75), and the number of months (e.g., 18), and displays a table as shown in the sample run.

```

Enter the initial deposit amount: 10000 Enter
Enter annual percentage yield: 5.75 Enter
Enter maturity period (number of months): 18 Enter
    
```

```

Month  CD Value
1      10047.91
2      10096.06
...
17     10846.56
18     10898.54
    
```



****5.34** (*Game: lottery*) Revise Listing 4.10, Lottery.py, to generate a lottery of a two-digit number. The two digits in the number are distinct. (Hint: Generate the first digit. Use a loop to continuously generate the second digit until it is different from the first digit.)

****5.35** (*Perfect number*) A positive integer is called a *perfect number* if it is equal to the sum of all of its positive divisors, excluding itself. For example, 6 is the first perfect number, because $6 = 3 + 2 + 1$. The next is $28 = 14 + 7 + 4 + 2 + 1$. There are four perfect numbers less than 10,000. Write a program to find these four numbers.

*****5.36** (*Game: scissor, rock, paper*) Programming Exercise 4.17 gives a program that plays the scissor, rock, paper game. Revise the program to let the user play continuously until either the user or the computer wins more than two times.

***5.37** (*Summation*) Write a program that computes the following summation:

$$\frac{1}{1 + \sqrt{2}} + \frac{1}{\sqrt{2} + \sqrt{3}} + \frac{1}{\sqrt{3} + \sqrt{4}} + \dots + \frac{1}{\sqrt{624} + \sqrt{625}}$$

***5.38** (*Simulation: clock countdown*) You can use the `time.sleep(seconds)` function in the `time` module to let the program pause for the specified seconds. Write a program that prompts the user to enter the number of seconds, displays a message at every second, and terminates when the time expires. Here is a sample run:

```

Enter the number of seconds: 3 Enter
2 seconds remaining
1 second remaining
Stopped
    
```



***5.39** (*Financial application: find the sales amount*) You have just started a sales job in a department store. Your pay consists of a base salary plus a commission. The base salary is \$5,000. The following scheme shows how to determine the commission rate:

Sales Amount	Commission Rate
\$0.01–\$5,000	8 percent
\$5,000.01–\$10,000	10 percent
\$10,000.01 and above	12 percent

Your goal is to earn \$30,000 a year. Write a program that finds out the minimum amount of sales you have to generate in order to make \$30,000.

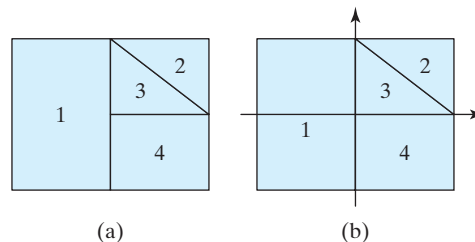
5.40 (*Simulation: heads or tails*) Write a program that simulates flipping a coin one million times and displays the number of heads and tails.

****5.41** (*Occurrence of max numbers*) Write a program that reads integers, finds the largest of them, and counts its occurrences. Assume that the input ends with number 0. Suppose that you entered 3 5 2 5 5 5 0; the program finds that the largest number is 5 and the occurrence count for 5 is 4. (Hint: Maintain two variables, **max** and **count**. The variable **max** stores the current maximum number, and **count** stores its occurrences. Initially, assign the first number to **max** and 1 to **count**. Compare each subsequent number with **max**. If the number is greater than **max**, assign it to **max** and reset **count** to 1. If the number is equal to **max**, increment **count** by 1.)



```
Enter a number (0: for end of input): 3 Enter
Enter a number (0: for end of input): 5 Enter
Enter a number (0: for end of input): 2 Enter
Enter a number (0: for end of input): 5 Enter
Enter a number (0: for end of input): 5 Enter
Enter a number (0: for end of input): 5 Enter
Enter a number (0: for end of input): 0 Enter
The largest number is 5
The occurrence count of the largest number is 4
```

****5.42** (*Monte Carlo simulation*) A square is divided into four smaller regions as shown in (a). If you throw a dart into the square one million times, what is the probability for the dart to fall into an odd-numbered region? Write a program to simulate the process and display the result. (Hint: Place the center of the square in the center of a coordinate system, as shown in (b). Randomly generate a point in the square and count the number of times for a point to fall in an odd-numbered region.)



***5.43** (*Math: combinations*) Write a program that displays all possible combinations for picking two numbers from integers 1 to 7. Also display the total number of combinations.



```
1 2
1 3
...
...
The total number of all combinations is 21
```

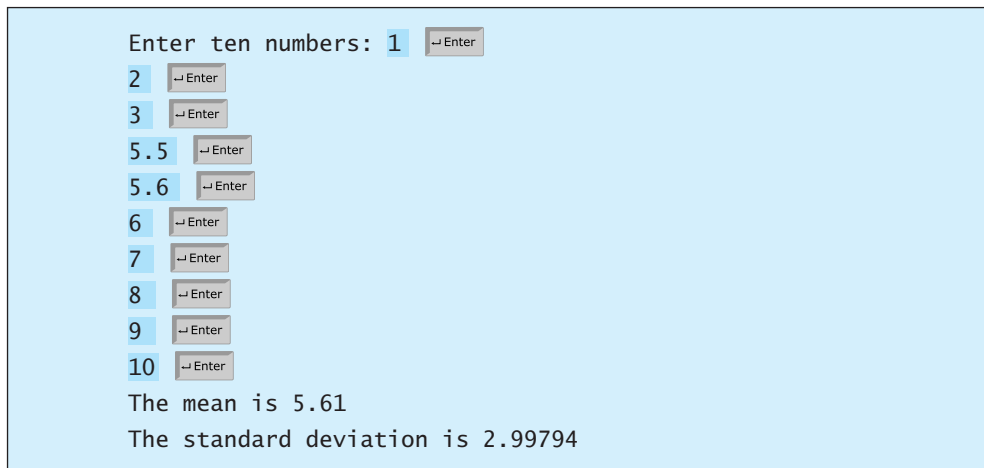
****5.44** (*Decimal to binary*) Write a program that prompts the user to enter a decimal integer and displays its corresponding binary value.

- **5.45** (*Decimal to hex*) Write a program that prompts the user to enter a decimal integer and displays its corresponding hexadecimal value.
- **5.46** (*Statistics: compute mean and standard deviation*) In business applications, you are often asked to compute the mean and standard deviation of data. The mean is simply the average of the numbers. The standard deviation is a statistic that tells you how tightly all the various data are clustered around the mean in a set of data. For example, what is the average age of the students in a class? How close are the ages? If all the students are the same age, the deviation is 0. Write a program that prompts the user to enter ten numbers, and displays the mean and standard deviations of these numbers using the following formula:

$$\text{mean} = \frac{\sum_{i=1}^n x_i}{n} = \frac{x_1 + x_2 + \dots + x_n}{n}$$

$$\text{deviation} = \sqrt{\frac{\sum_{i=1}^n x_i^2 - \frac{\left(\sum_{i=1}^n x_i\right)^2}{n}}{n - 1}}$$

Here is a sample run:



```

Enter ten numbers: 1 Enter
2 Enter
3 Enter
5.5 Enter
5.6 Enter
6 Enter
7 Enter
8 Enter
9 Enter
10 Enter
The mean is 5.61
The standard deviation is 2.99794
    
```



- **5.47** (*Turtle: draw random balls*) Write a program that displays 10 random balls in a rectangle with width 120 and height 100, centered at (0, 0), as shown in Figure 5.3a.

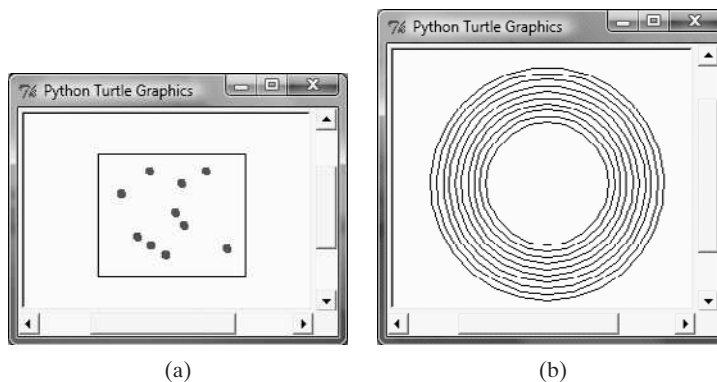
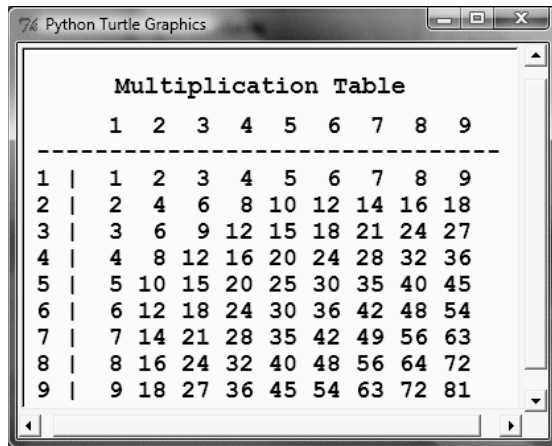


FIGURE 5.3 The program draws 10 random balls in (a), and 10 circles in (b).

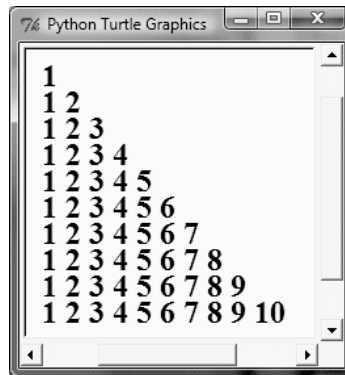
- **5.48** (Turtle: draw circles) Write a program that draws 10 circles with centers (0, 0), as shown in Figure 5.3b.
- **5.49** (Turtle: display a multiplication table) Write a program that displays a multiplication table, as shown in Figure 5.4a.
- **5.50** (Turtle: display numbers in a triangular pattern) Write a program that displays numbers in a triangular pattern, as shown in Figure 5.4b.



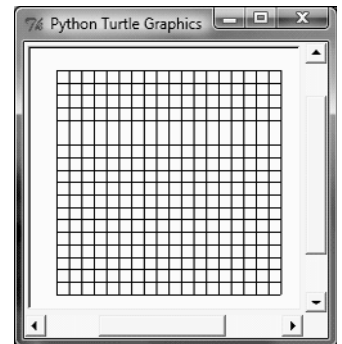
The screenshot shows a window titled "Python Turtle Graphics" containing a multiplication table. The table has a title "Multiplication Table" and columns labeled 1 through 9. The rows are also labeled 1 through 9. The numbers are displayed in a grid with vertical and horizontal lines separating the cells.

	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9
2	2	4	6	8	10	12	14	16	18
3	3	6	9	12	15	18	21	24	27
4	4	8	12	16	20	24	28	32	36
5	5	10	15	20	25	30	35	40	45
6	6	12	18	24	30	36	42	48	54
7	7	14	21	28	35	42	49	56	63
8	8	16	24	32	40	48	56	64	72
9	9	18	27	36	45	54	63	72	81

(a)



(b)



(c)

FIGURE 5.4 (a) The program displays a multiplication table. (b) The program displays numbers in a triangular pattern. (c) The program displays an 18-by-18 lattice.

- **5.51** (Turtle: display a lattice) Write a program that displays an 18-by-18 lattice, as shown in Figure 5.4c.
- **5.52** (Turtle: plot the sine function) Write a program that plots the sine function, as shown in Figure 5.5a.



Hint

The Unicode for π is `\u03c0`. To display -2π , use `turtle.write("-2\u03c0")`. For a trigonometric function like `sin(x)`, `x` is in radians. Use the following loop to plot the sine function:

```
for x in range(-175, 176):
    turtle.goto(x, 50 * math.sin((x / 100) * 2 * math.pi))
```

-2π is displayed at $(-100, -15)$, the center of the axis is at $(0, 0)$, and 2π is displayed at $(100, -15)$.

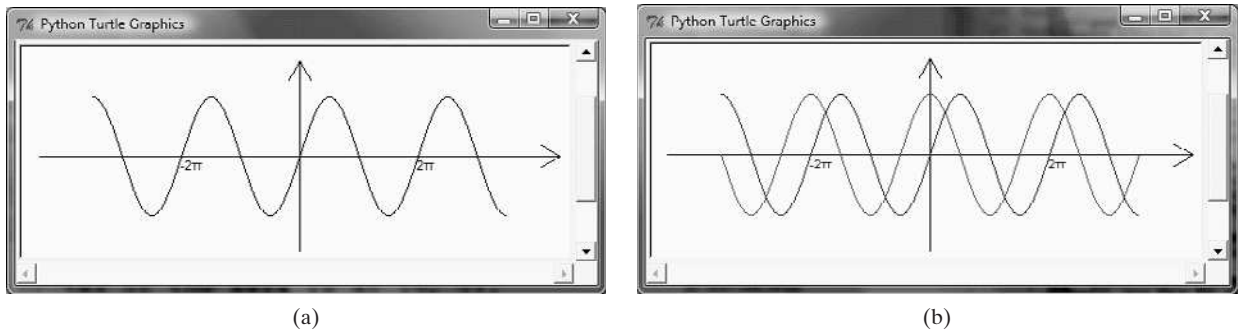


FIGURE 5.5 (a) The program plots a sine function. (b) The program plots sine function in blue and cosine function in red.

****5.53** (*Turtle: plot the sine and cosine functions*) Write a program that plots the sine function in red and cosine in blue, as shown in Figure 5.5b.

****5.54** (*Turtle: plot the square function*) Write a program that draws a diagram for the function $f(x) = x^2$ (see Figure 5.6a).

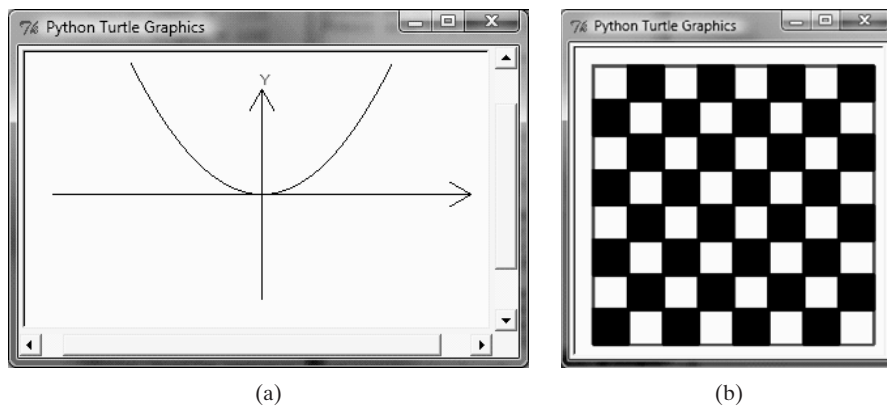


FIGURE 5.6 (a) The program plots a diagram for function $f(x) = x^2$. (b) The program draws a chessboard.

****5.55** (*Turtle: chessboard*) Write a program to draw a chessboard, as shown in Figure 5.6b.

This page intentionally left blank

CHAPTER 6

FUNCTIONS

Objectives

- To define functions with formal parameters (§6.2).
- To invoke functions with actual parameters (i.e., arguments) (§6.3).
- To distinguish between functions that return and do not return a value (§6.4).
- To invoke a function using positional arguments or keyword arguments (§6.5).
- To pass arguments by passing their reference values (§6.6).
- To develop reusable code that is modular and is easy to read, debug, and maintain (§6.7).
- To create modules for reusing functions (§§6.7–6.8).
- To determine the scope of variables (§6.9).
- To define functions with default arguments (§6.10).
- To define a function that returns multiple values (§6.11).
- To apply the concept of function abstraction in software development (§6.12).
- To design and implement functions using stepwise refinement (§6.13).
- To simplify drawing programs with reusable functions (§6.14).



6.1 Introduction



Functions can be used to define reusable code and organize and simplify code.

problem

Suppose that you need to find the sum of integers from **1** to **10**, **20** to **37**, and **35** to **49**. If you create a program to add these three sets of numbers, your code might look like this:

```
sum = 0
for i in range(1, 11):
    sum += i
print("Sum from 1 to 10 is", sum)

sum = 0
for i in range(20, 38):
    sum += i
print("Sum from 20 to 37 is", sum)

sum = 0
for i in range(35, 50):
    sum += i
print("Sum from 35 to 49 is", sum)
```

why functions?

You may have observed that the code for computing these sums is very similar, except that the starting and ending integers are different. Wouldn't it be nice to be able to write commonly used code once and then reuse it? You can do this by defining a function, which enables you to create reusable code. For example, the preceding code can be simplified by using functions, as follows:

define sum function

```
1 def sum(i1, i2):
2     result = 0
3     for i in range(i1, i2 + 1):
4         result += i
5
6     return result
7
8 def main():
9     print("Sum from 1 to 10 is", sum(1, 10))
10    print("Sum from 20 to 37 is", sum(20, 37))
11    print("Sum from 35 to 49 is", sum(35, 49))
12
13 main() # Call the main function
```

define main function
invoke sum

functions

Lines 1–6 define the function named **sum** with the two parameters **i1** and **i2**. Lines 8–11 define the **main** function that invokes **sum(1, 10)** to compute the sum from **1** to **10**, **sum(20, 37)** to compute the sum from **20** to **37**, and **sum(35, 49)** to compute the sum from **35** to **49**.

A *function* is a collection of statements grouped together that performs an operation. In earlier chapters, you learned about such functions as **eval("numericString")** and **random.randint(a, b)**. When you call the **random.randint(a, b)** function, for example, the system actually executes the statements in the function and returns the result. In this chapter, you will learn how to define and use functions and apply function abstraction to solve complex problems.

6.2 Defining a Function



A function definition consists of the function's name, parameters, and body.

The syntax for defining a function is as follows:

```
def functionName(list of parameters)
    # Function body
```



VideoNote
Use functions

Let's look at a function created to find which of two numbers is bigger. This function, named `max`, has two parameters, `num1` and `num2`, the larger of which is returned by the function. Figure 6.1 illustrates the components of this function.

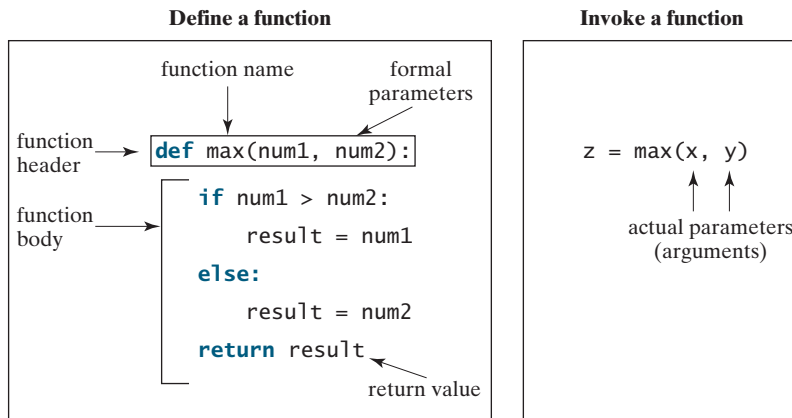


FIGURE 6.1 You can define a function and invoke it with arguments.

A function contains a header and body. The *header* begins with the `def` keyword, followed by the function's name and parameters, and ends with a colon.

The variables in the function header are known as *formal parameters* or simply *parameters*. A parameter is like a placeholder: When a function is invoked, you pass a value to the parameter. This value is referred to as an *actual parameter* or *argument*. Parameters are optional; that is, a function may not have any parameters. For example, the `random.random()` function has no parameters.

Some functions return a value, while other functions perform desired operations without returning a value. If a function returns a value, it is called a *value-returning function*.

The function body contains a collection of statements that define what the function does. For example, the function body of the `max` function uses an `if` statement to determine which number is larger and return the value of that number. A return statement using the keyword `return` is required for a value-returning function to return a result. The function terminates when a `return` statement is executed.

function header

formal parameters

parameters

actual parameter

argument

value-returning function

6.3 Calling a Function

Calling a function executes the code in the function.

In a function's definition, you define what it is to do. To use a function, you have to *call* or *invoke* it. The program that calls the function is called a *caller*. There are two ways to call a function, depending on whether or not it returns a value.

If the function returns a value, a call to that function is usually treated as a value. For example,

```
larger = max(3, 4)
```

calls `max(3, 4)` and assigns the result of the function to the variable `larger`.

Another example of a call that is treated as a value is

```
print(max(3, 4))
```

which prints the *return value* of the function call `max(3, 4)`.



Key Point

caller

return value

If a function does not return a value, the call to the function must be a statement. For example, the `print` function does not return a value. The following call is a statement:

```
print("Programming is fun!")
```



Note

A value-returning function also can be invoked as a statement. In this case, the return value is ignored. This is rare but is permissible if the caller is not interested in the return value.

When a program calls a function, program control is transferred to the called function. A called function returns control to the caller when its return statement is executed or the function is finished.

Listing 6.1 shows a complete program that is used to test the `max` function.

LISTING 6.1 TestMax.py

define max function

main function

invoke max

```
1 # Return the max of two numbers
2 def max(num1, num2):
3     if num1 > num2:
4         result = num1
5     else:
6         result = num2
7
8     return result
9
10 def main():
11     i = 5
12     j = 2
13     k = max(i, j) # Call the max function
14     print("The larger number of", i, "and", j, "is", k)
15
16 main() # Call the main function
```



The larger number of 5 and 2 is 5



	Line#	i	j	k	num1	num2	result
	11	5					
	12		2				
Invoke max	2				5	2	
	4						5
	13			5			

main function

This program contains the `max` and `main` functions. The program script invokes the `main` function in line 16. By convention, programs often define a function named `main` that contains the main functionality for a program.

How is this program executed? The interpreter reads the script in the file line by line starting from line 1. Since line 1 is a comment, it is ignored. When it reads the function header in line 2, it stores the function with its body (lines 2–8) in the memory. Remember that a function’s definition defines the function, but it does not cause the function to execute. The interpreter then reads the definition of the `main` function (lines 10–14) to the memory. Finally, the interpreter reads the statement in line 16, which invokes the `main` function and causes the `main` function to be executed. The control is now transferred to the `main` function, as shown in Figure 6.2.

execution

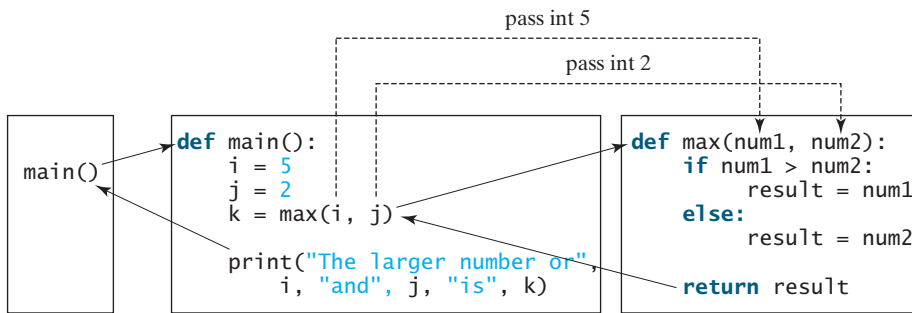


FIGURE 6.2 When a function is invoked, the control is transferred to the function. When the function is finished, the control is returned to where the function was called.

The execution of the `main` function begins in line 11. It assigns `5` to `i` and `2` to `j` (lines 11–12) and then invokes `max(i, j)` (line 13).

When the `max` function is invoked (line 13), variable `i`'s value is passed to `num1` and variable `j`'s value is passed to `num2`. The control is transferred to the `max` function, and the `max` function is executed. When the `return` statement in the `max` function is executed, the `max` function returns the control to its caller (in this case the `main` function).

After the `max` function is finished, the returned value from the `max` function is assigned to `k` (line 13). The `main` function prints the result (line 14). The `main` function is now finished, and it returns the control to its caller (line 16). The program is now finished.



Note

Here `main` is defined after `max`. In Python, functions can be defined in any order in a script file as long as the function is in the memory when it is called. You can also define `main` before `max`.

max function

order of functions

6.3.1 Call Stacks

Each time a function is invoked, the system creates an *activation record* that stores its arguments and variables for the function and places the activation record in an area of memory known as a *call stack*. A call stack is also known as an execution stack, runtime stack, or machine stack, and is often shortened to just “the stack.” When a function calls another function, the caller's activation record is kept intact and a new activation record is created for the new function call. When a function finishes its work and returns control to its caller, its activation record is removed from the call stack.

activation record

stack

A call stack stores the activation records in a last-in, first-out fashion. The activation record for the function that is invoked last is removed first from the stack. Suppose function `m1` calls function `m2`, and then `m3`. The runtime system pushes `m1`'s activation record into the stack, then `m2`'s, and then `m3`'s. After `m3` is finished, its activation record is removed from the stack. After `m2` is finished, its activation record is removed from the stack. After `m1` is finished, its activation record is removed from the stack.

Understanding call stacks helps us comprehend how functions are invoked. When the `main` function is invoked, an activation record is created to store variables `i` and `j`, as shown in Figure 6.3a. Remember that all data in Python are objects. Python creates and stores objects in a separate memory space called *heap*. Variables `i` and `j` actually contain reference values to `int` objects `5` and `2`, as shown in Figure 6.3a.

heap

Invoking `max(i, j)` passes the values `i` and `j` to parameters `num1` and `num2` in the `max` function. So now `num1` and `num2` reference `int` objects `5` and `2`, as shown in Figure 6.3b. The `max` function finds the maximum number and assigns it to `result`, so `result` now references `int` object `5`, as shown in Figure 6.3c. The result is returned to the `main` function

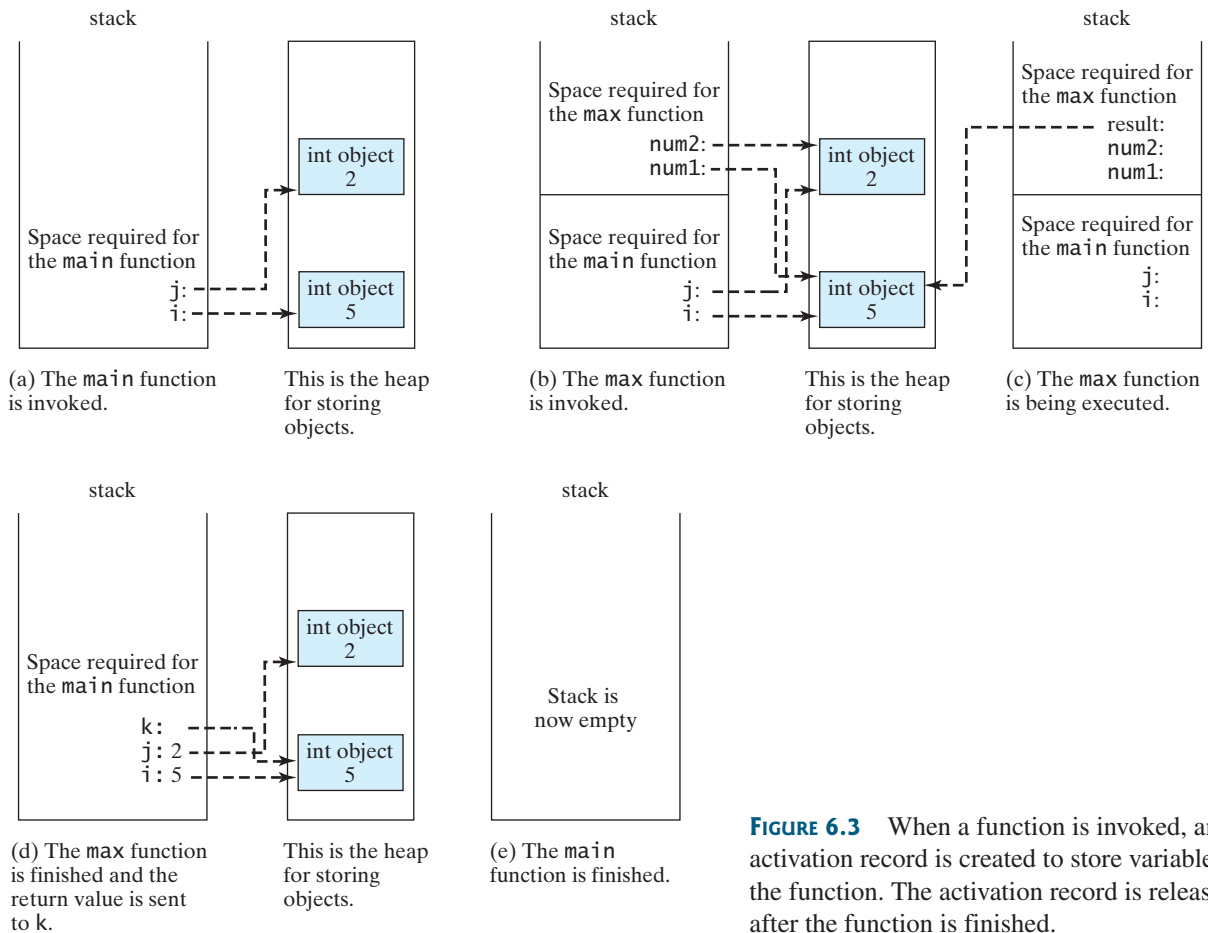


FIGURE 6.3 When a function is invoked, an activation record is created to store variables in the function. The activation record is released after the function is finished.

and assigned to variable `k`. Now `k` references `int` object 5, as shown in Figure 6.3d. After the `main` function is finished, the stack is empty, as shown in Figure 6.3e. The objects in the heap are automatically destroyed by the Python interpreter when they are no longer needed.

6.4 Functions with/without Return Values



A function does not have to return a value.

The preceding section gives an example of a value-returning function. This section shows how to define and invoke a function that does not return a value. Such a function is commonly known as a *void function* in programming terminology.

The program in Listing 6.2 defines a function named `printGrade` and invokes it to print the grade for a given score.

LISTING 6.2 PrintGradeFunction.py

`printGrade` function

```
1 # Print grade for the score
2 def printGrade(score):
3     if score >= 90.0:
4         print('A')
5     elif score >= 80.0:
6         print('B')
7     elif score >= 70.0:
8         print('C')
```

```

9     elif score >= 60.0:
10         print('D')
11     else:
12         print('F')
13
14 def main():
15     score = eval(input("Enter a score: "))
16     print("The grade is ", end = "")
17     printGrade(score)
18
19 main() # Call the main function

```

main function

invoke printGrade

```

Enter a score: 78.5
The grade is C

```



The `printGrade` function does not return any value. So, it is invoked as a statement in line 17 in the `main` function.

To see the differences between a function that does not return a value and a function that returns a value, let's redesign the `printGrade` function to return a value. We call the new function that returns the grade, `getGrade`, as shown in Listing 6.3.

LISTING 6.3 ReturnGradeFunction.py

```

1  # Return the grade for the score
2  def getGrade(score):
3      if score >= 90.0:
4          return 'A'
5      elif score >= 80.0:
6          return 'B'
7      elif score >= 70.0:
8          return 'C'
9      elif score >= 60.0:
10         return 'D'
11     else:
12         return 'F'
13
14 def main():
15     score = eval(input("Enter a score: "))
16     print("The grade is", getGrade(score))
17
18 main() # Call the main function

```

getGrade function

main function

invoke getGrade

```

Enter a score: 78.5
The grade is C

```



The `getGrade` function defined in lines 2–12 returns a character grade based on the numeric score value. It is invoked in line 16.

The `getGrade` function returns a character, and it can be invoked and used just like a character. The `printGrade` function does not return a value, and it must be invoked as a statement.

None function

**Note**

Technically, every function in Python returns a value whether you use **return** or not. If a function does not return a value, by default, it returns a special value **None**. For this reason, a function that does not return a value is also called a **None function**. The **None** value can be assigned to a variable to indicate that the variable does not reference any object. For example, if you run the following program:

```
def sum(number1, number2):
    total = number1 + number2

print(sum(1, 2))
```

you will see the output is **None**, because the **sum** function does not have a return statement. By default, it returns **None**.

return in None function

**Note**

A **return** statement is not needed for a **None** function, but it can be used for terminating the function and returning control to the function's caller. The syntax is simply

return

or

return None

This is rarely used, but it is sometimes useful for circumventing the normal flow of control in a function that does not return any value. For example, the following code has a return statement to terminate the function when the score is invalid.

```
# Print grade for the score
def printGrade(score):
    if score < 0 or score > 100:
        print("Invalid score")
        return # Same as return None

    if score >= 90.0:
        print('A')
    elif score >= 80.0:
        print('B')
    elif score >= 70.0:
        print('C')
    elif score >= 60.0:
        print('D')
    else:
        print('F')
```



MyProgrammingLab™

- 6.1** What are the benefits of using a function?
- 6.2** How do you define a function? How do you invoke a function?
- 6.3** Can you simplify the **max** function in Listing 6.1 by using a conditional expression?
- 6.4** True or false? A call to a **None** function is always a statement itself, but a call to a value-returning function is always a component of an expression.
- 6.5** Can you have a **return** statement in a **None** function? Does the **return** statement in the following function cause syntax errors?

```
def xFunction(x, y):
    print(x + y)
    return
```

- 6.6** Define the terms function header, parameter, and argument.
- 6.7** Write function headers for the following functions (and indicate whether the function returns a value):
- Computing a sales commission, given the sales amount and the commission rate.
 - Printing the calendar for a month, given the month and year.
 - Computing a square root.
 - Testing whether a number is even, and returning true if it is.
 - Printing a message a specified number of times.
 - Computing the monthly payment, given the loan amount, number of years, and annual interest rate.
 - Finding the corresponding uppercase letter, given a lowercase letter.
- 6.8** Identify and correct the errors in the following program:

```

1  def function1(n, m):
2  function2(3.4)
3
4  def function2(n):
5      if n > 0:
6          return 1
7      elif n == 0:
8          return 0
9      elif n < 0:
10         return -1
11
12 function1(2, 3)

```

- 6.9** Show the output of the following code:

```

1  def main():
2      print(min(5, 6))
3
4  def min(n1, n2):
5      smallest = n1
6      if n2 < smallest:
7          smallest = n2
8
9  main() # Call the main function

```

- 6.10** What error will occur when you run the following code?

```

def main():
    print(min(min(5, 6), (51, 6)))

def min(n1, n2):
    smallest = n1
    if n2 < smallest:
        smallest = n2

main() # Call the main function

```

6.5 Positional and Keyword Arguments

A function's arguments can be passed as positional arguments or keyword arguments.

The power of a function is its ability to work with parameters. When calling a function, you need to pass arguments to parameters. There are two kinds of arguments: *positional arguments* and *keyword arguments*. Using positional arguments requires that the arguments be passed in



positional arguments

the same order as their respective parameters in the function header. For example, the following function prints a message `n` times:

```
def nPrintln(message, n):
    for i in range(n):
        print(message)
```

You can use `nPrintln('a', 3)` to print `a` three times. The `nPrintln('a', 3)` statement passes `a` to `message`, passes `3` to `n`, and prints `a` three times. However, the statement `nPrintln(3, 'a')` has a different meaning. It passes `3` to `message` and `a` to `n`. When we call a function like this, it is said to use *positional arguments*. The arguments must match the parameters in *order*, *number*, and *compatible type*, as defined in the function header.

keyword arguments

You can also call a function using *keyword arguments*, passing each argument in the form `name = value`. For example, `nPrintln(n = 5, message = "good")` passes `5` to `n` and `"good"` to `message`. The arguments can appear in any order using keyword arguments.

mixing keyword and
positional arguments

It is possible to mix positional arguments with keyword arguments, but the positional arguments cannot appear after any keyword arguments. Suppose a function header is

```
def f(p1, p2, p3):
```

You can invoke it by using

```
f(30, p2 = 4, p3 = 10)
```

However, it would be wrong to invoke it by using

```
f(30, p2 = 4, 10)
```

because the positional argument `10` appears after the keyword argument `p2 = 4`.



MyProgrammingLab™

6.11 Compare positional arguments and keyword arguments.

6.12 Suppose a function header is as follows:

```
def f(p1, p2, p3, p4):
```

Which of the following calls are correct?

```
f(1, p2 = 3, p3 = 4, p4 = 4)
```

```
f(1, p2 = 3, 4, p4 = 4)
```

```
f(p1 = 1, p2 = 3, 4, p4 = 4)
```

```
f(p1 = 1, p2 = 3, p3 = 4, p4 = 4)
```

```
f(p4 = 1, p2 = 3, p3 = 4, p1 = 4)
```

6.6 Passing Arguments by Reference Values



When you invoke a function with arguments, each argument's reference is passed by value to the parameter in the function.

Because all data are objects in Python, a variable for an object is actually a reference to the object. When you invoke a function with arguments, the reference value of each argument is passed to the parameter. This is referred to as *pass-by-value* in programming terminology. For simplicity, we say that the value of an argument is passed to a parameter when invoking a function. The value is actually a reference value to the object.

pass-by-value

If the argument is a number or a string, the argument is not affected, regardless of the changes made to the parameter inside the function. Listing 6.4 gives an example.

LISTING 6.4 Increment.py

```

1 def main():
2     x = 1
3     print("Before the call, x is", x)
4     increment(x)
5     print("After the call, x is", x)
6
7 def increment(n):
8     n += 1
9     print("\tn inside the function is", n)
10
11 main() # Call the main function

```

invoke increment

increment n

```

Before the call, x is 1
      n inside the function is 2
After the call, x is 1

```



As shown in the output for Listing 6.4, the value of **x** (1) is passed to the parameter **n** to invoke the **increment** function (line 4). The parameter **n** is incremented by 1 in the function (line 8), but **x** is not changed no matter what the function does.

The reason is that numbers and strings are known as *immutable objects*. The contents of immutable objects cannot be changed. Whenever you assign a new number to a variable, Python creates a new object for the new number and assigns the reference of the new object to the variable.

immutable objects

Consider the following code:

```

>>> x = 4
>>> y = x
>>> id(x) # The reference of x
505408920
>>> id(y) # The reference of y is the same as the reference of x
505408920
>>>

```



You assign **x** to **y**, and both **x** and **y** now point to the same object for integer value 4, as shown in Figure 6.4a–b. But if you add 1 to **y**, a new object is created and assigned to **y**, as shown in Figure 6.4c. Now **y** refers to a new object, as shown in the following code:

```

>>> y = y + 1 # y now points to a new int object with value 5
>>> id(y)
505408936
>>>

```



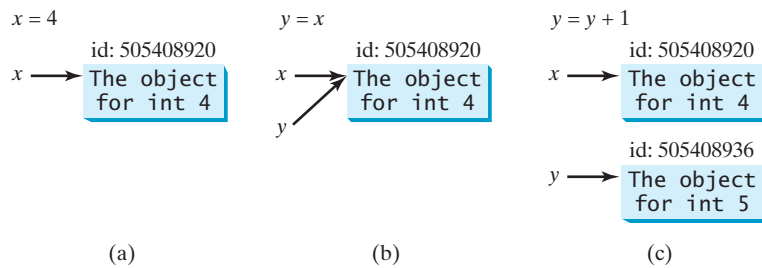


FIGURE 6.4 (a) 4 is assigned to `x`; (b) `x` is assigned to `y`; (c) `y + 1` is assigned to `y`.



6.13 What is pass-by-value?

6.14 Can the argument have the same name as its parameter?

6.15 Show the result of the following programs:

MyProgrammingLab™

```
def main():
    max = 0
    getMax(1, 2, max)
    print(max)

def getMax(value1, value2, max):
    if value1 > value2:
        max = value1
    else:
        max = value2

main()
```

(a)

```
def main():
    i = 1
    while i <= 6:
        print(function1(i, 2))
        i += 1

def function1(i, num):
    line = ""
    for j in range(1, i):
        line += str(num) + " "
        num *= 2
    return line

main()
```

(b)

```
def main():
    # Initialize times
    times = 3
    print("Before the call, variable",
          "times is", times)

    # Invoke nPrintln and display times
    nPrint("Welcome to CS!", times)
    print("After the call, variable",
          "times is", times)

    # Print the message n times
    def nPrint(message, n):
        while n > 0:
            print("n = ", n)
            print(message)
            n -= 1

    main()
```

(c)

```
def main():
    i = 0
    while i <= 4:
        function1(i)
        i += 1

        print("i is", i)

    def function1(i):
        line = " "
        while i >= 1:
            if i % 3 != 0:
                line += str(i) + " "
            i -= 1

        print(line)

    main()
```

(d)

- 6.16** For (a) in the preceding question, show the contents of the stack just before the function `max` is invoked, just as `max` is entered, just before `max` is returned, and right after `max` is returned.

6.7 Modularizing Code

Modularizing makes code easy to maintain and debug, and enables the code to be reused.



Functions can be used to reduce redundant code and enable code reuse. Functions can also be used to modularize code and improve a program's quality. In Python, you can place the function definition into a file called *module* with the file-name extension `.py`. The module can be later imported into a program for reuse. The module file should be placed in the same directory with your other programs. A module can contain more than one function. Each function in a module must have a different name. Note that the `turtle`, `random`, and `math` are the modules defined in the Python library, and thus they can be imported into any Python program.

module

Listing 5.8, `GreatestCommonDivisor.py`, shows a program that prompts the user to enter two integers and displays their greatest common divisor. You can rewrite the program to use a function and place it into a module named `GCDFunction.py`, as shown in Listing 6.5.

LISTING 6.5 GCDFunction.py

```
1 # Return the gcd of two integers
2 def gcd(n1, n2):
3     gcd = 1 # Initial gcd is 1
4     k = 2 # Possible gcd
5
6     while k <= n1 and k <= n2:
7         if n1 % k == 0 and n2 % k == 0:
8             gcd = k # Update gcd
9             k += 1
10
11     return gcd # Return gcd
```

define gcd function

return gcd

Now we write a separate program to use the `gcd` function, as shown in Listing 6.6.

LISTING 6.6 TestGCDFunction.py

```
1 from GCDFunction import gcd # Import the gcd function
2
3 # Prompt the user to enter two integers
4 n1 = eval(input("Enter the first integer: "))
5 n2 = eval(input("Enter the second integer: "))
6
7 print("The greatest common divisor for", n1,
8       "and", n2, "is", gcd(n1, n2))
```

import gcd

get input

invoke gcd

```
Enter the first integer: 45
Enter the second integer: 75
The greatest common divisor for 45 and 75 is 15
```



import from module

Line 1 imports the `gcd` function from the `GCDFunction` module, which enables you to invoke `gcd` in this program (line 8). You can also import it using the following statement:

```
import GCDFunction
```

Using this statement, you would have to invoke `gcd` using `GCDFunction.gcd`.

By encapsulating the code for obtaining the gcd in a function, this program has several advantages:

1. It isolates the problem for computing the gcd from the rest of the code in the program. Thus, the logic becomes clear and the program is easier to read.
2. Any errors for computing the gcd are confined to the `gcd` function, which narrows the scope of debugging.
3. The `gcd` function now can be reused by other programs.

What happens if you define two functions with the same name in a module? There is no syntax error in this case, but the latter function definition prevails.

Listing 6.7 applies the concept of code modularization to improve Listing 5.13, `PrimeNumber.py`. The program defines two new functions, `isPrime` and `printPrimeNumbers`. The `isPrime` function determines whether a number is prime, and the `printPrimeNumbers` function prints prime numbers.

LISTING 6.7 PrimeNumberFunction.py

```

1  # Check whether number is prime
2  def isPrime(number):
3      divisor = 2
4      while divisor <= number / 2:
5          if number % divisor == 0:
6              # If true, number is not prime
7              return False # number is not a prime
8              divisor += 1
9
10     return True # number is prime
11
12  def printPrimeNumbers(numberOfPrimes):
13      NUMBER_OF_PRIMES = 50 # Number of primes to display
14      NUMBER_OF_PRIMES_PER_LINE = 10 # Display 10 per line
15      count = 0 # Count the number of prime numbers
16      number = 2 # A number to be tested for primeness
17
18      # Repeatedly find prime numbers
19      while count < numberOfPrimes:
20          # Print the prime number and increase the count
21          if isPrime(number):
22              count += 1 # Increase the count
23
24              print(number, end = " ")
25              if count % NUMBER_OF_PRIMES_PER_LINE == 0:
26                  # Print the number and advance to the new line
27                  print()
28
29              # Check if the next number is prime
30              number += 1
31
32  def main():
33      print("The first 50 prime numbers are")

```

isPrime function

printPrimeNumbers function

invoke isPrime

```

34     printPrimeNumbers(50)
35
36 main() # Call the main function

```

invoke printPrimeNumbers



The first 50 prime numbers are

2	3	5	7	11	13	17	19	23	29
31	37	41	43	47	53	59	61	67	71
73	79	83	89	97	101	103	107	109	113
127	131	137	139	149	151	157	163	167	173
179	181	191	193	197	199	211	223	227	229

This program divides a large problem into two subproblems. As a result, the new program is easier to read and easier to debug. Moreover, the functions `printPrimeNumbers` and `isPrime` can be reused by other programs.

6.8 Case Study: Converting Decimals to Hexadecimals

This section presents a program that converts a decimal number to a hexadecimal number.



Hexadecimal numbers (introduced in Chapter 3) are often used in computer systems programming (see Appendix C for information on number systems). To convert a decimal number d to a hexadecimal number, you have to find the hexadecimal digits $h_n, h_{n-1}, h_{n-2}, \dots, h_2, h_1$, and h_0 such that

$$d = h_n \times 16^n + h_{n-1} \times 16^{n-1} + h_{n-2} \times 16^{n-2} + \dots + h_2 \times 16^2 + h_1 \times 16^1 + h_0 \times 16^0$$

These hexadecimal digits can be found by successively dividing d by 16 until the quotient is 0. The remainders are $h_0, h_1, h_2, \dots, h_{n-2}, h_{n-1}$, and h_n . The hexadecimal digits include the decimal digits 0,1,2,3,4,5,6,7,8,9 plus A which is the decimal value 10, B which is the decimal value 11, C which is 12, D which is 13, E which is 14, and F which is 15.

For example, the decimal number 123 is 7B in hexadecimal. The conversion is done as shown below:

$$\begin{array}{r}
 \begin{array}{r}
 0 \\
 16 \overline{) 7} \\
 \underline{0} \\
 7
 \end{array}
 \quad
 \begin{array}{r}
 7 \\
 16 \overline{) 123} \\
 \underline{112} \\
 11
 \end{array}
 \quad
 \begin{array}{l}
 \leftarrow \text{Quotient} \\
 \\
 \leftarrow \text{Remainder}
 \end{array}
 \end{array}$$

$\downarrow$ $\downarrow$
 h_1 h_0

The remainder of dividing 123 by 16 is 11, which is B in hexadecimal. The quotient of this division is 7. The remainder of dividing 7 by 16 is 7 and the quotient is 0. So, 7B is the hexadecimal number for 123.

The program in Listing 6.8 prompts the user to enter a decimal number and converts it into a hex number as a string.

LISTING 6.8 DecimalToHexConversion.py

```

1 # Convert a decimal to a hex as a string
2 def decimalToHex(decimalValue):

```

define decimalToHex

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

define toHexChar

get a letter

input decimal

```
hex = ""

while decimalValue != 0:
    hexValue = decimalValue % 16
    hex = toHexChar(hexValue) + hex
    decimalValue = decimalValue // 16

return hex

# Convert an integer to a single hex digit as a character
def toHexChar(hexValue):
    if 0 <= hexValue <= 9:
        return chr(hexValue + ord('0'))
    else: # 10 <= hexValue <= 15
        return chr(hexValue - 10 + ord('A'))

def main():
    # Prompt the user to enter a decimal integer
    decimalValue = eval(input("Enter a decimal number: "))

    print("The hex number for decimal",
          decimalValue, "is", decimalToHex(decimalValue))

main() # Call the main function
```



Enter a decimal number: 1234
The hex number for decimal 1234 is 4D2



	line#	decimalValue	hex	hexValue	toHexChar(hexValue)
	21	1234			
	3		""		
iteration 1 {	6			2	
	7		"2"		"2"
	8	77			
iteration 2 {	6			13	
	7		"D2"		"D"
	8	4			
iteration 3 {	6			4	
	7		"4D2"		"4"
	8	0			

The hex string is initially empty (line 3). The program uses the `decimalToHex` function (lines 2–10) to convert a decimal integer to a hex number as a string. The function gets the remainder of the division of the decimal integer by 16 (line 6). The remainder is converted into a character by invoking the `toHexChar` function and then appending to the hex string (line 7). Dividing the decimal number by 16 removes a hex digit from the number (line 8). The function repeatedly performs these operations in a loop until the quotient becomes 0 (lines 5–8).

The `toHexChar` function (lines 13–17) converts a `hexValue` between 0 and 15 into a hex character. If `hexValue` is between 0 and 9, it is converted to `chr(hexValue + ord('0'))` (line 15). For example, if `hexValue` is 5, `chr(hexValue + ord('0'))` returns 5. Similarly, if

`hexValue` is between 10 and 15, it is converted to `chr(hexValue - 10 + ord('A'))` (line 17). For example, if `hexValue` is 11, `chr(hexValue - 10 + ord('A'))` returns B.

6.9 The Scope of Variables

The scope of a variable is the part of the program where the variable can be referenced.



Chapter 2 introduced the scope of variables. This section discusses the scope of variables in the context of functions. A variable created inside a function is referred to as a *local variable*. Local variables can only be accessed within a function. The scope of a local variable starts from its creation and continues to the end of the function that contains the variable.

In Python, you can also use *global variables*. They are created outside all functions and are accessible to all functions in their scope. Consider the following examples.

scope of a variable
local variable
global variable

Example 1

```
1 globalVar = 1
2 def f1():
3     localVar = 2
4     print(globalVar)
5     print(localVar)
6
7 f1()
8 print(globalVar)
9 print(localVar) # Out of scope, so this gives an error
```

create a global variable

create a local variable

A global variable is created in line 1. It is accessed within the function in line 4 and outside the function in line 8. A local variable is created in line 3. It is accessed within the function in line 5. Attempting to access the variable from outside of the function causes an error in line 9.

Example 2

```
1 x = 1
2 def f1():
3     x = 2
4     print(x) # Displays 2
5
6 f1()
7 print(x) # Displays 1
```

create a global variable

create a local variable

Here a global variable `x` is created in line 1 and a local variable with the same name (`x`) is created in line 3. From this point on, the global variable `x` is not accessible in the function. Outside the function, the global variable `x` is still accessible. So, it prints 1 in line 7.

Example 3

```
1 x = eval(input("Enter a number: "))
2 if x > 0:
3     y = 4
4
5 print(y) # This gives an error if y is not created
```

create a variable

Here the variable `y` is created if `x > 0`. If you enter a positive value for `x` (line 1), the program runs fine. But if you enter a nonpositive value, line 5 produces an error because `y` is not created.

Example 4

```
1 sum = 0
2 for i in range(5):
```

variable i created

```

3     sum += i
4
5     print(i)

```

Here the variable `i` is created in the loop. After the loop is finished, `i` is `4`, so line 5 displays `4`.

global statement

You can bind a local variable in the global scope. You can also create a variable in a function and use it outside the function. To do either, use a `global` statement, as shown in the following example.

Example 5

global variable `x`

```

1  x = 1
2  def increase():
3      global x
4      x = x + 1
5      print(x) # Displays 2
6
7  increase()
8  print(x) # Displays 2

```

binding global variable

Here a global variable `x` is created in line 1 and `x` is bound in the function in line 3, which means that `x` in the function is the same as `x` outside of the function, so the program prints `2` in line 5 and in line 8.



Caution

Although global variables are allowed and you may see global variables used in other programs, it is not a good practice to allow them to be modified in a function, because doing so can make programs prone to errors. However, it is fine to define global constants so all functions in the module can share them.



Check
Point

MyProgrammingLab™

6.17 What is the printout of the following code?

```

def function(x):
    print(x)
    x = 4.5
    y = 3.4
    print(y)

x = 2
y = 4
function(x)
print(x)
print(y)

```

(a)

```

def f(x, y = 1, z = 2):
    return x + y + z

print(f(1, 1, 1))
print(f(y = 1, x = 2, z = 3))
print(f(1, z = 3))

```

(b)

6.18 What is wrong in the following code?

```

1  def function():
2      x = 4.5
3      y = 3.4
4      print(x)
5      print(y)
6
7  function()

```

```
8 print(x)
9 print(y)
```

6.19 Can the following code run? If so, what is the printout?

```
x = 10
if x < 0:
    y = -1
else:
    y = 1

print("y is", y)
```

6.10 Default Arguments

Python allows you to define functions with default argument values. The default values are passed to the parameters when a function is invoked without the arguments.



Listing 6.9 demonstrates how to define functions with *default argument* values and how to invoke such functions.

default argument

LISTING 6.9 DefaultArgumentDemo.py

```
1 def printArea(width = 1, height = 2):
2     area = width * height
3     print("width:", width, "\theight:", height, "\tarea:", area)
4
5 printArea() # Default arguments width = 1 and height = 2
6 printArea(4, 2.5) # Positional arguments width = 4 and height = 2.5
7 printArea(height = 5, width = 3) # Keyword arguments width
8 printArea(width = 1.2) # Default height = 2
9 printArea(height = 6.2) # Default width = 1
```

default argument

default arguments
positional arguments
keyword arguments
mixed arguments
mixed arguments

width: 1	height: 2	area: 2
width: 4	height: 2.5	area: 10.0
width: 3	height: 5	area: 15
width: 1.2	height: 2	area: 2.4
width: 1	height: 6.2	area: 6.2



Line 1 defines the `printArea` function with the parameters `width` and `height`. `width` has the default value `1` and `height` has the default value `2`. Line 5 invokes the function without passing an argument, so the program uses the default value `1` assigned to `width` and `2` to `height`. Line 6 invokes the function by passing `4` to `width` and `2.5` to `height`. Line 7 invokes the function by passing `3` to `width` and `5` to `height`. Note that you can also pass the argument by specifying the parameter name, as shown in lines 8 and 9.



Note

A function may mix parameters with default arguments and non-default arguments. In this case, the non-default parameters must be defined before default parameters.



Note

Many programming languages support a useful feature that allows you to define two functions with the same name in a module, but it is not supported in Python. With default arguments, you can define a function once, and call the function in many

different ways. This achieves the same effect as defining multiple functions with the same name in other programming languages. If you define multiple functions in Python, the later definition replaces the previous definitions.



MyProgrammingLab™

6.20 Show the printout of the following code:

```
def f(w = 1, h = 2):
    print(w, h)

f()
f(w = 5)
f(h = 24)
f(4, 5)
```

6.21 Identify and correct the errors in the following program:

```
1 def main():
2     nPrintln(5)
3
4 def nPrintln(message = "Welcome to Python!", n):
5     for i in range(n):
6         print(message)
7
8 main() # Call the main function
```

6.22 What happens if you define two functions in a module that have the same name?

6.11 Returning Multiple Values



*The Python **return** statement can return multiple values.*

Python allows a function to return multiple values. Listing 6.10 defines a function that takes two numbers and returns them in ascending order.

LISTING 6.10 MultipleReturnValueDemo.py

```
1 def sort(number1, number2):
2     if number1 < number2:
3         return number1, number2
4     else:
5         return number2, number1
6
7 n1, n2 = sort(3, 2)
8 print("n1 is", n1)
9 print("n2 is", n2)
```

return multiple values

receiving returned values



```
n1 is 2
n2 is 3
```

The **sort** function returns two values. When it is invoked, you need to pass the returned values in a simultaneous assignment.



MyProgrammingLab™

6.23 Can a function return multiple values? Show the printout of the following code:

```
1 def f(x, y):
2     return x + y, x - y, x * y, x / y
3
4 t1, t2, t3, t4 = f(9, 5)
5 print(t1, t2, t3, t4)
```

6.12 Case Study: Generating Random ASCII Characters

A character is coded using an integer. Generating a random character is to generate an integer.



Computer programs process numeric data and characters. You have seen many examples involving numeric data. It is also important to understand characters and how to process them. This section gives an example of generating random ASCII characters.

As introduced in Section 3.3, every ASCII character has a unique ASCII code between 0 and 127. To generate a random ASCII character, first generate a random integer between 0 and 127, and then use the `chr` function to obtain the character from the integer using the following code:

```
chr(randint(0, 127))
```

Let's consider how to generate a random lowercase letter. The ASCII codes for lowercase letters are consecutive integers starting with the code for `a`, then for `b`, `c`, ..., and `z`. The code for `a` is

```
ord('a')
```

So a random integer between `ord('a')` and `ord('z')` is

```
randint(ord('a'), ord('z'))
```

Therefore, a random lowercase letter is

```
chr(randint(ord('a'), ord('z')))
```

Thus, a random character between any two characters `ch1` and `ch2` with `ch1 < ch2` can be generated as follows:

```
chr(randint(ord(ch1), ord(ch2)))
```

This is a simple but useful discovery. In Listing 6.11 we create a module named `RandomCharacter.py` with five functions that randomly generate specific types of characters. You can use these functions in your future projects.

LISTING 6.11 RandomCharacter.py

```
1 from random import randint # import randint
2
3 # Generate a random character between ch1 and ch2
4 def getRandomCharacter(ch1, ch2):
5     return chr(randint(ord(ch1), ord(ch2)))
6
7 # Generate a random lowercase letter
8 def getRandomLowerCaseLetter():
9     return getRandomCharacter('a', 'z')
10
11 # Generate a random uppercase letter
12 def getRandomUpperCaseLetter():
13     return getRandomCharacter('A', 'Z')
14
15 # Generate a random digit character
16 def getRandomDigitCharacter():
17     return getRandomCharacter('0', '9')
18
19 # Generate a random character
20 def getRandomASCIICharacter():
21     return chr(randint(0, 127))
```

Listing 6.12 is a test program that displays 175 random lowercase letters.

LISTING 6.12 TestRandomCharacter.py

```

1  import RandomCharacter
2
3  constants
4  NUMBER_OF_CHARS = 175 # Number of characters to generate
5  CHARS_PER_LINE = 25 # Number of characters to display per line
6
7  # Print random characters between 'a' and 'z', 25 chars per line
8  for i in range(NUMBER_OF_CHARS):
9      lowercase letter
10     print(RandomCharacter.getRandomLowerCaseLetter(), end = " ")
11     if (i + 1) % CHARS_PER_LINE == 0:
12         print() # Jump to the new line

```



```

gmjshezfkg tazqgmswfc lrao
pnrunulnwmaztlfjedmpchcif
lalqdgivxkxpbzulrmqmbhikr
lbnrjlsopfxahssqhwuuljvbe
xbhdotzhpehbqmuwsfktwsoli
cbuwkzgxpm tzihgatds l vbwbz
bfesoklwbhnooygiigzdxuqni

```

Line 1 imports the `RandomCharacter` module, because the program invokes the function defined in this module.

Invoking `getRandomLowerCaseLetter()` returns a lowercase letter (line 8).

Note that the function `getRandomLowerCaseLetter()` does not have any parameters, but you still have to use the parentheses when defining and invoking it.



MyProgrammingLab™

- 6.24** Write an expression that returns a random integer between **34** and **55**, inclusively.
- 6.25** Write an expression that returns a random character between **B** and **M**, inclusively.
- 6.26** Write an expression that returns a random number between **6.5** and **56.5** (excluding 56.5).
- 6.27** Write an expression that returns a random lowercase letter.

6.13 Function Abstraction and Stepwise Refinement



Function abstraction is achieved by separating the use of a function from its implementation.

function abstraction

information hiding



VideoNote
Divide and conquer

divide and conquer

The key to developing software is to apply the concept of abstraction. You will learn many levels of abstraction from this book. *Function abstraction* separates the use of a function from its implementation. A client program, called simply the *client*, can use a function without knowing how it is implemented. The details of the implementation are encapsulated in the function and hidden from the client that invokes the function. This is known as *information hiding* or *encapsulation*. If you decide to change the implementation, the client program will not be affected, provided that you do not change the function header. The implementation of the function is hidden from the client in a “black box,” as shown in Figure 6.5.

You have already used many of Python’s built-in functions; you used these in client programs. You know how to write the code to invoke these functions in your program, but as a user of these functions, you are not required to know how they are implemented.

The concept of function abstraction can be applied to the process of developing programs. When writing a large program, you can use the “*divide-and-conquer*” strategy, also known as

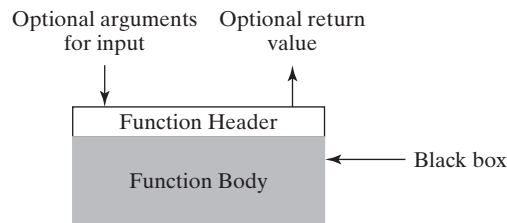


FIGURE 6.5 The function body can be thought of as a black box that contains the detailed implementation of the function.

stepwise refinement, to break down the problem into subproblems. The subproblems can be further divided into smaller, more manageable ones. stepwise refinement

Suppose you write a program that displays the calendar for a given month of the year. The program prompts the user to enter the year and the month, and then it displays the entire calendar for the month, as shown in the following sample run:

```
Enter full year (e.g., 2001): 2011 
Enter month as number between 1 and 12: 9 

September 2011
-----
Sun Mon Tue Wed Thu Fri Sat
      1  2  3
 4   5  6  7  8  9 10
11  12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30
```



Let's use this example to demonstrate the divide-and-conquer approach.

6.13.1 Top-Down Design

How would you get started writing such a program? Would you immediately start coding? Beginning programmers often start by trying to work out the solution to every detail. Although details are important in the final program, concern for detail in the early stages may block the problem-solving process. To make problem solving flow as smoothly as possible, this example begins by using function abstraction to isolate details from design and only later implements the details.

For this example, the problem is first broken into two subproblems: (1) get input from the user and (2) print the calendar for the month. At this stage, you should be concerned with what the subproblems will achieve, not with how to get input and print the calendar for the month. You can draw a structure chart to help visualize the decomposition of the problem (see Figure 6.6a).

You can use the `input` function to read input for the year and the month. The problem of printing the calendar for a given month can be broken into two subproblems: (1) print the month title and (2) print the month body, as shown in Figure 6.6b. The month title consists of three lines: month and year, a dashed line, and the names of the seven days of the week. You need to get the month name (e.g., January) from the numeric month (e.g., 1). This is accomplished in `getMonthName` (see Figure 6.7a).

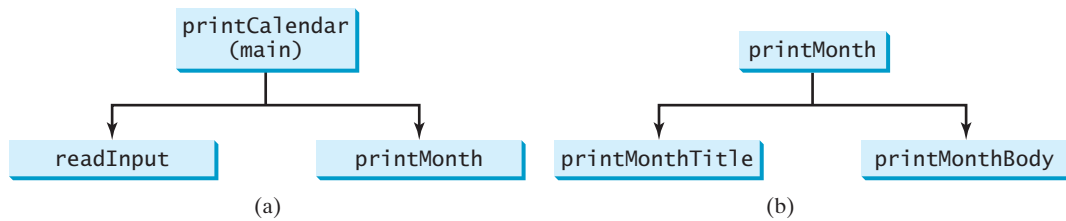


FIGURE 6.6 The structure chart shows that the `printCalendar` problem is divided into two subproblems, `readInput` and `printMonth`, and that `printMonth` is divided into two smaller subproblems, `printMonthTitle` and `printMonthBody`.

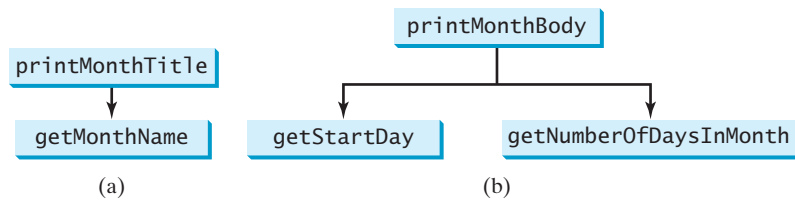


FIGURE 6.7 (a) To `printMonthTitle`, you need `getMonthName`. (b) The `printMonthBody` problem is refined into several smaller problems.

In order to print the month body, you need to know which day of the week is the first day of the month (`getStartDay`) and how many days the month has (`getNumberOfDaysInMonth`), as shown in Figure 6.7b. For example, December 2005 has 31 days, and December 1, 2005, is a Thursday.

How would you get the start day for the first date in a month? There are several ways to do so. Assume you know that the start day for January 1, 1800, was Wednesday (`START_DAY_FOR_JAN_1_1800 = 3`). You could compute the total number of days (`totalNumberOfDays`) between January 1, 1800, and the first date of the calendar month. The start day for the calendar month is $(\text{totalNumberOfDays} + \text{startDay1800}) \% 7$, since every week has seven days. Thus, the `getStartDay` problem can be further refined as `getTotalNumberOfDays`, as shown in Figure 6.8a.

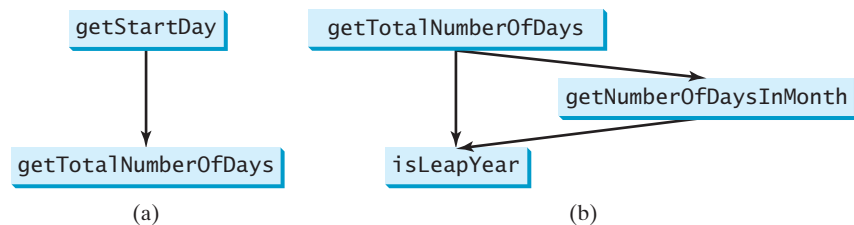


FIGURE 6.8 (a) To `getStartDay`, you need `getTotalNumberOfDays`. (b) The `getTotalNumberOfDays` problem is refined into two smaller problems.

To get the total number of days, you need to know whether the year is a leap year and the number of days in each month. Therefore, `getTotalNumberOfDays` needs to be further refined into two subproblems: `isLeapYear` and `getNumberOfDaysInMonth`, as shown in Figure 6.8b. The complete structure chart is shown in Figure 6.9.

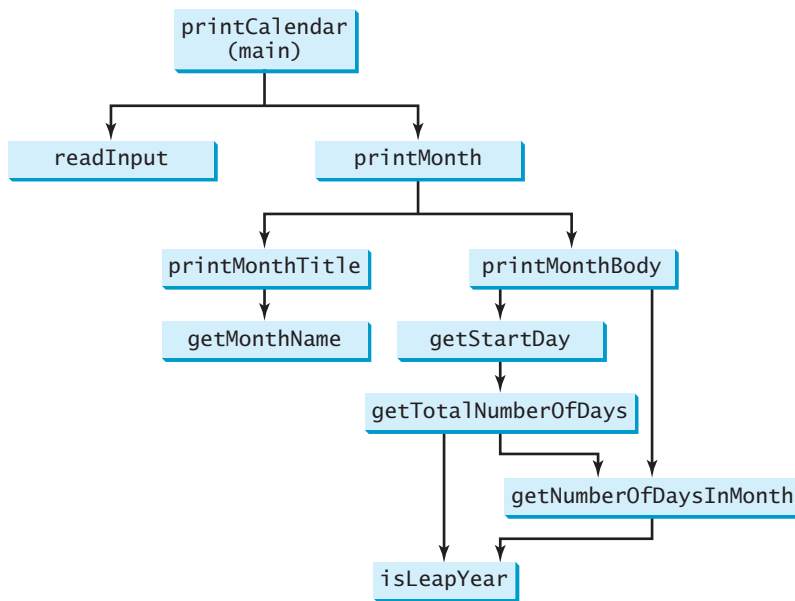


FIGURE 6.9 The structure chart shows the hierarchical relationship of the subproblems in the program.

6.13.2 Top-Down and/or Bottom-Up Implementation

Now let's turn our attention to implementation. In general, a subproblem corresponds to a function in the implementation, although some are so simple that this is unnecessary. You need to decide which modules to implement as functions and which to combine in other functions. Decisions of this kind should be based on the way that the overall program will be easier to read. In this example, the subproblem **readInput** can be simply implemented in the **main** function.

You can use either a “top-down” or a “bottom-up” approach. The top-down approach implements one function in the structure chart at a time from the top to the bottom. A *stub*, which is a simple but incomplete version of a function, can be used for the functions waiting to be implemented. Stubs enable you to build the framework of a program quickly. Implement the **main** function first, then use a stub for the **printMonth** function. For example, let **printMonth** display the year and the month in the stub. Thus, your program may begin like this:

top-down approach
stub

```

# A stub for printMonth may look like this
def printMonth(year, month):
    print(year, month)

# A stub for printMonthTitle may look like this
def printMonthTitle(year, month):
    print("printMonthTitle")

# A stub for getMonthBody may look like this
def getMonthBody(year, month):
    print("getMonthBody")

# A stub for getMonthName may look like this
def getMonthName(month):
    print("getMonthName")
  
```

```

# A stub for getStartDay may look like this
def getStartDay(year, month):
    print("getStartDay")

# A stub for getTotalNumberOfDays may look like this
def getTotalNumberOfDays(year, month):
    print("getTotalNumberOfDays")

# A stub for getNumberOfDaysInMonth may look like this
def getNumberOfDaysInMonth(year, month):
    print("getNumberOfDaysInMonth")

# A stub for isLeapYear may look like this
def isLeapYear(year):
    print("isLeapYear")

def main():
    # Prompt the user to enter year and month
    year = eval(input("Enter full year (e.g., 2001): "))
    month = eval(input(("Enter month as number between 1 and 12: ")))

    # Print calendar for the month of the year
    printMonth(year, month)

main() # Call the main function

```

bottom-up approach
driver

Run and test the program, and fix any errors. You can now implement the `printMonth` function. For functions invoked from the `printMonth` function, you can again use stubs.

The bottom-up approach implements one function in the structure chart at a time from the bottom to the top. For each function implemented, write a test program, known as the *driver*, to test it.

The top-down and bottom-up approaches are both fine. Both approaches implement functions incrementally, help to isolate programming errors, and make debugging easy. They can be used together.

6.13.3 Implementation Details

The `isLeapYear(year)` function can be implemented using the following code (see Section 4.12):

```
return year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)
```

Use the following facts to implement `getTotalNumberOfDaysInMonth(year, month)`:

- January, March, May, July, August, October, and December have 31 days.
- April, June, September, and November have 30 days.
- February has 28 days during a regular year and 29 days during a leap year. A regular year, therefore, has 365 days, and a leap year has 366 days.

To implement `getTotalNumberOfDays(year, month)`, you need to compute the total number of days (`totalNumberOfDays`) between January 1, 1800, and the first day of the calendar month. You could find the total number of days between the year 1800 and the calendar year and then figure out the total number of days prior to the calendar month in the calendar year. The sum of these two totals is `totalNumberOfDays`.

To print the calendar's body, first pad some space before the start day and then print the lines for every week.

The complete program is given in Listing 6.13.